MÁSTER EN REVERSING, ANÁLISIS DE MALWARE Y BUG HUNTING

MÓDULO 1. CRIPTOGRAFÍA

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*



Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Índice general

0. Herramientas **6**

1. Introducción a los fundamentos de la criptografía a través de los cifradores clásicos **7**
1.1. Criptovirología: la criptografía canalla detrás de la explosión del malware 7
1.2. ¿Qué puede hacer la criptografía por mí? 10
1.2.1. Los cuatro servicios de seguridad que te proporciona la criptografía 10
1.2.2. Las primitivas criptográficas que aportan los servicios de seguridad 11
1.2.3. Ya empiezo a sudar. ¿Me enteraré de algo? 11
1.3. Fundamentos del cifrado 12
1.4. Cifradores de sustitución 13
1.4.1. Sustitución monoalfabética: el cifrado de desplazamiento 13
1.4.2. Sustitución polialfabética: el cifrado de Vigenère 17
1.5. La libreta de un solo uso (*One-Time Pad*) 19
1.5.1. Cómo implementar la libreta de un solo uso con 1's y 0's 21
1.5.2. Seguridad de la libreta de un solo uso 22
1.5.3. Limitaciones prácticas de la libreta de un solo uso 23
1.6. Atacando la criptografía 24
1.6.1. Modelos de ataque contra el cifrado 24
1.6.2. Complejidad de los ataques 25

2. Aleatoriedad **28**
2.1. ¿Qué es la aleatoriedad? 29
2.2. Sin aleatoriedad, no hay seguridad 29
2.3. El desafío de la generación de secuencias aleatorias de bits 30
2.3.1. Generadores de bits verdaderamente aleatorios (TRNG) 30
2.3.2. Generadores de números aleatorios cuánticos (QRNG) 31
2.3.3. Generadores pseudoaleatorios (PRNG) 32
2.4. Generadores pseudoaleatorios inseguros 33
2.4.1. Generador Lineal Congruencial (LCG) 33
2.4.2. Mersenne Twister (MT) 35
2.5. Otros generadores pseudoaleatorios, ahora sí, seguros (CSPRBG) 35
2.5.1. RSA 36
2.5.2. Blum-Blum-Shub 36
2.5.3. Algoritmos criptográficos 37
2.6. Generadores de números pseudo-aleatorios criptográficamente seguros para programadores 37
2.6.1. Si la semilla es predecible, da igual el generador que uses 39
2.6.2. Y si reutilizas la semilla, estarás igual de mal 39
2.6.3. Y si además usas rand(), todavía peor 40
2.7. ¿Qué algoritmos de generación de números aleatorios utiliza el ransomware? 40
2.8. ¿Será esta secuencia aleatoria? Pruebas de aleatoriedad 40
2.8.1. Análisis de aleatoriedad con los postulados de Golomb 41
2.8.1.1. ¿Qué son los postulados de Golomb? 41
2.8.1.2. Importancia en criptografía 43
2.8.1.3. Limitaciones de los postulados de Golomb 43
2.8.2. Conclusión 43
2.9. Estándares de generación y tests de números pseudoaleatorios 44

3. Cifrado simétrico o de clave secreta **46**
3.1. Cifrado en flujo . . . 47
3.1.1. Registros de desplazamiento con retroalimentación lineal . . . 47
3.1.2. Registros de desplazamiento con retroalimentación no lineal . . . 49
3.1.3. Ejemplos de cifradores en flujo . . . 50
3.2. Cifrado en bloque . . . 52
3.2.1. Ejemplos de cifradores en bloque . . . 52
3.2.2. Los algoritmos de cifrado envejecen... y mueren . . . 54
3.2.3. Modos de operación del cifrado en bloque . . . 55
3.2.3.1. Modo Electronic Code Book (ECB) . . . 55
3.2.3.2. Modo Cipher Block Chaining (CBC) . . . 58
3.2.3.3. Modo Cipher Feedback (CFB) . . . 61
3.2.3.4. Modo Output Feedback (OFB) . . . 63
3.2.3.5. Counter mode (CTR) . . . 64
3.2.3.6. XTS . . . 66
3.2.3.7. Seguridad de los distintos modos de encadenamiento de bloques 67
3.3. Comparación entre cifradores en flujo y en bloque . . . 67
3.4. ¿Qué algoritmos de criptografía simétrica utiliza el ransomware? . . . 68
3.5. Cifrado autenticado con datos asociados . . . 68
3.6. Cómo dividir una clave secreta en múltiples claves para repartir entre varias personas 68
3.6.1. Reparto de secretos n de n . . . 69
3.6.2. Esquema de Reparto de Secretos de Shamir (SSSS) . . . 69
3.6.3. Criptografía Visual . . . 70
3.7. Tres tipos de aplicaciones para el cifrado simétrico . . . 71
3.8. Estandarización de algoritmos y pruebas . . . 71
3.8.1. Estándares de cifrado simétrico . . . 71
3.8.2. Cómo verificar que tu programa cifra/descifra correctamente . . . 72
3.9. El gran reto del ransomware: ¿cómo gestiono la clave? . . . 72

4. Cifrado asimétrico o de clave pública **73**
4.1. Nuevas direcciones en criptografía . . . 74
4.2. Aplicaciones de los criptosistemas de clave pública . . . 75
4.3. El criptosistema RSA . . . 75
4.3.1. Funcionamiento de RSA . . . 75
4.3.1.1. Cómo cifrar y descifrar con RSA . . . 75
4.3.1.2. Cómo generar parejas de claves para RSA . . . 76
4.3.1.3. Por sus primos los conoceréis . . . 78
4.3.2. Seguridad y ataques contra RSA . . . 79
4.3.2.1. A la búsqueda de primos enormes . . . 79
4.3.2.1.1. ¿Cómo de comunes son los números primos? . . . 80
4.3.2.1.2. Pruebas de primalidad . . . 81
4.3.2.2. La versión *libro de texto* de RSA es para la escuela, no para la vida real . . . 82
4.3.2.3. Los exponentes pequeños causan problemas grandes . . . 83
4.3.2.4. El tamaño del exponente privado también importa . . . 86
4.3.2.5. Padding: sin relleno adecuado no hay seguridad ninguna . . . 86
4.3.2.5.1. El relleno PKCS#1 v1.5 . . . 87
4.3.2.5.2. Relleno *Optimal Asymmetric Encryption Padding* (OAEP) 89
4.3.2.6. Otros ataques contra RSA . . . 90
4.4. El protocolo de acuerdo de secretos de Diffie-Hellman (DHKE) . . . 90
4.4.1. La teoría de grupos necesaria para entender el protocolo Diffie-Hellman . 90
4.4.2. Los pasos del protocolo de acuerdo de secretos de Diffie-Hellman (DHKE) 92
4.4.3. Seguridad de Diffie-Hellman: no todo vale . . . 95
4.4.3.1. ¿Secreto compartido o clave secreta? . . . 95
4.4.3.2. ¿El primer primo que encuentro vale? . . . 96
4.4.3.3. DH Anónimo: rápido, barato e ¿inseguro? . . . 98
4.4.3.4. Ataque Logjam contra el protocolo TLS . . . 98
4.5. El protocolo de acuerdo de secretos de Diffie-Hellman basado en Curvas Elípticas (ECDHKE) . . . 98
4.5.1. ¿Qué es una curva elíptica . . . 98
4.5.1.1. Curvas elípticas sobre los números reales . . . 98

4.5.1.2. Curvas elípticas sobre cuerpos finitos . . . 101
4.5.2. Criptografía de Curva Elíptica (ECC) . . . 102
4.5.3. Intercambio de claves Diffie-Hellman con curva elíptica (ECDHKE) . . . 105
4.5.4. Ataques contra ECDH . . . 106
4.6. Longitudes de clave y niveles de seguridad . . . 106
4.7. Criptografía, aleatoriedad y ataques de distinción . . . 107
4.7.1. El cifrado intermitente para evadir la detección estadística del ransomware 108
4.8. ¿Qué algoritmos de criptografía asimétrica utiliza el ransomware? . . . 108
4.9. Criptosistemas híbridos: la criptografía simétrica y la asimétrica son como la leche y el cacao en polvo . . . 108
4.9.1. Esquemas de cifrado y métodos de encapsulado de claves . . . 109
4.9.1.1. Esquemas de cifrado integrados . . . 110
4.9.1.2. Mecanismos de encapsulado de claves (KEM) . . . 110
4.9.2. Secreto perfecto hacia adelante gracias a las claves efímeras . . . 111
4.9.3. Estudio de caso #1: cifrado extremo-a-extremo de Whatsapp . . . 113
4.9.4. Estudio de caso #2: ransomware . . . 114
4.10. Estándares de cifrado asimétrico . . . 116

5. Funciones hash **118**
5.1. Funciones hash criptográficas . . . 118
5.1.1. Propiedad 1: resistencia a la preimagen . . . 120
5.1.2. Propiedad 2: resistencia a la 2ª preimagen . . . 121
5.1.3. Propiedad 3: resistencia a las colisiones . . . 121
5.2. La paradoja del cumpleaños . . . 123
5.3. Construcción de funciones hash . . . 124
5.3.1. Funciones hash dedicadas: la transformación de Merkle-Damgård (M-D) . 124
5.3.2. Funciones hash a partir de cifradores en bloque . . . 125
5.3.3. Crónica de la muerte anunciada de SHA-1 . . . 126
5.4. Aplicaciones de las funciones hash criptográficas . . . 126
5.4.1. Autenticación de mensajes . . . 126
5.4.2. Firmas digitales . . . 127
5.4.3. Otras aplicaciones . . . 127
5.5. Códigos de autenticación de mensajes . . . 128
5.5.1. MAC basado en hash criptográfico o HMAC . . . 130
5.5.1.1. Un intento ingenuo con funciones hash . . . 130
5.5.1.2. HMAC . . . 131
5.5.2. MAC basado en cifradores en bloque . . . 132
5.6. Cifrado autenticado . . . 133
5.6.1. GCM . . . 133
5.7. Estándares de funciones Hash . . . 135

6. Firma digital **136**
6.1. Esquema general de firma digital . . . 137
6.1.1. Primera aproximación a la firma digital . . . 137
6.1.2. Segunda aproximación a la firma digital . . . 138
6.1.3. Ataque de intermediario (o de la persona en el medio) . . . 138
6.1.4. Certificados digitales . . . 139
6.1.4.1. No todos los certificados fueron creados con el mismo nivel de confianza . . . 141
6.1.4.2. Cómo funcionan los certificados digitales . . . 142
6.1.5. Tercera aproximación a la firma digital . . . 145
6.1.6. Estudio de caso #1: cifrado/firmado de correos electrónicos con SMIME . 147
6.1.7. Estudio de caso #2: el estampillado de tiempo . . . 149
6.2. Esquema de firma digital con RSA . . . 150
6.2.1. Relleno de RSA . . . 152
6.2.1.1. Relleno PKCS#1 v1.5 . . . 152
6.2.1.2. The Probabilistic Signature Standard (PSS) . . . 154
6.2.2. Firmas ciegas con RSA . . . 154
6.2.3. Ataques a las firmas RSA . . . 155
6.2.3.1. Obtención del módulo a partir de varias firmas . . . 155
6.3. Esquema de firma digital con DSA . . . 155

6.3.0.1. Generación de claves . . . 156
6.3.0.2. Firma . . . 156
6.3.0.3. Verificación de firma . . . 156
6.4. Esquema de firma digital con ECDSA . . . 158
6.4.1. Generación de firmas con ECDSA . . . 158
6.4.2. Verificación de la firma con ECDSA . . . 158
6.4.3. Codificación de firmas ECDSA . . . 160
6.4.4. La importancia de k . . . 160
6.4.4.1. Nunca reutilices el número aleatorio . . . 160
6.4.4.2. Usa números aleatorios, sin partes conocidas . . . 161
6.4.4.3. Asegúrate de comprobar los valores de r y s . . . 162
6.4.4.4. Más ataques contra ECDSA . . . 162
6.5. Comparación de la eficiencia de firma entre RSA y ECDSA . . . 162
6.6. Criptografía con umbral y el futuro de las firmas digitales . . . 164
6.7. Estándares de firma digital . . . 165

A. Una cálida bienvenida a ASN.1 166

B. Biblioteca Jedi sobre criptografía para todos los niveles de inquietud y de conocimiento 172

Lección 0

Herramientas

A lo largo del curso de criptografía, utilizaremos diversas herramientas esenciales para analizar y aplicar diferentes técnicas criptográficas.

1. CyberChef es una plataforma en línea de procesamiento de datos que permite realizar operaciones de cifrado, decodificación, hash y manipulación de datos de manera sencilla y rápida.

2. Cryptool es un software educativo que facilita el aprendizaje de conceptos criptográficos a través de simulaciones y ejercicios interactivos.

3. OpenSSL es una robusta biblioteca de código abierto que permite implementar protocolos de seguridad como SSL/TLS, además de ofrecer funciones para generar claves, cifrar y descifrar mensajes.

4. SageMath es un sistema de software matemático que incluye herramientas de álgebra y teoría de números, fundamentales para abordar problemas de criptografía avanzada. Estas herramientas nos permitirán poner en práctica los conceptos teóricos, profundizando en su aplicación en la vida real.

5. Python no es necesario, pero sí altamente recomendble.

Lección 1

Introducción a los fundamentos de la criptografía a través de los cifradores clásicos

Objetivos de aprendizaje

1. Comprender la evolución histórica de la criptografía y su influencia en el desarrollo del malware moderno, especialmente el ransomware.
2. Conocer los cuatro servicios de seguridad que ofrece la criptografía: confidencialidad, integridad, autenticación y no repudio.
3. Analizar y aplicar los fundamentos de los cifradores clásicos, como los cifradores de sustitución y transposición.
4. Entender el funcionamiento y las limitaciones de la libreta de un solo uso y su papel en la seguridad criptográfica.

Competencias a desarrollar

1. Capacidad para implementar y romper cifradores clásicos utilizando técnicas de criptoanálisis básicas.
2. Habilidad para identificar y evaluar las diferentes amenazas y modelos de ataque en criptografía.
3. Competencia en el uso de primitivas criptográficas para proporcionar servicios de seguridad en sistemas informáticos.
4. Pensamiento crítico para analizar el uso ético y malicioso de la criptografía en contextos como la criptovirología.

1.1. Criptovirología: la criptografía canalla detrás de la explosión del malware

Durante siglos, la criptografía ha evocado imágenes de protección, privacidad y seguridad. Hasta que, en 1996, dos investigadores de la Universidad de Columbia en EEUU, Adam Young y Moti Yung, concibieron el primer criptovirus de la Historia:

> *En este trabajo presentamos la idea de la Criptovirología, demostrando que la criptografía también puede ser utilizada ofensivamente. Por ofensivo queremos decir que puede ser utilizada para montar ataques basados en la extorsión que causan pérdida de acceso a la información, pérdida de confidencialidad y fuga de información.*

En su artículo seminal, Young y Yung sentaron las bases teóricas de lo que sería el ransomware que hoy todos aborrecemos: la intersección entre la criptografía y el malware.

En su momento, el trabajo de Young y Yung no recibió atención por considerarse poco realista:

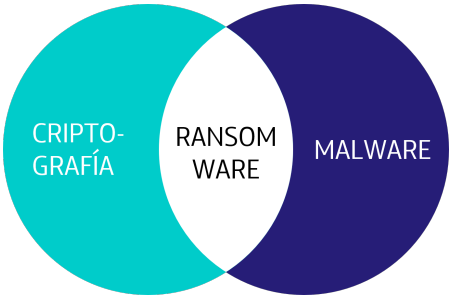


Figura 1.1: CRIPTOGRAFÍA + MALWARE = RANSOMWARE.

- Su propuesta requería el uso de criptografía asimétrica. Ni todo el hardware de la época poseía la capacidad computacional para trabajar con RSA, ni las restricciones de exportación de criptografía permitían el uso de longitudes de clave suficientemente robustas.
- No se vislumbraba medio impune de cobrar el rescate.

Por supuesto, con el aumento de la capacidad de cómputo y con el auge de las criptomonedas, el escenario cambió. A partir de mediados del 2010, tras la progresiva adopción de Bitcoin, el ransomware criptográfico experimentó un crecimiento sin precedentes. Su efecto ha sido, y sigue siendo, devastador. La Fig. 1.2 ilustra cómo a partir de 2016 el ransomware se convierte en una pandemia. El número de muestras encontradas diariamente se dispara hasta el punto de que ya no pueden mostrarse todas en una figura.

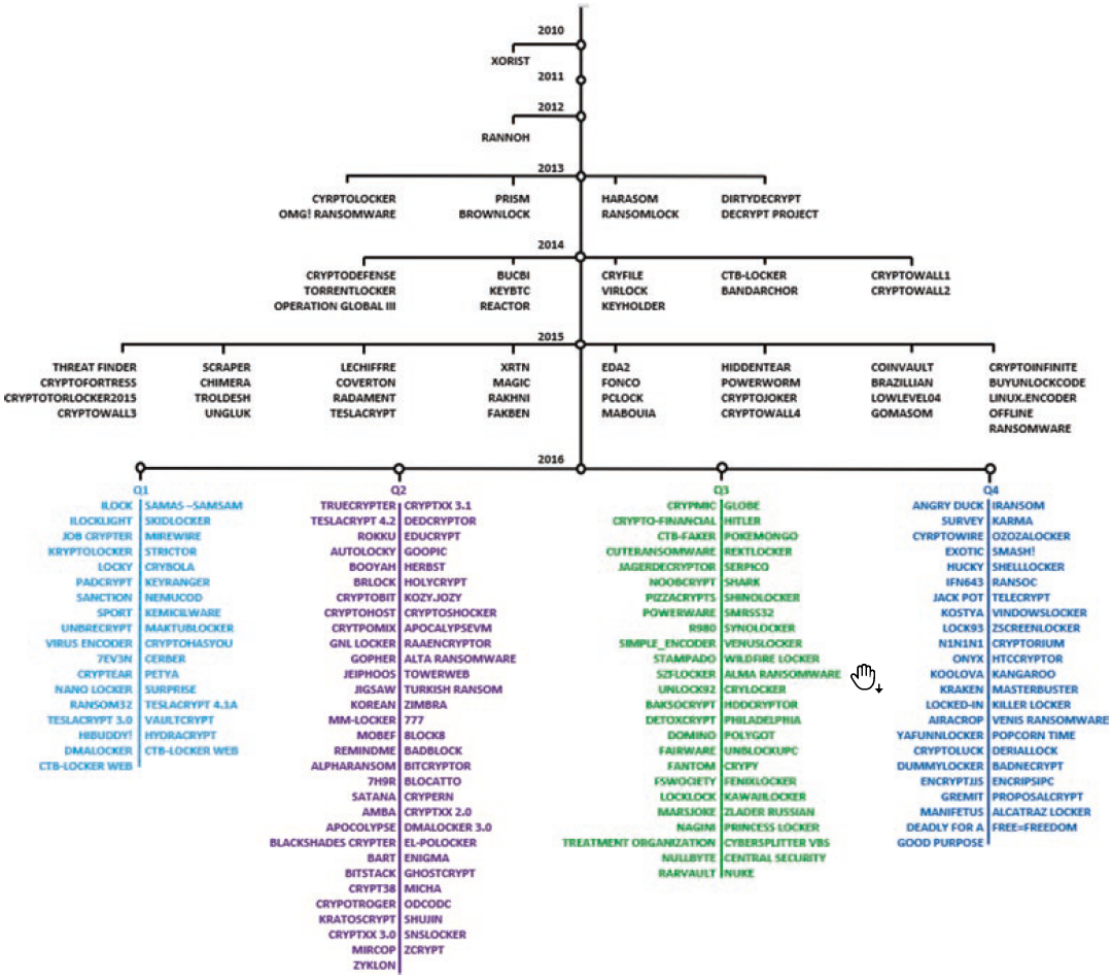


Figura 1.2: Línea temporal del ransomware desde 2010 hasta 2016 (Fuente: Matthew Ryan, *Ransomware Revolution: The Rise Of A Prodigious Cyber Threat*, Springer, 2021).

Un análisis más profundo revela que el cifrado está en el centro de casi todas las tecnologías que

han permitido el aumento del ransomware. La propagación del cifrado ha permitido y fomentado el desarrollo de una serie de tecnologías que sustentan los ataques de ransomware, como los que se detallan a continuación en la Fig. 1.3. Aunque muchas de estas tecnologías emergentes no se desarrollaron para impulsar los ciberataques, los atacantes las han adaptado y adoptado rápidamente para mejorar sus operaciones y retrasar la atribución de los ataques.



Figura 1.3: Bloques de construcción del ransomware (Fuente: Matthew Ryan, *Ransomware Revolution: The Rise Of A Prodigious Cyber Threat*, Springer, 2021).

A grandes rasgos, el proceso de ataque de los criptovirus bosquejado por Young y Yung atraviesa las tres fases ilustradas en la Fig. 1.4:

1. Infectar a la víctima.
2. Cifrar todos los archivos de ciertas extensiones a los que tenga acceso.
3. Pedir el rescate.

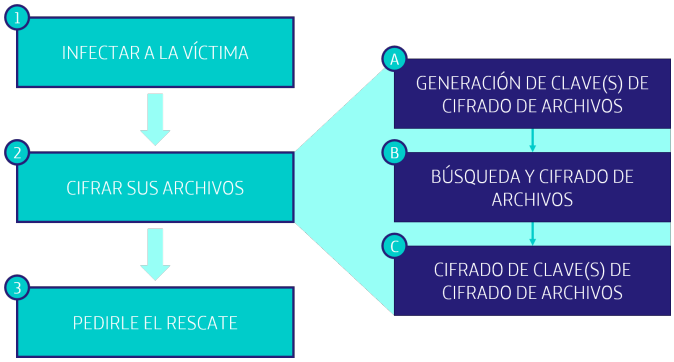


Figura 1.4: Proceso seguido por el ransomware para extorsionar a sus víctimas.

Para ser invulnerable, el ransomware criptográfico necesita dos elementos:

- **Criptografía robusta**: algoritmos de cifrado simétricos y asimétricos a prueba de balas.
- **Claves fuertes**: suficientemente largas y totalmente aleatorias.

En este curso explicaremos cómo funciona la criptografía y las técnicas de criptoanálisis para el reversing de muestras de malware que usan criptografía. Gracias a estos conocimientos, serás capaz de entender las fases de un ransomware bien diseñado para cifrar todos los archivos a los que tenga acceso:

1. **Generar de manera totalmente aleatoria e impredecible claves secretas para algoritmos simétricos robustos**: aprenderás cómo funcionan los generadores aleatorios y pseudoaleatorios.

2. **Cifrar con estas claves los archivos de la víctima**: te familiarizarás con la criptografía simétrica: cifradores en flujo, cifradores en bloque y sus modos de encadenamiento.

3. **Cifrar dichas claves secretas con claves públicas de algoritmos asimétricos**: entenderás los arcanos de RSA, Diffie-Hellam y el cifrado de curva elíptica.

En definitiva, en este curso aprenderás cuáles son los servicios de seguridad que aporta la criptografía y cómo los usan los cibercriminales.

1.2. ¿Qué puede hacer la criptografía por mí?

Al escuchar la palabra *criptografía*, la mayoría de la gente piensa en cifrar y descifrar. Pero la criptografía va mucho más allá de la protección de la confidencialidad de la información.

1.2.1. Los cuatro servicios de seguridad que te proporciona la criptografía

La criptografía puede utilizarse para ayudar a proporcionar los siguientes cuatro servicios de seguridad:

- **Confidencialidad**: se ocupa de detectar y disuadir la divulgación no autorizada de información, vamos, de mantener los secretos en secreto: tengo información y no quiero que nadie más la descubra a menos que yo se la revele. Los algoritmos de **cifrado** abordan directamente el problema de la **confidencialidad**. Estos algoritmos aceptan una clave secreta y luego proceden a cifrar el mensaje original, conocido como *texto claro*, y lo convierten en una información ininteligible conocida como *texto cifrado*, de tal manera que sólo el destinatario previsto, y nadie más, puede leerlo.

- **Integridad**: se ocupa de detectar e impedir la modificación no autorizada de la información. Con criptografía se realiza una **firma digital** de la información y basta que un solo bit de los datos cambie para que la firma sea diferente. Esto permite a las partes que se comunican realizar comprobaciones de integridad con respecto a su información y asegurarse de que nada ha cambiado durante el trayecto o una vez almacenada. Las **funciones de hash** suelen constituir una pieza clave de las firmas digitales.

- **Autenticación**: es la propiedad de atribuir una identidad al remitente de un mensaje o al autor de una acción. Uno de los métodos criptográficos preferidos para autenticar a las partes es mediante **certificados digitales**. Se basan en dos bloques de construcción básicos: las **funciones de hash** y el **cifrado de clave pública**, que se verán más adelante.

- El **no repudio** asegura que el remitente no puede negar haber enviado el mensaje.

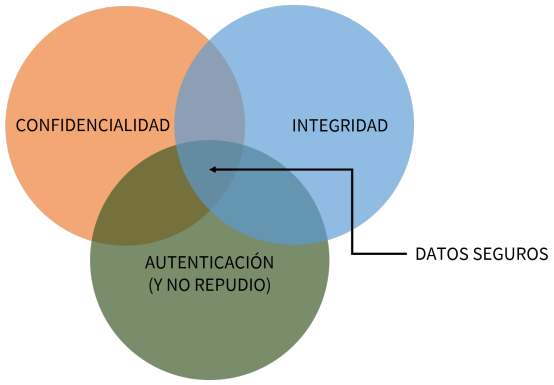


Figura 1.5: La criptografía proporciona los servicios de confidencialidad, integridad y autenticación (y no repudio).

1.2.2. Las primitivas criptográficas que aportan los servicios de seguridad

Estos servicios de seguridad se ofrecen a través de primitivas criptográficas. Las primitivas criptográficas son algoritmos criptográficos de bajo nivel bien establecidos que se utilizan como bloques básicos para construir protocolos criptográficos para sistemas de seguridad informática.

En este curso, resumido en la Fig. 1.6, echarás un vistazo a las tripas de las primitivas ya mencionadas y alguna más, dedicando una lección a cada una de ellas:

- **Lección 2 - Números aleatorios**: son necesarios para generar claves, vectores de inicialización, nonces, salts, y otros usos de importancia capital en criptografía.
- **Lección 3 - Cifrado de clave secreta**: permiten cifrar grandes volúmenes de datos a alta velocidad usando claves cortas.
- **Lección 4 - Cifrado de clave pública**: permiten cifrar/descifrar pequeñas cantidades de datos, fundamentalmente para intercambiar claves secretas y firmar hashes.
- **Lección 5 - Funciones hash**: aceptan un valor de longitud arbitraria a la entrada y devuelven un valor de longitud fija a la salida. Son unidireccionales en el sentido de que: 1) a partir de un valor de hash es computacionalmente inviable encontrar una entrada que genere esa salida; y 2) dados un valor de entrada y su salida es computacionalmente inviable encontrar otra entrada distinta que genere la misma salida.
- **Lección 6 - Firma digital**: consiste en una información digital añadida a los datos que permite verificar su integridad y su autoría.

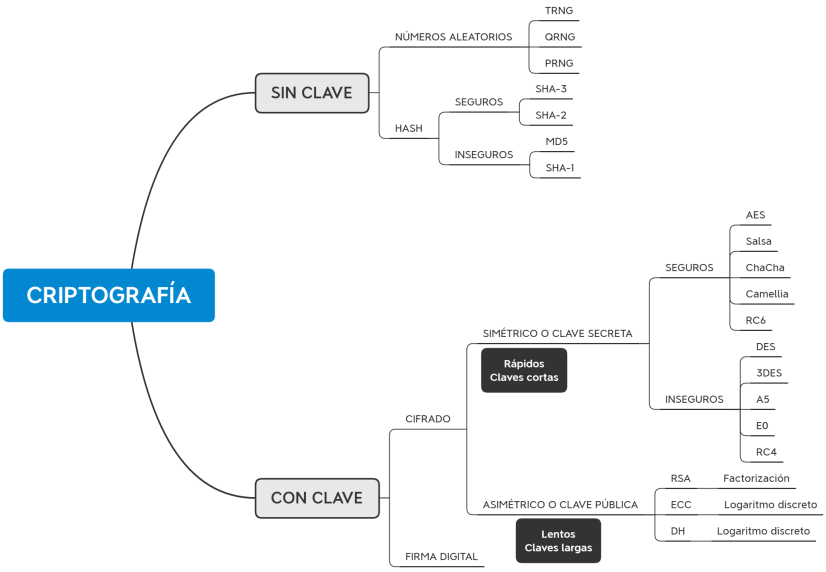


Figura 1.6: Resumen visual de las cinco primitivas criptográficas estudiadas en el curso.

1.2.3. Ya empiezo a sudar. ¿Me enteraré de algo?

Para entender a fondo la criptografía tienes que ser licenciado en matemáticas o un genio autodidacta de las matemáticas. Si te dicen cualquier otra cosa, ¡te están mintiendo! Pero, ¿quién necesita entender todos los entresijos matemáticos si no quiere dedicarse en cuerpo y alma a la criptografía? Para luchar contra el malware te basta con dominar las cuatro reglas $(+, -, \times, \div)$ y la operación módulo. Por supuesto, saber exponenciar viene muy bien, que en el fondo no es más que multiplicar repetidas veces. Si te manejas con estas operaciones (¿y quién no?), ya estás listo para zambullirte en el misterioso mundo de la criptografía.

¿Por dónde empezar? ¡Por los fundamentos!

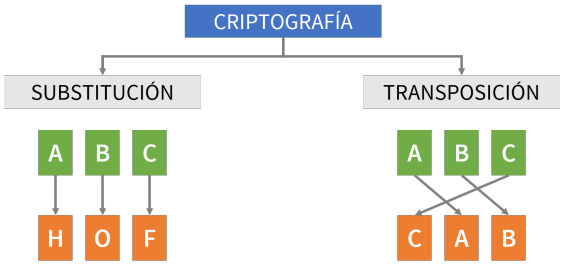


Figura 1.7: Las dos grandes ramas de la criptografía clásica. Hoy perviven a su manera en las cajas-S y cajas-P de los cifradores en bloque modernos basados en las redes de sustitución-permutación.

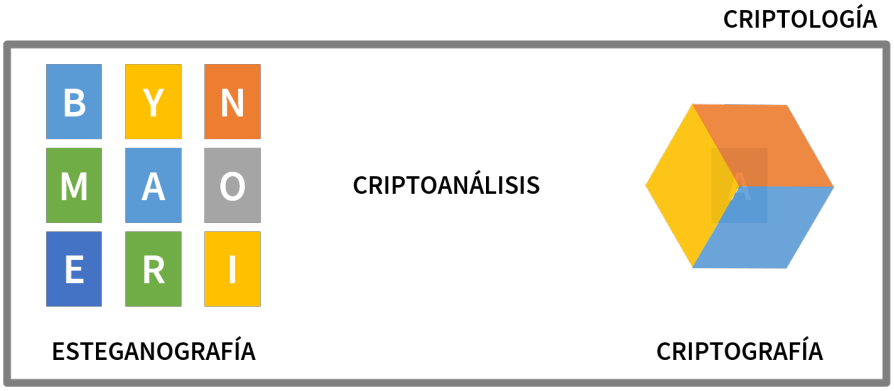


Figura 1.8: Criptología: la ciencia de la escritura secreta (criptografía), de la escritura oculta (esteganografía) y de su revelación no autorizada (criptoanálisis).

1.3. Fundamentos del cifrado

Históricamente, el primer método utilizado para proteger la información consistía en esconder el texto secreto. Estaba oculto a plena vista. Este método se llama esteganografía. Viene de la palabra griega *steganos*, que significa encubierto, escondido u oculto.

El segundo método inventado consistía en proteger el texto a través de un proceso de manipulación de las letras, denominado *cifrado*: hacer ilegible un mensaje para quien desconozca la clave de cifrado. Los diferentes algoritmos de cifrado propuestos a lo largo de los siglos funcionaban de dos maneras, mostradas en la Fig. 1.7:

1. Los cifradores de sustitución sustituían unos caracteres por otros, del mismo alfabeto o de otro enteramente distinto.

2. Los cifradores de transposición conservaban los mismos caracteres del texto original, pero cambiando su posición de formas más o menos complejas. En el vídeo `VideoTransposicion.mp4` te enseño un ejemplo casero de cifrado de transposición para que hagas las delicias de niños y mayores.

Obviamente, los atacantes no se quedaban de brazos cruzados viendo pasar los mensajes cifrados. Trataban de encontrar debilidades en el método criptográfico y obtener el contenido de los mensajes cifrados, incluso sin la clave. Este proceso de estudio se llama criptoanálisis.

Hoy nosotros llamamos criptología a estos tres campos: la ciencia de la escritura secreta (criptografía), la escritura escondida (esteganografía) y la búsqueda de debilidades y fallos en los sistemas criptográficos (criptoanálisis).

Un esquema de cifrado de clave secreta clásico o moderno se define especificando tres algoritmos, representados por cajitas grises circulares en la Fig. 1.9:

1. El *algoritmo de generación de claves* (G) produce aleatoriamente una clave k. A lo largo del curso tomarás conciencia de que el proceso debe ser aleatorio si no quieres que ocurran cosas feas, pero muy feas.

2. El *algoritmo de cifrado* (E): toma como entrada una clave k y un mensaje m y produce un texto cifrado c. Cifrar el texto claro m utilizando la clave k se representa como $E_k(m)$.

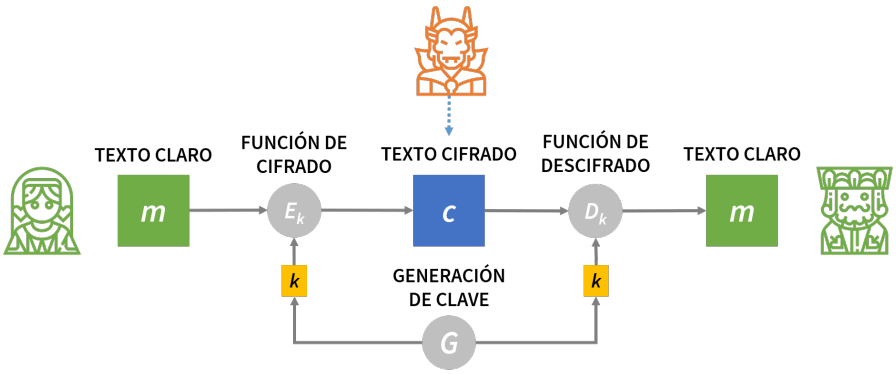


Figura 1.9: La criptografía clásica se ocupa de diseñar y utilizar cifradores que permiten que dos partes se comuniquen secretamente en presencia de un fisgón que puede vigilar toda la comunicación entre ellas.

3. El *algoritmo de descifrado* (D): toma como entrada una clave k y un texto cifrado c y produce un texto claro m. Descifrar el texto cifrado c utilizando la clave k se representa como $D_k(c)$.

Por supuesto, todo esquema de cifrado debe satisfacer el siguiente *requisito de corrección*:

Requisito de corrección

Para cada clave k generada por G y para cada mensaje m, se cumple que

$$D_k(E_k(m)) = m$$

El funcionamiento de estos esquemas es muy sencillo. De ahora en adelante, para explicar cualquier cuestión relacionada con criptografía recurriremos a Alice y Bob: los pobres intentan comunicarse en paz, pero siempre andan por ahí Eve, tratando de espiarlos, y Mallory, peor aún, intentando manipular sus mensajes.

Imagina que Alice quiere comunicarse de forma secreta con Bob. Primero, Alice ejecuta G para obtener una clave k que comparte con Bob. Después, cifra el mensaje secreto m, obteniendo $c = E_k(m)$, y envía este texto cifrado a Bob por el canal público. Al recibir c, Bob calcula $m = D_k(c)$ para recuperar el texto original.

En esta primera lección te empaparás de aquellos algoritmos clásicos que te ayudarán a introducirte sin dolor en los conceptos matemáticos básicos de criptografía. Concretamente, revisarás:

- Cifradores de sustitución: cifrado de desplazamiento y cifrado de Vigenère.
- La libreta de un solo uso (o cifrado de Vernam).

1.4. Cifradores de sustitución

Un cifrado de sustitución es aquel en el que cada carácter en el texto claro se sustituye por otro carácter en el texto cifrado. El receptor invierte la sustitución en el texto cifrado para recuperar el texto claro.

Aunque a lo largo de los siglos se han inventado docenas de cifradores de sustitución, en esta lección solo verás dos tipos, debido a la gran riqueza de conceptos matemáticos que aportan para entender los cifradores modernos:

- Sustitución monoalfabética.
- Sustitución polialfabética.

1.4.1. Sustitución monoalfabética: el cifrado de desplazamiento

Como irás viendo, los criptógrafos, como buenos matemáticos, son muy aficionados a los formalismos. Supongamos que los conjuntos de caracteres de texto cifrado y texto claro son los mismos. Sea $m = m_1 m_2 m_3 \ldots$ un mensaje de texto claro consistente en caracteres yuxtapuestos

$m_i \in \mathcal{A}$, donde $\mathcal{A}$ es un alfabeto de caracteres fijos como $\mathcal{A} = \{\text{A, B,}\ldots, \text{Z}\}$. Para operar con números en lugar de letras se necesita asignar un valor a cada letra. Por ejemplo, puede usarse el código[1] del Cuadro 1.1:

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12
N	O	P	Q	R	S	T	U	V	W	X	Y	Z
13	14	15	16	17	18	19	20	21	22	23	24	25

Cuadro 1.1: Código numérico empleado para representar los caracteres.

Si el número de caracteres en el alfabeto dado es $|\mathcal{A}| = N$, entonces la operación de cifrado por sustitución monoalfabética se representa matemáticamente como:

$$c_i = E_k(m_i) = (m_i + k) \mod N$$

Mientras que la operación de descifrado viene definida por

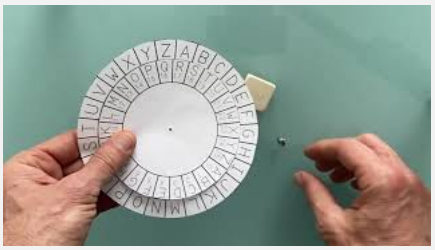
$$D_k(c_i) = (c_i - k) \mod N = (c_i + (N - k)) \mod N$$

El famoso cifrado de César es un caso particular del cifrado por desplazamiento, en el que las letras se desplazan 3 posiciones, $k = 3$. Por ejemplo, HOLA se convertirá en KROD. ¡Compruébalo!

Teniendo en cuenta que a la aritmética modular se la llama nada menos que aritmética del reloj (¿adivinas por qué?), desde el siglo XV se han venido utilizando discos de cifrado para simplificar el proceso de cifrar y descifrar, como se muestra en la Fig. 1.10. Se trata de una manera física muy ingeniosa de realizar la operación módulo sin saber matemáticas.

PRÁCTICA 1.1: Cifrando al estilo de los criptógrafos clásicos

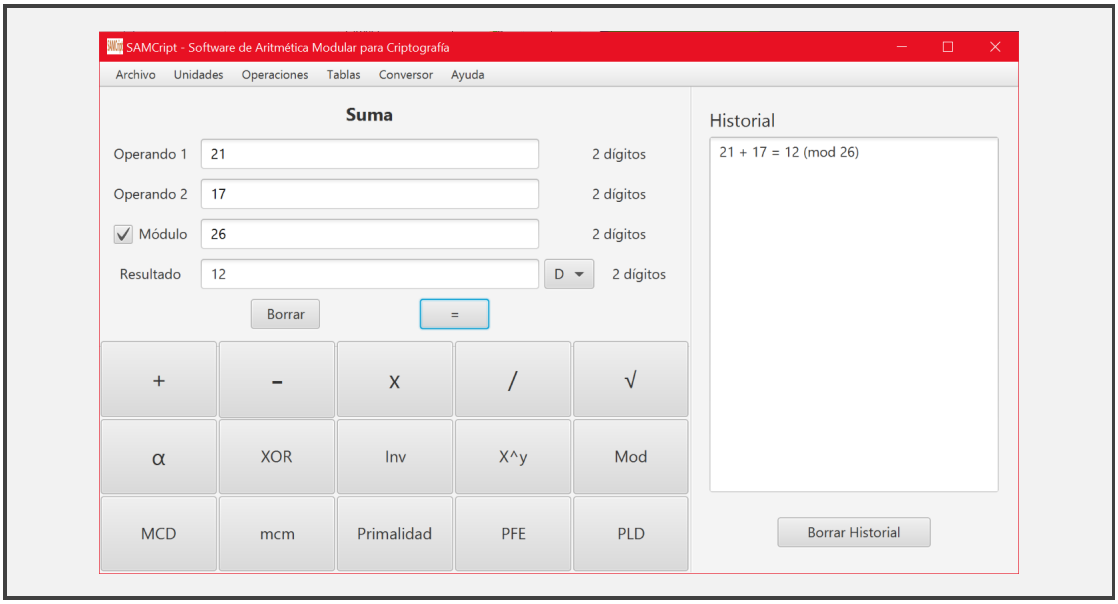
Si te animas a fabricar uno casero, en la Fig. 1.6.2 tienes dos discos de distintos tamaños para imprimir y montar uno sobre otro. No es que tenga mucho intríngulis, pero, por si acaso, en el vídeo `VideoDiscosCifrado.mp4` me verás montando uno.



PRÁCTICA 1.2: Poniéndote las pilas con la aritmética modular

Si te cuesta cogerle el tranquillo a esto de la aritmética modular, te recomiendo que veas la primera parte del siguiente vídeo: Class4crypt c4c2.1 Aritmética modular, conjunto de restos y función de Euler, y que descargues y practiques con la siguiente herramienta educativa en Java: SAMCript: Software de Aritmética Modular para Criptografía. Insisto, estas prácticas solo te serán necesarias si no te manejas a gusto con la operación módulo. Si lo tienes claro, sigue adelante con el curso.

[1] Un código es un sistema de reglas para convertir la información en otra forma o representación, a veces abreviada o secreta, para comunicarla a través de un canal de comunicación o para almacenarla.



Otro caso particular de interés es $k = 13$. Al desplazar 13 posiciones, obtenemos ROT13, que todavía se utiliza en los foros online como medio para evitar revelar spoilers, soluciones de puzles y materiales ofensivos. Por ejemplo, con ROT13, "Daenerys muere" se convierte en "Qnrarelf zhrer".

Cifrar un texto claro dos veces con ROT13 usando el alfabeto inglés restaura el texto original, ya que $k = 13$ y $N = 26$, de modo que :

$$m_i = E_{13}(E_{13}(m_i)) = ((m_i + 13) + 13) \mod 26 = m_i + 26 \mod 26 = m_i$$

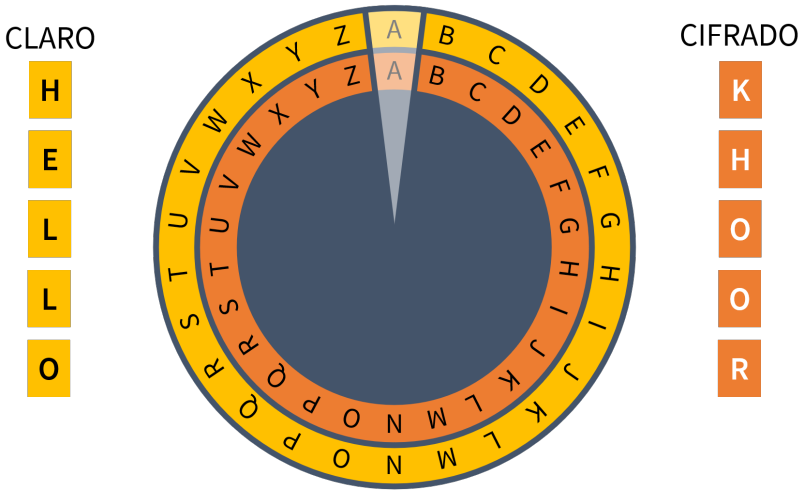
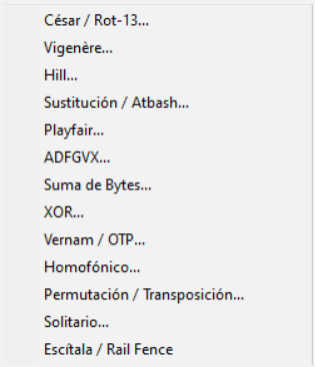


Figura 1.10: El Cifrado de César: uno de los cifradores conocidos más antiguos, cifra desplazando las letras del alfabeto 3 lugares hacia adelante.

PRÁCTICA 1.3: Descifrando textos cifrados mediante desplazamiento desconocido

Existen en Internet multitud de páginas dedicadas a la criptografía, donde podrás encontrar aplicaciones para cifrar/descifrar con algoritmos clásicos. Te recomiendo en particular dcode.fr. Si quieres usar CrypTool, también puedes. En la sección `Cifrar / Descifrar >Simétrico (clásico)` encontrarás un buen número de algoritmos, entre ellos, el de César. ¿Serías capaz de descifrar el siguiente texto cifrado con un desplazamiento desconocido?




```
LS KLZWSHGHTPLUAV LZ ZPLAL F ZHSAH H SH CPZAH WYVIHUKV AVKVZ
```

Como habrás podido comprobar al realizar la práctica, el cifrado de desplazamiento puede romperse trivialmente porque sólo hay que probar exhaustivamente $N = |\mathcal{A}|$ claves. Por ejemplo, $N = 26$ en el alfabeto inglés. Para resolver la práctica anterior habrás tenido que probar todos los posibles desplazamientos hasta dar con el correcto. En promedio, tendrás que probar la mitad. Examinemos con más detalle la cuestión de los ataques a la clave.

El conjunto de todas las claves posibles que produce el algoritmo de generación de claves G se llama *espacio de claves* y se denota por $\mathcal{K}$. Casi siempre, G simplemente elige al azar una clave uniforme k del espacio de claves $\mathcal{K}$. En la próxima lección verás cuáles son las desastrosas consecuencias por no elegir la clave aleatoriamente.

Un ataque que implica probar todas las claves posibles se llama *ataque de fuerza bruta* o *de búsqueda exhaustiva*. Claramente, para que un método de cifrado sea seguro no debe ser vulnerable a tal ataque. Esta observación se conoce como el *principio de espacio de claves suficiente*:

Principio de espacio de claves suficiente

Todo método de cifrado seguro debe tener un espacio de claves lo suficientemente grande como para hacer computacionalmente inviable un ataque de búsqueda exhaustiva.

El *principio de espacio de claves suficiente* da una condición necesaria para la seguridad, pero no suficiente. Vaya lío, ¿no?

En la práctica anterior has recurrido a la fuerza bruta para resolver el reto. En realidad, existe un método más sutil basado en la observación de que la frecuencia de aparición de las letras en el lenguaje humano no es aleatoria: se conoce perfectamente la distribución de frecuencia de las letras individuales en todos los idiomas. Por ejemplo, la letra Q en inglés viene prácticamente siempre seguida por la letra U. Más útil es el hecho de que ciertas letras como la E y la T aparecen con mucha más frecuencia que otras letras como la F y la C. La figura 1.11 enumera las letras con sus frecuencias típicas en el idioma inglés. Como puede verse, la letra más frecuente es la E, seguida de la T, la A, la O, la I, etc.

Así, el ataque por análisis de frecuencias funciona tabulando la distribución de frecuencias de los caracteres en el texto cifrado, para luego compararlas con las frecuencias de las letras conocidas de un texto normal. De esta manera, después de probar varias posibilidades y, tras una cierta cantidad de prueba y error, finalmente se descifrará el mensaje.

PRÁCTICA 1.4: Aplicando el análisis de frecuencias al cifrado de sustitución con CrypTool

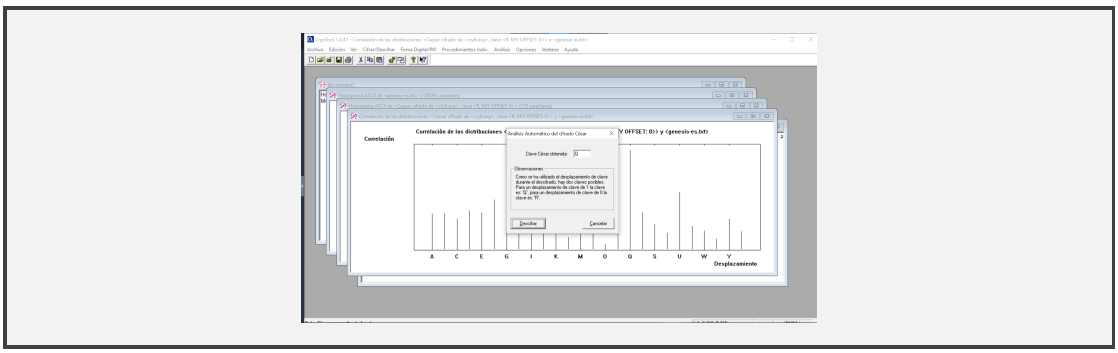
1. Abre el archivo `Sustitucion.txt`.
2. Cífralo con un desplazamiento de 17 posiciones (`Cifrar / Descifrar >Simétrico (clásico) >César / ROT-13`).
3. Analízalo estadísticamente: `Análisis >Cifrado Simétrico (clásico) >Solo texto cifrado >César`.

Si todo ha ido bien, te habrá devuelto el texto claro original. No está de más que le eches un vistazo a los histogramas y te pares a entender qué ha estado haciendo.

a	0,08167
b	0,01492
c	0,02782
d	0,04253
e	0,12702
f	0,02228
g	0,02015
h	0,06094
i	0,06966
j	0,00153
k	0,00772
l	0,04025
m	0,02406
n	0,06749
o	0,07507
p	0,01929
q	0,00095
r	0,05987
s	0,06327
t	0,09056
u	0,02758
v	0,00978
w	0,0236
x	0,0015
y	0,01974
z	0,00074



Figura 1.11: Se conoce la distribución de frecuencia de las letras individuales en el idioma inglés (y en todos los demás).



Si hubieras hecho algo parecido en la Edad Media, ¡te habrían quemado por brujo!

1.4.2. Sustitución polialfabética: el cifrado de Vigenère

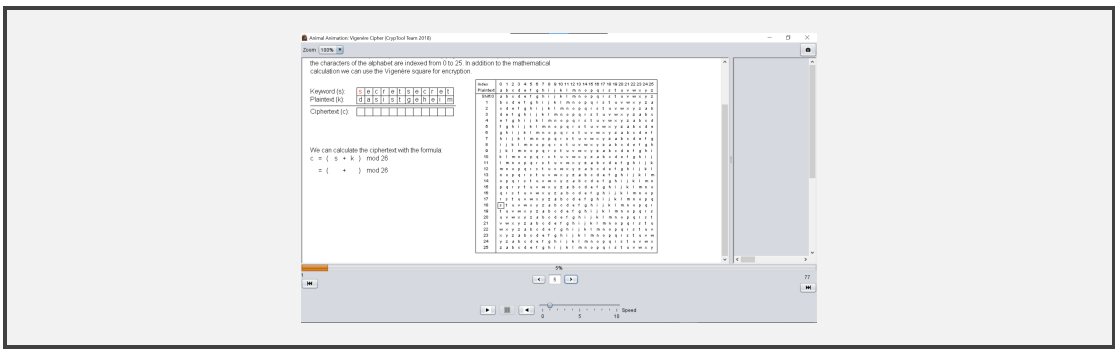
El cifrado de sustitución anterior implica un solo mapeo del alfabeto de texto claro en caracteres de texto cifrado, presa fácil del análisis de frecuencias. Una alternativa más compleja es utilizar diferentes mapeos de sustitución (llamados alfabetos múltiples) en varias porciones del texto claro. Esto da como resultado la llamada *sustitución polialfabética*.

El cifrado de Vigenère funciona utilizando diferentes cifrados de desplazamiento para cifrar diferentes letras. Un simple cifrado de Vigenère de período t sobre un alfabeto de N caracteres implica una clave de t caracteres $k_1k_2 \ldots k_t$. El mapeo del texto claro $m = m_1m_2m_3 \ldots$ al texto cifrado $c = c_1c_2c_3 \ldots$ se define sobre los caracteres individuales como $c_i = (m_i + k_i) \mod N$, donde el subíndice i en k_i se toma módulo t (la clave se reutiliza con período t).

Otra forma de verlo es que el Vigenère simple utiliza t cifrados de desplazamiento, definidos por t valores de desplazamiento k_i, cada uno de los cuales especifica una de las N sustituciones (monoalfabéticas); k_i se utiliza para los caracteres de la posición $i, i + t, i + 2t, \ldots$. En general, cada una de las t sustituciones es diferente.

PRÁCTICA 1.5: Visualizando el funcionamiento del cifrado de Vigenère con CrypTool

Esta visualización incluida en CrypTool te ayudará a entender mejor cómo funciona Vigenère:
`Procedimientos Indiv. >Visualización de Algoritmos >Vigenère`.



¿No has entendido nada? Lógico. Con la siguiente herramienta lo vas a entender a la primera. Se trata del *tablero de Vigenère*, ilustrado en el cuadro 1.2.

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a
c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b
d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c
e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d
f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e
g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f
h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g
i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h
j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i
k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j
l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k
m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l
n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m
o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n
p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p
r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q
s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r
t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s
u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t
v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u
w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v
x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w
y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x
z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y

Cuadro 1.2: El tablero de Vigenère.

El tablero de Vigenère para el idioma inglés consiste en 26 alfabetos dispuestos en un cuadrado, con cada alfabeto desplazado un carácter más que el alfabeto de su izquierda. Para utilizar una letra de una palabra clave determinada para cifrar una letra de texto claro determinado, debe seguirse el siguiente algoritmo:

1. Encuentra la letra del texto claro en la fila superior.
2. Encuentra la letra de la palabra clave en la primera columna.
3. La letra del texto cifrado está debajo de la letra de texto claro y a la derecha de la letra de la palabra clave.

¿Está claro ahora? Mejor un buen ejemplo. Para cifrar "HELLO" usando la clave "CAT", encuentra "H" en la fila superior, encuentra "C" en la primera columna, lo que da el texto cifrado "J", y así sucesivamente. En el vídeo `VideoTablaVigenere.mp4` explico cómo utilizar la tabla para realizar el ejemplo.

Incluso este sencillo ejemplo ilustra dos importantes características del cifrado de Vigenère:

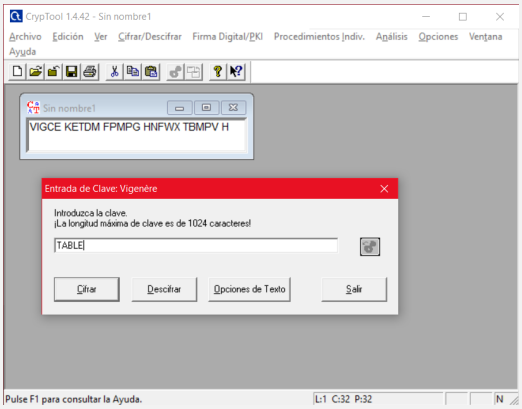
1. Las letras repetidas "LL" en el texto claro conducen a las letras distintas "EN" en el texto cifrado.

Texto claro:	H	E	L	L	O
Clave:	C	A	T	C	A
Texto cifrado:	J	E	E	N	O

2. Las letras repetidas "EE" en el texto cifrado corresponden a letras diferentes "EL" del texto claro.

PRÁCTICA 1.6: Cifrando y descifrando con Vigenère usando el tablero

Descifra el siguiente texto cifrado usando la clave TABLE. Para comprobar los resultados de los ejercicios, puedes acudir una vez más a dcode.fr o a CrypTool. Observa cómo cifrar con Vigenère equivale a cifrar con cinco cifrados de desplazamiento distintos.



VIGCE KETDM FPMPG HNFWX TBMPV H

Párate a reflexionar un momento. ¿Qué pasaría si utilizas un análisis de frecuencias como el que se utiliza para criptoanalizar cifrados de sustitución simple? ¿Funcionará con Vigenère o no funcionará?

Pues no funcionará porque Vigenère enmascara la frecuencia de los caracteres: ahora la letra E no corresponderá al mismo carácter, sino que corresponderá a t caracteres distintos, el período de la clave.

Por otro lado, ¿qué éxito tendría un ataque de fuerza bruta como el que se utilizó con la primera práctica contra el cifrado de desplazamiento? Pues tampoco resultará si se emplea una clave suficientemente larga, ya que el número de claves posibles será N^t.

Ahora entenderás por qué el cifrado de Vigenère se ganó durante siglos (desde 1553 hasta 1863) el apodo de *le chiffre indéchiffrable*.

Por último, fíjate que el cifrado de desplazamiento estudiado en la sección anterior es un Vigenère simple de periodo $t = 1$.

¡Y que no me olvide! Los discos de cifrado como el de la Fig. 1.10 también pueden utilizarse de manera conveniente con Vigenére: los discos pueden moverse para cada letra de la clave. En el video `VideoDiscoVigenere.mp4` puedes seguir el mismo ejemplo anterior realizado con el disco de cifrado en lugar de con la Tabla de Vigenère. ¿Qué te parece retomarlos y marcarte un Vigenère?

PRÁCTICA 1.7: Cifrando y descifrando con Vigenère usando los discos de cifrado

Usando tus flamantes discos de cifrado, descifra el siguiente texto cifrado mediante la clave GOLD. Para comprobar los resultados de los ejercicios, puedes acudir una vez más a tus viejos conocidos, dcode.fr o CrypTool.

RCD GOGNRY AZOGB

1.5. La libreta de un solo uso (*One-Time Pad*)

La debilidad fundamental del cifrado de Vigenère reside en su naturaleza cíclica, como observarás en la Fig. 1.12. Si la palabra clave es GON, de tres letras, entonces cada tercera letra del



Figura 1.12: La debilidad fundamental del cifrado de Vigenère es su naturaleza cíclica.



Figura 1.13: Un cifrado de Vigenère con clave aleatoria tan larga como el mensaje sería 100 % seguro.

texto claro estará cifrada según el mismo alfabeto de cifrado. Si el criptoanalista puede identificar la longitud t de la palabra clave, el texto cifrado puede tratarse como una serie de tres cifrados monoalfabéticos y cada uno puede romperse por análisis de frecuencias.

Sin embargo, ¿qué pasa si el cifrado de Vigenère se utiliza con una clave aleatoria k_i tan larga como el texto claro m_i, como en la Fig. 1.13?

$$c_i = m_i + k_i \mod N$$

En este caso, resulta imposible descifrar el texto cifrado sin la clave. Estamos ante un sistema 100 % seguro, demostrable matemáticamente. Veamos por qué.

Por ejemplo, para cifrar el mensaje

`NUNCA JUZGUES UN LIBRO POR SU PELI, ES MEJOR LEERLO ENTERO`

quitamos los espacios y la puntuación para obtener un mensaje que tiene 47 letras. Necesitamos una clave aleatoria que también tenga 47 letras, por ejemplo

`FJWEAXBCKUOLFGHIHVLNQCSNLSBGTTZYLHCZKRIVQIKLIFN`

con la cual se cifra el mensaje, dando como resultado el texto cifrado

`SDJGAGVBQOSDZTSQIMZCETKHAWMOXLLCUVTKOVZGEMXEMWB`

Ahora imagina que un criptoanalista se apodera del texto cifrado (SDJGA...). ¿Cómo podría atacar el cifrado? Un ataque de fuerza bruta probando todas las posibles claves no funcionaría porque hay demasiadas incluso para el ordenador más potente del universo. El número de claves sería igual a 26 elevado a la potencia del número total de letras del mensaje. En nuestro ejemplo, habría un total de 26^{47}, o sea 666.091.878.431.395.624.153.823.182.526.730.590.376.250.379.528.249.805.353.030.484.20 594.192.101.376 posibles claves (unos 220 bits).

Vale, vamos a un escenario de ciencia-ficción. Incluso si el criptoanalista contara con un ordenador lo suficientemente potente como para probar todas las claves, no podría romper el cifrado porque para cualquier texto cifrado, todos los posibles mensajes de texto claro son igualmente probables.

Por ejemplo, el texto cifrado `SDJGAG...` podría haberse obtenido fácilmente tras cifrar un texto claro completamente diferente, pero del mismo número de letras de longitud, como, por ejemplo

`SI NO TE GUSTA UN LIBRO NO LO LEAS HASTA ACABARLO, CIERRALO`

cifrado con la clave

`AVWSHCPHYVSJMIKPRYMOTFZDAEFOFSLCSVSKXKLEWIGNMLN`

(te dejo como ejercicio que lo verifiques, ¡no te creas algo porque yo lo diga!).

O sea, el mismo texto cifrado podría haberse obtenido usando dos mensajes de texto claro muy diferentes. Cuando usamos una clave aleatoria tan larga como el mensaje, el criptoanalista no tiene forma de saber cuál es el mensaje original. De hecho, cualquier mensaje legible de texto claro que tenga exactamente 47 letras de longitud es igual de probable que sea el texto claro original. Sólo porque una cierta clave pueda descifrar el texto cifrado no significa que sea la clave de cifrado original.

Resumiendo, debido a que cualquier texto claro podría haber sido utilizado para crear el texto cifrado con igual probabilidad, **es imposible criptoanalizar un mensaje cifrado cuando se ha cifrado con una clave aleatoria tan larga como el propio mensaje**.

Ya estás preparado para entender uno de los descubrimientos más importantes y excitantes de la criptografía: la libreta de un solo uso.



Figura 1.14: En la a libreta de un solo uso, las letras de la clave se generan al azar y se copian en hojas de papel en dos blocs idénticos.

1.5.1. Cómo implementar la libreta de un solo uso con 1's y 0's

Esta idea de emplear claves aleatorias se le ocurrió en 1918 al Mayor Joseph Mauborgne, del Ejército de los Estados Unidos. Primero, se necesita compilar un grueso bloc de notas compuesto por cientos de hojas de papel, cada una de las cuales contiene una clave única en forma de líneas de letras secuenciadas al azar, como en la Fig. 1.14. Hay dos copias del bloc, una para el emisor y otra para el receptor. Para cifrar un mensaje, el remitente aplica el cifrado de Vigenère utilizando la primera hoja del bloc como clave. Una vez que el mensaje se ha enviado, recibido y descifrado con éxito, tanto el emisor como el receptor destruyen la hoja que ha actuado como clave, para que no se vuelva a utilizar nunca más. Cuando se cifra el siguiente mensaje, se emplea la clave aleatoria de la siguiente hoja, que también se destruye posteriormente, y así sucesivamente. Debido a que cada clave se utiliza una vez, y sólo una vez, este sistema se conoce como el cifrado de la libreta de un solo uso.

Un año antes, en 1917, Gilbert Vernam de AT&T Bell Labs había patentado un cifrado de teleimpresora en el que una clave previamente generada, guardada en una cinta de papel, se combina poco a poco con el mensaje de texto claro para producir el texto cifrado, utilizando la operación XOR.

Por si no recuerdas qué es eso del XOR, te lo explico en dos palabras. Dado el alfabeto binario, $\mathcal{A} = \{0,1\}$ y $N = |\mathcal{A}| = 2$, la operación suma módulo 2 es equivalente a otra operación informática básica que es incluso más rápida que la adición. Se llama *O exclusivo* y se denota por XOR o $\oplus$.

INPUT		OUTPUT
A	B	A $\oplus$ B
0	0	0
0	1	1
1	0	1
1	1	0

El XOR de dos cadenas de bits es el resultado de realizar el XOR en cada par de bits correspondientes. Por ejemplo,

$$10110 \oplus 11010 = [1 \oplus 1][0 \oplus 1][1 \oplus 0][1 \oplus 1][0 \oplus 0] = 01100$$

Combinando las ideas de Vernam y Mauborgne, es decir, utilizando esta nueva operación sobre una clave generada al azar, nació el cifrado de Vernam, que puede definirse como

$$c_i = E_k(m_i) = k_i \oplus m_i$$

y

$$m_i = D_k(c_i) = k_i \oplus c_i.$$

Es fácil comprobar el *requisito de corrección*: $D_k(E_k(m_i)) = m_i$, ya que

$$D_k(c_i) = D_k(k_i \oplus m_i) = k_i \oplus (k_i \oplus m_i) = (k_i \oplus k_i) \oplus m_i = m_i,$$

ya que, como puedes comprobar tú mismo fácilmente, cualquier número sumado consigo mismo módulo 2 es igual a cero:

$$n \oplus n = 0$$

Este cifrado tiene la ventaja de ser altamente eficiente y completamente simétrico en el sentido de que E_k y D_k son la misma función. Además, es **perfectamente seguro** cuando:

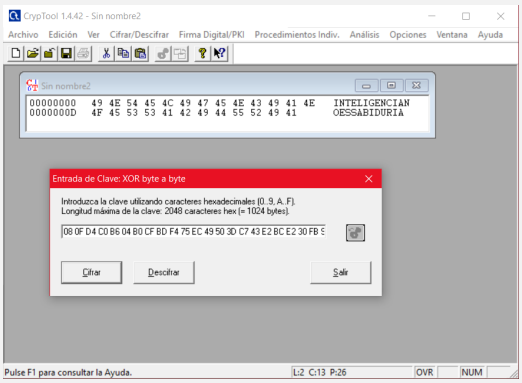
1. k se elige al azar,
2. k es tan largo como el mensaje m, y
3. k se utiliza una sola vez.

PRÁCTICA 1.8: Cifrando/descifrando con la libreta de un solo uso

Utiliza CrypTool para hacer el XOR de dos secuencias de entrada. Cifra el siguiente texto con la clave dada, ambos expresados en hexadecimal:

```
m = 49 4E 54 45 4C 49 47 45 4E 43 49 41 4E 4F 45 53 53 41 42 49 44 55 52
49 41
k = 08 0F D4 C0 B6 04 B0 CF BD F4 75 EC 49 50 3D C7 43 E2 BC E2 30 FB 97
66 E1
```

¿Cuál es el texto cifrado? A continuación, cífralo de nuevo con la misma clave. ¿Cuál es el resultado?



1.5.2. Seguridad de la libreta de un solo uso

Puede demostrarse que la libreta de un solo uso es teóricamente irrompible. Es decir, si un criptoanalista obtiene una cadena de texto cifrado $c_1c_2\cdots c_t$ cifrada usando una cadena de clave aleatoria que ha sido usada una sola vez, el criptoanalista no puede hacer nada mejor que adivinar el texto claro lanzando una moneda al aire.

La seguridad perfecta es genial en términos de seguridad, pero viene con un elevado precio: en cualquier esquema de seguridad perfecta, **la longitud de la clave debe ser al menos de la misma longitud que el mensaje a cifrar**. Esto significa que en la práctica **un esquema perfectamente seguro es prohibitivamente caro**, ya que requiere que las partes intercambien claves muy largas antes de que puedan comunicarse con seguridad.

Y si por ahorrar te da por utilizar la misma clave k para cifrar más de un texto claro, m y m', entonces te meterías en un embarazoso aprieto. Un criptoanalista podría explotar el hecho de que

$$c \oplus c' = (k \oplus m) \oplus (k \oplus m') = m \oplus m'$$

para extraer información sobre m o m'. Aunque este resultado a primera vista no parezca gran cosa, es suficiente para descartar cualquier pretensión de secreto perfecto cuando se cifran dos mensajes usando la misma clave. Si los mensajes corresponden a un texto en lenguaje natural, entonces, dado el XOR de dos mensajes suficientemente largos, es posible realizar un análisis de frecuencias y recuperar los mensajes propiamente dichos.

Dado que una imagen vale más que mil palabras, nada como ilustrar esta debilidad mediante el cifrado de imágenes, como se muestra en la Fig. 1.15.

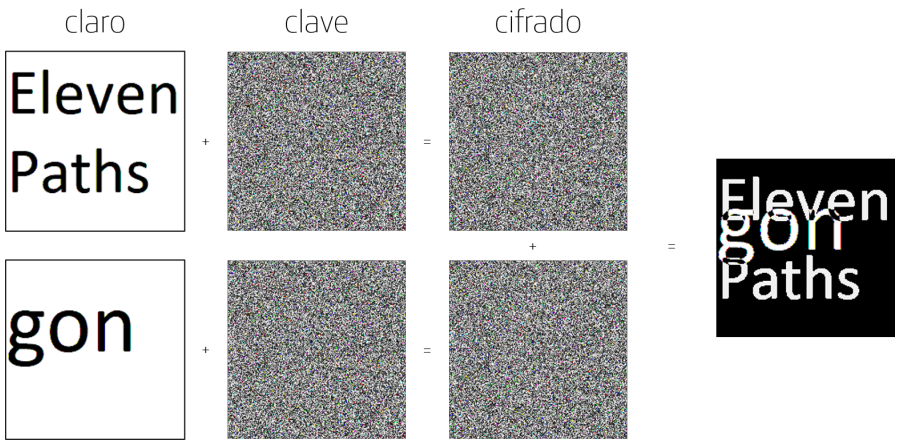


Figura 1.15: Reutilizando una clave en la libreta de un solo uso se filtra información sobre los mensajes cifrados con la misma clave.

PRÁCTICA 1.9: Cifrando imágenes mediante Vernam con CrypTool

Esta será una práctica un poco más complicada. Has interceptado dos imágenes, C101.bmp y C102.bmp, que han sido cifradas mediante Vernam usando la misma clave,  y  . Por lo tanto, es imposible descifrarlas sin el conocimiento de la clave. Pero ¿qué pasaría si haces el XOR de ambas? ¿Serías capaz de averiguar el contenido de ambas imágenes? La única herramienta necesaria es CrypTool.

1. Abre ambas imágenes BMP con CrypTool.
2. Las cabeceras BMP de estas imágenes ocupan los primeros 54 bytes (`0x36`). El contenido de la imagen se encuentra a partir del byte 54. Para cada una de las dos imágenes, copia los bytes a partir del 54 hasta el final de la imagen (selecciona la opción *Sin codificar*) y pégalos en dos nuevas ventanas (todo dentro de CrypTool). Aparecerá un galimatías. No te preocupes, selecciona *Ver* > *Mostrar como código Hexadecimal*. Ahora, cada ventana te mostrará en hexadecimal exclusivamente el contenido de cada imagen. Si quieres entender lo que estás haciendo en lugar de seguir pasos ciegamente (porque tú no eres un *script kiddie*, ¿verdad?), te ayudará consultar esta explicación sobre cómo funcionan los bitmaps.
3. A continuación, haz el XOR de una imagen con otra. Copia el contenido de una de las imágenes (ahora selecciona la opción *Codificación hexadecimal*). Después, haz clic sobre la ventana de la otra imagen y selecciona `Cifrar / Descifrar >Simétrico (clásico) >XOR`. Pega como clave el contenido de la otra imagen que acabas de copiar y cífrala.
4. Para poder visualizar la nueva imagen, copia el resultado del cifrado (selecciona *Codificación hexadecimal*) y pégalo en cualquiera de las imágenes a partir del byte 54, respetando las cabeceras (selecciona *Decodificación hexadecimal*). Guarda la imagen como un nuevo archivo .bmp. ¿Qué aparece?  .

Si te ves con ganas, puedes escribir un script en tu lenguaje de programación favorito que haga lo mismo. Te vendrá bien para futuras prácticas para evitar tanto vete y ven con CrypTool.

1.5.3. Limitaciones prácticas de la libreta de un solo uso

Por desgracia, a pesar de ser 100 % segura, la libreta de un solo uso requiere una clave tan larga como el mensaje, lo que limita la utilidad del esquema para enviar mensajes muy largos:

1. En primer lugar, existe el problema práctico de generar grandes cantidades de bits aleatorios para la clave. Como veremos en la siguiente lección, no resulta nada sencillo.

2. Además, se vuelve muy difícil compartir y almacenar de forma segura claves muy largas.
3. Y es problemático cuando las partes no pueden predecir por adelantado la duración del mensaje.

Para una vez que encontramos un método de cifrado 100 % seguro, ¡sería una lástima descartarlo por completo! No sufras más. Por fortuna, los criptógrafos encontraron un mecanismo sencillo para generar y transportar largas secuencias de bits aleatorios, superando así las limitaciones anteriores. En la lección 3 estudiarás cómo los cifradores en flujo dieron respuesta a esta dificultad y han pasado a ser los algoritmos favoritos de los militares por sus elevadas seguridad y velocidad.

Te recomiendo que veas el siguiente vídeo, Cómo vencer a los ordenadores cuánticos (sin fórmulas raras), con un resumen muy didáctico y fácil de seguir de todo lo explicado hasta aquí. Si viendo al vídeo se te escapa algún concepto, vuelve al manual para asegurarte de que antes de pasar a la siguiente lección lo llevas todo bien amarrado.

1.6. Atacando la criptografía

A estas alturas de la lección, ya te habrás percatado de que la robustez de un criptosistema debe residir en la fortaleza del algoritmo de cifrado usado y no en mantener en secreto el funcionamiento del algoritmo. La **seguridad a través de la oscuridad** tiene las alas cortas. Al final, el detalle de cualquier algoritmo saldrá a la luz, por mucho que intente guardarse en secreto. Y si el algoritmo era débil, se romperá en un santiamén. En lugar de confiar en la oscuridad es mejor confiar en la robustez, tal y como recoge uno de los principios de Kerckhoffs.

Principio de Kerckhoffs

Un criptosistema debería ser seguro incluso si se conocen todos los detalles sobre el sistema, excepto la clave secreta.

1.6.1. Modelos de ataque contra el cifrado

Existen varios modelos diferentes para atacar un algoritmo de cifrado, mostrados en la Fig. 1.16:

1. **Ataques de sólo texto cifrado** (*ciphertext-only attack*, COA): observan los textos cifrados, pero no conocen los textos claros correspondientes y no saben cómo se seleccionaron los textos claros. Los atacantes en el modelo COA son pasivos y no pueden realizar consultas de cifrado o descifrado.
2. **Ataques de texto claro conocido** (*known-plaintext attack*, KPA): observan los textos cifrados y conocen los textos claros correspondientes. Los atacantes en el modelo KPA obtienen así una lista de parejas de texto claro/cifrado, en la que se supone que los textos claros se seleccionan al azar. Una vez más, el KPA es un modelo de atacante pasivo.
3. **Ataques de texto claro elegido** (*Chosen-plaintext attack*, CPA): pueden realizar consultas de cifrado para los textos claros de su elección y observar los textos cifrados resultantes. Este modelo comprende situaciones en las que los atacantes pueden elegir todos o parte de los textos claros a cifrar y luego a ver el resultado del cifrado. A diferencia de COA o KPA, que son modelos pasivos, los CPA son atacantes activos, porque influyen en los procesos de cifrado en lugar de escuchar pasivamente.
4. **Ataques de texto cifrado elegido** (*chosen-ciphertext attack*, CCA): pueden tanto cifrar como descifrar; es decir, consiguen realizar consultas de cifrado y consultas de descifrado. El modelo CCA puede parecer ridículo al principio (si se puede descifrar, ¿qué más se necesita?), pero al igual que el modelo CPA, su objetivo es representar situaciones en las que los atacantes pueden tener alguna influencia en el texto cifrado y más tarde obtener acceso al texto claro. Además, descifrar algo no siempre es suficiente para romper un sistema. Por ejemplo, algunos dispositivos de protección por vídeo permiten a los atacantes realizar consultas de cifrado y consultas de descifrado utilizando el chip del dispositivo, pero en ese contexto los atacantes se interesan por la clave para redistribuirla; en este caso, el hecho de poder descifrar *gratuitamente* no basta para romper el sistema. Otro ejemplo es

el cifrado/descifrado de testigos de autenticación en un servicio web: el atacante persigue obtener la clave de cifrado usada y así poder suplantar a otros usuarios.

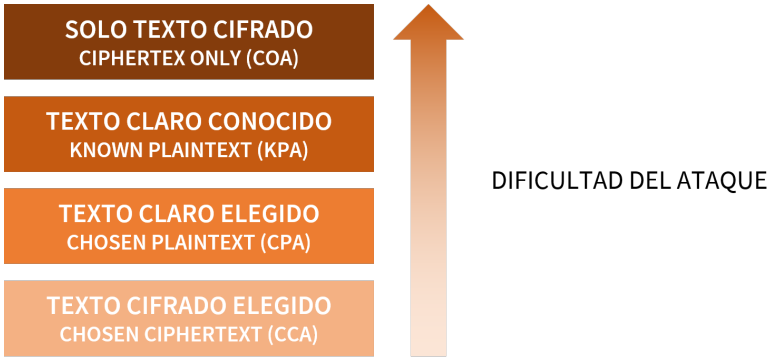


Figura 1.16: El objetivo de los modelos de ataque es recuperar sistemáticamente el texto claro a partir del cifrado o, incluso más drásticamente, deducir la clave.

En estos modelos de ataque, observar y solicitar textos cifrados no sale gratis. Cada texto cifrado proviene del cálculo de la función de cifrado. Esto significa que generar 2^n parejas de texto claro/cifrado a través de consultas de cifrado requiere tanto cálculo como probar 2^n claves, por ejemplo. El coste de las consultas debe tenerse en cuenta cuando se calcula el coste de un ataque. Y más si las consultas se realizan por Internet, por ejemplo a un servidor que devuelve testigos de sesión cifrados.

PRÁCTICA 1.10: Aplicando los modelos de ataque a los tres cifrados estudiados

Más que una práctica, es un ejercicio de reflexión, para que masques bien los modelos de ataque. Por supuesto, puedes poner a prueba tu hipótesis (y debes hacerlo si no lo tienes claro) con CrypTool.

1. Has usado el cifrador de sustitución para cifrar un mensaje. ¿Qué longitud de texto cifrado y texto claro necesita un atacante para obtener la clave de cifrado con un ataque de texto claro elegido? ¿Y para texto claro conocido? ¿Cambiaría algo?

2. En vista del poco éxito con el método anterior, te has pasado a Vigenère. Cifras un mensaje con una clave de 12 letras (sin repetición). ¿Qué longitud de texto cifrado y texto claro necesita un atacante para obtener la clave de cifrado con un ataque de texto claro elegido? ¿Y para texto claro conocido? ¿Cambiaría algo?

3. Vaya, hoy no es tu día. Decides probar con la libreta de un solo uso, a ver si hay más suerte. Después de todo, es 100 % segura, ¿no? De los cuatro modelos de ataque, ¿contra cuántos resistirá como una campeona si se cifran varios mensajes con la misma clave?

1.6.2. Complejidad de los ataques

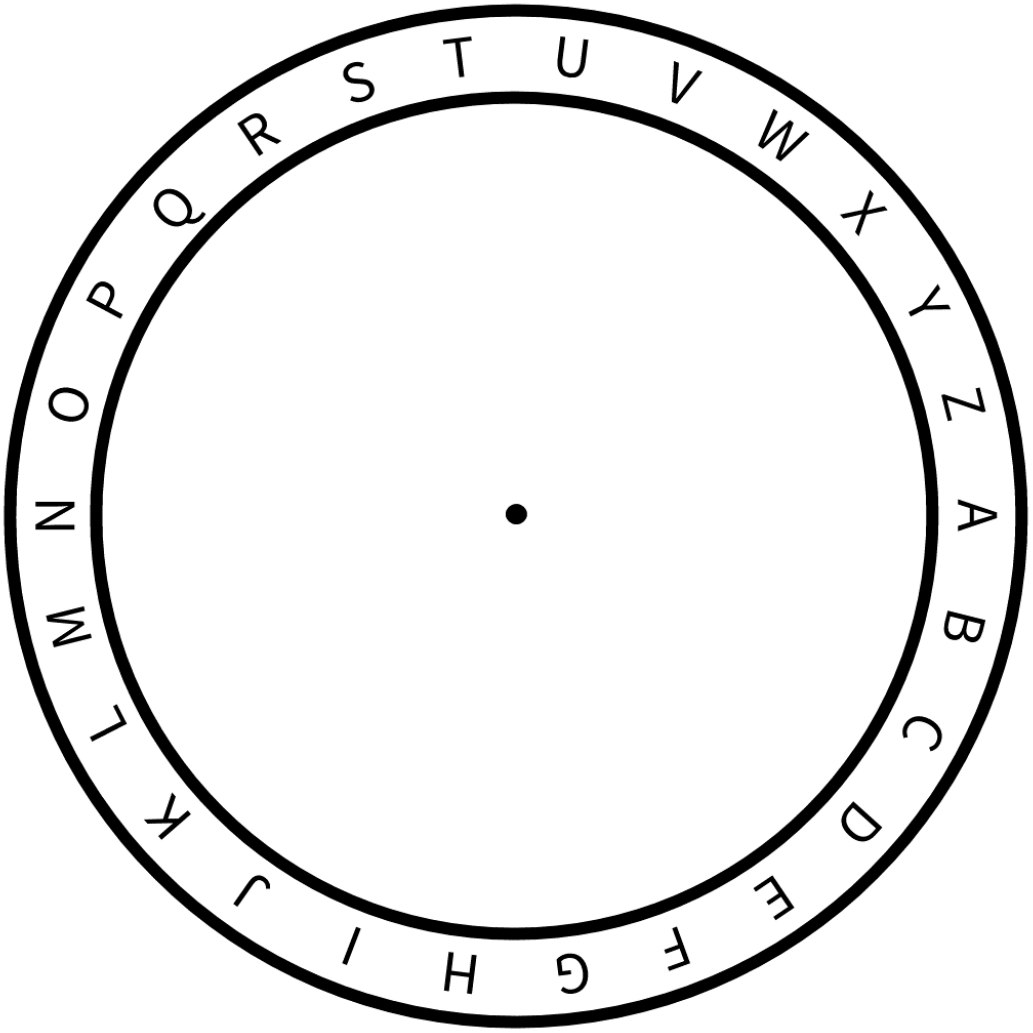
Existen diferentes métricas de complejidad para evaluar el coste de ejecutar un ataque contra un algoritmo criptográfico:

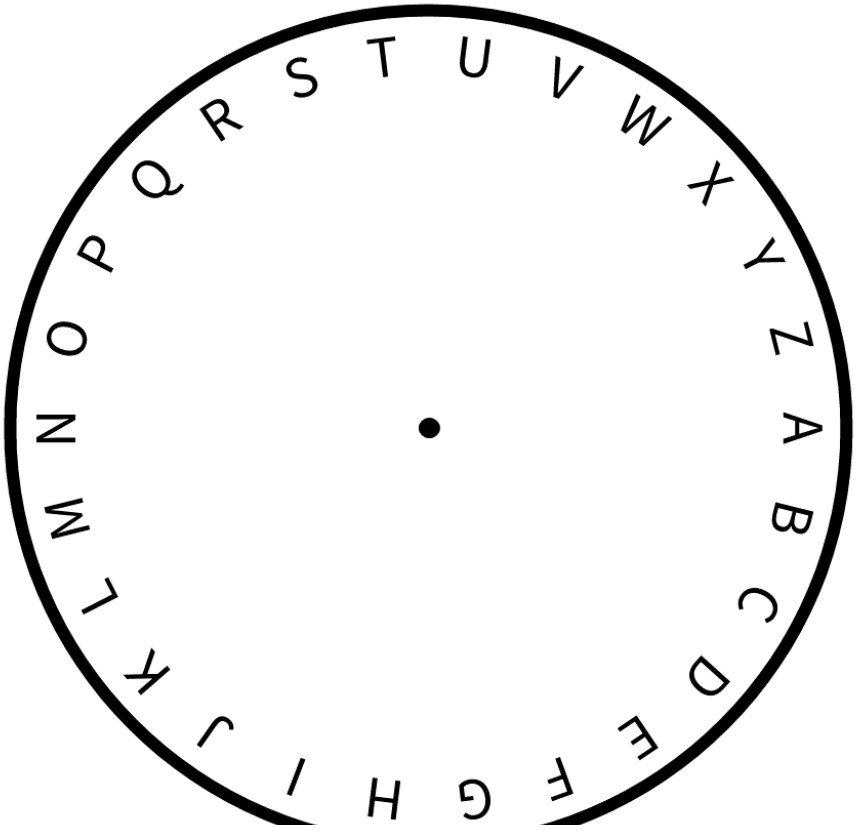
- **Complejidad temporal**: es la cantidad de cálculos fuera de línea necesarios para realizar con éxito un ataque criptográfico, por lo que el tiempo requerido para llevarlo a cabo dependerá de la potencia computacional a disposición del atacante. Por ejemplo, la complejidad temporal de una búsqueda exhaustiva de claves AES-128 es (aproximadamente) 2^{128} operaciones.

- **Complejidad en memoria**: es la cantidad de almacenamiento necesario para ejecutar exitosamente un ataque. Por ejemplo, para realizar un ataque de diccionario contra contraseñas de 8 caracteres usando el conjunto de caracteres ASCII (95 caracteres), hay 95^8 contraseñas posibles. Si se usa MD5, cada hash es de 128 bits o 16 bytes. La tabla tendría que almacenar $95^8 \times 16$ bytes. ¡Varios terabytes de datos!

- **Complejidad en datos**: es la cantidad de interacciones que el adversario necesita realizar con el mecanismo criptográfico para desarrollar el ataque. Por ejemplo, el ataque de oráculo PKCS#1 v1.5 (Public-Key Cryptography Standard) de Bleichenbacher al criptosistema RSA de 1024 bits necesita alrededor de un millón de consultas al oráculo de relleno.

En la próxima lección aprenderás el relevante papel que juega la aleatoriedad en criptografía. De hecho, como comprobarás, **sin aleatoriedad no hay seguridad**.


A B C D E F G H I J K L M N O P Q R S T U V W X Y Z


A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Lección 2

Aleatoriedad

Objetivos de aprendizaje

1. Comprender la importancia fundamental de la aleatoriedad en la criptografía y cómo afecta a la seguridad de los sistemas.
2. Diferenciar entre generadores de números aleatorios verdaderos (TRNG), cuánticos (QRNG) y pseudoaleatorios (PRNG), y conocer sus aplicaciones y limitaciones.
3. Identificar las vulnerabilidades asociadas con el uso de generadores pseudoaleatorios inseguros en criptografía.
4. Aplicar y analizar pruebas estadísticas para evaluar la calidad de la aleatoriedad en secuencias de bits.

Competencias a desarrollar

1. Capacidad para implementar generadores de números aleatorios criptográficamente seguros en distintos lenguajes de programación.
2. Habilidad para evaluar y asegurar la calidad de la aleatoriedad en la generación de claves y otros elementos criptográficos.
3. Competencia en la identificación y mitigación de riesgos asociados con fuentes de aleatoriedad débiles o predecibles.
4. Conocimiento de los estándares y mejores prácticas internacionales para la generación y prueba de números aleatorios en criptografía.

Cuentan las crónicas de los reyes de Noruega que el rey Olaf Haraldsson el Santo disputaba con el vecino rey de Suecia la posesión de la isla de Hísing. Decidieron resolver la disputa pacíficamente con un juego de dados. Tras acordar que la ganaría quien obtuviera la puntuación más alta, lanzó los dados en primer lugar el rey de Suecia.

–¡Doce! ¡He ganado! No hace falta que arrojéis los dados, rey Olaf.

Mientras agitaba los dados entre sus manos, Olaf el Santo replicó:

–Todavía quedan dos seises en los dados y no le será difícil a Dios, mi Señor, hacer que vuelvan a aparecer en mi favor.

Volaron los dados y volvieron a salir dos seises. De nuevo lazó los dados el rey de Suecia y de nuevo sacó dos seises. Cuando le llegó el turno a Olaf el Santo, uno de los dados arrojados se partió en dos, mostrando una mitad un 6 y la otra, un 1, sumando un total de 13. Como consecuencia, la titularidad de la isla le fue adjudicada a Noruega y ambos reyes se despidieron como amigos.

La aleatoriedad juega un papel fundamental en todos los juegos de azar. Y lo que tal vez te sorprenda más: la criptografía no podría existir sin la aleatoriedad. En esta lección descubrirás cómo se usa la aleatoriedad en criptografía y cómo obtener números aleatorios, tarea, como verás, nada sencilla.

2.1. ¿Qué es la aleatoriedad?

Existen tantas definiciones sobre aleatoriedad que podríamos llenar un libro con ellas. En criptografía es común la siguiente interpretación, que cito de Bruce Scheneier:

Aleatoriedad

La aleatoriedad se refiere al resultado de un proceso probabilístico que produce valores independientes, uniformemente distribuidos e impredecibles que no pueden ser reproducidos de manera fiable.

Quiero destacar los siguientes tres ingredientes que debe exhibir todo generador de aleatoriedad para usarse con garantías en criptografía:

1. **Independencia de valores**: no existe ninguna correlación entre los valores generados. Por ejemplo, si lanzas una moneda (sin trucar) al aire y sale cara nueve veces seguidas, ¿qué es más probable? ¿Cara o cruz en el décimo lanzamiento? Pues la probabilidad sigue siendo 1/2, porque el resultado de un lanzamiento es independiente de los lanzamientos precedentes.

2. **Impredecibilidad**: aunque te aburras observando valores y más valores, no puedes predecir con probabilidad superior al azar el próximo valor, sin importar lo larga que haya sido la secuencia precedente. Nuevamente, monedas, dados y ruletas constituyen excelentes generadores aleatorios porque, por muchas teorías que elabores, no sabrás qué saldrá a continuación (asumiendo que no están trucados).

3. **Distribución uniforme**: seguro que mientras leías la crónica del rey Olaf el Santo habrás pensado: "¡Imposible! ¿Cómo van a salir dos seises cuatro veces seguidas?". Y haces bien en dudar, porque la probabilidad de esta secuencia es $(1/36)\cdot(1/36)\cdot(1/36)\cdot(1/36) = (1/36)^4 = 0,00000059537\ldots$ o 1 entre 1,67 millones. No es probable que se dé esa secuencia, pero sí es posible. De hecho, si se tiran los dados mil millones de veces aparecería en promedio unas 600 veces. La aleatoriedad tal y como la imaginamos se manifiesta en los grandes números, no en los pequeños números. Cuantos más valores se generen, esperaremos ver todas las secuencias posibles, distribuidas uniformemente, sin ningún tipo de sesgo.

2.2. Sin aleatoriedad, no hay seguridad

Los números aleatorios tienen una importancia capital en criptografía: constituyen la base misma de la seguridad. La criptografía no puede incorporarse a los productos y servicios sin números aleatorios. **Sin aleatoriedad, no hay seguridad**. Y el malware no es ninguna excepción.

Por ejemplo, **para cifrar un archivo se requiere una clave de cifrado aleatoria**. Si te parece trivial usar una clave elegida al azar, ponte en el lugar del creador del malware: una vez desplegado en un dispositivo infectado, cuyas especificaciones técnicas desconoces por completo, ¿cómo obtiene tu programa malicioso esa clave para cifrar con ella los archivos de la víctima? Tienes varias opciones:

- **La codificas directamente en el malware**: hmm, mala idea. El análisis de código binario mediante reversing del ejecutable dará con las claves. Por ejemplo, DMA Locker v1 incluía la clave de cifrado en el propio código, lo que permitía su fácil recuperación.

- **La solicitas a un servidor C&C**: es una solución aceptable, aunque plantea sus propios inconvenientes: la víctima debe estar conectada a Internet; el acceso a ese servidor no puede estar bloqueado por un cortafuegos; el tráfico debe cifrarse, lo que añade nuevas complejidades; el sitio se aloja en la red Tor; suelen necesitar Algoritmos de Generación de Dominio para evitar el cierre de servidores por las autoridades; etc. Por ejemplo, CryptoWall utilizaba este enfoque, con grandes réditos.

- **Cada copia genera su propia clave de cifrado**: para ser robustas, las claves criptográficas necesitan generarse de forma verdaderamente aleatoria. En caso contrario, podrían

adivinarse o regenerarse. La cuestión es: ¿dónde encuentras en el dispositivo infectado la entropía[1] necesaria para crear claves robustas?

Además de para generar claves, ¿qué papel juega la aleatoriedad en criptografía? No solo se necesitan números aleatorios para generar claves simétricas, sino también parejas de claves asimétricas, para generar vectores de inicialización (IV) para modos de operación criptográfica, en protocolos de respuesta a desafíos, como entradas adicionales a la mayoría de los algoritmos estandarizados de cifrado y firma de claves públicas, para generar *salts*, para generar *nonces* y para generar valores efímeros en protocolos de intercambio de claves.

La existencia de fuentes aleatorias adecuadas suele darse por sentado. Nada más lejos de la realidad. Los esquemas criptográficos más seguros fracasan si no se cumplen los supuestos de aleatoriedad perfecta. En esta lección entenderás la diferencia entre los generadores de bits verdaderamente aleatorios y los pseudoaleatorios y cómo usar estos últimos donde se necesite buena aleatoriedad en criptografía.

2.3. El desafío de la generación de secuencias aleatorias de bits

En el corazón de la generación de secuencias aleatorias de bits para su uso en criptografía se encuentran dos componentes fundamentales: la fuente de entropía y el generador de bits aleatorios determinista (DRBG), que toma esa entropía y la *estira* a tantos bits como se necesiten. En otras palabras, la entropía en un DRBG es la cantidad de incertidumbre o sorpresa genuina que se usa como *semilla* inicial para garantizar que las secuencias generadas sean difíciles de predecir.

En la lección anterior viste que, a pesar de ser 100 % seguro, el cifrado de Vernam requiere **una clave verdaderamente aleatoria tan larga como el mensaje y que nunca se reutilice**, lo que lo vuelve demasiado engorroso para aplicaciones prácticas. El primer gran reto del cifrado de Vernam es: ¿cómo generar secuencias de bits aleatorias que sean fácilmente regenerables a voluntad? En otras palabras: ¿dónde encontramos una fuente de entropía capaz de generar aleatoriedad?

2.3.1. Generadores de bits verdaderamente aleatorios (TRNG)

La **entropía de Rényi**, especialmente la **entropía mínima** (H_∞), se usa para medir la incertifumbre de un generador de ruido aleatorio al enfocarse en el evento más probable en la distribución de salida. En términos sencillos, si X es una variable aleatoria que representa la salida del generador de ruido, con probabilidades de los posibles valores $P(X = x)$, la entropía mínima de Rényi se calcula como:

$$H_\infty(X) = -\log \max_x P(X = x)$$

Esta métrica evalúa la seguridad del generador en el peor caso:

- Si $H_\infty(X)$ es alto, significa que no hay valores con probabilidades muy altas, lo que indica buena aleatoriedad y baja previsibilidad.
- Si $H_\infty(X)$ es bajo, indica que hay un valor probable dominante, sugiriendo que el generador podría ser predecible y menos seguro.

En resumen, se utiliza H_∞ para asegurar que una fuente de entropía tiene suficiente incertidumbre para aplicaciones seguras, evaluando cuán improbable es el valor más predecible. A estas fuentes de entropía naturales también se las llama *generadores de bits verdaderamente aleatorio.*

Ejemplo: entropía mínima de Rényi de una secuencia aleatoria

Supongamos que tenemos una variable aleatoria X que puede tomar los valores $\{a, b, c\}$ con probabilidades:

- $P(X = a) = 0{,}7$

[1] Aleatoriedad recogida por un sistema operativo o una aplicación para su uso en criptografía o para otros usos que requieren datos aleatorios. Esta aleatoriedad a menudo se obtiene de fuentes de hardware, ya sea preexistentes, tales como los movimientos del ratón o, si se dispone, de generadores de azar.

- $P(X = b) = 0{,}2$
- $P(X = c) = 0{,}1$

La entropía mínima (min-entropy) de X se calcula como:

$$H_\infty(X) = -\log \max(P(X = a), P(X = b), P(X = c)) = -\log 0{,}7 \approx 0{,}514$$

Este valor relativamente bajo de entropía mínima indica que la variable aleatoria X es predecible en cierta medida, ya que hay un valor que tiene una probabilidad considerablemente alta de ocurrir.

Un generador de bits verdaderamente aleatorio (TRNG) requiere una fuente natural de aleatoriedad. Diseñar un dispositivo de hardware o un programa de software para explotar esta aleatoriedad y producir una secuencia de bits libre de sesgos y correlaciones es una tarea difícil. Por ejemplo, se sabe que el ruido térmico de una resistencia es una buena fuente de aleatoriedad. Incluso se han utilizado plátanos para generar números aleatorios.

Los TRNGs también pueden recolectar entropía en un sistema operativo en ejecución mediante el uso de sensores conectados, dispositivos de E/S, actividad de la red o del disco, registros del sistema, procesos en ejecución y actividades del usuario como pulsaciones de teclas y movimientos del ratón. Estas actividades generadas por el sistema y por el humano pueden ser una buena fuente de entropía, pero pueden ser frágiles y ser manipuladas por un atacante. Además, son lentas para producir bits aleatorios.

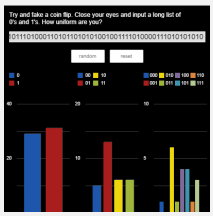
En los sistemas comunes de *nix, típicamente se utiliza el archivo de dispositivo `/dev/urandom` para generar bits aleatorios fiables. Debido a que es un archivo de dispositivo, la solicitud de bits aleatorios de `/dev/urandom` se hace leyendo como un archivo. Por ejemplo, el siguiente comando usa `/dev/urandom` para escribir 10 MB de bits aleatorios en un archivo:

```
dd if=/dev/urandom of=<archivo de salida>bs=1M count=10
```

Una excelente fuente de números aleatorios es la página web RANDOM.ORG, que los genera a partir del ruido atmosférico. Puedes generarlos desde el sitio web, a través de un widget en tu propia página o mediante llamadas a su API.

PRÁCTICA 2.1: ¿Eres verdaderamente aleatorio?

Conéctate a la página ¿Qué tan uniforme eres? y, con los ojos cerrados, trata de generar una secuencia aleatoria de unos y ceros, a ver qué tal se te da. El objetivo es que todas las barras de cada grupo tengan la misma altura. Si eres humano, ¡no podrás! Prueba lanzando una moneda al aire y verás la diferencia.



2.3.2. Generadores de números aleatorios cuánticos (QRNG)

Los generadores de números aleatorios cuánticos (QRNG) son un tipo de RNG que se basa en la aleatoriedad que surge de los fenómenos de la mecánica cuántica como la desintegración radiactiva, las fluctuaciones del vacío y la observación de la polarización de los fotones. Estos pueden proporcionar una aleatoriedad real, en lugar de sólo una aleatoriedad aparente. Sin embargo, en la práctica, los QRNG pueden estar sesgados y no producen bits rápidamente. La mayoría de los dispositivos QRNG utilizan las propiedades cuánticas de los fotones de una u otra manera como fuente primaria de entropía.

Un método extendido consiste en utilizar una fuente de fotones individuales, producidos uno por uno para evitar interferencias entre ellos, y un divisor de haz, que divide un rayo de luz en dos posibles trayectorias con una probabilidad del 50 %. Cuando un fotón alcanza el divisor, no sabemos de antemano por cuál trayectoria irá: su comportamiento es intrínsecamente aleatorio. Se colocan detectores de fotones en las dos posibles trayectorias, de tal manera que si el fotón

Clasificación de RNG	Nivel	Descripción	Ejemplos
Incertidumbre cuantificada	Nivel 4	Capacidad para modelar y verificar la imprevisibilidad de los procesos físicos subyacentes en el proceso de generación de entropía en tiempo real.	QRNG con metrología
Caos diseñado	Nivel 3	Los componentes están diseñados para generar entropía; sin embargo, los ataques aún son posibles. Su imprevisibilidad depende en gran medida de pruebas estadísticas, ya que las pruebas físicas de imprevisibilidad son imposibles o demasiado complejas.	TRNG (osciladores de anillo, diodos de avalancha...)
Aleatoriedad oportunista	Nivel 2	Los sistemas aprovechan procesos que muestran un comportamiento aparentemente impredecible como fuentes de entropía no dedicadas. Dependen de factores externos, a menudo no controlados.	Movimiento del ratón, pulsaciones de teclado, fluctuaciones de reloj...
Ilusión de aleatoriedad	Nivel 1	Hay una fachada de aleatoriedad más que una presencia real. Se utiliza entropía baja o inexistente. Por ejemplo, se reutilizan los mismos números o se utiliza un RNG determinista/pseudo-RNG.	RNG basado en software, archivos de números aleatorios (RN)

Cuadro 2.1: Clasificación de fuentes de RNG (fuente: Quantum Random Number Generators—What's all the fuss about?)

es detectado en el detector A, se interpreta como un 0 y si el fotón es detectado en el detector B, se interpreta como un 1. Este proceso es impredecible porque está basado en la superposición cuántica y el colapso de la función de onda del fotón. Esto significa que cada detección ocurre de manera aleatoria y no puede ser influida por ningún factor externo clásico.

Divisor de haz (beam splitter): Un divisor de haz es un componente óptico que divide un rayo de luz en dos posibles trayectorias con una probabilidad del 50

Cuando un fotón alcanza el divisor, no "decide"de antemano por cuál trayectoria irá; en cambio, su comportamiento es intrínsecamente aleatorio. Detección de trayectoria:

Se colocan detectores de fotones en las dos posibles trayectorias. Si el fotón es detectado en el detector A, se interpreta como un "0". Si el fotón es detectado en el detector B, se interpreta como un "1". El proceso es impredecible porque está basado en la superposición cuántica y el colapso de la función de onda del fotón. Esto significa que cada detección ocurre de manera aleatoria y no puede ser influenciada por ningún factor externo clásico.

Como las fuentes de entropía anteriormente citadas, necesitan un componente adicional para producir bits de forma fiable a alta velocidad. Varias empresas ofrecen dispositivos QRNG, como QuSide, ID Quantique o Qrypt. Incluso puedes generar y utilizar tu propio número aleatorio generado por la cuántica en varios sitios en Internet, incluyendo ANU QRNG. Y si quieres incluirlos en tus propios programas, puedes probar con la biblioteca para Python qrng.

2.3.3. Generadores pseudoaleatorios (PRNG)

Por desgracia, al igual que el lanzamiento de monedas para producir bits aleatorios no sería práctico para las aplicaciones criptográficas, la mayoría de las fuentes naturales de entropía no son prácticas debido a la lentitud inherente al muestreo del proceso y a la dificultad de asegurar que un adversario no observe el proceso. Además, sería imposible de reproducir, lo que exigiría dos copias de la misma secuencia: una para Alice y otra para Bob, lo que entraña la dificultad casi insalvable de hacérselas llegar a ambos. Por lo tanto, se requiere un método para generar aleatoriedad que se pueda implementar en software y que sea fácilmente reproducible, tantas veces como sea necesario.

La respuesta para generar una secuencia aleatoria de 0's y 1's de longitud arbitraria es un

generador pseudoaleatorio: un algoritmo eficiente y determinístico para transformar una cadena corta y uniforme de longitud k, llamada la *semilla*, en una cadena de salida más larga, "de aspecto uniforme" (o *pseudoaleatorio*), de longitud $l \gg k$. Dicho de otra manera:

Un generador pseudoaleatorio utiliza una pequeña cantidad de verdadera aleatoriedad para generar una gran cantidad de pseudoaleatoriedad.

Los generadores de números pseudoaleatorios son funciones deterministas y sólo pueden producir números aleatorios con tanta entropía como la de su semilla.

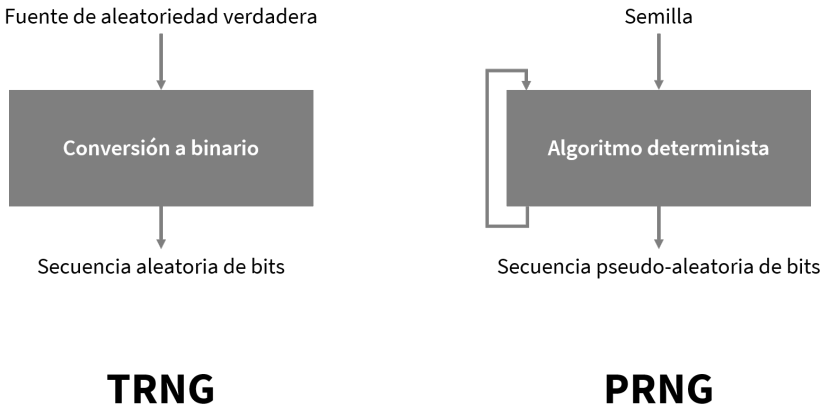


Figura 2.1: El reto de la generación de números aleatorios: generadores verdaderamente aleatorios (TRNG) y generadores pseudoaleatorios (PRNG).

Eso sí, debe quedar claro que las secuencias así generadas no son aleatorias en sentido estricto, sino *estadísticamente aleatorias*, es decir, *parecen* aleatorias. Cómo afirmó el gran matemático John von Neumann, famoso por su participación en el Proyecto Manhattan:

"Cualquiera que considere métodos aritméticos para producir dígitos aleatorios está, por supuesto, en estado de pecado. Porque, como se ha señalado varias veces, no existe un número aleatorio, sólo hay métodos para producir números aleatorios, y un procedimiento aritmético estricto, por supuesto, no es tal método."

2.4. Generadores pseudoaleatorios inseguros

La mayoría de bibliotecas de programación estándar contienen una función `rand()` que genera números pseudo-aleatorios. Esta función pseudo-aleatoria toma una semilla como entrada y produce una secuencia de bits como salida.

En esta sección aprenderás cómo funcionan dos algoritmos ampliamente utilizados para este fin y que **nunca deberían utilizarse en criptografía**: el Generador Lineal Congruencial (LCG) y Mersenne Twister (MT). Se han producido numerosas vulnerabilidades por ignorar este precepto: CVE-2013-2803 o CVE-2013-6386. Resulta muy educativa la historia de cómo un investigador hackeó una web de lotería con un bote de 1.500 millones de dólares que usaba la función `Math.random()` para generar los números.

2.4.1. Generador Lineal Congruencial (LCG)

Por ejemplo, la clase predeterminada para generar números aleatorios en Java, `java.util.Random`, implementa un generador lineal congruencial (*linear congruential generator*, LCG). Estos generadores producen una secuencia de números $x_1, x_2, \ldots$, mediante la siguiente recurrencia:

$$x_{i+1} = ax_i + b \mod m.$$

El número x_0 es la semilla inicial, mientras que los números a, b y m son parámetros que rigen la relación. La selección de valores para a, b y m resulta fundamental para desarrollar un buen generador de números aleatorios.

Por ejemplo, para $a = b = 1$. La secuencia producida obviamente no es satisfactoria. En cambio, para $a = 7, b = 0, m = 32$, y $x_0 = 1$ se genera la secuencia $\{7, 17, 23, 1, 7, \ldots\}$, que también es claramente insatisfactoria. De los 32 valores posibles, sólo se usan cuatro; por lo tanto, se dice que la secuencia tiene un período de 4. Si, en cambio, se usa el valor $a = 5$,

entonces la secuencia generada es $\{5, 25, 29, 17, 21, 9, 13, 1, 5, \ldots\}$, lo que aumenta el período a 8. En general, se busca que m sea lo más grande posible, para que exista el potencial de producir una larga serie de números aleatorios distintos.

En el caso de Java, a =25.214.903.917, $b = 11$ y $m = 2^{48}$. `java.util.Random` nunca da sus 48 bits de estado completos, da como máximo 32 en cada llamada a `nextInt()`. Otras llamadas que devuelven más bits, por ejemplo, `nextLong()`, generan múltiples números de 32 bits y los combinan entre sí.

PRÁCTICA 2.2: Generando secuencias pseudoaleatorias del LCG con Excel

Vamos a implementar un LCG usando Excel. Si no te manejas con Excel, casi mejor prográmalo en tu lenguaje favorito.

1. Asigna nombre a cuatro celdas: a, b, m, x0 e introduce un valor para cada una de ellas. Serán los parámetros de operación del LCG.
2. En una celda cualquiera, por ejemplo la A3, escribe: =x0. Tomará el valor que has asignado a esta constante.
3. En la celda inmediatamente debajo, escribe: =RESIDUO(A3*a+b;m).
4. Estira hacia abajo esta celda y obtendrás la serie generada por el LCG.

Cada vez que cambies el valor de alguno de los parámetros, se actualizarán los valores generados por el LCG. Puedes probar a representarlos gráficamente, como se explicó en la práctica anterior. Y ya, para nota, anímate a escribir un programa para implementar un LCG. No te llevará más de un par de líneas de código.

El uso de generadores de números pseudoaleatorios basados en LCG es adecuado para fines experimentales, como las simulaciones, pero se desaconseja totalmente para fines criptográficos, debido a que son *predecibles*: aunque no se conozcan los parámetros a, b y m, un atacante puede utilizar el conocimiento de algunos números generados para predecir números futuros.

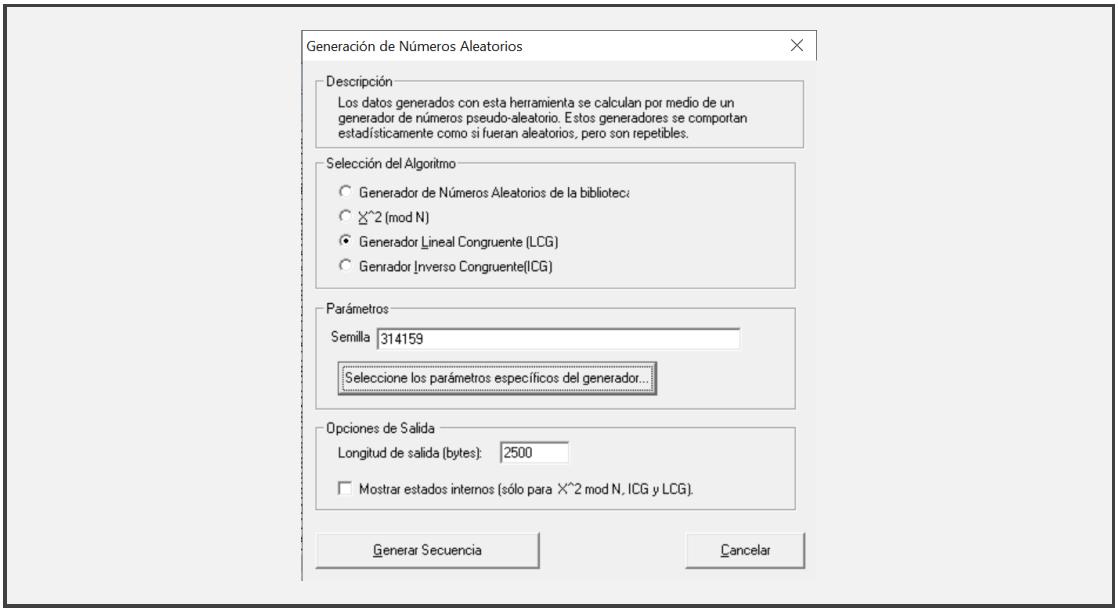
Si un adversario sabe que se está utilizando un LCG y si se conocen los parámetros (por ejemplo, $a = 7^5, b = 0, m = 2^{31} - 1$), entonces una vez que se descubre un solo número, se conocen todos los números subsiguientes.

En el caso de Java resulta un pelín más complicado porque, aunque la semilla tiene 48 bits de precisión, se convierte en una de 32 bits, desplazando 16 bits hacia la derecha. Por consiguiente, no es posible determinar la semilla a partir de un único entero generado por `java.util.Random`, pero con el siguiente entero, es fácil adivinar esos 16 bits descartados, forzando todos los valores posibles, y viendo si el siguiente valor de ese entero coincide con el segundo valor obtenido.

Incluso si el oponente sólo sabe que se está utilizando un algoritmo lineal congruencial, el conocimiento de una pequeña parte de la secuencia es suficiente para determinar los parámetros del algoritmo.

PRÁCTICA 2.3: Generando secuencias pseudoaleatorias del LCG con CrypTool

CrypTool proporciona distintos algoritmos para generar números aleatorios. Puedes probar el LCG en `Procedimientos Indiv. >Herramientas >Generar números aleatorios`. También tienes la opción de experimentar con los distintos parámetros.



2.4.2. Mersenne Twister (MT)

El algoritmo Mersenne Twister (MT) es un PRNG no criptográfico utilizado en PHP, Python, R, Ruby y otros muchos sistemas. MT genera bits aleatorios uniformemente distribuidos sin sesgo estadístico, pero es predecible: dados unos pocos bits producidos por MT, es bastante fácil predecir qué bits seguirán. Veamos por qué.

El estado interno de MT es una matriz, S, consistente en 624 palabras de 32 bits. Esta matriz se inicializa con $S_1, S_2, \ldots, S_{624}$ y evoluciona a $S_2, S_3, \ldots, S_{625}$, luego a $S_3, S_4, \ldots, S_{626}$, y así sucesivamente, de acuerdo con la ecuación

$$S_{k+624} = S_{k+397} \oplus A((S_k \wedge \text{0x80000000}) \vee (S_{k+1} \wedge \text{0x7fffffff}))$$

donde $\oplus$ denota la operación XOR bit a bit, $\wedge$ denota el AND y $\vee$ denota el OR y A es una función que transforma una palabra de 32 bits, x, en $(x \gg 1)$, si el bit más significativo de x es 0, o en $(x \gg 1) \oplus$0x9908b0df en caso contrario.

Si te fijas bien, verás que en esta ecuación los bits de S interactúan entre sí sólo a través de los XOR. Los operadores $\wedge$ y $\vee$ nunca combinan dos bits de S juntos, sino sólo bits de S con bits de las constantes 0x80000000 y 0x7fffffff. De esta manera, cualquier bit de S_{625} puede expresarse como un XOR de bits de S_{398}, S_1 y S_2, y cualquier bit de cualquier estado futuro puede ser expresado como una combinación XOR de bits del estado inicial $S_1, \ldots, S_{624}$.

Debido a que hay exactamente $624 \times 32 = 19.968$ bits en el estado inicial (o 624 palabras de 32 bits), cualquier bit de salida puede expresarse como una ecuación con un máximo de 19.969 términos (19.968 bits más un bit constante). Eso son unos 2,5 kilobytes de datos. El recíproco también es cierto: los bits del estado inicial pueden ser expresados como un XOR de bits de salida.

Si eres de los que les gusta probar por sí mismos los ataques, disfrutarás con la lectura de Cracking Random Number Generators, donde el autor explica paso a paso y con ejemplos de código cómo programar un ataque para romper MT.

2.5. Otros generadores pseudoaleatorios, ahora sí, seguros (CSPRBG)

Por supuesto, existen otros muchos métodos para generar secuencias pseudoaleatorias a partir de una semilla que se consideran seguros para aplicaciones criptográficas, como por ejemplo en el cifrado en flujo, que conocerás en la próxima lección. A estos generadores se los denomina Generadores de Números Pseudo-Aleatorios Criptográficamente Seguros (*Cryptographically Secure Pseudo Random Number Generator*, CSPRNG). Un CSPRBG se define como aquel que pasa *la prueba del siguiente bit*:

La prueba del siguiente bit

Dados los primeros k bits de la secuencia, no hay un algoritmo práctico que permita afirmar que el siguiente bit será 1 (o 0) con una probabilidad superior a 1/2. A todos los efectos prácticos, la secuencia es *impredecible*.

Aunque en este curso no entraremos a revisar su funcionamiento, sí conviene al menos listar algunos muy conocidos y utilizados.

2.5.1. RSA

El generador de bits pseudoaleatorios RSA es un CSPRBG bajo la suposición de que el problema de RSA es intratable. Para producir una secuencia, se aplica el siguiente algoritmo.

1. Genera dos primos aleatorios de gran tamaño p y q y calcula $n = p \cdot q$ y $\phi(n) = (p-1)(q-1)$. Selecciona un entero aleatorio $e \in \{1, 2, \ldots, \phi(n) - 1\}$ tal que $gcd(e, \phi(n)) = 1$. El símbolo $gcd()$ representa el **máximo común divisor**: definido como el mayor número entero que divide a dos o más números enteros sin dejar residuo alguno.
2. Selecciona un entero aleatorio x_0 (la semilla) en el intervalo $[1, n-1]$.
3. Para i desde 0 hasta $l-1$:
   a) $x_{i+1} = x_i^e \mod n$
   b) $b_{i+1} \leftarrow$ el bit menos significativo de x_{i+1}
4. La secuencia de salida es $b_1, b_2, \ldots, b_l$.

Pueden realizarse modificaciones para mejorar su eficiencia.

2.5.2. Blum-Blum-Shub

También conocido como el generador $x^2 \mod n$ o el generador BBS, es un CSPRBG bajo el supuesto de que la factorización de enteros es intratable. Constituye la base del esquema de cifrado probabilístico de clave pública de Blum-Goldwasser.

En este esquema, primero genera dos primos grandes p y q congruentes con $3 \mod 4$. Es decir,

$$p \equiv q \equiv 3 \mod 4$$

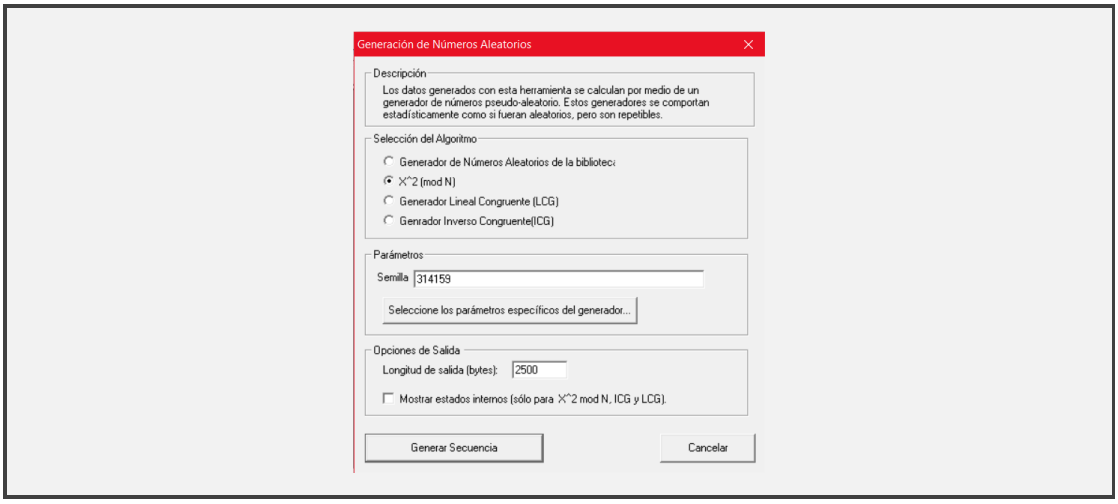
Este requisito asegura que cualquier residuo cuadrático de un valor modular $n = p \cdot q$ tiene una cantidad específica y controlada de raíces cuadradas, facilitando la generación de una secuencia de bits pseudoaleatoria segura.

Para operar, calcula $n = p \cdot q$ y elige una semilla aleatoria s en el intervalo $[1, n-1]$ relativamente prima con n, es decir, $gcd(s, n) = 1$. Para inicializar el generador BBS, establece el valor inicial a $x_0 \equiv s^2 \mod n$. El generador BBS produce una secuencia de bits aleatorios $b_1, b_2, \ldots$ dada por

1. $x_{i+1} = x_i^2 \mod n$
2. $b_{i+1} \leftarrow$ el bit menos significativo de x_{i+1}.

PRÁCTICA 2.4: Generando números aleatorios con BBS en CrypTool

También puedes probar BBS en CrypTool: `Procedimientos Indiv. >Herramientas >Generar números aleatorios >X ^ 2 (mod N)` y configurar el valor del módulo.



2.5.3. Algoritmos criptográficos

Los algoritmos de cifrado y los hashes criptográficos pueden utilizarse como generadores de números pseudoaleatorios de muy alta calidad. Sin embargo, suelen ser considerablemente más lentos (normalmente por un factor 2 a 10) que los generadores de números aleatorios rápidos no criptográficos. Estos algoritmos parten de una cierta entropía inicial (semilla) y calculan el siguiente número aleatorio mediante algún cálculo que es imprevisible sin conocer la semilla. Estos cálculos se denominan *funciones pseudoaleatorias* (PRF).

Las funciones pseudoaleatorias suelen utilizar un estado interno. Al principio, el estado se inicializa con una semilla inicial. Cuando se genera el siguiente número aleatorio, se calcula a partir del estado interno (utilizando algún cálculo o fórmula), y luego se cambia el estado interno de la función pseudoaleatoria (utilizando algún cálculo o fórmula). Cuando se genera el siguiente número aleatorio, se calcula de nuevo a partir del estado interno de la función y este estado se cambia de nuevo y así sucesivamente. Este proceso, en su forma más simple, puede implementarse como sigue:

```
init(entropy):
  state = entropy, counter = 0
nextNum():
  state = PRF(state, ++counter)
  return state}
```

Como función pseudoaleatoria puede utilizarse una función hash o un hmac, las cuales se estudiarán en la lección 5.

PRÁCTICA 2.5: Generando números aleatorios criptográficamente seguros con Python

Copia el siguiente código a un archivo. Si no tienes instalado Python en tu sistema, cópialo a un playground, como el de W3Schools, y ejecútalo.

```
import hashlib, hmac
entropy = b"hola"
seed = hashlib.sha256(entropy).hexdigest()
print("seed = ", seed)
for counter in range(1,10):
    h = hmac.new(seed.encode(),str(counter).encode(),hashlib.sha256)
    .hexdigest()
    print(counter, " | ", h)
```

2.6. Generadores de números pseudo-aleatorios criptográficamente seguros para programadores

Los PRNG no criptográficos están diseñados para producir distribuciones uniformes para aplicaciones como simulaciones científicas o videojuegos. Sin embargo, nunca se deben usar en

aplicaciones criptográficas, porque son inseguras: sólo se preocupan por la calidad de la distribución de probabilidad de los bits y no por su previsibilidad.

Tal es el caso de la típica función `rand()` de los lenguajes de programación, que falla miserablemente a la hora de generar claves aleatorias, como le pasó a DMA Locker v2. Por ejemplo, en 2013 se descubrió una vulnerabilidad crítica en **todos** los monederos de Bitcoin para Android. La vulnerabilidad radicaba en el método `java.security.SecureRandom` de Android, plagado de errores. Como aprenderás en la lección 6, las firmas ECC necesitan un valor aleatorio que, si se repite, permite violar la firma. Por desgracia, la función `java.security.SecureRandom` generaba valores repetidos con alarmante frecuencia.

La forma más segura es utilizar funciones robustas y bien testadas, que implementen CSPRNG:

- En los sistemas operativos de la familia Windows, las funciones CSPRNG están disponibles a través de APIs dedicadas, como `CryptGenRandom()`, vía la Cryptographic API (CAPI), ahora obsoleta; la posterior `RtlGenRandom()`, llamada también `SystemFunction036`; o, la más reciente, `BCryptGenRandom()`, proporcionada por la Cryptography API: Next Generation (CNG). Windows toma la entropía del controlador de modo del kernel cng.sys (antes ksecdd.sys), cuyo recolector de entropía está vagamente basado en Fortuna.

- El generador aleatorio proporcionado por el módulo `random` de Python es un generador de números pseudo-aleatorios que no es criptográficamente seguro. Como resultado, se ha añadido el módulo `secrets` en Python 3.6 y en adelante para asegurar el generador de números aleatorios. El módulo `secrets` es CSPRNG, es decir, genera números pseudoaleatorios criptográficamente seguros. Este módulo se basa en `os.urandom()` y `random.SystemRandom()`, una interfaz para la mejor fuente de aleatoriedad criptográfica del sistema operativo. En Windows, `os.urandom()` utiliza internamente `CryptGenRandom()`. En Linux 3.17 y posteriores, se utiliza la llamada al sistema `getrandom()` cuando está disponible. En OpenBSD 5.6 y posteriores, se utiliza la función C `getentropy()`.

- Por supuesto, existen soluciones análogas en otras familias de sistemas operativos y en otros lenguajes, aunque con otros nombres. En general, estas funciones devuelven números aleatorios tan buenos como la mejor fuente de entropía que pueda proporcionar tu sistema operativo.

Así pues, la mejor opción será que el malware invoque al ejecutarse a una función CSPRNG para generar la clave aleatoria. De hecho, este paso resulta tan crucial para el éxito del ataque, que algunas propuestas anti-ransomware, como USHALLNOTPASS, Redemption, PayBreak o DRBG, proponen controlar el acceso a estas funciones mediante *hooks* o instalar en ellas puertas traseras para coartar así la capacidad del ransomware de generar claves robustas. Como reza el título de uno de esos trabajos: *no random, no ransom.*

PRÁCTICA 2.6: Generando números aleatorios criptográficamente seguros con OpenSSL

Puedes utilizar OpenSSL para generar números aleatorios mediante el siguiente comando, que genera un número de 16 bytes (128 bits) y lo muestra en formato hexadecimal.

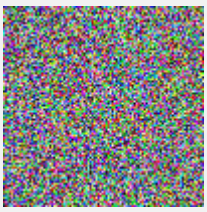
```
$ openssl rand -hex 16
57e4aa42c6e93582838d211b0f060311
```

1. ¿Pasa el test de inspección visual? Anímate a representar gráficamente la secuencia en un BMP, al estilo de lo que hiciste en la primera lección. Pruébalo con una imagen de 100×100 píxeles.

   a) En Paint, crea una imagen en blanco de 100×100 y guárdala como BMP color de 24 bits.

   b) Genera con OpenSSL un número aleatorio de longitud múltiplo de 30.000 bytes. Así, podrás hacer que cada grupo de tres bytes corresponda a los tres colores RGB de cada pixel.

   c) Abre el BMP con CrypTool y pega la secuencia aleatoria sobre los bits de imagen (si has creado la imagen en blanco, los 30.000 bytes deberían tener valor `0xFF`). Mira lo que me sale a mí:



2. Experimenta con las opciones del comando `rand`, porque te vas a hartar a usarlo durante el curso.

2.6.1. Si la semilla es predecible, da igual el generador que uses

Para que un PRNG sea efectivo en un entorno criptográfico, la semilla deber ser verdaderamente aleatoria, procedente por ejemplo de `/dev/urandom`. Un error común es usar como semilla para el PRNG valores predecibles como:

- La hora actual: por ejemplo, CVE-2011-0766.
- El identificador de proceso PID: por ejemplo, CVE-2011-2190.
- Un valor constante: por ejemplo, CVE-2013-6394.

Algunas versiones de Libc en OS X utilizan datos no inicializados como semilla. La lectura de datos no inicializados es un comportamiento indefinido en C, por lo que el código para inicializar el PRNG podría ser eliminado por las optimizaciones del compilador.

De manera similar, el tristemente famoso bug de SSL en Debian (CVE-2008-0166) fue causado por un programador que intentaba suprimir las advertencias de Valgrind en el código que leía los datos no inicializados para sembrar un PRNG.

Ejemplo: desenmascarando fuentes de entropía pobres

En 1996, Netscape se vio obligado a sacar la nueva versión de SSL tras descubrirse que inicializaba la semilla de su PRNG con una combinación de PIDs y tiempo fáciles de adivinar. Estudia el siguiente pseudocódigo del viejo navegador Netscape e identifica la línea con el error garrafal que cometió el programador.

```
global variable seed;
RNG_CreateContext()
    (seconds, microseconds) = time(); /* Time elapsed since 1970 */
    pid = process ID;
    ppid = parent process ID;
    a = mklcpr(microseconds);
    b = mklcpr(pid + seconds + (ppid << 12));
    seed = MD5(a, b); /* Derivation of a 128-bit value using MD5 */
    mklcpr(x) /* not cryptographically significant */
    return ((0xDEECE66D * x + 0x2BBB62DC) >> 1);
```

2.6.2. Y si reutilizas la semilla, estarás igual de mal

Para asegurar que los números aleatorios son independientes en los procesos, un PRNG debe ser resembrado en cada nuevo proceso. Por ejemplo, stunnel antes de la versión 5.00 (CVE-2014-0016) y libssh antes de la 0.6.3 (CVE-2014-0017) no reiniciaron el PRNG en procesos hijo bifurcados, de modo que un atacante que conoce las salidas del PRNG de un proceso puede predecir la aleatoriedad del otro proceso hijo. Las versiones antiguas de Ruby presentaban un problema similar (CVE-2011-2686).

2.6.3. Y si además usas rand(), todavía peor

Una vulnerabilidad descubierta en la generación de claves AES de un sistema de alarma muy utilizado en Alemania permitió clonar los llaveros utilizados con este sistema. La vulnerabilidad se asignó a CVE-2021-34600. A continuación se muestra el código deficiente usado para la generación de las claves:

Ejemplo: generación de claves AES inseguras

```
static uint8_t key[16];
memset(key, 0, 16);
srand(time());
for (int i = 0; i < 16; i++) {
    key[i] = rand() % 0xFF;
}
return key;
}
```

Las claves AES generadas con estas funciones están

1. basadas en la marca de tiempo Unix actual del sistema y
2. generadas usando un generador de números pseudoaleatorios criptográficamente inseguro.

2.7. ¿Qué algoritmos de generación de números aleatorios utiliza el ransomware?

En la tabla 2.2 se listan las funciones de generación de claves para algoritmos simétricos y asimétricos usadas por el malware analizado. La mayoría de muestras llevan precalculadas las claves asimétricas, lo cual resulta más inseguro, como aprenderás más adelante en la Sec. 4.9.4.

Ransomware	Clave secreta	Clave pública
Ryuk	CryptGenKey	Harcoded
Dharma	get_random_NZ	Harcoded
LockBit	CryptGenRandom	Harcoded
SamSam	RNGCryptoServiceProvider	Harcoded
GandCrab	CryptGenRandom	CryptGenKey
Clop	CryptGenKey	Harcoded
Katyusha	rand()	Harcoded
Snatch	rand()*	Harcoded
Phobos	CryptGenRandom	Harcoded
Nemty	rand()	CryptGenKey

Cuadro 2.2: Funciones de generación de claves utilizadas por el ransomware moderno (*) Del paquete `crypto/rand`, que implementa un generador de números aleatorios criptográficamente seguro. (Fuente: Roderik Ploszek, Peter Švec and Patrik Debnár, Analysis of encryption schemes in modern ransomware, Rad Hazu. Matematicke Znanosti, 25 (546), 1-13, 2021. https://doi.org/10.21857/mn1qgc58gy).

2.8. ¿Será esta secuencia aleatoria? Pruebas de aleatoriedad

Los valores aleatorios son fundamentales para que la criptografía pueda resistir el análisis y los ataques de la criptografía. La comprobación del carácter aleatorio de las claves criptográficas es una actividad crucial para determinar la calidad de la implementación de toda aplicación criptográfica. Existen varias suites para pruebas estadísticas como TestU01, Diehard o las del National Institute of Standards and Technology (NIST). Calculan una serie de estadísticos sobre



Figura 2.2: El problema con la aleatoriedad es que nunca estás seguro.

las secuencias y los comparan con los resultados típicos que se obtendrían de una distribución perfectamente uniforme. Por ejemplo, se realizan los conocidos test *monobit*, test de *rachas* y test de *autocorrelación*.

2.8.1. Análisis de aleatoriedad con los postulados de Golomb

Para determinar si una secuencia de bits generada por un PRNG es adecuada para su uso en criptografía, es fundamental analizar sus propiedades de aleatoriedad. Una herramienta clásica para este propósito son los **postulados de Golomb**, propuestos por Solomon W. Golomb en 1967. Estos postulados proporcionan criterios matemáticos para evaluar la calidad de las secuencias pseudoaleatorias, especialmente aquellas utilizadas en sistemas de cifrado de flujo y secuencias de expansión de espectro.

2.8.1.1. ¿Qué son los postulados de Golomb?

Los postulados de Golomb son tres propiedades que una secuencia binaria larga debe cumplir para ser considerada pseudoaleatoria:

1. **Balance de unos y ceros**: En una secuencia de longitud N, el número de unos (n_1) y ceros (n_0) debe ser aproximadamente igual. Matemáticamente, esto se expresa como:

$$|n_1 - n_0| \leq 1 \tag{2.1}$$

2. **Propiedad de rachas**: Las rachas (*runs*) son subsecuencias de bits iguales consecutivos. Los postulados establecen que el número de rachas de longitud k debe seguir una distribución específica:
   - Aproximadamente la mitad de las rachas deben ser de longitud 1.
   - Aproximadamente una cuarta parte deben ser de longitud 2.
   - En general, el número de rachas de longitud k debe ser $\frac{1}{2^k}$ del número total de rachas.
3. **Autocorrelación**: La autocorrelación desplazada de la secuencia debe ser mínima, lo que significa que la secuencia no debe mostrar patrones repetitivos cuando se compara consigo misma desplazada en el tiempo.

Veamos cómo aplicar cada uno de estos postulados a una secuencia de bits para evaluar su aleatoriedad.

Ejemplo: test de frecuencia (o monobit)

Su propósito es determinar si el número de unos y ceros en una secuencia es aproximadamente el mismo que se esperaría para una secuencia verdaderamente aleatoria. El test evalúa la cercanía de la fracción de unos a 1/2, es decir, el número de unos y ceros en una secuencia debería ser aproximadamente el mismo. Normalmente es la primera prueba que se realiza porque si la secuencia no pasa esta prueba, tampoco se espera que pase las otras pruebas.
El test de frecuencia funciona de la siguiente manera:

1. Dada una secuencia s de n bits, convierte los ceros y unos a los valores -1 y $+1$, o lo que es lo mismo, suma los valores 0's y 1's para producir $S_n = \sum(2s_i - 1)$.
2. Calcula el estadístico $S_{obs} = \frac{|S_n|}{\sqrt{n}}$.
3. Calcula $P = erfc(\frac{S_{obs}}{\sqrt{2}})$, donde *erfc* es la función error complementaria.

Si el valor $P < 0,01$, entonces se concluye que la secuencia no es aleatoria. En caso contrario, la secuencia es aleatoria.
Por ejemplo, para la secuencia
`01000111101111010110011100010000010001011101110011111010111000010`
`11000100000111001001000110111011101100100111010011000111010110010`
$P = 0,109599 > 0,01$ y, por lo tanto, pasa el test.

Ejemplo: test de rachas

Su finalidad es determinar si el número de rachas de unos y ceros de diversas longitudes es el esperado para una secuencia aleatoria. En particular, este test determina si la oscilación entre esos ceros y unos es demasiado rápida o demasiado lenta.
Una *racha* de una secuencia s es una sub-secuencia de s que consiste en 0's consecutivos o en 1's consecutivos que no están precedidos ni sucedidos por el mismo símbolo. Una serie de ceros se denomina *hueco*, mientras que una serie de unos se denomina *bloque*.
Por ejemplo, la secuencia $s = \{0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1\}$ tiene 8 rachas: 4 rachas de longitud 1 (2 huecos y 2 bloques), 2 rachas de longitud 2 (1 hueco y 1 bloque), 1 racha de longitud 3 (1 hueco) y 1 racha de longitud 4 (1 bloque).
A menudo la criptografía opera sobre bloques de datos de longitud específica; por lo tanto, resulta pertinente evaluar las características de la cadena de datos cuando se observa en bloques similares a los que utilizará el algoritmo.
El test de rachas funciona de la siguiente manera:

1. Dada una secuencia s de n bits, calcula la proporción π de unos en s: $\pi = \frac{\sum s_i}{n}$.
2. Si $|\pi - 1/2| \geq 2/\sqrt{n}$, entonces no es necesario seguir adelante porque no pasa el test.
3. Calcula el estadístico $V_n = \sum_{i=1}^{n-1} r(i) + 1$, donde $r(i) = 0$ si $s_i = s_{i+1}$ y $r(i) = 1$ en caso contrario.
4. Calcula $P = erfc(\frac{|V_n - 2n\pi(1-\pi)|}{2\sqrt{2n}\pi(1-\pi)})$.

Si el valor $P < 0,01$, entonces se concluye que la secuencia no es aleatoria. En caso contrario, la secuencia es aleatoria.
Por ejemplo, para la secuencia
`00100101001010001001101010011101100111001001110011111010011111110`
`00011101001111110011010001000110111000101000010111011000000101000`
$P = 0,85748 > 0,01$ y, por lo tanto, pasa el test.

Ejemplo: test de Autocorrelación

La autocorrelación desplazada mide la similitud entre la secuencia original y una versión desplazada de sí misma. Para una secuencia aleatoria, la autocorrelación debe ser mínima (idealmente cero) para todos los desplazamientos distintos de cero.
Para calcular la autocorrelación para un desplazamiento d, se utiliza:

$$C(d) = \sum_{i=1}^{N} s_i \oplus s_{i+d} \tag{2.2}$$

donde $\oplus$ es la suma módulo 2 (XOR), y s_i es el i-ésimo bit de la secuencia.
Calculamos la autocorrelación para un desplazamiento $d = 1$:

$$\begin{aligned} C(1) &= \sum_{i=1}^{19} s_i \oplus s_{i+1} \\ &= (1 \oplus 1) + (1 \oplus 0) + (0 \oplus 1) + \ldots + (1 \oplus 0) \\ &= 0 + 1 + 1 + \ldots + 1 \\ &= \text{Número total de diferencias entre bits adyacentes} \end{aligned}$$

En una secuencia completamente aleatoria de longitud N, se espera que $C(d) \approx \frac{N}{2}$ para $d \neq 0$.

2.8.1.2. Importancia en criptografía

Cumplir con los postulados de Golomb es esencial para garantizar que una secuencia puede resistir análisis y ataques criptográficos. Si una secuencia falla en alguno de estos criterios, podría ser predecible o mostrar patrones explotables por un atacante, comprometiendo la seguridad del sistema.

2.8.1.3. Limitaciones de los postulados de Golomb

Aunque los postulados de Golomb son útiles para evaluar ciertas propiedades de aleatoriedad, no son suficientes por sí solos para garantizar la seguridad criptográfica. Es posible que una secuencia cumpla con estos postulados y aún así sea predecible si el generador subyacente es débil. Por lo tanto, es crucial utilizar PRNGs criptográficamente seguros y complementarlos con otros análisis y pruebas estadísticas.

PRÁCTICA 2.7: Experimentando con PRNGs y los postulados de Golomb

Utiliza un lenguaje de programación de tu preferencia para generar secuencias de bits con un PRNG conocido. Luego, aplica los postulados de Golomb para analizar su aleatoriedad.

1. Genera una secuencia de 1000 bits utilizando un LCG y otra utilizando un CSPRNG.
2. Aplica los tres postulados de Golomb a ambas secuencias.
3. Compara los resultados y discute cuál secuencia muestra mejor aleatoriedad según los postulados.

2.8.2. Conclusión

Los postulados de Golomb proporcionan una base matemática para evaluar la aleatoriedad de las secuencias de bits. Sin embargo, en el contexto de la criptografía moderna, es importante complementar estos análisis con pruebas más robustas y asegurarse de utilizar generadores de números aleatorios criptográficamente seguros para mantener la integridad y confidencialidad de la información.

Aunque es imposible dar una prueba matemática de que un generador es, en efecto, un generador de bits aleatorios, las pruebas descritas aquí ayudan a detectar ciertos tipos de debilidades que el generador puede tener. Si se considera que la secuencia ha fallado en alguna de las pruebas estadísticas, el generador puede ser rechazado como no aleatorio.

En cambio, si la secuencia supera todas las pruebas estadísticas, el generador no se rechaza, ya que el hecho de superar las pruebas sólo aporta pruebas probabilísticas de que el generador produce secuencias que tienen ciertas características estadísticas de las secuencias aleatorias. Pasar estas pruebas estadísticas es una condición necesaria, pero no suficiente para que un generador sea seguro.

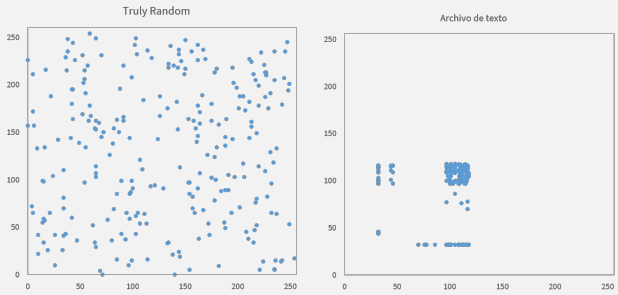
Por este motivo, la utilidad de estas pruebas en criptografía es limitada: **se puede diseñar un PRNG criptográficamente débil capaz de pasar todos los tests**.

He aquí un algoritmo para un PRNG que pasará cualquier prueba estadística que se le haga: empezar a contar enteros desde uno, hacer un hash de ese valor del contador, aumentar el contador en uno, y añadir el nuevo valor del contador al hash anterior y hacer un hash de nuevo. El resultado será una cadena de hash, una serie de números de **aspecto aleatorio**, pero **totalmente predecible**.

Además de pasar todos los tests, un requisito mínimo de seguridad para un generador de bits pseudoaleatorios es que la longitud k de la semilla aleatoria sea lo suficientemente grande para que una búsqueda exhaustiva de 2^k elementos (el número total de semillas posibles) sea computacionalmente inviable para el adversario. En otras palabras, debe ser resistente a los *ataques de fuerza bruta*.

PRÁCTICA 2.8: La prueba de aleatoriedad que solo los humanos hacen de maravilla: inspección visual de archivos en Excel

Si una secuencia de números se representa en un gráfico, nos permite evaluar visualmente su aleatoriedad. Si detectas patrones a simple vista, ¡no es aleatoria! Por ejemplo, aquí muestro la representación bidimensional de dos secuencias s, donde en el eje x represento s_i y en el eje y represento s_{i+1}. Salta a la vista que la secuencia de la izquierda *parece* aleatoria (de hecho, lo es), mientras que la de la derecha no lo es en absoluto. No es que sea muy matemático, la verdad, ¡pero funciona!



1. Abre el archivo SecuenciasAleatorias.txt y pega la primera secuencia en una hoja de Excel. A continuación, vuélvela a pegar en la columna contigua, pero desplazada una posición hacia abajo, como en el ejemplo que te muestro aquí. Ya tienes los datos para el gráfico.

124	
39	124
213	39
168	213
110	168
117	110
13	117
	13

2. Inserta un gráfico de dispersión, tomando como valores de entrada las dos columnas anteriores (menos la primera y última filas).

3. Repite el proceso para la segunda secuencia.

¿Cuál de las dos te parece más aleatoria?

2.9. Estándares de generación y tests de números pseudo-aleatorios

Varios organismos de estandarización han publicado recomendaciones sobre cómo generarlos y qué pruebas deben superar para su uso en aplicaciones criptográficas.

- NIST
  - FIPS 140-2: Security Requirements for Cryptographic Modules, May 25, 2001.
  - SP 800-90A: Recommendation for Random Number Generation Using Deterministic Random Bit Generators, June 25, 2015
  - SP 800-90B: Recommendation for the Entropy Sources Used for Random Bit Generation, January 10, 2018
  - SP 800-90C: Recommendation for Random Bit Generator (RBG) Constructions, September 7, 2022
  - SP 800-22 Rev. 1a: A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications, April 2010

- ISO
  - ISO/IEC 20543:2019: Information technology — Security techniques — Test and analysis methods for random bit generators within ISO/IEC 19790 and ISO/IEC 15408
- BSI
  - BSI TR-02102-1: "Cryptographic Mechanisms: Recommendations and Key Lengths" Version: 2022-1
  - A Proposal for Functionality Classes for Random Number Generators - V2.36
  - Functionality Classes and Evaluation Methodology for Deterministic Random Number Generators - Version 2.0
  - Functionality classes and evaluation methodology for true (physical) random number generators - Version 3.1

PRÁCTICA 2.9: Analizando la aleatoriedad con la batería de pruebas del FIPS PUB-140-2 en CrypTool

1. Genera una secuencia aleatoria de 2.500 bytes de longitud: `Procedimientos Indiv. >Herramientas >Generar números aleatorios`.
2. Pásale los tests definidos por el NIST, a ver qué tal se comporta: `Análisis >Análisis de aleatoriedad >Batería de pruebas del FIPS PUB-140-1`.



Lección 3

Cifrado simétrico o de clave secreta

Objetivos de aprendizaje

- Reconocer cómo funciona la criptografía de clave secreta, sus tipos (cifrado en flujo y en bloque) y su relevancia histórica y actual.
- Distinguir entre ECB, CBC, CFB, OFB, CTR, XTS y conocer sus ventajas, limitaciones y contextos de uso.
- Saber por qué algoritmos como AES o DES/3DES son considerados seguros o inseguros, y entender cómo influyen factores como la longitud de la clave o la reutilización del vector de inicialización.
- Usar herramientas (p. ej., OpenSSL, CrypTool) para cifrar, descifrar y realizar pruebas de seguridad con distintos algoritmos y modos.

Competencias a desarrollar

- Elegir el algoritmo simétrico y el modo de operación más apropiados para un escenario concreto (por ejemplo, cifrado de disco, comunicaciones en tiempo real, etc.).
- Manejar software especializado como OpenSSL y CrypTool para implementar, comprobar y diagnosticar esquemas de cifrado simétrico.
- Reconocer malas prácticas (p. ej., el uso de ECB en imágenes o la reutilización de IV) y proponer mejoras para reforzar la seguridad de los sistemas.
- Entender la importancia de proteger la clave simétrica (p. ej., emplear cifrado asimétrico para transportarla) y aplicar técnicas de reparto seguro de secretos, como el esquema de Shamir, según las necesidades del entorno.

La criptografía se divide en dos ramas principales:

1. **Criptografía simétrica o de clave secreta**. Lo que la gente asume que es la criptografía: cifrar y descifrar información usando la misma clave secreta. Toda la criptografía desde la

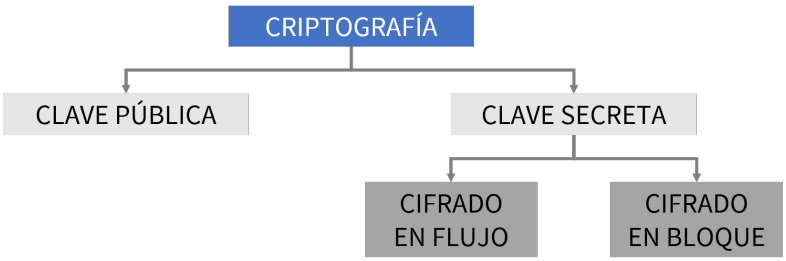


Figura 3.1: Las dos ramas principales de la criptografía y sus subramas.

antigüedad hasta 1976 se basó exclusivamente en algoritmos simétricos. El cifrado simétrico todavía se utiliza ampliamente, especialmente para el cifrado de datos y la comprobación de la integridad de los mensajes.

2. **Criptografía asimétrica o de clave pública**. En 1976, Whitfield Diffie, Martin Hellman y Ralph Merkle introdujeron un tipo de cifrado totalmente diferente y revolucionario. En la criptografía de clave pública, cada usuario posee una pareja de claves: una privada y otra pública. Los algoritmos asimétricos se utilizan para aplicaciones como el cifrado de datos, la firma digital y el acuerdo de claves. La criptografía de clave pública se estudiará en la siguiente lección.

A su vez, los algoritmos de cifrado simétrico se subdividen en dos grupos, cifrado en *flujo* y cifrado en *bloque*, como se muestra en la Fig. 3.1:

1. Los algoritmos de cifrado en flujo cifran de uno en uno los caracteres individuales, normalmente dígitos binarios (bits), de un mensaje de texto claro, utilizando una transformación de cifrado que varía con el tiempo.

2. Por el contrario, los algoritmos de cifrado en bloque tienden a cifrar simultáneamente grupos de caracteres (bloques) de un mensaje de texto claro utilizando una transformación de cifrado fija.

En esta lección aprenderás a trabajar con ambas aproximaciones al cifrado, ya que se usan masivamente en todo tipo de aplicaciones.

3.1. Cifrado en flujo

El cifrado en flujo por excelencia es la libreta de un solo uso, explicada en la primera lección (v. 1.5), en el que el mensaje se mezcla con una clave igual de larga.

$$c_i = m_i \oplus k_i$$

Ya viste en la lección anterior que el reto consistía en cómo generar una clave aleatoria tan larga como el propio mensaje. La respuesta residía en los generadores pseudoaleatorios, los cuales generan la secuencia cifrante (*keystream*), k_i, a partir de una semilla, s, que constituye la clave de cifrado. Posteriormente, esta secuencia cifrante se mezcla con el mensaje. Como Vernam, la mayoría de los cifradores en flujo propuestos hasta la fecha en la literatura son aditivos.

Un *cifrado en flujo aditivo binario* es un cifrado en flujo síncrono (la secuencia cifrante k_i se calcula independientemente del texto claro m_i y cifrado c_i), en el que los dígitos de la secuencia cifrante, del texto claro y del texto cifrado son binarios y la función de salida h en $c_i = h(m_i, k_i)$ es la función XOR, tal y como se representa en la Fig. 3.2. Las funciones de cifrado y descifrado son las mismas porque ambas hacen lo mismo, es decir, el XOR del mensaje con la secuencia cifrante.

Implementados en hardware (ASIC, PLD, FPGA, etc.) generalmente son más rápidos que los algoritmos de cifrado en bloque y tienen circuitos de hardware menos complejos, ya que las operaciones de desplazamiento y XOR son rápidas y requieren pocos recursos. También son más apropiados, y en algunos casos obligatorios (por ejemplo, en algunas aplicaciones de telecomunicaciones), cuando el buffer es limitado o cuando los caracteres deben ser procesados individualmente a medida que se reciben. Debido a que tienen una propagación de errores limitada o nula, los cifradores en flujo también pueden ser ventajosos en situaciones en las que los errores de transmisión son muy probables.

3.1.1. Registros de desplazamiento con retroalimentación lineal

Desde el veterano A5/1 usado en redes 2G al reciente Grain-128a, la mayoría de cifradores en flujo se basan en los registros de desplazamiento con retroalimentación, en particular, los registros de desplazamiento con retroalimentación lineal (*linear feedback shift registers*, LFSRs), mostrados en la Fig. 3.3.

Un LFSR de longitud L consiste en L registros numerados $0, 1, \ldots, L-1$, cada uno capaz de almacenar un bit y tener una entrada y una salida; y un reloj que controla el movimiento de los datos. Durante cada unidad de tiempo se realizan las siguientes operaciones:

1. El contenido del registro 0 es la salida y forma parte de la secuencia de salida

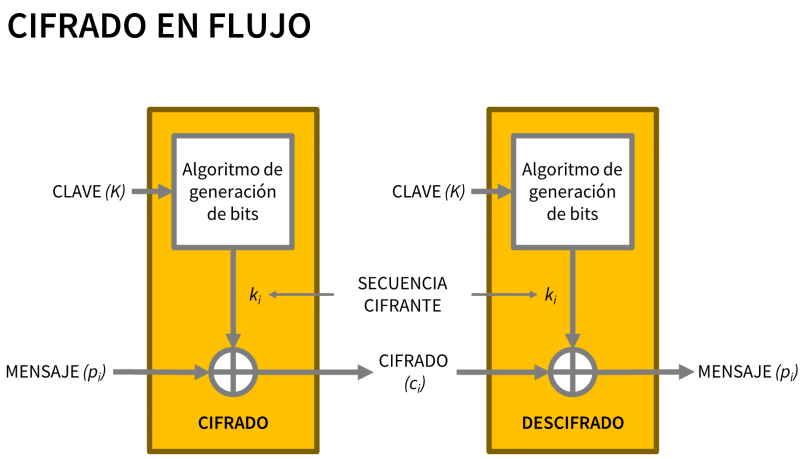


Figura 3.2: Cifrado en flujo usando un generador de bits pseudoaleatorio determinístico.

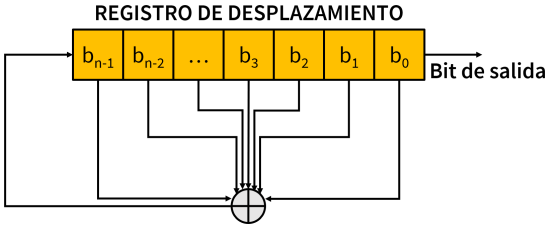


Figura 3.3: Registro de desplazamiento con retroalimentación lineal (LFSR).

2. El contenido del registro i se cambia al registro $i-1$ para cada $i, 1 \leq i \leq L-1$.
3. El nuevo contenido del registro $L-1$ es el bit de retroalimentación s_j que se calcula sumando módulo 2 los contenidos previos de un subconjunto fijo de registros $0, 1, \ldots, L-1$.

Si el estado inicial del LFSR en la Fig. 3.3 es $[s_{L-1}, \ldots, s_1, s_0]$, entonces la secuencia de salida $s = s_0, s_1, s_2, \ldots$ está determinada unívocamente por la siguiente recursión:

$$s_j = (c_1 s_{j-1} + c_2 s_{j-2} + \ldots + c_L s_{j-L}) \mod 2 \quad \text{para } j \geq L.$$

Una manera alternativa de representar la evolución del LFSR es mediante un *polinomio de retroalimentación* $P(x)$:

$$P(x) = x^j + c_{j-1}x^{j-1} + \cdots + c_1 x + 1$$

Ejemplo: funcionamiento de un LFSR

El LFSR representado en la Fig. 3.4 está definido por el siguiente polinomio:

$$P(x) = x^4 + x + 1$$

Alternativamente, también está definido por:

$$s_j = s_{j-1} + s_{j-4} \mod 2,$$

e inicializado con el estado inicial $[1, 1, 1, 1]$, produce la siguiente secuencia de estados internos antes de repetirse:

1 1 1 1
0 1 1 1
1 0 1 1
0 1 0 1
1 0 1 0
1 1 0 1

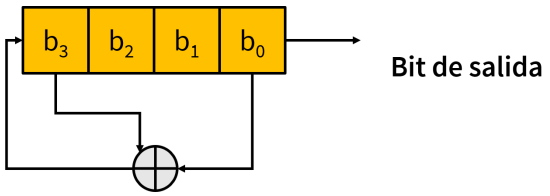


Figura 3.4: LFSR definido por $s_j = s_{j-1} + s_{j-4}$ mód 2.

0 1 1 0
0 0 1 1
1 0 0 1
0 1 0 0
0 0 1 0
0 0 0 1
1 0 0 0
1 1 0 0
1 1 1 0

La secuencia de salida es la cadena de bits menos significativos:

111101011001000 . . .

Un LFSR de L-bits puede estar en uno de $2^L - 1$ estados internos. Esto significa que puede, en teoría, generar una secuencia pseudoaleatoria de $2^L - 1$ bits de largo antes de repetirse. Un LFSR alcanza su período máximo de $2^L - 1$ si y solo si el polinomio $P(x)$ es *primitivo*, lo que significa que la secuencia no se repite hasta que todos los estados posibles (excepto el estado cero) hayan sido generados. Un polinomio primitivo posee dos propiedades fundamentales:

1. Irreducibilidad: un polinomio es irreducible si no puede factorizarse en el campo $\mathbb{F}_2$ (o más generalmente en un campo $\mathbb{F}_q$) en productos de polinomios de menor grado.
2. Orden máximo (primitividad): si $P(x)$ es irreducible, se dice que es primitivo si genera una secuencia cíclica con el período máximo posible en el campo $\mathbb{F}_2$ (o $\mathbb{F}_q$).

Los LFSRs presentan una serie de limitaciones y vulnerabilidades:

- Ataques por correlación: pueden ser vulnerables a ataques si la estructura del polinomio de retroalimentación es conocida. Los atacantes pueden reconstruir la secuencia pseudoaleatoria y la clave.
- No son criptográficamente seguros por sí solos: no son adecuados para cifrado seguro, ya que su naturaleza lineal los hace susceptibles a ataques como el ataque de Berlekamp-Massey, que puede deducir el estado interno a partir de una porción de la salida.
- Composición no lineal: para mejorar su seguridad, los sistemas modernos usan combinaciones de múltiples LFSRs y agregan funciones no lineales, como el Generador de Geffe o el Generador de Shrinking.

3.1.2. Registros de desplazamiento con retroalimentación no lineal

Los registros de desplazamiento con retroalimentación no lineal (*Nonlinear-feedback shift register*, NFSR) son como los LFSRs, pero con una función de retroalimentación no lineal en lugar de una lineal. Es decir, en lugar de usar sólo XORs bit a bit, la función de retroalimentación puede incluir operaciones AND y OR bit a bit, con sus ventajas y desventajas.

Una ventaja de añadir funciones de retroalimentación no lineal es que hacen que los NFSRs sean criptográficamente más robustos que los LFSRs porque los bits de salida dependen del estado secreto inicial de una manera compleja, de acuerdo con ecuaciones de tamaño exponencial.

Una desventaja de los NFSRs es que no hay una forma eficiente de determinar el período de un NFSR o simplemente saber si su período es máximo. Para un NFSR de n bits, necesitarías hacer cerca de 2^n pruebas para verificar que su período es máximo. Este cálculo es imposible para NFSRs de 80 bits o más.

Algoritmo	Uso legado	Uso futuro
HC-128	✓	✓
Salsa20/20	✓	✓
Chacha	✓	✓
SNOW 2.0	✓	✓
SNOW 3G	✓	✓
SOSEMANUK	✓	✓
Grain-218a	✓	✓
Grain	✓	✗
Mickey 2.0	✓	✗
Trivium	✓	✗
Rabbit	✓	✗
A5/1	✗	✗
A5/2	✗	✗
E0	✗	✗
RC4	✗	✗

Cuadro 3.1: Seguridad de los cifradores en flujo (fuente: Algorithms, Key Size and Protocols Report (2018).

Afortunadamente, hay un truco para usar un NFSR sin preocuparse por los períodos cortos: se pueden combinar los LFSRs y los NFSRs para obtener tanto el período máximo garantizado como la fuerza criptográfica extra y así es como funciona por ejemplo Grain-128a.

Tanto en los LFSRs como en los NFSRs, resulta evidente que la seguridad del cifrado en flujo depende enteramente del interior del generador de la secuencia cifrante.

3.1.3. Ejemplos de cifradores en flujo

Los cifradores en flujo se utilizan a menudo por su velocidad y simplicidad de implementación en hardware y en aplicaciones en las que el texto claro llega en cantidades de longitud desconocida, como una conexión inalámbrica segura o una llamada de teléfono. Se utilizan ampliamente en los protocolos y productos de seguridad actuales. Algunos ejemplos ampliamente usados son:

- A5/1 es una familia de cifradores en flujo utilizados para cifrar comunicaciones GSM. Es el estándar europeo para los teléfonos móviles digitales. Se utiliza para cifrar el enlace del teléfono a la estación base. El estándar A5/1 fue creado en 1987, pero sólo se publicó a finales de los años 90 después de que cayera ante el *reversing*. Los ataques aparecieron a principios de la década de 2000 y A5/1 fue roto finalmente de una manera que permite el descifrado real (no solo teórico) de las comunicaciones cifradas.

- PKZIP es el algoritmo de cifrado incorporado en el programa de compresión de datos PKZIP. Es un cifrado en flujo que cifra los datos byte a byte. Se han publicado varios trabajos con ataques parciales o totales al algoritmo.

- RC4 es un cifrado en flujo de tamaño variable desarrollado en 1987 por Ron Rivest para RSA Data Security, Inc. RC4 fue usado en docenas de productos comerciales de criptografía, en el protocolo de cifrado WEP en redes WiFi, en el protocolo de cifrado de tráfico web SSL/TLS, etc. Hoy en día ya no se utiliza debido a muchos problemas de seguridad descubiertos a lo largo de los años. Ha sido reemplazado por RC6.

- Content Scramble System (CSS) es un sistema utilizado para proteger las películas en discos DVD. Utiliza un cifrado en flujo, llamado CSS, para cifrar el contenido de las películas. El CSS fue diseñado en los años 80 cuando el cifrado exportable se restringía a claves de 40 bits. Como resultado, CSS cifra las películas usando una clave secreta de 40 bits. Además, el cifrado en flujo CSS es particularmente débil y puede ser roto en mucho menos tiempo que una búsqueda exhaustiva en las 2^{40} claves.

- Salsa20 y ChaCha son dos cifrados en flujo diseñados por Daniel J. Bernstein para su implementación eficiente en software. ChaCha20 fue seleccionado por Google para reemplazar a RC4 en TLS. También se utiliza en protocolos de comunicación como OpenSSH, IKE, IPsec y algunas VPNs. También utiliza ChaCha20 el cifrado de Android Adiantum para

ejecutarse en teléfonos y dispositivos inteligentes de todo tipo (televisores, relojes, dispositivos médicos, etc.) que no tienen el hardware especializado para utilizar los métodos actuales para cifrar con AES datos almacenados localmente de manera eficiente.

- Trivium, diseñado como parte del proyecto eSTREAM, es un cifrador de flujo eficiente basado en registros de desplazamiento de retroalimentación no lineales (NLFSR) que genera claves pseudoaleatorias a partir de una clave y un vector de inicialización, siendo seleccionado en el perfil 2 de eSTREAM por su excelente desempeño en hardware restringido y su diseño compacto y seguro..
- Resulta muy ilustrativo visitar el sitio web de eSTREAM, donde se encuentra el ECRYPT Stream Cipher Project, para promover el diseño de cifrados en flujo eficientes y compactos, adecuados para una amplia adopción.

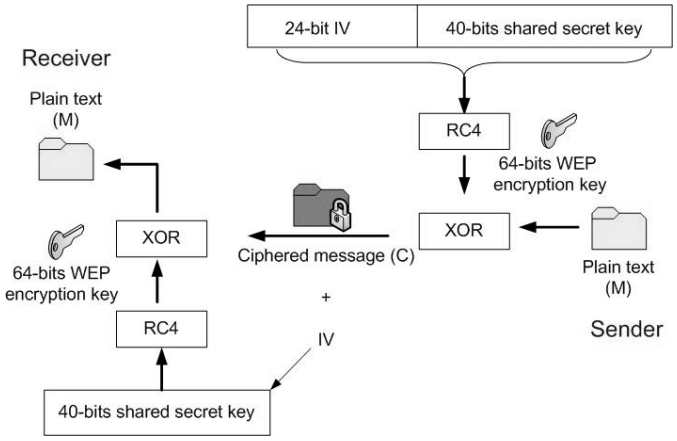


Figura 3.5: Esquema del protocolo WEP utilizado para controlar el acceso a la WLAN y para cifrar la información confidencial (fuente: IEEE 802.11 Wireless LAN Security Overview).

PRÁCTICA 3.1: XOReando con CrypTool

1. Abre con CrypTool un archivo cualquiera de menos de 1.024 bytes.
2. En `Procedimientos Indiv. >Herramientas >Generar números aleatorios`, genera una secuencia aleatoria del mismo número de bytes que el tamaño del archivo.
3. Cifra el archivo con la secuencia aleatoria, usando `Cifrar / descifrar >Simétrico (clásico) >XOR`.

Repite el proceso anterior para distintos tipos de archivo.

No hará falta que te diga que si usas algoritmos de cifrado en flujo reutilizando la misma clave, será trivial recuperar la secuencia cifrante (no la clave). Por ejemplo, TorrentLocker o DirCrypt cometieron este fallo de principiante. En la práctica, teniendo acceso a un único archivo sin cifrar y su correspondiente archivo cifrado de más de 2 MB, se podía recuperar la secuencia cifrante mediante un ataque trivial de texto claro conocido (KPA) y, con ella, descifrar todos los demás archivos cifrados.

PRÁCTICA 3.2: Recuperando archivos cifrados con TorrentLocker

Los archivos de tu disco han sido cifrados con TorrentLocker, usando la misma secuencia cifrante para todos ellos: file01.enc, file02.enc, ... Por fortuna, conservas una copia de seguridad de uno de los archivos cifrados: file01.txt. ¿Podrías descifrar file02.enc? Tienes los tres archivos en la carpeta de ejercicios.

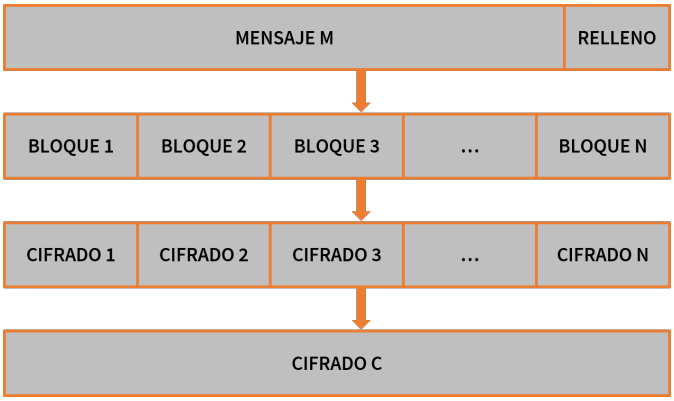


Figura 3.6: Los cifradores en bloque operan sobre grupos de bits de longitud fija, llamados *bloques*.

3.2. Cifrado en bloque

Acabas de ver cómo los cifradores en flujo cifran el texto claro bit a bit o carácter a carácter.

En cambio, en el cifrado en bloque, el texto claro se divide en bloques de igual longitud para ser cifrados bloque a bloque. Típicamente, se utiliza un tamaño de bloque de 64 o 128 bits.

Normalmente, la longitud del texto claro original no será un múltiplo exacto del tamaño del bloque, lo que exige recurrir a esquemas de relleno: los bits restantes hasta completar el tamaño de bloque se rellenan (*padding*) con bits extra siguiendo algún tipo de convención (v. Fig. 3.6).

Existen diferentes esquemas para rellenar el último bloque, como el PKCS#7, en el que el relleno se hace en bytes (grupos de 8 bits). El valor de cada byte añadido es el número de bytes que se añaden hasta completar el tamaño de bloque, es decir, se añaden n bytes, cada uno de valor n.

En función de cuántos bytes queden por completar, el relleno será uno de:

```
01
02 02
03 03 03
04 04 04 04
...
```

¿Y si la longitud del texto es múltiplo del tamaño de bloque? Entonces, no se rellena, ¿verdad? Pues no. En ese caso, se añade un bloque de relleno completo, con sus 16 bytes establecidos al valor `0x10`. Para evitar incertidumbre, siempre hay relleno.

El cifrado en bloque es una función que asigna bloques de texto claro de n bits a bloques de texto cifrado de n bits; donde n representa la *longitud de bloque*. Puede verse como un simple cifrado de sustitución con un gran tamaño de caracteres. La función de cifrado se parametriza con una clave K de k bits, tomando valores de un subconjunto $\mathcal{K}$ (el espacio de claves). En general, se asume que la clave se elige aleatoriamente. Para permitir un descifrado único, la función de cifrado debe ser uno a uno, es decir, invertible.

Al igual que con el cifrado en flujo, en el cifrado en bloque ambos usuarios comparten una clave de cifrado simétrica. Utilizando algunos de los modos de operación explicados en la Sección 3.2.3, se puede utilizar un cifrado en bloque para conseguir el mismo efecto que un cifrado en flujo.

3.2.1. Ejemplos de cifradores en bloque

Data Encryption Standard (DES) fue desarrollado en la década de 1970. Aunque su corta longitud de clave de 56 bits lo hace demasiado inseguro para las aplicaciones actuales, ha sido muy influyente en el desarrollo de la criptografía moderna. **El DES no es débil por sí mismo, sino por su clave corta**, de tan solo 56 bits. Esta limitación condujo a utilizar variantes del algoritmo con claves más largas, como por ejemplo Triple DES (o 3DES), con una clave de 168 bits.

En 2001, DES fue sustituido por el algoritmo seleccionado como Advanced Encryption Standard (AES), también conocido por su nombre original Rijndael, con un tamaño de bloque de 128 bits y con tres longitudes de clave diferentes: 128, 192 y 256 bits. Estos tamaños podrían cambiar en breve a tenor de una reciente publicación del NIST: PRE-DRAFT Call for Comments: NIST Proposes to Standardize a Wider Variant of AES

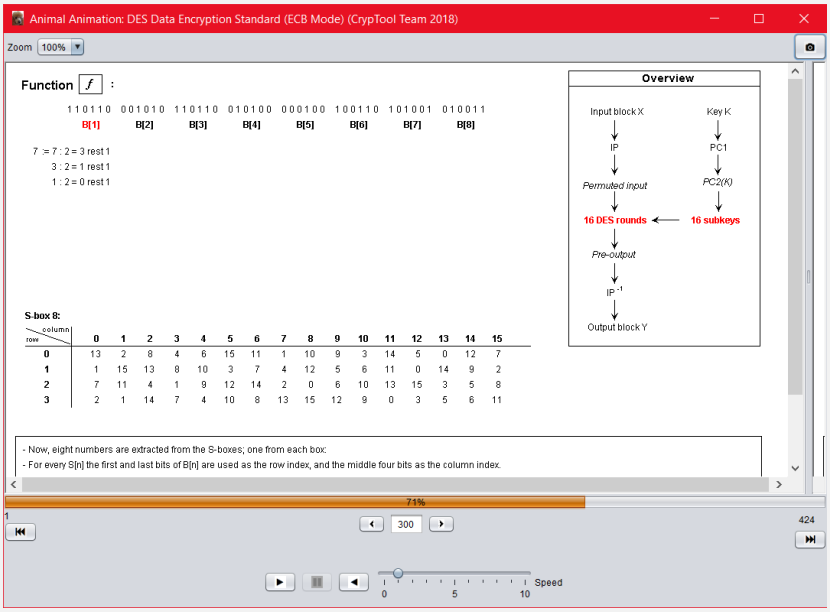
Algoritmo	Uso legado	Uso futuro
AES	✓	✓
Camellia	✓	✓
Serpent	✓	✓
3DES	✓	✗
Kasumi	✓	✗
Blowfish	✓	✗
DES	✗	✗

Cuadro 3.2: Seguridad de los cifradores en bloque (fuente: Algorithms, Key Size and Protocols Report (2018)).

PRÁCTICA 3.3: Viendo las tripas de DES y AES

En este curso no entrarás a estudiar los detalles del funcionamiento de DES ni AES. No obstante, te recomiendo leer en la wikipedia cómo funcionan sus componentes y, posteriormente, para aterrizar con firmeza todos los conceptos, ver sendas visualizaciones del comportamiento interno de DES y AES, incluidas en la estupenda herramienta CrypTool.

1. Para DES: `Procedimientos Indiv. >Visualización de Algoritmos >DES`.
2. Para AES: `Procedimientos Indiv. >Visualización de Algoritmos >AES`, o bien, AES Animation.



Además de estos dos algoritmos, existen cientos de ellos. Entre los que se consideran seguros hoy y en el futuro previsible destacan: Camellia, Serpent, etc.

Por si no había quedado claro, estos algoritmos se caracterizan por:

- Ser muy rápidos, pudiendo alcanzar velocidades de hasta 1 GB/s, dependiendo de la implementación en software y del procesador del dispositivo.
- Requerir claves cortas, por ejemplo, de 128 o 256 bits.

Así que puedes encontrar el cifrado simétrico en aplicaciones que requieran alta velocidad de cifrado y claves cortas como, por ejemplo:

- Herramientas de comunicación, como Whatsapp, Apple Messages o Zoom.
- Herramientas de archivo y compresión.
- Herramientas de cifrado de archivos.
- Encryption File System (EFS) en Microsoft Windows proporciona cifrado a nivel de sistema de archivos.

- Cifrado de discos y particiones, como BitLocker en Windows o FileVault en Mac OS X.
- Seguridad para las comunicaciones en redes de área local, como en el estándar IEEE 802.11i.
- El nuevo estándar de seguridad WiFi, WPA3, utiliza AES con 256 bits de clave.
- Protocolos de comunicaciones seguras, como el TLS utilizado por ejemplo en páginas web seguras con HTTPS.
- Ransomware: obviamente, para cifrar lo archivos de un disco duro para exigir su rescate, nada como la criptografía simétrica.

PRÁCTICA 3.4: OpenSSL responde: ¿cómo de rápido es rápido?

OpenSSL realiza un test de velocidad de todos sus algoritmos. Por ejemplo, para el cifrador en flujo RC4 arroja los siguientes resultados:

```
$ openssl speed rc4
Doing rc4 for 3s on 16 size blocks: 125816026 rc4's in 3.02s
Doing rc4 for 3s on 64 size blocks: 35706043 rc4's in 3.02s
Doing rc4 for 3s on 256 size blocks: 7028515 rc4's in 3.00s
Doing rc4 for 3s on 1024 size blocks: 1607018 rc4's in 3.00s
Doing rc4 for 3s on 8192 size blocks: 177045 rc4's in 3.00s
Doing rc4 for 3s on 16384 size blocks: 90593 rc4's in 3.00s
OpenSSL 1.1.1g 21 Apr 2020
...
The 'numbers' are in 1000s of bytes per second processed.
type 16 bytes 64 bytes 256 bytes 1024 bytes 8192 bytes 16384 bytes
rc4 667542.02k 757782.14k 599766.61k 548528.81k 483450.88k 494758.57k
```

Puedes probar con otros algoritmos y así comparas.

3.2.2. Los algoritmos de cifrado envejecen... y mueren

¿Cómo es que un algoritmo envejece y deja de ser seguro con los años? ¿Le pasará a AES con el tiempo lo mismo que le pasó a DES? Existen varios avances que debilitan a todo algoritmo criptográfico con el paso del tiempo:

- **Avances en computación convencional**: todo algoritmo de cifrado depende de una clave. Un ataque de búsqueda exhaustiva prueba todos sus posibles valores hasta dar con el correcto. Para llevarlo a cabo solo hace falta potencia bruta de cómputo, no fineza matemática. De ahí que se llame también ataque de fuerza bruta. No importa lo bien diseñado que esté un algoritmo ni lo inteligentes que sean sus operaciones, el avance inexorable de la capacidad de cómputo va erosionando lentamente su seguridad, hasta que finalmente puede realizarse la búsqueda exhaustiva en tiempo y coste razonables. Por mucho que aumentes la longitud de la clave, ¿cómo preservar la confidencialidad ante un atacante armado con recursos computacionales infinitos? Salvo con Vernam, no parece posible con ningún otro algoritmo.
- **Avances en criptoanálisis**: todos los algoritmos criptográficos de dominio público se someten al escrutinio despiadado de toda la comunidad científica. Con los años, terminan descubriéndose debilidades que disminuyen la complejidad de los ataques. Recordemos al pobre César, considerado seguro durante 15 siglos, o a Vigenère, seguro durante 300 años. Si la clave tiene una longitud n, cualquier vía de ataque que reduzca el número necesario de pruebas por debajo de 2^n supone un avance. Cuando la reducción es tan drástica que con la potencia de cómputo del momento un ataque se vuelve factible, entonces el algoritmo se considera roto. Un algoritmo bien diseñado podría no tener ningún resquicio. Ejemplos de algoritmos de cifrado rotos son RC2, RC4, A5/1, A5/2, E0, por citar algunos.
- **Avances en matemáticas**: las matemáticas no son inmutables. Se producen avances que permiten resolver problemas antes insolubles. Muchos algoritmos criptográficos se basan en la asunción de que $P \neq NP$: hasta ahora nadie ha podido demostrar que si resulta fácil de comprobar que la solución de un problema es correcta, entonces debe ser igualmente fácil de resolver. No se descarta que se produzcan avances en matemáticas que permitan

resolver en tiempo polinómico problemas como el del viajante, el de la factorización, el del logaritmo discreto o el de la búsqueda de los vectores más cortos en una celosía.

- **Avances en computación no convencional**: problemas considerados hoy NP podrían ser de tipo P bajo paradigmas computacionales radicalmente diferentes del Silicio. El ejemplo más claro surgió con la computación cuántica y sus algoritmos de Shor, para factorizar números; y de Grover, para acelerar búsquedas en listas. Su consecuencia más devastadora para la criptografía es que los ordenadores cuánticos arrasarán con los algoritmos de cifrado de clave pública RSA, DSA, ECDSA, Diffie-Hellman y ECC, los estándares de facto actuales. Existen por supuesto otras formas de computación no convencional, como la computación biológica basada en ADN y su capacidad de paralelización masiva, o la basada en nanohilos, o en nanotubos de carbono, o en moléculas orgánicas, o en óptica, o en micro/nanofluidos, o incluso en amebas caóticas, y muchas más que se están investigando. ¿Aparecerán entre ellas algoritmos eficientes para resolver otros problemas matemáticos centrales en tiempo polinómico? Nadie lo sabe aún. El futuro está lleno de posibilidades.

Todo algoritmo conocido ha sucumbido o se cree que sucumbirá ante uno o más de los avances anteriores. Y ninguno resistiría a una infinita potencia de cómputo. Con la excepción del cifrado de Vernam, claro, el algoritmo eterno que nunca se rompe..., pero que resulta impráctico.

3.2.3. Modos de operación del cifrado en bloque

En la práctica, normalmente desearás cifrar más de un solo bloque de 8 o 16 bytes de texto claro, por ejemplo, al cifrar un correo electrónico o un archivo. Existen varias formas de cifrar textos claros de longitud arbitraria con un cifrado en bloque, entre ellas:

- Modo Electronic Code Book (ECB)
- Modo Cipher Block Chaining (CBC)
- Modo Cipher Feedback (CFB)
- Modo Output Feedback (OFB)
- Modo Contador (CTR)

Los tres últimos modos permiten además utilizar el cifrado en bloque como bloque de construcción para el cifrado en flujo. En las siguientes descripciones, E_K representa la operación de cifrado con una clave K y E_K^{-1} representa el descifrado con la misma clave. Como siempre, se debe satisfacer el *requisito de corrección*: $E_K^{-1}(E_K(m)) = m$.

3.2.3.1. Modo Electronic Code Book (ECB)

> Everybody knows ECB mode is bad because we can see the penguin.
>
> *Marsh Ray*
> *Cryptographer at Microsoft*

El modo Electronic Code Book (ECB) es la forma más sencilla de cifrar un mensaje: divides el mensaje en t bloques de n bits $m_1, m_2, \ldots, m_t$ y cifras cada uno por separado, como se puede ver en las figuras 3.7 y 3.8.

1. Cifrado: para $1 \leq i \leq t, c_i = E_K(m_i)$, de manera que $c = c_1||c_2||\cdots||c_t$.
2. Descifrado: para $1 \leq i \leq t, m_i = E_K^{-1}(c_i)$, de manera que $m = m_1||m_2||\cdots||m_t$.

Como consecuencia, bloques de texto claro idénticos (cifrados con la misma clave) dan como resultado textos cifrados idénticos, es decir, no hay dependencias de encadenamiento: cada bloque se cifra independientemente de otros bloques. El reordenamiento de los bloques de texto cifrado da lugar a un reordenamiento correspondiente de los bloques de texto claro.

Dado que los bloques de texto cifrado son independientes, la sustitución maliciosa de los bloques ECB (por ejemplo, la inserción de un bloque frecuente) no afecta al descifrado de los bloques adyacentes. Además, los cifrados en modo ECB no ocultan los patrones de datos: los bloques de texto cifrado idénticos implican bloques de texto claro idénticos. En otras palabras, como puedes comprobar en la Fig. 3.9: ¡se ve el pingüino!

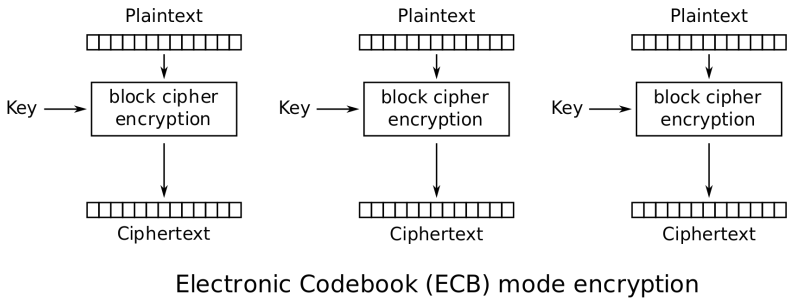


Figura 3.7: Modo Electronic Code Book (ECB): cifrado (fuente: Wikipedia).

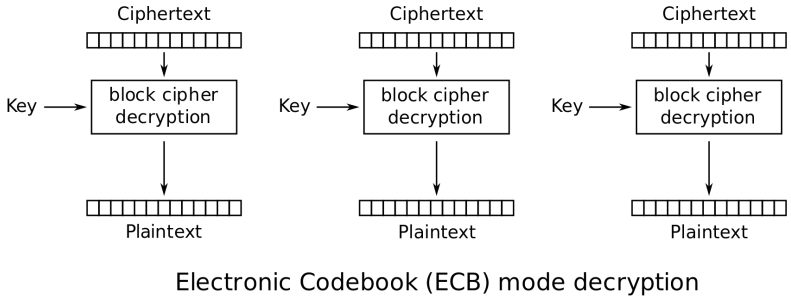


Figura 3.8: Modo Electronic Code Book (ECB): descifrado (fuente: Wikipedia).

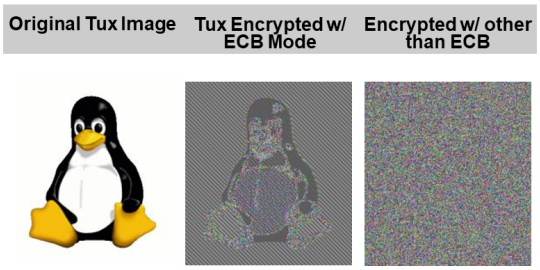


Figura 3.9: El modo ECB puede filtrar patrones de datos de texto claro en el texto cifrado utilizado para cifrar una imagen de mapa de bits que utiliza grandes áreas de color uniforme.

PRÁCTICA 3.5: Cifrando imágenes en modo ECB con OpenSSL

Vas a cifrar la famosa imagen del pingüino usando DES en modo ECB, a ver qué pasa.

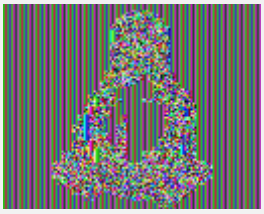
1. Abre el archivo `LinuxPenguin.bmp` con CrypTool, copia (Sin codificar) la información de la imagen a una nueva ventana, descartando la cabecera (recuerda que las cabeceras BMP de estas imágenes ocupan los primeros 54 bytes (`0x36`), y guárdala como un archivo, por ejemplo, `LinuxPenguin.raw`.

2. Cifra el archivo mediante OpenSSL con DES en modo ECB y 128 bits de clave, derivada a partir de la contraseña que quieras:

   ```
   $ openssl aes-128-ecb -e -in LinuxPenguin.raw -out LinuxPenguin.enc
   -nosalt -pbkdf2 -k gonzalo.
   ```

3. Abre el archivo `LinuxPenguin.enc` con CrypTool, copia (Codificación hexadecimal) el contenido completo del archivo y pégalo (Decodificación hexadecimal) en la parte anteriormente seleccionada del archivo `LinuxPenguin.bmp`.

4. Guárdalo y ábrelo con Paint. A mí me ha salido la siguiente imagen. ¿Y a ti? ¿También se te ve el pingüino?



Repite el ejercicio cifrando con DES en modo ECB mediante CrypTool, sin recurrir a OpenSSL. Usa cualquier clave aleatoria, da igual. Y, si te animas, prográmalo todo en tu lenguaje favorito.

El modo ECB también puede hacer que los protocolos sin protección de integridad sean aún más susceptibles de ataques de repetición, ya que cada bloque se descifra exactamente de la misma manera. Por esta razón, el modo ECB no se recomienda para mensajes más largos que un bloque o si las claves se reutilizan para más de un solo mensaje de un único bloque. La seguridad puede mejorarse un poco mediante la inclusión de bits de relleno aleatorios en cada bloque.

En resumen, los principales problemas de seguridad de ECB y que desaconsejan su uso son:

1. **Fuga de información por colisiones de bloque**: si dos bloques de texto claro son iguales (digamos $m_i = m_j$), entonces los dos bloques de texto cifrado correspondientes son iguales ($c_i = c_j$). La relación de igualdad es una información que se filtra. Esto no sería un problema si los bloques de texto claro fueran totalmente aleatorios, ya que la probabilidad de igualdad sería razonablemente baja. Sin embargo, los textos claros habituales presentan mucha redundancia en la práctica, por lo que las igualdades son frecuentes (recuerda, ¡se ve el pingüino!).

2. **Cuestiones de integridad**: aunque se supone que el cifrado protege la confidencialidad, pero no la integridad, un atacante puede interceptar el texto cifrado y permutar dos bloques. El destinatario legítimo del texto cifrado modificado descifrará el mensaje correctamente y obtendrá dos bloques de texto claro permutados. Del mismo modo, un bloque puede ser eliminado, sustituido por otro, etc. De este modo, el texto claro es fácilmente *maleable* por un adversario.

Quiero recalcar que estos problemas no tienen que ver con el algoritmo en concreto (DES, AES, ...), sino con el modo de encadenamiento. El algoritmo más seguro del mundo en modo ECB seguirá siendo inseguro. Por ejemplo, el ransomware LooCipher usa AES-ECB.

PRÁCTICA 3.6: Maleando el texto cifrado en modo ECB con CrypTool

Considera el siguiente mensaje m:
`Gon gana 25000 E al mes.Eva gana 15.000 E al mes.`

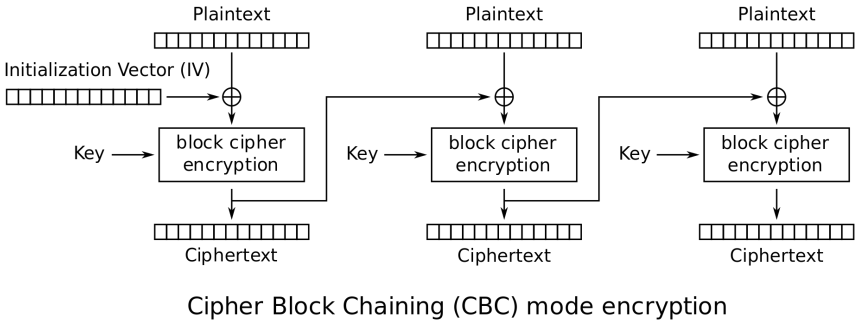
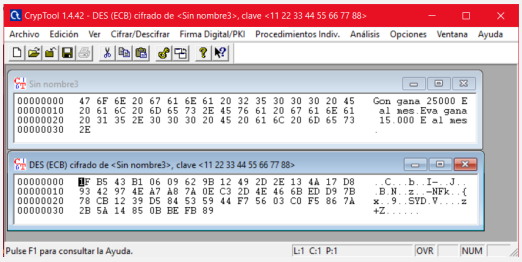


Figura 3.10: Modo Cipher Block Chaining (CBC): cifrado (fuente: Wikipedia).

En DES cada bloque tiene 64 bits de longitud, es decir, cada bloque está formado por 8 bytes. Dado que cada carácter en el texto claro está representado por un byte de ASCII, cada grupo de 8 caracteres se tratará como un único bloque. Vas a malear el mensaje cifrado para que al ser descifrado arroje un resultado sorprendente.

1. Utilizando CrypTool, cifra el mensaje anterior con DES en modo ECB utilizando la clave `K = 11 22 33 44 55 66 77 88`, mediante `Cifrar / Descifrar >Simétrico (moderno) >DES (ECB)`.
2. El texto cifrado resultante será de la forma $c = c_1||c_2|||c_3||\ldots||c_n$. Intercambia los bloques de texto cifrado c_1 y c_4 en la secuencia de bloques de texto cifrado anterior para obtener la siguiente secuencia de bloques de texto cifrado: $c_4||c_2||c_3||c_1||c_5||\ldots||c_n$.
3. Descifra el texto cifrado resultante usando la misma clave. ¿Qué mensaje obtienes? ¿Has conseguido cambiarme el sueldo?



3.2.3.2. Modo Cipher Block Chaining (CBC)

El modo de funcionamiento CBC implica el uso de un vector de inicialización de n bits, denominado IV.

1. Cifrado: $c_0 = \text{IV}$, para $1 \leq i \leq t, c_i = E_K(c_{i-1} \oplus m_i)$.
2. Descifrado: $c_0 = \text{IV}$, para $1 \leq i \leq t, m_i = c_{i-1} \oplus E_K^{-1}(c_i)$.

El modo de operación CBC exhibe las siguientes propiedades:

- **Textos claros idénticos**: cuando se cifran bloques idénticos de texto claro bajo la misma clave y mismo IV, resultan bloques de texto cifrado idénticos. Cambiar el IV, la clave o el primer bloque de texto claro (por ejemplo, usando un contador o un campo aleatorio) da como resultado un texto cifrado diferente. ¡Ya no se ve el pingüino!



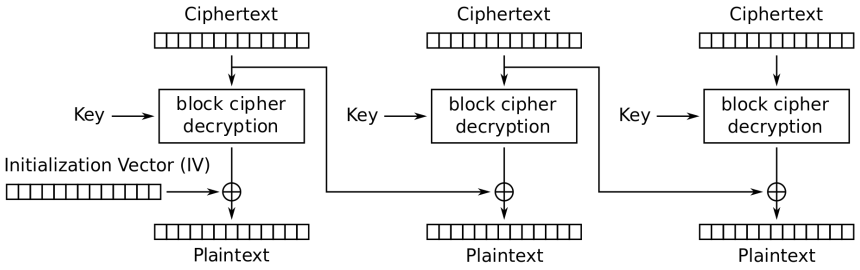


Figura 3.11: Modo Cipher Block Chaining (CBC): descifrado (fuente: Wikipedia).

- **Dependencias de encadenamiento**: el mecanismo de encadenamiento hace que el texto cifrado c_i dependa de m_i y de todos los bloques de texto claro precedentes. Un cambio de un solo bit en el IV o en el texto claro afecta a todos los bloques cifrados posteriores. Sin embargo, toda la dependencia de los bloques precedentes está contenida en el valor del bloque de texto cifrado anterior. Por consiguiente, la reorganización del orden de los bloques de texto cifrado afecta al descifrado. Para descifrar correctamente un bloque de texto cifrado se requiere un bloque de texto cifrado precedente correcto.
- **Propagación de errores**: un error de un solo bit en el bloque de texto cifrado c_i afecta al descifrado de los bloques c_i y c_{i+1}, ya que m_i depende de c_i y c_{i-1}. El bloque m_i recuperado de c_i es típicamente totalmente aleatorio (50 % de error), mientras que el texto claro recuperado m_{i+1} tiene errores de bits precisamente donde c_i lo hizo. Por lo tanto, un adversario puede provocar cambios de bits predecibles en m_{i+1} alterando los bits correspondientes de c_i.
- **Recuperación de errores**: el modo CBC es *autosincronizante*[1] o de *autoclave*, en el sentido de que si se produce un error, incluida la pérdida de uno o más bloques enteros, en el bloque c_i, pero no en c_{i+1}, c_{i+2} se descifra correctamente de m_{i+2} en adelante.
- **Paralelismo**: con CBC, el descifrado puede ser mucho más rápido que el cifrado debido al paralelismo. Mientras que el cifrado de un nuevo bloque, m_i, necesita esperar al bloque anterior, c_{i-1}, el descifrado de un bloque se calcula $m_i = c_{i-1} \oplus E_K^{-1}(c_i)$, donde no hay necesidad del bloque anterior de texto claro, m_{i-1}. Esto significa que todos los bloques pueden ser descifrados en paralelo simultáneamente, siempre y cuando se conozca también el bloque de texto cifrado anterior, lo que suele suceder.

PRÁCTICA 3.7: Maleando el texto cifrado en CBC con CrypTool y OpenSSL

1. Repite el mismo ejercicio de la práctica 3.6, pero ahora usa DES en modo CBC. Contrasta el texto descifrado con el obtenido cuando usaste ECB. ¿Qué ha cambiado?
2. Desgraciadamente, el CBC se utiliza a menudo con un IV constante en lugar de uno aleatorio, lo que expone a textos claros idénticos y textos claros que empiezan con bloques idénticos. Por ejemplo, digamos que el texto claro de dos bloques m_1m_2 está cifrado en modo CBC como c_1c_2. Si otro texto claro m_1m_2' se cifra con el mismo IV, donde m_2' es un bloque distinto de m_2, entonces el texto cifrado se verá como c_1c_2', con c_2' diferente de c_2, pero con el mismo primer bloque c_1. Así, un atacante puede adivinar que el primer bloque es el mismo para los dos textos claros, aunque sólo vea los textos cifrados.

   a) Genera una clave y un IV de 128 bits para AES.

   ```
   $ openssl rand -hex 16
   69deadf1fa46df699054f8909b5ba8d4
   ```

[1] Si se pierde parte del texto cifrado (por ejemplo, debido a errores de transmisión), el receptor perderá sólo una parte del mensaje original y debería poder seguir descifrando correctamente el resto de los bloques después de procesar cierta cantidad de datos de entrada.

```
$ openssl rand -hex 16
66ee3959c8eda2e479843e4ec85c9a06
```

b) Cifra ambos archivos, `AA.txt` y `AB.txt` (ambos en la carpeta de prácticas), con AES de 128 bits en modo CBC, con la misma clave y mismo IV.

```
$ openssl aes-128-cbc -e -in AA.txt -out
AA.enc -iv 69deadf1fa46df699054f8909b5ba8d4 -K
66ee3959c8eda2e479843e4ec85c9a06 -nopad
$ openssl aes-128-cbc -e -in AB.txt -out
AB.enc -iv 69deadf1fa46df699054f8909b5ba8d4 -K
66ee3959c8eda2e479843e4ec85c9a06 -nopad
```

c) Haz un volcado hexadecimal de ambos archivos y comprobarás cómo los primeros 16 bytes son idénticos.

```
$ hexdump -C AA.enc
000000 36 e8 39 04 94 8b b6 a9 5a 65 f9 b6 da 52 11 97
000010 f3 4d 09 eb 1f a4 bd eb 75 68 b9 0b 82 87 13 e1
$ hexdump AB.enc
000000 36 e8 39 04 94 8b b6 a9 5a 65 f9 b6 da 52 11 97
000010 26 e9 3e 3f 4d cf 15 b3 b7 62 55 08 81 31 c1 60
```

De ahí la importancia de cambiar siempre el IV, especialmente cuando se debe usar la misma clave de cifrado una y otra vez.

El vector de inicialización no solo debe cambiar, sino que debe ser impredecible. Fíjate qué podría ocurrir en caso contrario.

Supongamos que un texto m_1 se cifra con un vector IV_1:

$$c_1 = E_K(m_1 \oplus IV_1)$$

Mientras que otro texto m_2 se cifra con otro vector IV_2:

$$c_2 = E_K(m_2 \oplus IV_2)$$

Si conoces cuál será el siguiente vector de inicialización, IV_3, puedes construir un texto claro $m_3 = m_1 \oplus IV_1 \oplus IV_3$ cuyo cifrado c_3 será idéntico a c_1:

$$c_3 = E_K(m_1 \oplus IV_1 \oplus IV_3 \oplus IV_3) = E_K(m_1 \oplus IV_1) = c_1$$

En 2022, varios investigadores de la Universidad de Tel Aviv publicaron un documento explicando cómo millones de *smartphones* de Samsung (entre ellos los Galaxy S8, S9, S10, S20 y S21) eran vulnerables a un ataque de reutilización de vectores iniciales proporcionados al AES en el Modo Contador de Galois (GCM), que se utilizaba para cifrar claves criptográficas en el almacén de claves del *smartphone*.

Sin embargo, no hay problema en revelar el IV a posteriori, **después** de haber cifrado el texto. Hasta cierto punto, ya que si se modifica el IV que se envía al receptor del mensaje cifrado, cambiará el texto descifrado, como comprobarás en la siguiente práctica. De ahí la necesidad de utilizar cifrado autenticado, para garantizar la integridad del IV y del texto cifrado. Es importante recalcar que este ataque solo funciona con el primer bloque del texto cifrado.

PRÁCTICA 3.8: Manipulando el IV al descifrar en CBC con OpenSSL

1. Crea un IV y una clave secreta aleatorios de 128 bits.

```
$ openssl rand -hex 16
487a9473d1e38f5e4ab701cb2ef1a5f4
$ openssl rand -hex 16
799a5ee49089ebb026cba11e78200668
```

2. Cifra el archivo `mensaje.txt`, cuyo contenido es "Mandame 4.000 E.".

```
$ openssl aes-128-cbc -e -in mensaje.txt -out mensaje.enc -iv
487a9473d1e38f5e4ab701cb2ef1a5f4 -K 799a5ee49089ebb026cba11e78200668
```

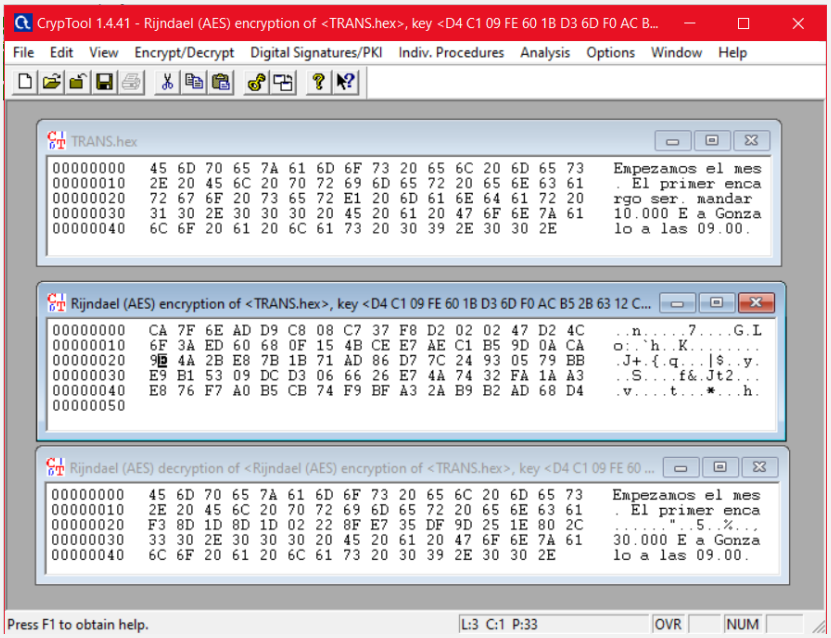
3. Descifra el texto cifrado usando el siguiente valor modificado para el IV:

```
$ openssl aes-128-cbc -d -in mensaje.enc -iv
487a9473d1e38f5e4bb701cb2ef1a5f4 -K 799a5ee49089ebb026cba11e78200668
```

¿Cómo ha cambiado el texto claro recuperado?

PRÁCTICA 3.9: Maleando el texto cifrado en CBC con Cryptool

1. Crea un archivo de texto en Cryptool.
2. Cífralo en modo CBC con una clave aleatoria de 128 bits.
3. Manipula un bit del texto cifrado de un bloque.
4. Descifra el criptograma usando la misma clave secreta.



Comprobarás cómo el bloque c_i manipulado devuelve un bloque claro m_i correspondiente totalmente alterado, pero en el bloque claro siguiente m_{i+1} solo cambia el mismo byte correspondiente al alterado en c_i, lo que permite manipular a voluntad ese bloque. Dependiendo de la aplicación, este maleado podría tener consecuencias desastrosas. Te dejo que explores este reto de un CTF basado en este ataque. En mi ejemplo, he cambiado un solo bit del byte en la posición 33 del texto cifrado, con el resultado de que ha cambiado el byte en la posición 49 del texto claro. Ahora, en el mensaje original, el valor 10.000 ha pasado a ser 30.000. Este ataque se conoce como *bit flipping*.
Repite este ejercicio con tu propio mensaje y experimenta cambiando los valores de algunos bytes del texto cifrado para provocar cambios deliberados en el mensaje claro recuperado.

Además, CBC es susceptible a ataques de Oráculo de relleno, capaces de recuperar el texto claro completo.

3.2.3.3. Modo Cipher Feedback (CFB)

Mientras que el modo CBC procesa n bits de texto claro cada vez (usando un cifrado en bloque de n bits), algunas aplicaciones requieren el cifrado de unidades de texto claro de r bits para su transmisión sin demora, para algún valor fijo de $r < n$. A menudo $r = 1$ o $r = 8$. En este caso, puede utilizarse el modo CFB de la siguiente manera:

1. **Cifrado**: deseas cifrar un mensaje dividido en bloques de r bits: $m_1, \ldots, m_u$, $I_1 = \text{IV}$ (I_i es el valor de entrada de un registro de desplazamiento de n bits). Para $1 \leq i \leq u$,
   (a) $O_i = E_K(I_i)$ (calcula la salida del cifrado en bloque).

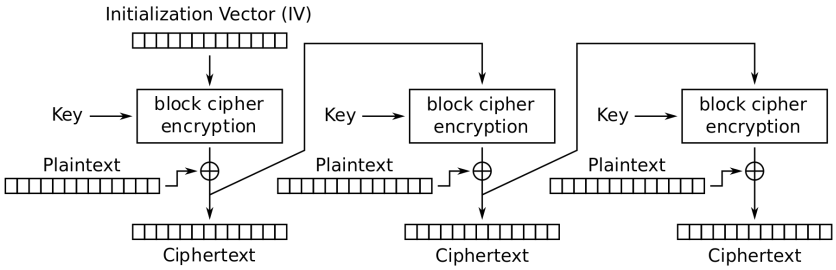


Figura 3.12: Modo Cipher Feedback (CFB): cifrado (fuente: Wikipedia).

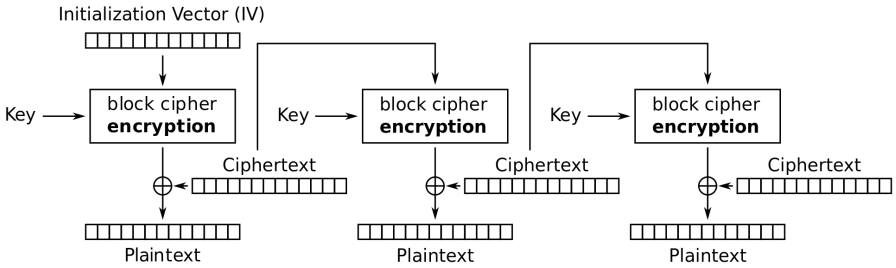


Figura 3.13: Modo Cipher Feedback (CFB): descifrado (fuente: Wikipedia).

(b) t_i = los r bits más a la izquierda de O_i (asumiendo que el bit más a la izquierda se identifica como el bit 1).

(c) $c_i = m_i \oplus t_i$ (transmite el bloque de texto cifrado de r bits, c_i).

(d) $I_{i+1} = 2^r \cdot I_i + c_j \mod 2^n$ (desplaza c_i al extremo derecho del registro de desplazamiento).

2. **Descifrado**: $I_1 = \text{IV}$, para $1 \leq i \leq u$, al recibir c_i:
$m_i = c_i \oplus t_i$, donde t_i, O_i y I_i se calculan como antes.

Por su lado, el modo de operación CFB exhibe las siguientes propiedades:

- **Textos claros idénticos**: al igual que con CBC, al cambiar el IV se obtiene que la misma entrada de texto claro se cifra a una salida diferente. No es necesario que el IV sea secreto, pero en algunas aplicaciones se puede desear que el IV sea impredecible.
- **Dependencias de encadenamiento**: al igual que el cifrado CBC, el mecanismo de encadenamiento hace que el bloque de texto cifrado c_i dependa tanto de m_i como de los bloques de texto claro precedentes; en consecuencia, el reordenamiento de los bloques de texto cifrado afecta al descifrado. Para descifrar sin errores un bloque de texto cifrado correcto, es necesario que los $\lceil n/r \rceil$ bloques de texto cifrado precedentes sean correctos (para que el registro de desplazamiento contenga el valor adecuado).
- **Propagación de errores**: uno o más errores de bits en un solo bloque de texto cifrado de r bits c_i afecta al descifrado de ese y de los siguientes $\lceil n/r \rceil$ bloques de texto cifrado, es decir, hasta que se procesen n bits de texto cifrado, después de lo cual el bloque de error c_i se ha desplazado completamente fuera del registro de desplazamiento. El texto claro recuperado m'_i diferirá de m_i precisamente en las posiciones de los bits c_i erróneos; los otros bloques de texto claro recuperados incorrectamente serán típicamente vectores aleatorios, es decir, tendrán un 50 % de bits erróneos. Así, un adversario puede provocar cambios de bits predecibles en m_i alterando los bits correspondientes de c_i.
- **Recuperación de errores**: el modo CFB se auto-sincroniza de forma similar al CBC, pero requiere $\lceil n/r \rceil$ bloques de texto cifrado para recuperarse.

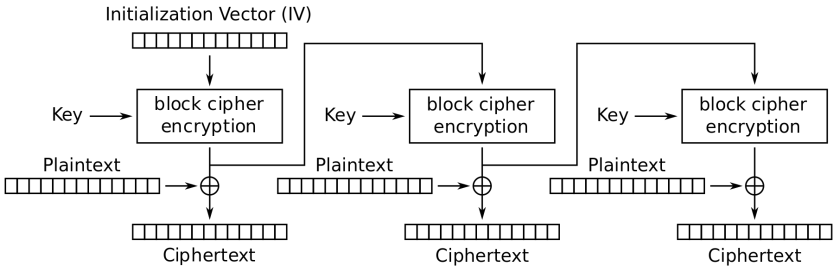


Figura 3.14: Modo Output Feedback (OFB): cifrado (fuente: Wikipedia).

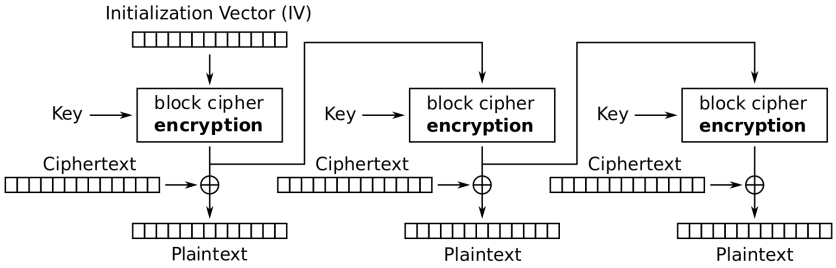


Figura 3.15: Modo Output Feedback (OFB): descifrado (fuente: Wikipedia).

- **Rendimiento**: para $r < n$, el rendimiento se reduce en un factor de n/r (vs. CBC) en la medida en que cada ejecución de E produce sólo r bits de salida de texto cifrado.

3.2.3.4. Modo Output Feedback (OFB)

El modo OFB puede utilizarse para aplicaciones en las que debe evitarse toda propagación de errores. Es similar al CFB y permite el cifrado de varios tamaños de bloque (caracteres), pero difiere en que la salida de la función de cifrado E sirve como retroalimentación, en lugar del texto cifrado.

1. **Cifrado**: el mensaje se divide en bloques de r bits: $m_1, \ldots, m_u$, y $I_1 = \text{IV}$ (I_i es el valor de entrada al registro de desplazamiento). Para $1 \leq i \leq u$, dado el bloque de texto claro m_i:
   a) $O_i = E_K(I_i)$ (calcula la salida del cifrado en bloque).
   b) $t_i =$ los r bits más a la izquierda de O_i (asumiendo que el bit más a la izquierda se identifica como el bit 1).
   c) $c_i = m_i \oplus t_i$ (transmite el bloque de texto cifrado de r bits, c_i).
   d) $I_{i+1} = O_i$ (actualiza la entrada al cifrado en bloque para el siguiente bloque).
2. **Descifrado**: $I_1 = \text{IV}$, para $1 \leq i \leq u$, al recibir c_i:
   $m_i = c_i \oplus t_i$, donde t_i, O_i y I_i se calculan como antes.

Estas son las propiedades más destacadas del modo de operación OFB:

1. **Textos claros idénticos**: al igual que con los modos CBC y CFB, al cambiar el IV el mismo texto se cifra a una salida diferente.
2. **Dependencias de encadenamiento**: la secuencia cifrante es independiente del texto claro. Esta característica hace que este modo de cifrado sea vulnerable a los ataques de texto claro conocido siempre que se reutilicen el IV y la clave, ya que permiten recuperar la secuencia cifrante (pero no lo clave ni el IV).

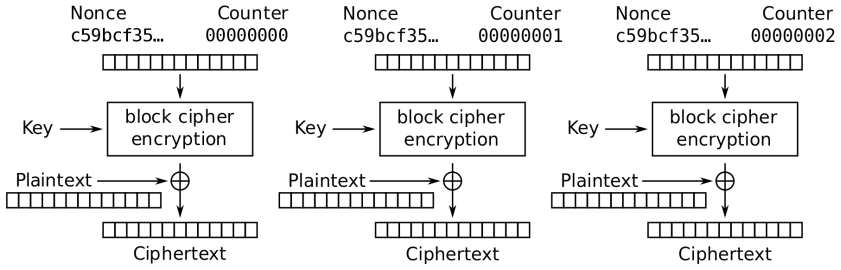


Figura 3.16: Counter mode (CTR): cifrado (fuente: Wikipedia).

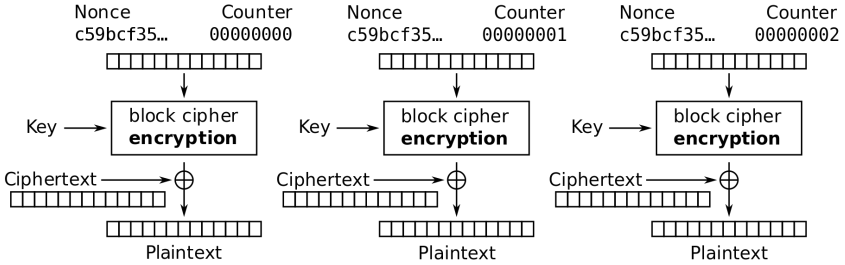


Figura 3.17: Counter mode (CTR): descifrado (fuente: Wikipedia).

3. **Propagación de errores**: uno o más errores de bits en cualquier carácter de texto cifrado c_i afecta al descifrado de sólo ese carácter, en la posición o posiciones precisas de los bits donde está el error en c_i, haciendo que el correspondiente bit o los bits de texto claro recuperados se complementen. Esta característica hace que el cifrado sea muy maleable, más que CBC.

4. **Recuperación de errores**: el modo OFB se recupera de los errores de bits de texto cifrado, pero no puede autosincronizarse tras la pérdida de bits de texto cifrado, lo que destruye la alineación de la secuencia descifrante (en cuyo caso se requiere una resincronización explícita).

5. **Rendimiento**: para $r < n$, el rendimiento se reduce como en el modo CFB.

6. **Paralelismo**: en todos los casos, como la secuencia cifrante es independiente del texto claro o del texto cifrado, puede precalcularse dadas la clave y el IV. Eso sí, recuerda no reutilizarlos nunca y hacerlos siempre aleatorios.

3.2.3.5. Counter mode (CTR)

El modo Contador (CTR) presenta muchas de las ventajas de ECB, pero sin sus desventajas. Al igual que en los modos OFB y CFB, la secuencia cifrante se calcula bloque a bloque. La entrada al cifrado en bloque es un contador que toma un valor diferente cada vez que el cifrado en bloque calcula un nuevo bloque de secuencia cifrante. La Fig. 3.16 muestra el principio.

1. **Cifrado**: para $1 \leq i \leq t, c_i = E_K(IV||CTR_i) \oplus m_i$.

2. **Descifrado**: para $1 \leq i \leq t, m_i = E_K(IV||CTR_i) \oplus c_i$.

A diferencia del valor inicial del CBC, la cadena $(IV||CTR_1)$ no necesita ser aleatoria, simplemente tiene que ser un valor único, nunca repetido, por la misma razón que la *libreta de un solo uso* no debe ser reutilizada.

En el modo CTR, en realidad nunca se utiliza AES para el cifrado o descifrado de los datos. En su lugar, este modo genera una secuencia cifrante que tiene la misma longitud que el texto claro y luego utiliza XOR para combinarlas. Es decir, CTR imita el funcionamiento de la libreta

de un solo uso, pero en lugar de requerir una clave tan larga como el mensaje (una verdadera molestia cuando se cifra un archivo de 1 TB), utiliza AES (o cualquier otro cifrador en bloque) y un contador para generar una secuencia cifrante de longitud casi arbitraria a partir de una clave secreta tan pequeña como 128 bits.

Como el contador cambia en una pequeña cantidad cada vez, el cifrador en bloque debe exhibir buenas propiedades de avalancha por bloque, como es el caso de AES. Por lo tanto, cada bloque de salida parece completamente diferente del último (el 50 % de los bits habrá cambiado en promedio) y la secuencia cifrante en su conjunto parece ser una cadena aleatoria, capaz de superar todas las pruebas de aleatoriedad. Una vez que se genera una cantidad suficiente de material de clave, se utiliza la operación XOR para combinarla con el texto claro y producir el texto cifrado.

Se debe tener cuidado al inicializar la entrada al cifrado en bloque y evitar usar el mismo valor de entrada dos veces. De lo contrario, si un atacante conoce uno de los dos textos claros que fueron cifrados con la misma entrada, puede calcular el bloque de la secuencia cifrante y así descifrar inmediatamente el otro texto cifrado. Debido a que el modo CTR es un cifrado en flujo, simplemente se hace el XOR del texto claro con la secuencia cifrante. Conocido el texto claro, puede recuperarse la clave trivialmente: $c \oplus m = m \oplus k \oplus m = k$.

PRÁCTICA 3.10: Recuperando la secuencia cifrante de AES-CTR mediante un ataque de texto claro conocido

Imagina que un ransomware está cifrando archivos diferentes usando mismo IV y misma clave, generados, eso sí, aleatoriamente. Tienes acceso a numerosos archivos cifrados en el disco de la víctima: `CTR1.enc`, `CTR2.enc`, etc. Resulta que la víctima conserva en un USB una copia en claro de uno de los archivos cifrados: `CTR1.txt`. Como posees el texto claro y cifrado, si haces el XOR de ambos habrás obtenido la secuencia cifrante.

1. Abre en CrypTool los archivos `CTR1.txt` y `CTR1.enc`. Calcula el XOR entre ambos. El resultado es la secuencia cifrante.

   ```
   000000 2f f1 61 ed 5e ff ce 5e 25 a5 d5 ed 99 4e 0d 30
   000010 f3 e1 c6 ec 42 25 d9 2a 6f 5b 28 5d 4b 45 78 38
   ...
   ```

2. Abre el segundo archivo cifrado, `CTR2.enc`, y calcula el XOR con la secuencia cifrante anterior. Obtendrás el archivo original.

   ```
   El vídeo proporciona una manera eficaz ...
   ```

Los tamaños de los mensajes pueden ser un poco diferentes, pero si te faltan algunos bytes de la secuencia cifrante, recupera lo que puedas. Como has podido comprobar, AES-CTR no resiste un ataque KPA, como aprendiste en la Sec. 1.6.1. Para evitar estos ataques, el ransomware Snake Locker, por ejemplo, utiliza una clave y IV generados aleatoriamente para cada archivo cifrado.

Para combatir esta debilidad, a menudo se adopta en la práctica el siguiente enfoque. Supón un cifrado en bloque con un tamaño de bloque de 128 bits, como AES. Primero eliges un IV que es un *nonce*[2] con una longitud menor que la del bloque, por ejemplo, 96 bits. Los 32 bits restantes son utilizados por un contador con el valor CTR que se inicializa a cero. Por cada bloque que se cifra durante la sesión, el contador se incrementa, pero el IV permanece igual. En este ejemplo, el número de bloques que podemos cifrar sin elegir un nuevo IV es de 2^{32}. Como cada bloque consta de 8 bytes, se pueden cifrar un máximo de $8 \times 2^{32} = 2^{35}$ bits o unos 32 Gigabytes, antes de que se deba generar un nuevo IV.

Como cada bloque se procesa independientemente, al igual que en ECB, y a diferencia de CBC, CFB y OFB, pueden cifrarse (y descifrarse) todos los bloques del texto claro en paralelo. Sin embargo, a diferencia del modo ECB, dos bloques idénticos de texto claro darán como resultado dos bloques distintos de texto cifrado (tampoco se ve el pingüino). En cierto modo, se está usando la operación de cifrado de los distintos bloques para generar una secuencia cifrante como en el caso del cifrado en flujo. Además, a diferencia de ECB, cada bloque de texto cifrado

[2]En criptografía, un *nonce* es un número arbitrario que puede ser utilizado una sola vez en una comunicación criptográfica. Suele ser un número aleatorio o pseudoaleatorio emitido en un protocolo de autenticación para asegurar que comunicaciones pasadas no se reutilicen en ataques de repetición. También pueden ser útiles como vectores de inicialización y en funciones de cifrado (fuente: Wikipedia).

corresponde a una posición específica dentro del texto cifrado, por lo que no pueden reordenarse caprichosamente sin generar errores (no se puede malear).

Otra diferencia con los modos ECB y CBC consiste en que el modo CTR sólo requiere la implementación del algoritmo de cifrado y no del algoritmo de descifrado, lo cual supone una importante ventaja cuando el algoritmo de descifrado difiere sustancialmente del algoritmo de cifrado, como ocurre con AES.

Felizmente, los cifrados en flujo no requieren relleno (*padding*). Es bastante simple hacer el XOR solo de un bloque parcial, descartando las partes posteriores de la clave que no se necesitan. Por ejemplo, los siguientes estándares y aplicaciones utilizan AES-CTR para dotar de confidencialidad a las comunicaciones, solo por citar unos pocos entre los más conocidos:

- 5G
- TLS 1.3
- Whatsapp
- Signal
- Telegram
- Protonmail
- 1Password
- NordVPN
- Etc.

3.2.3.6. XTS

Ninguno de los modos de cifrado anteriores resulta directamente adecuado como base de una solución de cifrado de discos duros. En 2007, XTS-AES fue estandarizado por el IEEE como 1619-2007 - IEEE Standard for Cryptographic Protection of Data on Block-Oriented Storage Devices. Posteriormente, en 2010 el NIST publicó The XTS-AES Mode for Confidentiality on Storage Devices, catapultando su adopción. El modo XTS-AES es una instanciación del cifrado por bloques modificable XEX (XOR Encrypt XOR) de Rogaway, complementado con un método llamado *robo de texto cifrado* para ampliar el dominio de posibles cadenas de datos de entrada. XTS es el más que conveniente acrónimo de *XEX-based tweaked codebook mode with ciphertext stealing*. Veámoslo.

Supón que cada sector del disco tiene un número único, al igual que cada bloque de datos dentro de un sector. La idea central del modo XTS es utilizar el número de sector del disco y la posición de un bloque de datos dentro del sector como información predecible que varía con cada cifrado de bloque. Por ejemplo. Bitlocker en Windows utiliza XTS-AES (y también una versión modificada de CBC), así como otros muchos productos de cifrado de disco, como TrueCrypt o VeraCrypt.

El modo XTS-AES se basa en el concepto de cifrado por bloques modificable (*tweakable*), que funciona de forma muy similar a los *salts* utilizados con las contraseñas para calcular un hash distinto incluso con contraseñas idénticas. Un cifrado por bloques modificable es aquel que tiene tres entradas: un texto claro P, una clave simétrica K y una modificación (*tweak*) T; y produce un texto cifrado de salida C. Podemos escribirlo como $C = E(K, T, P)$. No es necesario mantener en secreto la modificación T. Mientras que el propósito de la clave K es proporcionar seguridad, el propósito del ajuste T es proporcionar variabilidad. Es decir, el uso de diferentes ajustes con el mismo texto claro y con la misma clave produce diferentes resultados.

En esencia, en XTS se utiliza el modo ECB, pero para cada bloque se cambian dos *tweaks*, correspondientes al sector y posición dentro del mismo. Este enfoque supera la principal debilidad de seguridad de ECB que ya conoces: dos cifrados del mismo bloque producen el mismo texto cifrado. Cuando el último bloque del disco tiene menos de 128 bits, los dos últimos bloques se cifran/descifran utilizando la técnica de *robo de texto cifrado* (*ciphertext stealing*) en lugar de relleno (*padding*).

Como puede verse, el modo XTS-AES, al igual que el modo CTR, resulta adecuado para el funcionamiento en paralelo. Como no hay encadenamiento, se pueden cifrar o descifrar varios bloques simultáneamente.

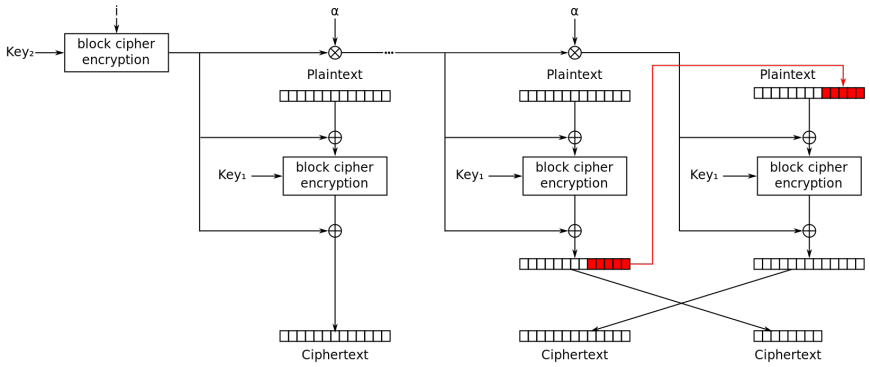


Figura 3.18: Modo XTS: cifrado (fuente: Wikipedia).

Modo	Seguridad frente a CPA	Seguridad frente a CCA	Requiere IV/Nonce	Paralelizable	Comentarios sobre la Seguridad
ECB	No	No	No	Sí	Revela patrones de datos, no recomendado para uso general.
CBC	Sí (con IV aleatorio)	No	Sí	No	Vulnerable a ataques de reordenamiento y oráculos de padding.
CFB	Sí (con IV aleatorio)	No	Sí	No	Sensible a la manipulación de bloques.
OFB	Sí (con IV aleatorio)	No	Sí	Sí	Produce una secuencia pseudoaleatoria para el XOR con el texto claro.
CTR	Sí (con nonce)	No	Sí	Sí	Requiere manejo cuidadoso del nonce. Altamente paralelizable.
XTS	Específico del sector	No	Sí (tweak)	Medio	Protege contra ciertos ataques de manipulación en almacenamiento.

Cuadro 3.3: Comparativa de seguridad de los distintos modos de encadenamiento de bloques para confidencialidad, basada en Evaluation of Some Blockcipher Modes of Operation.

Siempre que un atacante no pueda combinar imágenes del estado del disco duro en varios momentos diferentes, XTS-AES ofrece unas propiedades de seguridad relativamente buenas y una buena eficacia. Sin embargo, si el atacante puede crear copias del medio de almacenamiento cifrado en un mayor número de puntos diferentes en el tiempo, hay que suponer una cierta fuga de información, no necesariamente insignificante. Por ejemplo, al comparar dos imágenes de un disco duro cifrado con XTS-AES realizadas en distintos momentos, el atacante puede ver inmediatamente qué bloques de texto claro del disco duro se han modificado en ese periodo y cuáles no.

Encontrarás una discusión sobre la seguridad de este modo de cifrado en You Don't Want XTS.

3.2.3.7. Seguridad de los distintos modos de encadenamiento de bloques

El criptógrafo Phillip Rogaway publicó en 2011 Evaluation of Some Blockcipher Modes of Operation, uno de los análisis de seguridad más completos de los modos de encadenamiento de bloques para confidencialidad (los explicados en esta sección), así como para autenticidad y cifrado autenticado. Su lectura es muy recomendable si quieres profundizar en su funcionamiento y seguridad.

3.3. Comparación entre cifradores en flujo y en bloque

Los cifradores en bloque operan sobre grandes bloques de texto claro usando una transformación fija; los cifradores en flujo operan con una transformación variable con el tiempo sobre dígitos individuales de texto claro.

Las mayores diferencias entre los cifradores en flujo y en bloque residen en la implementación. Los cifradores en flujo que sólo cifran y descifran datos un bit a la vez no son realmente adecuados para su implementación en software. En cambio, los cifradores en bloque pueden ser más fáciles de implementar en software, porque a menudo evitan las manipulaciones de bits individuales que llevan mucho tiempo y, en su lugar, funcionan con bloques del tamaño de palabra de un ordenador.

Por su parte, los cifradores en flujo suelen ser más adecuados para la implementación en hardware. Tiene sentido que un dispositivo de cifrado de hardware en un canal de comunicaciones digitales cifre los bits individuales a medida que pasan. Por otro lado, tiene menos sentido que un dispositivo de cifrado de software cifre cada bit individual por separado. En general, el tamaño de bloque debería ser al menos el del bus de datos.

Ransomware	Algoritmo	Modo	IV
Ryuk	AES-256	CBC	Cero
Dharma	AES-128	CBC	Aleatorio
LockBit	AES-128	CBC	Aleatorio
SamSam	AES-128	CBC	Aleatorio
GandCrab	Salsa20, RC4	-	-
Clop	RC4	-	-
Katyusha	AES-256	CBC	Aleatorio
Snatch	AES-128	OCFB	Aleatorio
Phobos	AES-256	CBC	Aleatorio
Nemty	AES-128	CBC	Aleatorio

Cuadro 3.4: Cifradores simétricos en el ransomware moderno y sus parámetros (Fuente: Roderik Ploszek, Peter Švec and Patrik Debnár, Analysis of encryption schemes in modern ransomware, Rad Hazu. Matematicke Znanosti, 25 (546), 1-13, 2021. https://doi. org/10.21857/mn1qgc58gy).

Otra diferencia importante es que se ha hecho un esfuerzo mucho mayor para analizar los cifradores en bloque. En general, parecen aplicables a una gama más amplia de aplicaciones que el cifrado en flujo. La gran mayoría de las aplicaciones de criptografía simétrica para redes telemáticas utilizan cifrado en bloque.

No existe una prueba matemáticamente completa de la seguridad de ningún cifrado en bloque ni en flujo. Lo mejor que puede hacerse es considerar que un cifrado simétrico es seguro si ninguno de los ataques conocidos es significativamente más rápido que una búsqueda exhaustiva del espacio de claves (también conocido como *ataque de fuerza bruta*).

3.4. ¿Qué algoritmos de criptografía simétrica utiliza el ransomware?

En la tabla 3.4 se observa que la mayoría de muestras de ransomware analizadas utilizan AES, mientras que el método de encadenamiento de bloque más utilizado es CBC. ¿Para qué reinventar la rueda?

3.5. Cifrado autenticado con datos asociados

Hasta ahora sólo hemos hablado de cifrado sin ninguna forma de autenticación. Eso significa que aunque podemos cifrar y descifrar los mensajes, no podemos verificar que el mensaje es lo que el remitente realmente envió (integridad).

Si bien el cifrado no autenticado puede proporcionar confidencialidad, ya hemos visto en los numerosos ejercicios de maleado realizados en las prácticas anteriores que, sin autenticación, un atacante activo puede modificar con éxito mensajes cifrados válidos, a pesar de que no conozca necesariamente el texto claro correspondiente. La aceptación de estos mensajes a menudo puede conducir a la filtración de información secreta, lo que significa que ni siquiera obtenemos confidencialidad. Los ataques de relleno de CBC que ya hemos discutido ilustran esta debilidad.

Como resultado, necesitamos formas de autenticar Y cifrar nuestras comunicaciones secretas. ¿Cómo? Añadiendo información extra al mensaje que sólo el remitente podría haber calculado. Al igual que el cifrado, la autenticación viene tanto en forma de clave secreta (simétrica) como de clave pública (asimétrica). Los esquemas de autenticación simétrica se denominan típicamente *códigos de autenticación de mensajes*, mientras que el equivalente de clave pública se denomina típicamente *firma digital*.

En la lección 5 profundizarás en estos esquemas, así como en una nueva primitiva criptográfica, la función *hash*, que permitirá tanto crear esquemas de autenticación de mensajes como esquemas de firma digital.

3.6. Cómo dividir una clave secreta en múltiples claves para repartir entre varias personas

Te estás dando cuenta de lo difícil que es gestionar claves: si guardas los datos cifrados con un algoritmo simétrico, ¿dónde guardas la clave de cifrado? En última instancia, **la seguridad**

de un sistema de cifrado reside en la gestión de sus claves. Las claves pasan a ser el talón de Aquiles de la criptografía. De hecho, no están seguras ni en la memoria del ordenador: Heartbleed, Spectre y Meltdown vienen a la cabeza como ejemplos recientes de vulnerabilidades que permitían leer espacios privados de la memoria y obtener, entre otros datos, claves de cifrado.

A su vez, los ataques de canal lateral pueden filtrar información sobre claves gracias a variaciones electromagnéticas o de consumo de energía. Más aún, las claves pueden quedarse grabadas en una memoria DRAM incluso después de apagar el equipo. ¿No existe forma entonces de garantizar la seguridad de las claves? Una solución pasa por dividir la clave en dos o más partes, de manera que la información cifrada no pueda descifrarse a menos que se junten todas (o un número mínimo de) las partes de la clave.

3.6.1. Reparto de secretos n de n

Por ejemplo, si la quieres repartir entre tres personas, podrías dividirla en tres trozos y mandar cada trozo a cada persona, de manera que para recuperar la clave original tendrían que juntar los tres trozos en el orden correcto. A continuación tienes otro ejemplo más versátil, aunque aún muy básico.

Ejemplo: reparto de la clave secreta k entre tres personas

Para dividir la clave k en tres partes, k_1, k_2 y k_3, genera aleatoriamente dos claves, k_1 y k_2, de la misma longitud que k. La tercera parte de la clave se calcula como $k_3 = k_1 \oplus k_2 \oplus k$. No hay dos partes que proporcionen ninguna información sobre la clave secreta: las tres partes son necesarias para recuperar k.

$$k_1 \oplus k_2 \oplus k_3 = k_1 \oplus k_2 \oplus k_1 \oplus k_2 \oplus k = k$$

El esquema descrito exhibe la propiedad *3 de 3* o $(3, 3)$, es decir, hacen falta las tres partes para reconstruir el secreto, si una falla, no hay secreto. Esta misma idea puede extenderse fácilmente a la distribución secreta de un valor entre n partes, requiriendo que todas ellas lo reconstruyan. Sería una estructura (n, n). Para ello, basta con elegir números aleatorios para las primeras $n-1$ subclaves y establecer que la subclave de la última parte sea la clave a repartir menos la suma de los números aleatorios de las primeras $n-1$ partes.

3.6.2. Esquema de Reparto de Secretos de Shamir (SSSS)

Estos esquemas pueden generalizarse aún más para ser (k, n) (siendo $n \geq k \geq 1$): con juntar k partes puede recuperarse un secreto compartido entre n partes, pero juntando $k-1$ partes no se sabe nada sobre el secreto.

El método más popular es sin duda el esquema de reparto de secretos de Shamir (SSSS), basado en las propiedades de los polinomios. Se explica fácilmente para compartir 2 de n, pero se extiende de forma sencilla a cualquier k de n. Empecemos con el caso $(2, n)$.

Imagina que quieres compartir un valor s entre dos partes. Defines una línea aleatoria que corte el eje y en el punto s: $f(x) = ax + s$, donde a es un número aleatorio. Después, les entregas un punto de esta línea a cada una de las dos partes. Puedes entregar a la parte i el punto $(i, f(i))$. Observa que dado cualquier punto de la línea, se puede rotar la línea en cualquier dirección y este punto seguiría siendo válido, lo que significa que la línea todavía podría cortar el eje y en cualquier punto. Por lo tanto, ninguna de las partes sabe nada sobre el secreto. Sin embargo, dados dos puntos cualesquiera en la línea, las dos partes pueden reconstruir la línea y luego calcular $s = f(0)$. Obviamente se pueden entregar más puntos de la curva a otras personas, ya que siempre harán falta dos para recuperar s. De ahí que sea un esquema $(2, n)$.

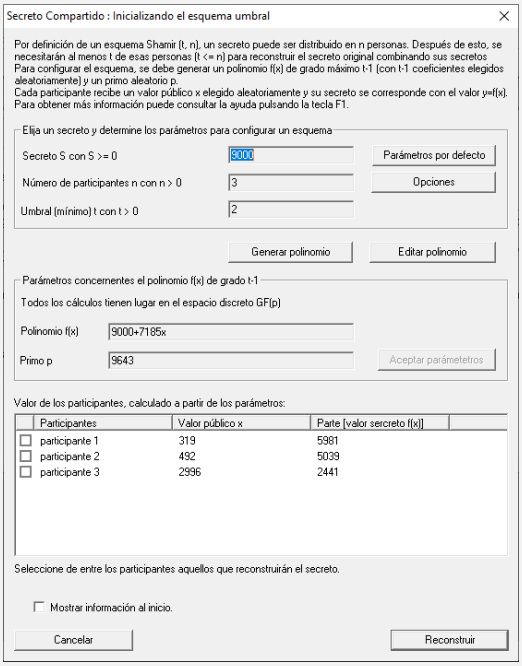
Para extender este esquema a (k, n), la única diferencia es que en lugar de usar una línea, se usa un polinomio de coeficientes aleatorios de grado $(k-1)$: $f(x) = s + a_1 x + a_2 x^2 + a_3 x^3 + \cdots + a_{k-1} x^{k-1}$. Al igual que antes, $f(0) = s$. ¿Por qué funciona? Porque un polinomio de grado $(k-1)$ puede reconstruirse (técnicamente llamado *interpolación de Lagrange*) conocidos k puntos del mismo. Además, al igual que pasaba con la recta, dado cualquier $k-1$ o menos puntos, todos los valores del eje y son todavía posibles y ninguna de las partes sabe nada sobre el secreto.

En la práctica, para dotar al esquema de mayor seguridad, las operaciones se hacen sobre un campo finito de tamaño $q = p^r$, donde p es un primo, r es un entero positivo y $q > n$. Suele tomarse $r = 1$, de manera que $q = p$ es un primo.

PRÁCTICA 3.11: Repartiendo secretos con el esquema de reparto de secretos de Shamir (SSSS) en Cryptool

1. Selecciona `Procedimientos Indiv. >Demostración de secreto compartido (Shamir).`
2. Introduce el secreto (CrypTool solo acepta números menores que 10.000), en cuántas partes quieres repartirlo y cuántas harán falta para recuperarlo.
3. Sigue el resto de instrucciones y el programa hará el resto.

Echa un vistazo también al fichero de log porque resulta muy instructivo.



PRÁCTICA 3.12: Recuperando secretos con el esquema de reparto de secretos de Shamir (SSSS) en Sagemath

Conocidos k puntos y el primo del campo finito, el siguiente código te permitirá recuperar un polinomio y, por tanto, el secreto compartido, usando el método de interpolación de Lagrange:

```
q = 79 # primo para el campo finito
shares = [(11,46),(41,44),(51,12),(9,29)] # los puntos en los que se ha
dividido el secreto
R = PolynomialRing(GF(q),"x") # se crea un anillo de polinomios para ese
campo finito
p = R.lagrange_polynomial(shares) # reconstrucción del polinomio original
usando la interpolación de Lagrange
print( "p(x) =", p )
print( "Secreto = p(0) =", p(0))
```

El resultado de la evaluación es:

```
p(x) = 74*x^3 + 40*x^2 + 42*x + 56
Secreto = p(0) = 56
```

3.6.3. Criptografía Visual

En 1994, Shamir y Moni Naor desarrollaron y propusieron una variante visual del reparto de secretos: la criptografía visual, que permite repartir una imagen secreta entre n participantes. La imagen se divide en n transparencias, de tal manera que si se apilan unas encima de otras cualesquiera k transparencias, la imagen se hace visible, pero si se apilan menos de k transparencias, no se puede ver nada en absoluto. La Fig. 3.19 muestra un ejemplo para $n = 2$.

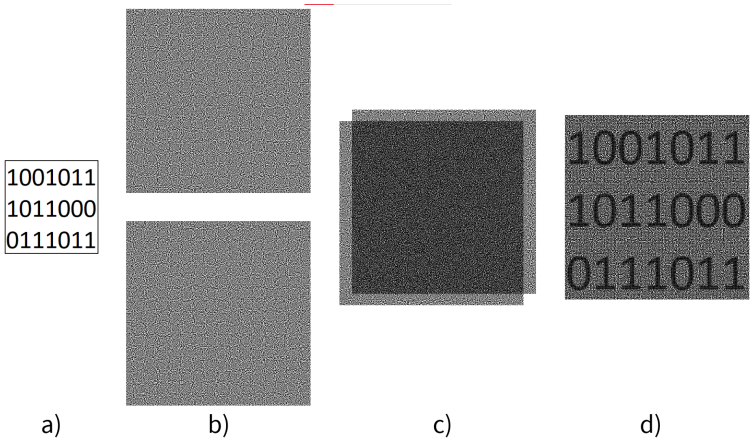


Figura 3.19: Criptografía visual: a) imagen original de 200x200; b) las dos transparencias de 400x400 creadas a partir de la imagen original; c) si las dos transparencias no están perfectamente sobrepuestas, no se recupera la imagen original; y d) solo cuando las dos transparencias están perfectamente apiladas se recupera la imagen original, siempre con ruido.

La mayoría de los criptosistemas visuales propuestos en la literatura son perfectamente seguros y pueden implementarse fácilmente sin ningún cálculo criptográfico. Una mejora adicional permite que cada transparencia represente una imagen de aspecto inocente, como una imagen de un paisaje o una imagen de una flor), ocultando así el hecho de que se está compartiendo el secreto. Por consiguiente, las técnicas esteganográficas se utilizan a veces en combinación con la criptografía visual.

3.7. Tres tipos de aplicaciones para el cifrado simétrico

Hay tres tipos principales de aplicaciones de cifrado simétrico de datos:

- El **cifrado en tránsito** protege los datos enviados de una máquina a otra: los datos se cifran antes de ser enviados y se descifran tras recibirse, como en las conexiones cifradas a sitios web de comercio electrónico mediante TLS. El cifrado en flujo se utiliza típicamente en estas aplicaciones.
- El **cifrado en reposo** protege los datos almacenados en un sistema de información. Los datos se cifran antes de ser escritos en la memoria (volátil o permanente) y se descifran antes de ser leídos. Algunos ejemplos son el cifrado de discos en ordenadores portátiles, así como el cifrado de máquinas virtuales para instancias virtuales en la nube. En estas aplicaciones suele utilizarse el cifrado en bloque.
- Todos estamos familiarizados con la idea de cifrar los datos en disco o al transmitirlos entre dispositivos. En cambio, el **cifrado en uso** protege los datos en memoria, mientras se están procesando. Por ejemplo, imagina que has cifrado tus datos en tus instalaciones y que sólo tú tienes las claves. Subes esos datos a un servicio de almacenamiento en la nube: sencillo, seguro y protegido. Pero ahora quieres entrenar modelos de *machine learning* basados en esos datos. Antes de que puedan ser procesados por la aplicación de aprendizaje, los datos deben ser descifrados en memoria. Tus datos se vuelven vulnerables justo antes, durante y después del procesamiento: a los volcados de memoria, al robo por el usuario administrador y a otros exploits maliciosos. La **computación confidencial** resuelve este problema aprovechando un entorno de ejecución de confianza basado en hardware, o TEE (*trusted execution environment*), que es un enclave seguro dentro de una CPU.

3.8. Estandarización de algoritmos y pruebas

3.8.1. Estándares de cifrado simétrico

Varios organismos de estandarización han publicado recomendaciones sobre cómo utilizar los algoritmos de cifrado simétrico más seguros y cómo gestionar sus claves de manera segura.

- NIST
  - FIPS 197: Advanced Encryption Standard (AES), Nov 2001
  - SP 800-67 Revision 2: Triple DES, Nov 2017
  - SP 800-57 Part 1 Rev. 5: Recommendation for Key Management: Part 1 – General, May 2020
- ISO
  - ISO/IEC 18033-3:2010: Information technology — Security techniques — Encryption algorithms — Part 3: Block ciphers
  - ISO/IEC 18033-4:2011: Information technology — Security techniques — Encryption algorithms — Part 4: Stream ciphers
  - ISO/IEC 19592-1:2016: Information technology — Security techniques — Secret sharing — Part 1: General
  - ISO/IEC 19592-2:2017: Information technology — Security techniques — Secret sharing — Part 2: Fundamental mechanisms
- BSI
  - BSI TR-02102-1: Cryptographic Mechanisms: Recommendations and Key Lengths - Version: 2022-1
- CCN
  - Guía de Mecanismos Criptográficos autorizados por el CCN (CCN-STIC 221)

3.8.2. Cómo verificar que tu programa cifra/descifra correctamente

El Instituto Nacional de Normas y Tecnología (NIST) de los Estados Unidos publica una lista de los KAT (*known answer tests*) para la validación de la implementación de algoritmos criptográficos. El archivo contiene archivos de respuesta (.rsp) con los resultados previstos para determinadas entradas.

PRÁCTICA 3.13: Poniendo a prueba el cifrado AES-CBC de CrypTool

1. Descomprime el archivo `KAT_AES.zip` y abre el archivo `CBCVarTxt128.rsp` con un editor de texto cualquiera.
2. En CrypTool, crea un archivo nuevo, visualízalo en modo hexadecimal y pega el primer texto claro del archivo de respuesta (Decodificación hexadecimal):

   `PLAINTEXT = 80000000000000000000000000000000`
3. Cífralo con AES-CBC: `Cifrar / Descifrar >Simétrico (moderno) >AES (CBC)`. La clave predeterminada es `0x00...` y el IV, también (CrypTool lo tiene así configurado y no puede cambiarse). El resultado coincidirá con el del archivo .rsp.

Nota: verás que el cifrado de CrypTool tiene 16 bytes adicionales. Se debe a que CrypTool usa relleno 01 00 (añade el byte `01` y rellena el resto con bytes `00`, por ejemplo, `61 62 63 01 00 00 00 00 ...`). Aunque el tamaño del texto claro a cifrar sea múltiplo entero del tamaño de bloque, añadirá el relleno, como acabas de comprobar.

3.9. El gran reto del ransomware: ¿cómo gestiono la clave?

Ya solo queda un pequeño detalle: ¿cómo proteger la clave usada para cifrar los archivos? Si está en el código, será recuperada mediante *reversing*. Si se destruye sin más, luego será imposible descifrar los archivos y nadie pagaría el rescate. Si se envía en claro desde el servidor C&C podría ser interceptada.

Tal y como sugirieron Young y Yung, nada como cifrarla a su vez mediante un algoritmo de cifrado asimétrico. Los estudiarás en la próxima lección.

Lección 4

Cifrado asimétrico o de clave pública

Objetivos de aprendizaje

- Identificar los principales problemas relacionados con la distribución de claves, el crecimiento del número de claves y la imposibilidad de garantizar el no repudio.
- Entender en qué consiste la clave pública, la clave privada y la función unidireccional, así como los problemas matemáticos (factorización y logaritmo discreto) en los que se basan.
- Reconocer cómo se aplican los esquemas de cifrado, firma digital y acuerdo de claves (como Diffie-Hellman) en la práctica.
- Reconocer la complementariedad entre ambos tipos de criptografía y cómo se integran en soluciones híbridas.

Competencias a desarrollar

- Elegir el método criptográfico más apropiado para cada necesidad (cifrado, firma, distribución de clave) considerando factores como velocidad, seguridad y requisitos de infraestructura.
- Implementar y probar algoritmos de clave pública (RSA, Diffie-Hellman) en entornos de desarrollo y con herramientas estándar.
- Diferenciar escenarios en los que conviene utilizar claves efímeras (por ejemplo, para proporcionar secreto perfecto hacia adelante) o claves de larga duración (certificados).
- Integrar criptografía asimétrica y simétrica para cubrir necesidades de rendimiento, seguridad y escalabilidad en aplicaciones reales.

En la lección anterior estudiaste la criptografía simétrica o de clave secreta y sus dos enfoques: cifrado en flujo y cifrado en bloque. La criptografía simétrica es muy potente, pero no está exenta de limitaciones:

- **Problema de distribución de la clave**: la clave se debe establecer entre ambas partes usando un canal seguro. Si las dos partes no se conocen e interactúan a través de Internet, por ejemplo, sería imposible hacerlo de forma segura.
- **Crecimiento descontrolado del número de claves**: incluso si resolvemos el problema de distribución de la clave, debemos tratar con un número potencialmente muy alto de claves. Si cada par de usuarios necesita un par de claves separadas, en una red con n usuarios, el número de pares de claves es

$$\frac{n \cdot (n-1)}{2}$$

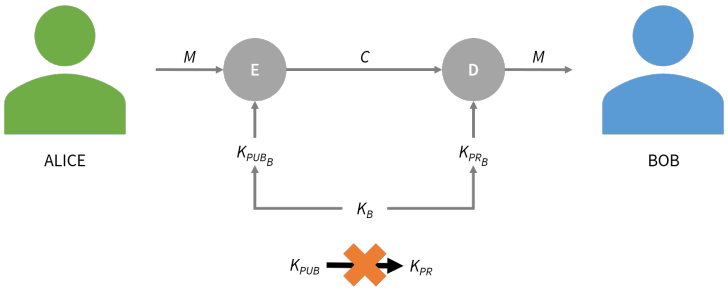


Figura 4.1: Criptografía asimétrica: se usa una clave pública para cifrar y una clave privada para descifrar.

y cada usuario tiene que almacenar $n - 1$ claves de forma segura.

- **Imposibilidad de garantizar el no-repudio**: si ambas partes conocen la clave secreta, una podría cifrar un mensaje y decir que fue la otra parte quien lo cifró. Prevenir este engaño se llama *no repudio* y se puede lograr con criptografía asimétrica usando firmas digitales, tratadas en la lección 6.

4.1. Nuevas direcciones en criptografía

Para superar estos inconvenientes, Diffie, Hellman y Merkle plantearon en 1976 una propuesta revolucionaria. Idearon una clave k de dos partes: una pública, k_{pub}, que como su nombre indica es conocida por todo el mundo; y otra privada, k_{pr}, que solo conoce su legítimo propietario y nunca debe revelarse (v. Fig. 4.1).

Por supuesto, a partir del conocimiento de la clave pública tiene que ser imposible obtener la correspondiente clave privada.

Además, algunos algoritmos, como el RSA (v. Sec. 4.3), también muestran la siguiente característica: cualquiera de las dos claves de la pareja puede utilizarse para cifrar y la otra para descifrar.

Existe una sencilla analogía para la criptografía asimétrica: una puerta con una clave de combinación. La acción de cifrar un mensaje puede considerarse como un simple cierre de la puerta, ya que no se necesita el conocimiento de la clave para realizar esta operación porque este mecanismo es de conocimiento público. Para descifrar el mensaje, es decir, abrir la puerta, se utiliza la clave. Sólo la persona que conoce la clave secreta podrá abrir la puerta (sin romperla).

Los algoritmos de clave pública se construyen a partir de un principio común, la función *unidireccional*. Una función f es una función unidireccional si:

1. $y = f(x)$ es fácil de calcular, pero
2. $x = f^{-1}(y)$ resulta inviable computacionalmente.

Generalmente, *fácil* se define como un problema que puede ser resuelto en tiempo polinómico en función de la longitud de la entrada. Así, si la longitud de la entrada es de n bits, entonces el tiempo para calcular la función es proporcional a n^a, donde a es una constante fija. Se dice que tales algoritmos pertenecen a la clase **P**. El término *inviable* es un concepto mucho más difuso. En general, podemos decir que un problema es inviable si el esfuerzo para resolverlo crece más rápidamente que el tiempo polinómico en función del tamaño de la entrada.

En los esquemas prácticos de clave pública se utilizan dos funciones unidireccionales:

- El problema de la *factorización* de enteros, en el que se basa RSA: dados dos números primos muy grandes, es fácil calcular su producto; sin embargo, es muy difícil factorizar el producto resultante.
- El problema del *logaritmo discreto*, en el que se basa Diffie-Hellman: consiste en encontrar la y para la cual $g^y = x$, dada una base g dentro de algún grupo $\mathbb{Z}_p^*$, donde p es un número primo, y dado un elemento de grupo x. El *logaritmo discreto* se llama *discreto* porque tratas con números enteros en vez de números reales (continuos) y se llama *logaritmo* porque buscas el logaritmo de x en base g. Por ejemplo, el logaritmo de 256 en base 2 es 8 porque $2^8 = 256$. Obviamente, cuanto mayor sea el valor de p, más difícil será resolver el logaritmo.

Algoritmo	Cifrado / Descifrado	Firma digital	Acuerdo de claves
RSA	SÍ	SÍ	SÍ
Curva elíptica	SÍ	SÍ	SÍ
Diffie–Hellman	NO	NO	SÍ
DSS	NO	SÍ	NO

Cuadro 4.1: Aplicaciones de los criptosistemas de clave pública.

4.2. Aplicaciones de los criptosistemas de clave pública

Como queda dicho, los sistemas de clave pública se caracterizan por el uso de un algoritmo criptográfico con dos claves, una privada, k_{pr}, y otra disponible públicamente, k_{pub}. Según la aplicación, el emisor utiliza su clave privada o la clave pública del receptor, o ambas, para realizar algún tipo de función criptográfica. En términos generales, se puede clasificar el uso de criptosistemas de clave pública en tres categorías:

- **Cifrado/descifrado**: el emisor cifra el mensaje con la clave pública del destinatario y éste lo descifra con su clave privada.
- **Firma digital**: el emisor *firma* un mensaje con su clave privada. La firma se logra mediante un algoritmo criptográfico aplicado al mensaje o a un pequeño bloque de datos que es una función del mensaje, como un hash de todo el mensaje.
- **Acuerdo de claves**: dos partes cooperan para acordar una clave de sesión, que es una clave secreta para cifrado simétrico generada para su uso en una transacción (o sesión) determinada y válida durante un breve período de tiempo. Son posibles varios enfoques diferentes, que implican la(s) clave(s) privada(s) de una o ambas partes.

La tabla 4.1 muestra cómo algunos algoritmos son adecuados para las tres aplicaciones, mientras que otros sólo pueden utilizarse para una o dos de esas aplicaciones.

Pero basta de cháchara. Ya va siendo hora de verles las tripas a los algoritmos.

4.3. El criptosistema RSA

El esquema criptográfico RSA, propuesto por los criptógrafos Rivest, Shamir y Adleman en 1977, es actualmente el esquema criptográfico asimétrico más utilizado, aunque los esquemas basados en curvas elípticas están ganando terreno rápidamente. Luego entenderás por qué. De momento, familiarízate con su funcionamiento. ¡Es una obra de arte!

4.3.1. Funcionamiento de RSA

4.3.1.1. Cómo cifrar y descifrar con RSA

La aritmética modular juega un papel central en RSA. El texto claro para cifrar, m, se considera que representa un elemento en $\mathbb{Z}_n^* = \{0, 1, \ldots, n-1\}$. Vamos, que es un número entero. Como consecuencia, el valor binario del texto claro m debe ser menor que n. Lo mismo ocurre con el texto cifrado.

Dada la clave pública $k_{pub} = (n, e)$ y el texto claro m, la función de cifrado es:

$$c = e_{k_{pub}}(m) \equiv m^e \mod n$$

donde $m, c \in \mathbb{Z}_n^*$.

Dada la clave privada $k_{pr} = d$ y el texto cifrado c, la función de descifrado es:

$$m = d_{k_{pr}}(c) \equiv c^d \mod n$$

donde $m, c \in \mathbb{Z}_n^*$.

Ejemplo: cifrando y descifrando con RSA

Sean las claves $k_{pub} = (33, 3)$ y $k_{pr} = 7$. El mensaje para cifrar es $m = 4$.

$$c = m^e = 4^3 \equiv 31 \mod 33$$

$$c^d = 31^7 \equiv 4 = m \mod 33$$

En la práctica, m, c, n y d son números muy grandes, normalmente de 1.024 bits o más. El valor e se denomina a veces *exponente de cifrado* o *exponente público*, y la clave privada d se denomina a veces *exponente de descifrado* o *exponente privado*.

Incluso sin conocer más detalles, ya puedes establecer algunos requisitos para el criptosistema RSA:

1. Dado que un atacante tiene acceso a la clave pública, debe ser inviable computacionalmente determinar la clave privada d a partir de los valores e y n de la clave pública.
2. Dado que m debe ser menor que el módulo n, no podrás cifrar más de l bits con una operación RSA, donde l es la longitud en bits de n.
3. Debería ser relativamente fácil calcular $m^e \mod n$ para cifrar, y $c^d \mod n$ para descifrar. Esto significa que necesitas un método para exponenciación rápida de números de semejante tamaño.
4. Para un tamaño de n dado, debería haber muchos pares de clave privada y pública, de lo contrario un atacante podría realizar un ataque de fuerza bruta.

4.3.1.2. Cómo generar parejas de claves para RSA

¿De dónde salen los valores de la clave pública y la privada? Fácil. Estos son los pasos para calcular la clave pública y privada de un criptosistema RSA:

1. Elige dos primos grandes p y q.
2. Calcula el producto $n = p \cdot q$.
3. Calcula el producto $\phi(n) = (p-1)(q-1)$ ($phi(n)$ es la *función de Euler*, definida como la cantidad de enteros positivos menores o iguales a n y coprimos con n).
4. Selecciona un exponente público al azar $e \in \{1, 2, \ldots, \phi(n)-1\}$, tal que $gcd(e, \phi(n)) = 1$, es decir, que sea relativamente primo con $\phi(n)$. El símbolo $gcd()$ representa el **máximo común divisor** (*greatest common divisor*): definido como el mayor número entero que divide a dos o más números enteros sin dejar residuo alguno.
5. Utilizando el algoritmo de Euclides extendido, calcula la clave privada d, de tal manera que
$$d \cdot e \equiv 1 \mod \phi(n).$$
En otras palabras, d es el inverso módulo $\phi(n)$ de e. De ahí la condición de que $gcd(e, \phi(n)) = 1$: para asegurar que el inverso de e existe módulo $\phi(n)$, de modo que siempre hay una clave privada d.

Por supuesto, el descifrado invierte la operación de cifrado, ya que:

$$d_{k_{pr}}(c) = d_{k_{pr}}(e_{k_{pub}}(m)) \equiv (m^e)^d \equiv m^{d \cdot e} \equiv m \mod n$$

Ejemplo: generando parejas de claves para RSA

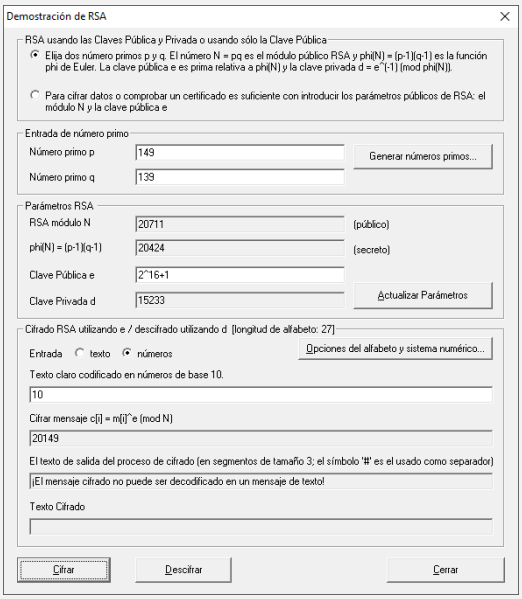
Vas a calcular los valores para la pareja de claves del ejemplo anterior.

1. Si $p = 3$ y $q = 11$, entonces $n = p \cdot q = 33$ y $\phi(n) = (p-1) \cdot (q-1) = 20$.
2. En principio, cualquier e que satisfaga la condición $gcd(e, \phi(n)) = 1$ servirá, como $e = 3$, que es relativamente primo con $\phi(n) = 20$ y menor que $\phi(n)$.
3. A continuación, calcula el exponente d encontrando el inverso de $3 \mod 20$. Puedes aplicar el algoritmo de Euclides extendido, que arroja como resultado $d = 7$. En este curso no entraré en los detalles de cómo funciona este algoritmo, pero si quieres calcular inversos multiplicativos modulares, puedes probar directamente con Sage-math mediante la función `inverse_mod(numero,modulo)`. Haz la prueba. Verás que da ese resultado.

4. La pareja de claves resultantes es por tanto $k_{pub} = (33, 3)$ y $k_{pr} = 7$.
5. Es fácil comprobar que para $m = 4$, $c = m^e = 4^3 \equiv 31 \mod 33$ y $c^d = 31^7 \equiv 4 = m \mod 33$.

PRÁCTICA 4.1: Demostración de RSA con CrypTool

1. Selecciona `Cifrar / Descifrar >Asimétrico >RSA demostración`. Se abre un cuadro de diálogo.
2. Pulsa el botón `Generar números primos`. Se abre un nuevo cuadro de diálogo.
3. Determina el límite para los valores de p y q y pulsa el botón `Generar números primos`. Si te gustan, pulsa el botón `Utilizar primos`.
4. De vuelta en el primer cuadro de diálogo, comprobarás que CrypTool ha calculado el resto de los parámetros, usando como exponente público $e = 2^{16} + 1$. Puedes cambiarlo si así lo deseas.
5. Pulsa el botón `Opciones del alfabeto y sistema numérico`, que te permite configurar cómo interpretar los datos. Por ejemplo, yo he seleccionado `Alfabeto específico` para trabajar solo con las letras de la A a la Z.
6. De vuelta en el cuadro de diálogo principal, selecciona la opción `números` y escribe un valor para el texto claro. Pulsa el botón `Cifrar` y obtendrás el resultado.



PRÁCTICA 4.2: Generando claves RSA y cifrando/descifrando con OpenSSL

Practica todo lo que hemos estado viendo usando OpenSSL:

1. Genera una pareja de claves RSA de 512 bits, volcadas a un archivo.

```
$ openssl genrsa -out private.key 512
Generating RSA private key, 512 bit long modulus (2 primes)
......++++++++++++++++++++++++++
...........++++++++++++++++++++++++++
e is 65537 (0x010001)
```

2. Visualiza el contenido del archivo con toda la información de las claves.

```
$ openssl rsa -in private.key -text -noout
modulus:
00:db:2c:c0:1f:93:9f:84:6b:d9:2a:37:03:19:50:
```

```
    5a:c3:1b:50:c3:d2:fd:a5:ef:fe:02:4b:e6:ca:56:
    a6:06:e4:db:e3:da:08:33:03:fd:16:0d:28:ad:26:
    52:8c:ac:7a:2b:1b:56:4b:c6:0e:26:3e:4d:b3:fa:
    2b:e0:d0:d9:37
    publicExponent: 65537 (0x10001)
    privateExponent:
    1e:f0:23:32:91:e1:0b:8d:39:bb:66:c6:8e:35:34:
    8a:a3:f9:73:9a:fd:e9:05:86:82:86:7a:4e:87:8a:
    aa:20:33:2f:d0:b1:5e:61:a1:d7:c9:d4:60:88:7f:
    07:87:0b:2d:ef:35:6b:e8:82:50:ff:ea:fd:72:72:
    f5:1f:ba:31
    ...
```

3. Exporta la clave pública a un archivo separado.

```
$ openssl rsa -in private.key -pubout -out public.key
writing RSA key
```

4. Genera un archivo que contenga 32 bytes aleatorios.

```
$ openssl rand -out random.file 32
```

5. Cifra dicho archivo utilizando la clave pública generada anteriormente.

```
$ openssl pkeyutl -encrypt -in random.file -out random.file.enc
-inkey public.key -pubin
```

6. Descifra el archivo usando ahora la clave privada.

```
$ openssl pkeyutl -decrypt -in random.file.enc -out random.file.dec
-inkey private.key
```

7. Verifica que el archivo descifrado es el mismo que el archivo original.

```
$ hexdump random.file
000000 66 09 b1 7d be 5d 1a 34 c7 3c 7a 28 20 c5 c0 3b
000010 a8 6e 91 ac 2a 40 94 ae 0a 8f 16 43 ef cc 78 90
$ hexdump random.file.dec
000000 66 09 b1 7d be 5d 1a 34 c7 3c 7a 28 20 c5 c0 3b
000010 a8 6e 91 ac 2a 40 94 ae 0a 8f 16 43 ef cc 78 90
```

4.3.1.3. Por sus primos los conoceréis

En el artículo The Million-Key Question – Investigating the Origins of RSA Public Keys de 2016, los investigadores estudiaron sesenta millones de claves generadas por una serie de bibliotecas de código abierto y tarjetas inteligentes de código cerrado. Analizando el byte más significativo del módulo de la clave pública, encontraron patrones distintos o *huellas dactilares* (*fingerprints*) que identifican la biblioteca criptográfica que las generó.

Estas *huellas dactilares* pueden utilizarse para identificar qué biblioteca generó una clave determinada. Los patrones subyacentes se deben a distintos enfoques de la difícil tarea de generar números primos, como ha quedado probado en las secciones anteriores. Los autores del artículo crearon mapas de calor, que muestran la distribución probable de los bits.

Estas *huellas dactilares* no son sólo una curiosidad: representan un ciberriesgo potencial. Si se descubriera una vulnerabilidad en una biblioteca popular, estas huellas podrían utilizarse para identificar claves vulnerables. Del mismo modo, las *huellas dactilares* podrían ayudar a la desanonimización cuando se utilizan servicios como Tor. Sin ir más lejos, los mismos autores encontraron la famosa vulnerabilidad ROCA de generación de clave en RSA gracias a la información sobre la biblioteca usada que filtraban las claves por ella generadas. La herramienta online BadKeys permite detectar estas claves débiles.

Puede ver más gráficos interesantes y poner a prueba sus algoritmos de identificación en su sitio web: RSABias.

4.3.2. Seguridad y ataques contra RSA

4.3.2.1. A la búsqueda de primos enormes

Hay un importante aspecto práctico de RSA difícil de implementar en protocolos reales: la generación de los primos p y q en el paso 1 de la generación de la clave. Dado que su producto es el módulo RSA $n = p \cdot q$, los dos primos deberían tener aproximadamente la mitad de la longitud de bits de n. Por ejemplo, si quieres configurar RSA con un módulo de longitud $\lceil \log_2 n \rceil = 1.024$, p y q deberían tener una longitud de bits de unos 512 bits. El enfoque general es generar números enteros al azar y comprobar si son primos. El generador de números aleatorios debe ser impredecible porque si un atacante puede calcular o adivinar uno de los dos primos, RSA puede romperse fácilmente. Se imponen dos restricciones:

1. **El generador de números debe ser aleatorio e impredecible**. Si existe algún tipo de sesgo o defecto en el generador, pueden ocurrir todo tipo de desastres. Por ejemplo, podrían ser predecibles, como ocurrió con el error de la distribución de OpenSSL de Debian, que redujo las semillas de inicialización del generador a tan solo 32.768 valores.

   O podría suceder que en algún momento se repitan dos primos, de modo que dos módulos $n_1 = p \cdot q_1$ y $n_2 = p \cdot q_2$ compartan el mismo factor p, en cuyo caso mediante el ataque Batch-GCD se pueden recuperar los primos compartidos calculando el mayor divisor común entre todos los pares de n. Encontrar el primo común en dos módulos en Sagemath es tan sencillo como calcular su máximo común divisor: `gcd(n1,n2)`.

   En 2012, dos grupos de investigación, constataron independientemente que en torno al 0,75% de certificados TLS de Internet compartía factores primos debido a una entropía insuficiente durante la generación de los primos.

   En 2016, otro trabajo mostró que el 0,4 % continuaba vulnerable. Este es el problema denunciado en 2019 por los autores del estudio sobre la supuesta vulnerabilidad de RSA que tanto ruido hizo. No es una vulnerabilidad como tal de RSA, sino un fallo de implementación. En algunos casos, incluso si dos valores n_1 y n_2 no tienen exactamente el mismo factor, pero están muy próximos, ambos n_1 y n_2 pueden factorizarse.

   El ataque ROCA (Return of Coppersmith's Attack) es una vulnerabilidad descubierta en 2017 que afecta claves RSA generadas por ciertos chips de hardware (como los de Infineon), debido a un patrón estructurado en la generación de números primos. Esto permite factorizar las claves públicas de forma eficiente utilizando una variante del algoritmo de Coppersmith, comprometiendo la seguridad de dispositivos como tarjetas inteligentes y pasaportes electrónicos. La mitigación requiere reemplazar las claves afectadas y actualizar el firmware de los dispositivos.

2. p **y** q **deben ser independientes**. Si comparten aproximadamente la primera mitad de sus dígitos, podría factorizarse n mediante el método de factorización de Fermat. Algunos programadores buscan atajar al generar primos buscando a partir del valor objetivo: por ejemplo, arrancan desde $2^{1024} + 1$ y van iterando de dos en dos comprobando si el número es primo. Un ejemplo reciente se encontró en la generación de primos de las bibliotecas SafeZone Crypto. ¡¡¡Desastre!!!

PRÁCTICA 4.3: Probando en Sagemath el ataque de Fermat contra primos próximos

El método de factorización de Fermat se basa en la observación de que todo número impar puede expresarse como la diferencia de dos cuadrados:

$$n = a^2 - b^2$$

Si n es el producto de dos primos, p y q, entonces:

$$n = \left(\frac{p+q}{2}\right)^2 \cdot \left(\frac{p-q}{2}\right)^2$$

Si $p - q < n^{1/4}$, entonces el algoritmo de Fermat para factorizar n funciona como un tiro:

1. $a = \lceil \sqrt{n} \rceil$.
2. $b = a \cdot a - n$.

3. Mientras b no sea un cuadrado perfecto:
   a) Incrementa a.
   b) $b = a \cdot a - n$.
4. El factor es $a - \sqrt{b}$.

A continuación vas a ponerlo a prueba usando Sagemath. Imagina que un programador ha generado un primo de 256 bits aleatoriamente y luego, por mala suerte o por puro desconocimiento, ha escogido un segundo primo cercano, tal que $p - q < n^{1/4}$. Concretamente, el valor del módulo generado para RSA es:
`n=0xfc1f9cc16552fedb0239d02b0b8d5f38223dd5485b10c3c6b4a46e041d67c0e86cd1`
`3c97036f4d326e0ee6d7510c0bec74300cab0719105eda2e8d8530d9b8f3`
Pega la siguiente implementación del método de Fermat para Sagemath. Si introduces el valor para `n`, comprobarás cómo te devuelve los factores en un santiamén.

```
a=ceil(sqrt(n))
b=a*a-n
while b != isqrt(b) ** 2:
      a+=1
      b=a*a-n
f1=a-sqrt(b)
f2=n/f1
[hex(f1),hex(f2)]
```

Para que la generación de claves RSA funcione en la práctica, antes se vuelve necesario responder a dos preguntas:

1. ¿Cuántos números enteros aleatorios hay que probar antes de obtener un primo? Si la probabilidad de un primo es muy pequeña, podría tomar demasiado tiempo.
2. ¿Cómo de rápido se puede comprobar si un entero aleatorio es primo? De nuevo, si la prueba es demasiado lenta, el enfoque es poco práctico.

Por fortuna para nuestros secretos, resulta que ambos pasos son razonablemente rápidos.

4.3.2.1.1. ¿Cómo de comunes son los números primos? Salta a la visa al mirar los primeros números enteros positivos que los primos se vuelven menos densos a medida que el valor aumenta:

$$2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, \ldots$$

La cuestión es si todavía queda una probabilidad razonable de que un número aleatorio con, digamos, 512 bits, sea un primo. Afortunadamente, este es el caso. La posibilidad de que un entero elegido al azar $\tilde{p}$ sea primo se deriva del famoso teorema de los números primos y es de aproximadamente $1/ln(\tilde{p})$. Si sólo se prueban los números impares, la probabilidad se duplica. Así, la probabilidad de que un número impar al azar p sea primo es

$$Pr(\tilde{p} \quad \text{es primo}) \approx \frac{2}{ln(\tilde{p})}.$$

La probabilidad de que los números enteros sean primos disminuye lentamente, proporcionalmente a la longitud del número entero. Esto significa que incluso para parámetros RSA muy largos, digamos con 4.096 bits, la densidad de los primos es aún suficientemente alta.

PRÁCTICA 4.4: Generando números primos en Sagemath

Puedes generarlos usando el comando `random_prime(lim_sup(,lim_inf))`, pasando como parámetro el límite superior o bien un límite superior y uno inferior. Por ejemplo:
`random_prime(2**128)`
ha devuelto como resultado 171200035713777282007340250195102008797. Si quieres confinarlo entre unos límite, entonces puedes usar:
`random_prime(2**128,2**127)`
que ha devuelto 206660720847999610117096771210924664933.

4.3.2.1.2. Pruebas de primalidad Para decidir si los números enteros generados aleatoriamente son primos, una primera aproximación podría ser factorizar el número en cuestión. Sin embargo, para los números usados en RSA, la factorización no es computacionalmente viable, ya que p y q son demasiado grandes. De hecho, ésa es la gracia de RSA: se eligen números que no pueden ser factorizados porque la factorización de n es el ataque más conocido contra RSA.

Sin embargo, la situación no es desesperada. En realidad, no estamos interesados en la factorización de $\tilde{p}$, sólo necesitamos la declaración de si el número que se está probando es un primo o no. Resulta que tales pruebas de primalidad son mucho más fáciles de calcular que la factorización.

Eso sí, los tests de primalidad son probabilísticos. Deciden con una probabilidad muy alta que el número es primo, pero no pueden asegurarlo. Para aumentar la seguridad, se pasan repetidas veces. Los más usados son el test de Miller-Rabin y el test de Solovay-Strassen. No es tanto un problema de seguridad, como de eficiencia. ¡Sería desastroso que p o q no fueran primos!

Puedes encontrar instrucciones, explicaciones, ejemplos y código sobre todos los asuntos tratados en esta sección en la completísima página de FactHacks.

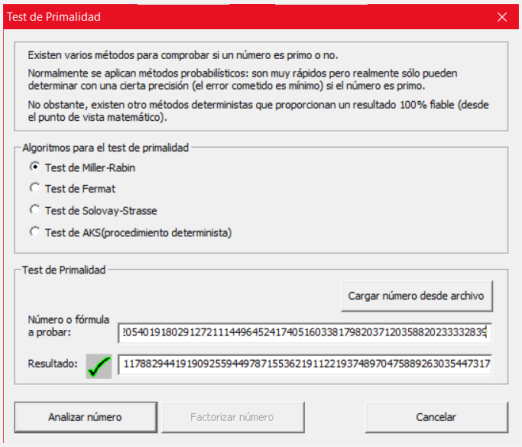
PRÁCTICA 4.5: Verificando la primalidad con CrypTool

CrypTool proporciona varias pruebas de primalidad.

1. Selecciona `Procedimiento Indiv. >RSA Criptosistema >Test de primalidad`.
2. Introduce el número entero

   ```
   1178829441919092559449787155362191122193748970475889 /
   263035447317331871097044774387209183674205401918029 1 /
   272111449645241740516033817982037120358820233332839
   ```

   y pulsa el botón `Analizar número`.
3. Experimenta con distintos tests. ¿Qué pasa si usas un test determinístico, como AKS?



PRÁCTICA 4.6: Creando primos y verificando la primalidad con OpenSSL

OpenSSL también incorpora el comando `prime` tanto para crear primos de la longitud deseada como para probar la primalidad de enteros de cualquier longitud.

1. Genera un primo del tamaño deseado.

   ```
   $ openssl prime -generate -bits 32
   3891623467
   ```

2. Comprueba si un número es o no primo.

   ```
   $ openssl prime 3891623467
   E7F5762B (3891623467) is prime
   ```

PRÁCTICA 4.7: Creando primos y verificando la primalidad con SageMath

Abre SageMath y usa el comando `is_prime(num)`, por ejemplo:
`is_prime(970757761969932065704796361105879082 37)`
que devuelve `True`.

4.3.2.2. La versión *libro de texto* de RSA es para la escuela, no para la vida real

Nunca, bajo ningún pretexto, utilices en aplicaciones reales lo que se conoce como la versión *libro de texto* de RSA, es decir, tal cual se ha explicado en la Sec. 6.2, ya que sería susceptible a todo tipo de ataques:

- Al ser determinista, dado un mensaje cifrado, podrías buscar exhaustivamente el correspondiente mensaje claro. O podrías construir un diccionario de parejas de textos claro/cifrado.
- Por el mismo motivo, es susceptible a ataques de reinyección: si un atacante intercepta paquetes cifrados con RSA, podría reutilizarlos inyectándolos en otra comunicación.
- Sucumbe a los ataques de encuentro en el medio (a no confundir con ataques de intermediario), que funcionan precalculando listas de mensaje cifrados con (n, e) y mensajes cifrados con $(n, e-1)$ y buscando colisiones entre las listas.

  RSA se utiliza habitualmente para enviar mensajes cortos que incluyen claves de cifrado para otros algoritmos, como DES o AES. Esto significa que el cifrado debe realizarse con cuidado, ya que los mensajes más pequeños pueden dar lugar a posibles vulnerabilidades. Considera enviar el mensaje $m \approx 10^{38}$, correspondiente a una clave de cifrado AES. Cifra m mediante RSA, utilizando un valor de n de aproximadamente 300 dígitos.

  $$c \equiv m^e \mod n$$

  Lo que significa que c tiene aproximadamente 300 dígitos, al igual que n, a pesar del pequeño tamaño de m, siempre y cuando e sea suficientemente grande. Eve intercepta este mensaje y ahora conoce c, n y e. Para atacar el sistema, Eve confecciona dos listas:

  1. $cx^{-e} \mod n$ para todo x tal que $1 \leq x \leq 10^{19}$.
  2. $y^e \mod n$ para todo y tal que $1 \leq y \leq 10^{19}$.

  Eve busca entonces coincidencias en la primera y segunda lista. Si se encuentra una, sabe:

  $$c \equiv (xy)^e \mod n$$

  y, por tanto, puede deducir que:

  $$m \equiv xy \mod n$$

  Aunque este ataque no siempre funcionará, demuestra que Eve podrá obtener el mensaje original (sin romper del todo el criptosistema factorizando n o encontrando d) cuando el mensaje sea igual al producto de dos valores x e y, ambos menores que 10^{19}. Muchos valores de m cumplirán este requisito y, por lo tanto, se manifiesta la vulnerabilidad del sistema cuando se envían mensajes pequeños. Una forma de evitar este tipo de ataque es alargar arbitrariamente el mensaje añadiendo y/o anteponiendo un número de dígitos aleatorios al mensaje m.

  Este ataque *meet-in-the-middle* es mucho más eficiente que intentar todos los 10^{38} valores posibles para m. Este ataque requiere el cálculo de las dos listas, por lo que hay que hacer $2 \cdot 10^{19}$ cálculos en lugar de 10^{38}.

  Si quieres profundizar en este ataque, puedes leer Why Textbook ElGamal and RSA Encryption are Insecure.

PRÁCTICA 4.8: Construyendo diccionarios de parejas de textos claro/cifrado para RSA

Supón que Alice utiliza el método RSA de la siguiente manera. Comienza con un mensaje que consiste en varias letras, codificadas como $A = 0, B = 1, \ldots, Z = 25$. Luego cifra cada letra por separado. Por ejemplo, si su mensaje es CAT, ella calcula $2^e \mod n$, $0^e \mod n$, y $19^e \mod n$. Luego envía el mensaje cifrado a Bob. En particular, Alice ha usado como clave pública ($n = 8881$,$e = 13$).
Eve intercepta intercepta un mensaje cifrado con dicho esquema:
`231 4028 476 476 699`
¿Será Eve capaz de descifrar el mensaje sin factorizar el valor de $n = 8881$?

Vas a resolver el reto usando una sencilla hoja de Excel.

1. Define las dos constantes n y e con los valores de la clave pública.
2. Escribe en una columna los números de 0 a 25.
3. En la columna contigua, escribe la fórmula para cifrar con el esquema anterior de RSA: `=RESIDUO(POTENCIA(B5;e);n)`.
4. Arrastra y tienes el diccionario calculado.

	n	e
	8881	13
	CLARO	CIFRADO
A	0	0
B	1	1
C	2	8192
D	3	4624
E	4	4028
F	5	794
G	6	2343
H	7	231
I	8	4461
J	9	4809
K	10	3556
L	11	476
M	12	2015
N	13	513
O	14	699
...		

El mensaje secreto es `HELLO`.

En resumen, para evitar estos problemas, en lugar de usar RSA tal cual, resulta crítico utilizar relleno (*padding*). Lo estudiarás en la Sec. 4.3.2.5.

4.3.2.3. Los exponentes pequeños causan problemas grandes

Típicamente se escoge uno de los siguientes valores para el exponente público: $e = 3$ (en binario, `11`), $e = 17$ (en binario, `1 0001`) o $e = 65537$ (en binario, `1 0000 0000 0000 0001`), conocidos como primos de Fermat. ¿Por qué? Porque cuantos menos unos (1's) tenga el exponente, más rápido será luego realizar las operaciones matemáticas de exponenciación, ya que se utiliza el algoritmo de exponenciación binaria.

Un valor pequeño para el exponente implica una alta velocidad de operación, pero, y éste es un GRAN PERO, acarrea graves problemas de seguridad. Es importante recalcar que todos los ataques mencionados a continuación obtienen el mensaje original m a partir del mensaje cifrado c, pero no obtienen la clave privada (d).

- Ataques de raíz e-ésima: teniendo en cuenta que un valor común para el exponente es $e = 3$, para mensajes cortos tales que $m \ll n$, podría darse el caso de que $m^e < n$, por lo que

sin más que extraer la raíz e-ésima del cifrado se recupera el mensaje original. De ahí la importancia del relleno (*padding*) para prevenirlo, que verás en la Sec. 4.3.2.5.

PRÁCTICA 4.9: Probando el ataque de raíz *e*-ésima contra RSA

Piensa en una situación totalmente corriente. Quieres cifrar una clave secreta para AES de 128 bits usando RSA de 2.048 bits, de manera que puedas compartirla con otro usuario. Pruébalo a ver qué pasa.

1. Crea aleatoriamente la clave simétrica para AES de 128 bits.

```
$ openssl rand -out secret.key 16
$ hexdump secret.key
000000 a2 18 0a cc 4e a5 06 b6 84 23 d4 d3 19 54 7d 38
```

2. Genera una pareja de claves RSA de 2.048 bits y exponente público $e = 3$.

```
$ openssl genrsa -3 -out private.key 2048
Generating RSA private key, 2048 bit long modulus (2 primes)
......+++++
e is 3 (0x03)
```

3. Cifra la clave secreta con la clave pública con RSA sin relleno usando la opción `-pkeyopt rsa_padding_mode:none`:

```
$ openssl pkeyutl -encrypt -inkey private.key -in secret.key
-out secret.key.enc -pkeyopt rsa_padding_mode:none
RSA operation error
5808:error:0406B07A:rsa routines:RSA_padding_add_none:data too
small for key size:crypto\rsa\rsa_none.c:23:
```

   Hmmm, da un error. Cuando opera sin relleno, OpenSSL solo admite valores de m de exactamente el mismo número de bytes que n; un byte de más o de menos, y no lo cifrará. La solución es sencilla, rellena con tantos ceros como necesites por delante del número m, de manera que OpenSSL se lo trague y las matemáticas sigan cuadrando. He escrito este script para PowerShell de Windows que rellena el archivo con la clave con 240 bytes a cero $(2048/8-16)$, si usas otro sistema operativo, escribe el tuyo propio:

```
Set-Content -Path .\secret.key -Value ([byte[]]::new(240) +
(Get-Content -Path .\secret.key -Raw -Encoding Byte)) -Encoding
Byte
```

4. Repite la operación de cifrado y examina el resultado del cifrado:

```
$ openssl pkeyutl -encrypt -inkey private.key -in secret.key
-out secret.key.enc -pkeyopt rsa_padding_mode:none
$ hexdump secret.key.enc
000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
...
0000c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0000d0 40 fc 6e 62 79 2a c6 82 f6 01 89 55 a6 7c 6f 22
0000e0 e9 bf 07 62 28 c9 c3 fc 2d bf 75 1f 28 ac 20 7d
0000f0 98 b5 05 d6 85 98 c5 19 c1 6b d5 99 93 dc 6e 00
```

   El archivo prácticamente sigue a ceros. La explicación es que el mensaje m (en este caso, la clave secreta) es tan corta que al elevarla al cubo no llega ni de lejos al valor de n.

5. Basta con extraer la raíz cúbica de ese valor para obtener el mensaje original. Yo he utilizado Sagemath:

```
hex(0x40FC6E62792AC682F6018955A67C6F22E9BF076228C9C3FC2DBF751F28
AC207D98B505D68598C519C16BD59993DC6E00^(1/3))
```

   Como es de esperar, el valor devuelto es la clave simétrica. ¿Te sale a ti también con tus propios valores?

- Ataques de broadcast de Håstad: si se cifra el mismo mensaje m con distintas claves públicas (n_i, e), que difieren en el valor de n, pero todas usan $e = 3$, entonces para $i \geq 3$ y según el teorema chino del resto (CRT), es posible calcular $c' = m^3 \mod n_1 n_2 n_3$, tal que $m = c'^{1/3}$. Generalizando para cualquier valor de e, se requieren e textos cifrados para obtener el mensaje original m. Para evitarlo, se requiere usar un relleno aleatorizado, no uno fijo.

PRÁCTICA 4.10: Probando el ataque de broadcast de Håstad contra RSA

Imagina que has interceptado un mensaje de correo cifrado dirigido a tres destinatarios. El cuerpo del mensaje está cifrado con AES-CBC-256, vamos, inatacable por ahí. Obviamente, para que los tres puedan descifrar el mensaje, la clave simétrica AES ha sido cifrada con la clave pública de cada destinatario, de manera que cada uno de ellos pueda descifrarla usando su clave privada respectiva. Resumiendo, tienes acceso al mensaje cifrado ($E_{k_s}(m)$); a la clave simétrica usada para cifrarlo, cifrada a su vez tres veces, c_1, c_2, c_3, una por cada usuario, con sus claves públicas respectivas; obviamente, también tienes las claves públicas, n_1, n_2, n_3, porque, como su nombre indica, son públicas, y resultan ser de 512 bits de longitud (podrían factorizarse, pero para este ejemplo, supón que son inatacables hoy por hoy); y al IV.

- Mensaje cifrado (suministrado en el archivo `email.enc` en la carpeta de prácticas):
```
000000 3f 46 d2 aa f5 41 db 90 7d ae c5 0a eb c5 63 d1
000010 b3 43 73 98 c7 6c f6 24 f9 74 a2 5f 0f 13 42 7d
000020 45 1f 15 06 9c 29 af d7 fc 80 68 0a 71 0e 24 5f
000030 df d9 da 40 46 f7 47 53 0b 41 1d 7d e4 16 fc 68
```
- IV: `84610df7db4c330be57c6f35e8c9b6fb`
- Clave simétrica cifrada con RSA para el destinatario 1:
```
c1=29c10741f5fc8b1e06e98a1adfdcf9f7ada61bdcd8e93f2e4eca02bf44fcb75fdd
3d4120d03242a14fb9e82ea0ebae016271496f97ae0fc197b021125b6be8e1
```
- Clave simétrica cifrada con RSA para el destinatario 2:
```
c2=90a1b71c29e68c96d797b6955c44f3a97a2198c7354998e4d5c60184d7fd22d7ab
ed06a8c99c2b670628a82d43c075456f5712635f76158194b8d0f895852af3
```
- Cave simétrica cifrada con RSA para el destinatario 3:
```
c3=b139aa0e35f84e674e438cb399a250220f6410c46cca55c0c3f74a00f85f88e404
1902cd92254b2135d8b0cd4ed3e8d26c1d2069e6b1e3bd4fe79418375ae534
```
- Clave pública del destinatario 1:
```
n1=D085C103705A84FDA30BA1B33CDB47CCBC3E210C3114B5B1E4E300C893D2E7D
458E9090018CDD3FEB6FAC78EA10A8649DAB0E4E202BBC6819CD2CB86B72E2E7F
```
- Clave pública del destinatario 2:
```
n2=A54B455E3117E7FB963E6AA401C642CA166A65FB156D8A48B44B5780B77A9C4
F2FCF61E76FF71A0E5FA0FB30AF61D49981E448DFADC71210F781425FE3929BC5
```
- Clave pública del destinatario 3:
```
n3=D29BD42F77098231131926A3B17A2F22EBBB7ABFE8633D3D6606BE3C2F061FB
B1EE2AB84F33E1A87E5EA404843569324A41755472007D9DA9B11E1111A22E45B
```

Con toda esta información, ¿serías capaz de obtener el mensaje original? Hay un par de detalles más que aún no había mencionado:

- El exponente público utilizado por las claves públicas resulta ser $e = 3$.
- Se ha cifrado con RSA sin ningún tipo de relleno.

¡Estos dos últimos datos suponen toda la diferencia!
Dado que $c = m^e \mod n$, el problema se reduce a resolver el siguiente conjunto de ecuaciones:

$$m^3 \equiv c_1 \mod n_1$$
$$m^3 \equiv c_2 \mod n_2$$
$$m^3 \equiv c_3 \mod n_3$$

Aplicando el Teorema chino del resto en Sagemath, la ecuación se resuelve con una sola operación:
`m=crt([c1,c2,c3],[n1,n2,n3])`
Basta con extraer la raíz cúbica de este resultado y obtendrás la solución buscada para la clave simétrica:
`m**(1/3)`
Ahora, mete todos estos valores en Sagemath y aplica el CRT. ¿Qué resultado obtienes para la clave simétrica?
Utiliza OpenSSL para descifrar el mensaje de correo con dicha clave:
`openssl aes-256-cbc -d -in email.enc -iv 84610df7db4c330be57c6f35e8c9b6fb -K [valor de la clave secreta]`
¿Qué dice el mensaje de correo original?

- Ataques de Franklin-Reiter: Franklin-Reiter identificó un nuevo ataque contra RSA con $e = 3$. Si dos mensajes m_1 y m_2 que difieren sólo en una cierta distancia (no entraremos en los detalles matemáticos de esa distancia) se cifran con el mismo módulo n, entonces es posible recuperar ambos.
- Ataques de Coppersmith: si se usan dos rellenos aleatorios para el mismo mensaje en dos ocasiones sucesivas (p.e. porque fueron interceptados y el primero no llegó a su destinatario), aunque se desconozca el relleno aleatorio, se puede recuperar el mensaje utilizando el teorema de Coppersmith, siempre que el relleno aleatorio sea corto. Concretamente, cuando $e = 3$, la longitud del relleno debe ser inferior a la novena parte de la longitud del mensaje.
- Ataque de Bleichenbacher contra las firmas: como verás en la lección 6, para verificar una firma hay que exponenciar al exponente e. De nuevo, si $e = 3$, existe un ataque muy sencillo de implementar debido al criptólogo Daniel Bleichenbacher que permite falsificar firmas RSA basadas en el esquema de relleno definido en el estándar PKCS#1 v1.5.

Es justo mencionar que con el relleno adecuado los ataques anteriores se neutralizan. Si te han picado la curiosidad los entresijos matemáticos de todos estos ataques, puedes leer una descripción relativamente asequible en el trabajo clásico de David Boneh sobre ataques contra RSA. Otra obra de referencia merecedora de una buena lectura es RSA, DH and DSA in the Wild.

4.3.2.4. El tamaño del exponente privado también importa

Aunque el proceso habitual para generar las claves de RSA es el descrito en la Sec. 4.3.1.2, algún desarrollador, presionado por la necesidad de velocidad en dispositivos con potencia de cómputo limitada, podría querer fijar primero un valor pequeño para d y calcular e después. Así agilizaría drásticamente la velocidad de exponenciación, hasta por un factor de 10.

El criptólogo Michael Wiener ideó un ataque que permite recuperar d a partir de la clave pública (n, e) siempre que $d < \frac{n^{1/4}}{3}$. Para un módulo típico de 2.048 bits, significa que la longitud de d debe ser superior a 512 bits, lo que aun así sigue siendo largo para dispositivos muy modestos, como las tarjetas inteligentes.

Si d es pequeño, entonces e es necesariamente grande. Si, contrariamente a la tradición, una clave pública (n, e) presenta un valor de e enorme ($e \gg 65.537$), puede valer la pena comprobar el ataque de Wiener contra ella.

4.3.2.5. Padding: sin relleno adecuado no hay seguridad ninguna

Para que RSA funcione, el tamaño del mensaje m no puede exceder el tamaño del módulo n. Por consiguiente, cuando sea más pequeño, habrá que rellenar m (*padding*). Pero ¿cómo? ¿Tal vez rellenar con 0's por delante del número hasta llegar a los 2.048 bits? Craso error, como comprobaste en una práctica anterior.

4.3.2.5.1. El relleno PKCS#1 v1.5 En la práctica, RSA tiene que ser usado con un esquema de *relleno*. Los esquemas de relleno son extremadamente importantes y si no se implementan adecuadamente, la implementación de RSA puede ser insegura. El relleno incrusta una estructura aleatoria en el texto claro antes del cifrado.

En PKCS#1 v1.5, el relleno se compone de las siguientes partes:

00	BT	RELLENO ALEATORIO	00	MENSAJE

- El primer byte `0x00` asegura que el bloque de cifrado, convertido en un entero, es menor que el módulo.
- El siguiente byte `BT` identifica el esquema. Para la versión 1.5, el byte tendrá el valor 00, 01 o 02. Para una operación de clave privada, el tipo de bloque será 00 o 01. Para una operación de clave pública, será 02.
- Luego viene la cadena de relleno `PS`. Si la longitud del módulo n es L bytes, `PS` consistirá en $L - 3 - |M|$ bytes. Para el bloque de tipo 00, los bytes tendrán el valor `00`; para el de tipo 01, tendrán el valor `FF`; y para el de tipo 02, estarán generados aleatoriamente y no serán cero.
- A continuación, un separador `0x00`.
- Sigue el mensaje real. Esto hace que la longitud final del bloque a cifrar sea L, exactamente al tamaño de la clave n.

PRÁCTICA 4.11: Creando tu propio relleno según la especificación PKCS#1 v1.5

Imagina que quieres cifrar una clave AES de 128 bits (16 bytes) con una clave RSA de 1.024 bits (128 bytes). ¡Necesitas relleno! ¿De qué longitud? Pues de $128 - 16 - 3 = 109$ bytes.

1. Crea una pareja de claves RSA de 1.024 bits.

   ```
   $ openssl genrsa -out private.key 1024
   ```

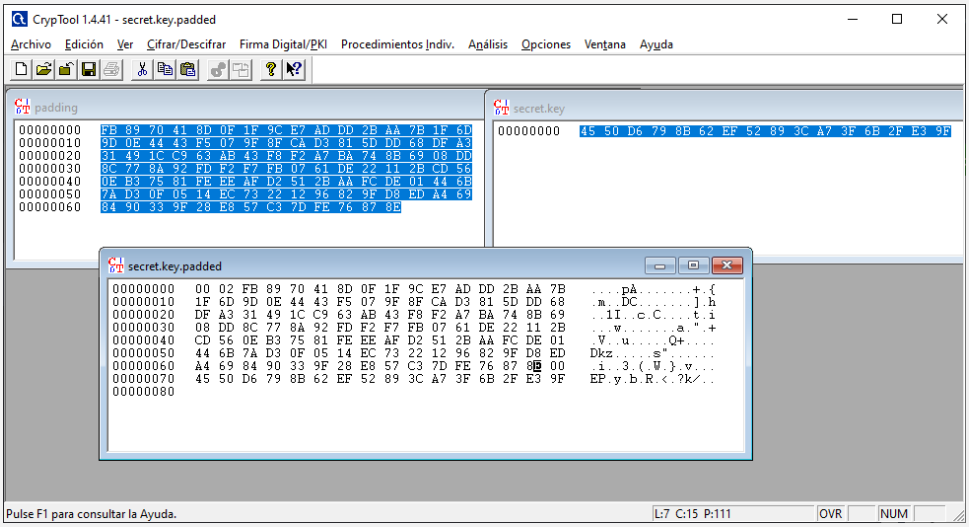
2. Crea la clave AES y asegúrate de que no contiene el byte `00`.

   ```
   $ openssl rand -out secret.key 16
   $ hexdump secret.key
   000000 45 50 d6 79 8b 62 ef 52 89 3c a7 3f 6b 2f e3 9f
   ```

3. Crea el relleno aleatorio de 109 bytes y asegúrate de que no contiene el byte `00`.

   ```
   $ openssl rand -out padding 109
   ```

4. Crea un archivo nuevo en Cryptool y visualízalo en modo hexadecimal.
5. Escribe `00 02` porque se trata de una operación de clave pública.
6. Abre el archivo con el relleno `padding` y cópialo sin codificar a continuación del `00 02`.
7. Añade `00`.
8. Abre el archivo `secret.key` y cópialo sin codificar a continuación del separador anterior `00`.
9. Guarda el resultado final como `secret.key.padded`.



10. Cífralo usando RSA ¡sin relleno! con la opción `-pkeyopt rsa_padding_mode:none`:

    ```
    $ openssl pkeyutl -encrypt -inkey private.key -in secret.key.padded
    -pkeyopt rsa_padding_mode:none -out secret.key.enc
    ```

11. Descífralo ahora usando RSA ¡con relleno PKCS#1 v1.5!:

    ```
    openssl pkeyutl -decrypt -inkey private.key -in secret.key.enc
    -hexdump -pkeyopt rsa_padding_mode:pkcs1
    0000 - 45 50 d6 79 8b 62 ef 52-89 3c a7 3f 6b 2f e3 9f
    ```

    ¡¡¡Coincide!!! Has completado con éxito tu primer relleno PKCS#1 v1.5.

12. Te propongo como ejercicio que cambies el valor de los dos primeros bytes o del separador y compruebes qué pasa.

Una forma rápida de comprobar el relleno PKCS#1 v1.5. es mediante la siguiente concatenación de comandos:

```
$ openssl rand -out secret.key 16
$ hexdump secret.key
000000 6a 04 33 84 d5 45 c4 de cd 8f f7 3b d8 e8 e4 1d
$ type secret.key | openssl pkeyutl -encrypt -inkey private.key -pkeyopt
rsa_padding_mode:pkcs1 | openssl pkeyutl -decrypt -inkey private.key
-pkeyopt rsa_padding_mode:none -hexdump
0000 - 00 02 51 19 3e 54 fc e2-31 e0 4c 70 2a 43 5a 0c
0010 - 44 e1 38 c7 a6 73 62 92-01 88 3d 7f cd 59 7a 97
0020 - 5b 59 35 4a 3f 71 9a 51-53 9d 4f 6b 10 fc b6 7b
0030 - dd b2 23 74 2e 27 79 e3-ee 69 7e 5c d4 91 2d d3
0040 - dd 6f 51 bc a5 a2 d1 bf-dd 47 38 43 17 9e 01 31
0050 - 38 31 69 63 29 97 4f a8-ac 9f 09 b4 56 61 a4 e8
0060 - 98 46 25 0a a5 6d d7 1e-a1 21 cc e0 3d 51 5b 00
0070 - 6a 04 33 84 d5 45 c4 de-cd 8f f7 3b d8 e8 e4 1d
```

El relleno detallado en la especificación PKCS#1 v1.5 resultó ser vulnerable a los ataques de Bleichenbacher. Si un atacante envía a un servidor TLS paquetes cifrados con RSA y el servidor devuelve un mensaje de error si un determinado paquete cifrado ha sido incorrectamente rellenado bajo las reglas de PKCS#1 v1.5, puede necesitar tan sólo 15.000 paquetes para obtener información suficiente para descifrar un texto cifrado con RSA o para falsificar una firma.

Y existen otras muchas versiones de este tipo de ataques contra los esquemas de relleno, como el célebre ataque ROBOT. Años después resurgió como el ataque Marvin, un ataque tan preciso que es capaz de percibir cambios en el tiempo de procesamiento del orden de unos pocos ciclos de CPU de diferencia en la respuesta, a través de una red en producción a kilómetros saltando seis enrutadores en el camino. De hecho, este tipo de ataques resulta tan insidioso y tan difícil de eliminar desde su descubrimiento en 1998, que en la nueva versión de TLS, la 1.3, se ha eliminado a RSA por completo de la ecuación para intercambio de claves, confiando en

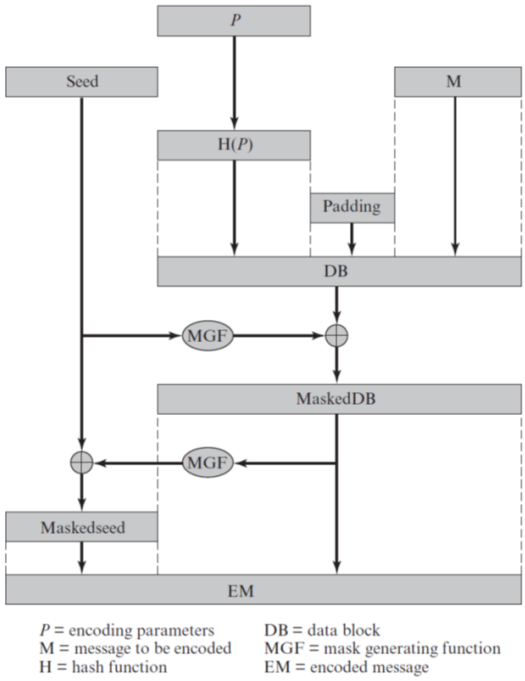


Figura 4.2: Optimal Asymmetric Encryption Padding (OAEP).

Diffie-Hellman Efímero (DHE) como único método.

PKCS#1 v1.5 presentó otros problemas. En el protocolo TLS, el cliente genera aleatoriamente una clave de sesión que cifra con la clave pública del servidor (en realidad el proceso es algo más complicado, pero lo dejaremos aquí). Cualquier persona con acceso a la clave privada correspondiente puede recuperar la clave de sesión, comprometiendo la seguridad de la sesión. El ataque no tiene que ocurrir en tiempo real. Un adversario podría almacenar todo el tráfico cifrado y esperar pacientemente hasta que obtenga la clave privada. Por ejemplo, los avances en potencia de cómputo, analizados en la Sec. 3.2.1, podrían hacer factible un ataque de fuerza bruta en el futuro. O tras la llegada de los ordenadores cuánticos podría factorizarse n. Alternativamente, la clave puede obtenerse utilizando poderes legales, coacción, soborno, irrumpiendo en un servidor que la utiliza o explotando alguna vulnerabilidad en el software TLS, al estilo de Heartbleed. Después del compromiso de la clave, es posible descifrar todo el tráfico previamente registrado.

Los otros mecanismos comunes de intercambio de claves usados en TLS no sufren este problema, por lo que se dice que exhiben secreto perfecto hacia adelante: cada conexión utiliza una clave de sesión independiente. Una clave de servidor comprometida podría utilizarse para hacerse pasar por el servidor, pero no para descifrar retroactivamente el tráfico.

4.3.2.5.2. Relleno *Optimal Asymmetric Encryption Padding* (OAEP) Entenderás que el relleno PKCS#1 v1.5 se desaconseje, salvo para retrocompatiblidad con aplicaciones antiguas. Para todas las aplicaciones nuevas se recomienda **para cifrar con RSA** el Optimal Asymmetric Encryption Padding (OAEP), especificado y estandarizado a partir de PKCS#1 v2.0). Para firmar con RSA se recomienda otro relleno, PSS, que ya estudiarás en la Sec. 6.2.1.2.

OAEP satisface los siguientes dos objetivos:

1. Agregar un elemento de aleatoriedad para convertir el esquema de cifrado determinista de RSA en modo libro de texto en un esquema probabilístico.

2. Impedir el descifrado parcial de los textos cifrados (u otras fugas de información) asegurando que un adversario no pueda recuperar ninguna porción del texto claro sin poder invertir la función unidireccional f.

PRÁCTICA 4.12: Rellenando mensajes con OAEP en OpenSSL

1. Utiliza las claves RSA y el mensaje de la práctica anterior para cifrarlo con el relleno OAEP.

```
$ openssl pkeyutl -encrypt -in secret.key -inkey private.key
-pkeyopt rsa_padding_mode:oaep -out secret.enc
```

2. Descífralo de nuevo con el mismo modo de relleno.

```
$ openssl pkeyutl -decrypt -inkey private.key -pkeyopt
rsa_padding_mode:oaep -in secret.enc -out secret.dec
```

Obviamente, el valor descifrado tiene que coincidir con el cifrado. También puedes hacerlo en una sola línea:

```
$ type secret.key | openssl pkeyutl -encrypt -inkey private.key -pkeyopt
rsa_padding_mode:oaep | openssl pkeyutl -decrypt -inkey private.key
-pkeyopt rsa_padding_mode:oaep -hexdump
0000 - 6a 04 33 84 d5 45 c4 de-cd 8f f7 3b d8 e8 e4 1d
```

4.3.2.6. Otros ataques contra RSA

Y aún quedan por mencionar los ataques contra implementaciones físicas deficientes, derivadas de la exponenciación modular.

Los ataques de temporización se aprovechan del hecho de que para descifrar hay que exponenciar con la clave privada (d) como exponente. Midiendo el tiempo empleado en descifrar se puede estimar el valor de la clave. Los más conocidos son el ataque de Kocher, el ataque de Schindler o el de Brumley-Boneh. Todos ellos pueden contrarrestarse mediante descifrado ciego: para descifrar el texto cifrado c, primero se calcula el valor intermedio $y = r^e c \mod n$, donde r es un número entero aleatorio. A continuación, y se descifra de la manera usual, seguido por la multiplicación por $r^{-1} \mod n$:

$$r^{-1}y^d = r^{-1}(r^e c)^d = r^{-1}rc^d = c^d \mod n.$$

Puesto que r es aleatorio, y es aleatorio y la medición del tiempo de descifrado de y no revela ninguna información sobre la clave privada (d). No hace falta repetir que debe usarse un número aleatorio r distinto para cada descifrado.

Por otro lado, los ataques de canal lateral obtienen información de un criptosistema a partir de sus parámetros físicos, como temperatura, consumo energético, emisiones electromagnéticas, etc. Después de todo, RSA no existe solo en los libros de matemáticas, debe materializarse en el Mundo Real. En el caso de RSA, el consumo de energía cuando está operando con un 1 es distinto que con un 0.

Por último, los ataques de glitch buscan inducir errores (*glitches*) en las operaciones del dispositivo, como por ejemplo una tarjeta inteligente. Aunque parezca mentira, en algunas implementaciones de RSA un simple error inducido puede permitir a un atacante factorizar n.

4.4. El protocolo de acuerdo de secretos de Diffie-Hellman (DHKE)

El intercambio de claves Diffie-Hellman (*Diffie–Hellman key exchange*, DHKE) es una aplicación verdaderamente impresionante del problema del logaritmo discreto a la criptografía. El propósito del algoritmo es permitir que dos usuarios intercambien de forma segura una clave que pueda utilizarse para el posterior cifrado simétrico de los mensajes. El algoritmo en sí mismo se limita al **intercambio de valores secretos, no de claves secretas**.

4.4.1. La teoría de grupos necesaria para entender el protocolo Diffie-Hellman

El protocolo de intercambio de secretos de Diffie-Hellman se basa en un campo de las matemáticas llamado *teoría de grupos*. Resulta muy similar al concepto de *intercambio de claves mezclando colores*, que tiene una buena representación visual, lo que simplifica su comprensión, como se muestra en el vídeo `VideoDiffieHellman.mp4`.

La idea básica detrás de DHKE es que la exponenciación en $\mathbb{Z}_p^*$, siendo p primo, es una función unidireccional y que la exponenciación es conmutativa, es decir,

$$k = (g^x)^y \equiv (g^y)^x \mod p.$$

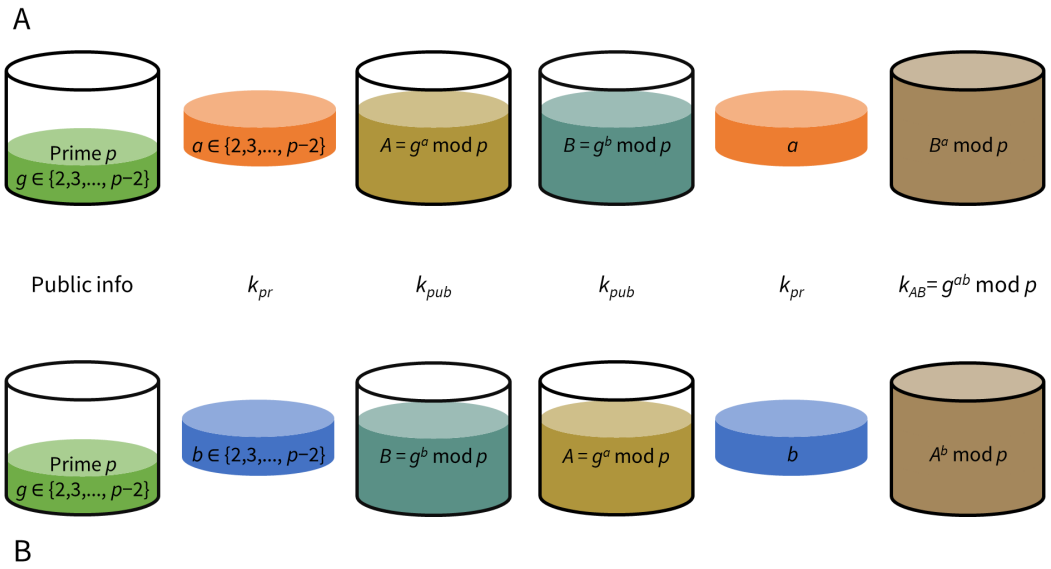


Figura 4.3: El diseño del esquema de intercambio de secretos de mezcla de colores asume que si tenemos dos líquidos de diferentes colores, puedes fácilmente mezclar los colores y obtener un nuevo color, pero la operación inversa es casi imposible: no hay manera de separar los colores mezclados de vuelta a sus componentes de color originales.

El valor $k = (g^x)^y \equiv (g^y)^x \mod p$ es el secreto conjunto que las dos partes pueden utilizar para derivar una clave de sesión. El valor g se llama *generador del grupo*. Diffie-Hellmann opera en grupos cíclicos. Todos estos grupos tienen en común que hay al menos un generador, es decir, un elemento que puede utilizarse para generar todos los demás elementos del grupo.

Ejemplo: generador de grupo en $\mathbb{Z}_{11}^*$

Considera $p = 11$ y, por tanto, el grupo $\mathbb{Z}_{11}^*$, es decir, el conjunto de números enteros $i = 0, 1, \ldots, 10$ para los cuales $gcd(i, 11) = 1$. Este es un grupo abeliano bajo la operación de multiplicación módulo 11. El número $g = 2$ es un generador de $\mathbb{Z}_{11}^*$ porque genera todos los elementos del grupo al ser exponenciado repetidamente:

$$
\begin{aligned}
g^1 &= 2 \mod 11 \\
g^2 &= 4 \mod 11 \\
g^3 &= 8 \mod 11 \\
g^4 &= 16 \equiv 5 \mod 11 \\
g^5 &= 32 \equiv 10 \mod 11 \\
g^6 &= 64 \equiv 9 \mod 11 \\
g^7 &= 128 \equiv 7 \mod 11 \\
g^8 &= 256 \equiv 3 \mod 11 \\
g^9 &= 512 \equiv 6 \mod 11 \\
g^{10} &= 1024 \equiv 1 \mod 11
\end{aligned}
$$

¿Cuántos generadores contiene un grupo $\mathbb{Z}_p^*$? Contiene exactamente $\phi(p-1)$, es decir, $p-1$. Sucede que cuando nuestro módulo p es primo, cada elemento de nuestro grupo es un generador de un subgrupo. Estos diferentes subgrupos pueden tener diferentes tamaños, que llamamos *órdenes*.

Ejemplo: subgrupos en $\mathbb{Z}_5^*$

Los diferentes subgrupos del grupo multiplicativo módulo 5 son:

- Grupo de **orden 1** generado por **1**:

$$1$$

- Grupo de **orden 4** generado por **2**:

$$2^1 = 2$$
$$2^2 = 4$$
$$2^3 = 3$$
$$2^4 = 1$$

- Grupo de **orden 4** generado por **3**:

$$3^1 = 3$$
$$3^2 = 4$$
$$3^3 = 2$$
$$3^4 = 1$$

- Grupo de **orden 2** generado por **4**:

$$4^1 = 4$$
$$4^2 = 1$$

Todos incluyen el número 1 (llamado elemento *identidad*) y tienen *órdenes* (número de elementos) diferentes.

4.4.2. Los pasos del protocolo de acuerdo de secretos de Diffie-Hellman (DHKE)

Estrictamente hablando, DHKE consiste en dos protocolos, el protocolo de *configuración* y el protocolo *principal*, el cual realiza propiamente el intercambio de secretos.

El **protocolo de configuración** consiste en los siguientes pasos:

1. Elige un primo grande p.
2. Elige un entero $g \in \{2, 3, \ldots, p-2\}$.
3. Publica p y g.

Estos dos valores a veces se denominan *parámetros de dominio*. Pueden calcularse para cada nueva sesión o estar precalculados.

Si ambas partes conocen los parámetros públicos p y g calculados en la fase de configuración, pueden generar un secreto conjunto k_s con el siguiente **protocolo anónimo de intercambio de secretos**[1]:

1. Alice selecciona un entero aleatorio $a = k_{pr_A}, a \in \{2, \ldots, p-2\}$ y calcula $A = k_{pub_A} = g^a$ mód p.
2. Bob selecciona independientemente un entero aleatorio $b = k_{pr_B}, b \in \{2, \ldots, p-2\}$ y calcula $B = k_{pub_B} = g^b$ mód p.
3. Cada parte mantiene en privado el valor k_{pr} y hace público el valor k_{pub}.
4. Alice calcula el secreto compartido como $k_s = k_{pub_B}^{k_{pr_A}} \equiv B^a$ mód p.

[1] Se llama *anónimo* porque no está autenticado: los participantes no tienen una identidad que pueda ser verificada por ninguna de las partes y ninguna de las partes posee una clave a largo plazo. En otras palabras, Alice no puede probar a Bob que ella es Alice y viceversa.

5. Bob calcula el secreto compartido como $k_s = k_{pub_A}^{k_{pr_B}} \equiv A^b \mod p$.

Estos dos cálculos producen resultados idénticos:

$$B^a \equiv (g^b)^a \equiv g^{ab} \mod p$$

$$A^b \equiv (g^a)^b \equiv g^{ab} \mod p$$

y por lo tanto ambas partes comparten el mismo secreto $k_s \equiv g^{ab} \mod p$. Este secreto puede utilizarse para establecer una comunicación segura entre Alice y Bob, por ejemplo, usando k_s para derivar una clave de sesión para un algoritmo simétrico como AES.

Ejemplo: acordando claves de sesión con Diffie-Hellman

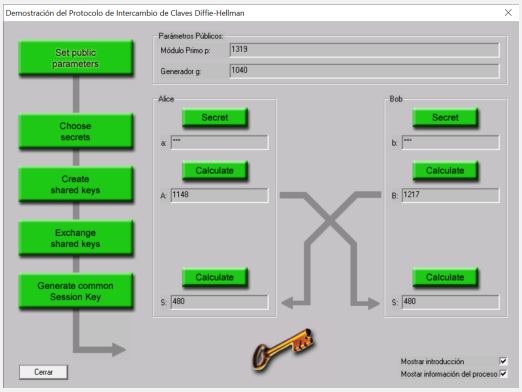
Si los parámetros de dominio son $p = 29$ y $g = 2$, el protocolo procede de la siguiente manera:

1. Alice elige $a = k_{pr_A} = 5$, mientras que Bob elige $b = k_{pr_B} = 12$.
2. Ambos calculan sus claves públicas respectivas, $A = k_{pub_A} = 2^5 \equiv 3 \mod 29$ y $B = k_{pub_B} = 2^{12} \equiv 7 \mod 29$ y las comunican abiertamente.
3. Finalmente, la clave acordada por ambas partes es $k_s = B^a \equiv 7^5 = 16 \mod 29$ o $k_s = A^b = 3^{12} \equiv 16 \mod 29$.

Como puedes observar, Alice y Bob han terminado el protocolo compartiendo el mismo valor secreto $k_s = 16$.

PRÁCTICA 4.13: Visualizando el protocolo Diffie-Hellman con CrypTool

Selecciona `Procedimientos Indiv. >Protocolos >Demostración Diffie-Hellman`. Aparecerá una nueva ventana que te irá guiando paso a paso en la selección de los parámetros de dominio y en la ejecución del protocolo.



El intercambio de claves Diffie-Hellman se ha convertido en uno de los algoritmos criptográficos más populares, que permite a los protocolos de Internet acordar una clave compartida y negociar una conexión segura. Es fundamental para muchos protocolos, como HTTPS, SSH, IPsec, SMTPS y todos los protocolos basados en TLS.

PRÁCTICA 4.14: Implementando el protocolo Diffie-Hellman con OpenSSL

1. Genera los parámetros públicos globales de DH de la longitud deseada, p.e. 1.024 bits, y guárdalos en un archivo.

```
$ openssl genpkey -genparam -algorithm DH -out dhgp.pem -pkeyopt
dh_paramgen_prime_len:1024
```

2. Muestra los parámetros generados.

```
$ openssl pkeyparam -in dhgp.pem -text -noout
DH Parameters: (1024 bit)
```

```
prime:
00:b7:9d:69:b7:90:96:8a:52:63:fc:ea:05:0d:59:
a4:29:74:1f:46:f5:19:7a:dc:c9:20:ab:00:91:f4:
df:e4:0e:7f:94:ef:67:61:ac:dd:86:d2:a3:43:95:
b2:63:49:df:88:a6:89:d0:c4:5b:7b:27:05:6d:bd:
be:de:7d:ad:c3:b6:48:42:4f:58:cc:80:e4:99:ba:
84:1d:58:fb:77:39:15:b4:c1:27:aa:31:b7:5a:4c:
29:0a:0b:80:99:b7:3e:5f:d9:14:da:a9:62:35:7f:
e4:32:5d:bc:b9:cd:d1:ed:37:74:20:c9:6c:d4:23:
5e:81:25:c8:3e:22:ef:c0:33
generator: 2 (0x2)
```

3. Alice y Bob utilizan estos parámetros públicos, p y g, para generar sus propias claves pública y privada. Primero va Alice:

```
$ openssl genpkey -paramfile dhgp.pem -out dhkeyA.pem

$ openssl pkey -in dhkeyA.pem -text -noout
DH Private-Key: (1024 bit)
private-key:
60:7c:2b:40:08:92:00:4a:cf:fb:bb:50:66:f9:c3:
...
public-key:
4b:ca:b2:cb:0c:e2:ec:47:00:8e:50:97:95:68:4b:
...
prime:
00:b7:9d:69:b7:90:96:8a:52:63:fc:ea:05:0d:59:
...
generator: 2 (0x2)
```

4. Bob utiliza los mismos parámetros públicos del archivo `dhgp.pem` para generar sus propias claves pública y privada.

```
$ openssl genpkey -paramfile dhgp.pem -out dhkeyB.pem
$ openssl pkey -in dhkeyB.pem -text -noout
DH Private-Key: (1024 bit)
private-key:
41:bf:f2:11:bd:07:ad:b3:08:d6:27:dc:ba:1b:8f:
...
public-key:
60:e6:74:cc:86:c5:71:0f:3f:70:5a:16:96:1b:f7:
...
prime:
00:b7:9d:69:b7:90:96:8a:52:63:fc:ea:05:0d:59:
...
generator: 2 (0x2)
```

5. Alice y Bob extraen la clave pública y se la pasan el uno al otro.

```
$ openssl pkey -in dhkeyA.pem -pubout -out dhpubA.pem
$ openssl pkey -pubin -in dhpubA.pem -text -noout
DH Public-Key: (1024 bit)
public-key:
4b:ca:b2:cb:0c:e2:ec:47:00:8e:50:97:95:68:4b:
...
prime:
00:b7:9d:69:b7:90:96:8a:52:63:fc:ea:05:0d:59:
...
generator: 2 (0x2)
$ openssl pkey -in dhkeyB.pem -pubout -out dhpubB.pem
$ openssl pkey -pubin -in dhpubB.pem -text -noout
DH Public-Key: (1024 bit)
```

```
public-key:
60:e6:74:cc:86:c5:71:0f:3f:70:5a:16:96:1b:f7:
...
prime:
00:b7:9d:69:b7:90:96:8a:52:63:fc:ea:05:0d:59:
...
generator: 2 (0x2)
```

6. Una vez intercambiadas las claves públicas, Alice y Bob pasan ahora a derivar el secreto compartido.

```
$ openssl pkeyutl -derive -inkey dhkeyA.pem -peerkey dhpubB.pem -out
secretA.bin
$ openssl pkeyutl -derive -inkey dhkeyB.pem -peerkey dhpubA.pem -out
secretB.bin
```

7. Por supuesto, si todo ha ido bien, ambos secretos serán idénticos.

```
$ hexdump secretA.bin
000000 57 17 2a 08 1c 00 a8 94 61 05 b9 0c 3e 5e 24 f6
...
$ hexdump secretB.bin
000000 57 17 2a 08 1c 00 a8 94 61 05 b9 0c 3e 5e 24 f6
...
```

4.4.3. Seguridad de Diffie-Hellman: no todo vale

La dificultad del problema del logaritmo discreto garantiza que no pueden deducirse las claves privadas a partir del conocimiento de la clave pública. Elegir una clave privada en DH es como elegir un índice en una lista de números producidos por un generador g. El logaritmo discreto equivale a encontrar el índice solo a partir de la clave pública:

$$g, g^2, g^3, \ldots, g^{340282366920938463463374607431768211456}, \ldots \bmod p$$

Clave pública DH

Hasta aquí la teoría. En la práctica, esta descripción (de libro de texto) de Diffie-Hellman deja un amplio número de opciones a las implementaciones: el tipo de grupo, cómo se acuerdan los grupos y la generación de exponentes. Dependiendo de cómo se implementen estas opciones, la seguridad finalmente obtenida puede variar drásticamente.

4.4.3.1. ¿Secreto compartido o clave secreta?

En la práctica, nunca se debe usar k_s directamente como clave secreta, sino como semilla para una función de derivación de claves (KDF), sobre las que aprenderás en la próxima lección.

Ejemplo: el secreto compartido no sirve como clave

Imagina, por ejemplo, que estás trabajando dentro del grupo multiplicador $\mathbb{Z}_{13}^* = \{1, 2, 3, ..,12\}$ usando $g = 2$ como generador del grupo, lo que significa que g^i abarca todos los valores de $\mathbb{Z}_{13}^*$ para $i = 1, 2, \ldots 12$: $g^1 = 2, g^2 = 4, g^3 = 8, g^4 = 13$, y así sucesivamente. Si el exponente de g es aleatorio, se obtendrá un elemento aleatorio de $\mathbb{Z}_{13}^*$, pero la codificación de un elemento de $\mathbb{Z}_{13}^*$ como una cadena de 4 bits no será uniformemente aleatoria: no todos los bits tendrán la misma probabilidad de ser un 0 o un 1. En $\mathbb{Z}_{13}^*$, siete valores tienen el 0 como su bit más significativo (los números del 1 al 7 del grupo), pero sólo cinco tienen el 1 como su bit más significativo (del 8 al 12). Es decir, este bit es 0 con una probabilidad $7/12 \approx 0,58$, mientras que, idealmente, un bit aleatorio debería ser 0 con una probabilidad 0,5. Además, las secuencias de 4 bits 1101, 1110 y 1111 no aparecerán nunca.

4.4.3.2. ¿El primer primo que encuentro vale?

Cuanto más grande el primo usado, mejor para evitar los ataques de cálculo de logaritmos. Ahora bien, no es recomendable usar cualquier valor para g y p. Para garantizar la máxima seguridad, los parámetros de DH seguros deben funcionar con un primo p de 2.048 bits (o más), tal que $(p-1)/2$ es también primo, lo que se conoce como primo seguro, que garantiza que el grupo no tiene subgrupos pequeños que volverían a DH más fácil de romper, sino que el orden del subgrupo generado por g es $(p-1)/2$ (y otro de tamaño 2 generado por -1). De hecho, existe un ataque a los intercambios de claves conocido como *ataque de subgrupo pequeño* en el que un atacante envía varias claves públicas no válidas para filtrar bits de su clave privada gradualmente, y donde las claves públicas no válidas son generadoras de subgrupos pequeños. Por este motivo siempre es una buena idea verificar que la clave pública que recibes no está en un subgrupo pequeño.

Más aún, con un primo seguro, DH puede trabajar con $g = 2$, lo que facilita calcular multiplicaciones un poco más rápidamente y resulta utilizable incluso cuando no es una raíz primitiva, ya que sigue cubriendo la mitad del espacio de residuos posibles.

En contrapartida, generar un primo seguro p consume más tiempo que generar un primo totalmente aleatorio, pero tampoco tanto. Típicamente, se parte de un primo aleatorio q de $2t$ bits de longitud para una seguridad objetivo de t bits, se calcula $p = 2q + 1$ y se verifica si este p es primo. En caso afirmativo, se utiliza como parámetro de dominio; en caso negativo, se busca otro.

PRÁCTICA 4.15: Comparando con OpenSSL el tiempo de generación de primos seguros para DH y primos aleatorios para RSA

Buscar primos seguros tiene un precio. Vas a comprobarlo con tu propio ordenador. Abre una ventana de Windows PowerShell y ejecuta el siguiente comando para que OpenSSL busque un primo seguro:

```
$ Measure-Command {openssl dhparam -noout 2048}
Generating DH parameters, 2048 bit long safe prime, generator 2
This is going to take a long time
.....................................................++++++++++*
TotalSeconds : 40,5314882
```

Compara este tiempo con el invertido en generar dos primos de la mitad de longitud en el mismo ordenador.

```
$ Measure-Command {openssl genrsa 2048}
Generating RSA private key, 2048 bit long modulus (2 primes)
......+++++
.....................................+++++
e is 65537 (0x010001)
TotalSeconds : 0,1218452
```

Otra forma de probarlo con OpenSSL es mediante el comando `prime`. Haz la prueba.

```
$ Measure-Command {openssl prime -generate -safe -bits 2048}
TotalSeconds : 14,360957
$ Measure-Command {openssl prime -generate -bits 2048}
TotalSeconds : 0,2314802
```

De hecho, la forma más rápida de obtener p y g es abstenerse de generarlos. No hay ningún problema de seguridad con Diffie-Hellman (o DSA) si reutilizas p, q y g generados previamente. Por lo tanto, puedes utilizar valores *estándar*, por ejemplo, los descritos en el RFC 2409 o en el RFC 5114.

Generar números tan grandes es computacionalmente costoso, lo que lo vuelve prohibitivo para negociar sesiones TLS, por ejemplo. Como solución suelen precalcularse o usarse parámetros bien conocidos. A modo de ejemplo, los siguientes valores están tomados de RFC 5114 **2048-bit MODP Group with 256-bit Prime Order Subgroup**:

```
   p =
87A8E61D B4B6663C FFBBD19C 65195999 8CEEF608 660DD0F2
5D2CEED4 435E3B00 E00DF8F1 D61957D4 FAF7DF45 61B2AA30
16C3D911 34096FAA 3BF4296D 830E9A7C 209E0C64 97517ABD
```

```
5A8A9D30 6BCF67ED 91F9E672 5B4758C0 22E0B1EF 4275BF7B
6C5BFC11 D45F9088 B941F54E B1E59BB8 BC39A0BF 12307F5C
4FDB70C5 81B23F76 B63ACAE1 CAA6B790 2D525267 35488A0E
F13C6D9A 51BFA4AB 3AD83477 96524D8E F6A167B5 A41825D9
67E144E5 14056425 1CCACB83 E6B486F6 B3CA3F79 71506026
C0B857F6 89962856 DED4010A BD0BE621 C3A3960A 54E710C3
75F26375 D7014103 A4B54330 C198AF12 6116D227 6E11715F
693877FA D7EF09CA DB094AE9 1E1A1597
```

g =

```
3FB32C9B 73134D0B 2E775066 60EDBD48 4CA7B18F 21EF2054
07F4793A 1A0BA125 10DBC150 77BE463F FF4FED4A AC0BB555
BE3A6C1B 0C6B47B1 BC3773BF 7E8C6F62 901228F8 C28CBB18
A55AE313 41000A65 0196F931 C77A57F2 DDF463E5 E9EC144B
777DE62A AAB8A862 8AC376D2 82D6ED38 64E67982 428EBC83
1D14348F 6F2F9193 B5045AF2 767164E1 DFC967C1 FB3F2E55
A4BD1BFF E83B9C80 D052B985 D182EA0A DB2A3B73 13D3FE14
C8484B1E 052588B9 B7D2BBD2 DF016199 ECD06E15 57CD0915
B3353BBB 64E0EC37 7FD02837 0DF92B52 C7891428 CDC67EB6
184B523D 1DB246C3 2F630784 90F00EF8 D647D148 D4795451
5E2327CF EF98C582 664B4C0F 6CC41659
```

Y como resultado el generador genera un subgrupo de orden:

q =

```
8CF83642 A709A097 B4479976 40129DA2 99B1A47D 1EB3750B
A308B0FE 64F5FBD3
```

La contrapartida de usar claves precalculadas, es que un atacante puede dedicar todos sus recursos computacionales durante años para tratar de romperlas. Millones de servidores HTTPS, SSH y VPN utilizan los mismos números primos para el intercambio de claves Diffie-Hellman. Un grupo de investigadores analizó el primo más común de 512 bits utilizado para TLS y demostró que el ataque Logjam puede utilizarse para romper conexiones en el 80 % de los servidores TLS que soportan `DHE_EXPORT`. Además, estimaron que un equipo académico puede romper un primo de 768 bits y que un estado nacional puede romper un primo de 1.024 bits. Romper el primo más común de 1.024 bits utilizado por los servidores web permitiría espiar de forma pasiva las conexiones al 18 % del millón de dominios HTTPS más importantes. Un segundo primo permitiría el descifrado pasivo de las conexiones al 66 % de los servidores VPN y al 26 % de los servidores SSH. Los autores conjeturan que la NSA ha logrado dicha ruptura.

Con el tiempo se ha ido descubriendo que algunos grupos recogidos en viejos estándares son débiles o están totalmente rotos, como en RFC 5114. Se prefiere el estándar RFC 7919.

PRÁCTICA 4.16: Ataque de subgrupos pequeños usando el algoritmo de Pohlig-Hellman en Sagemath

El algoritmo de Pohlig-Hellman permite resolver el problema del logaritmo discreto para cuerpos finitos cuyo orden puede factorizarse en potencias de primos más pequeños.
Considera un número primo de la forma $p = q + 1$, donde $q = q_1^{n_1} \cdot q_2^{n_2} \cdot q_3^{n_3} \cdots$.
El algoritmo PH permite resolver el DLP en los subgrupos más pequeños de orden $q_1^{n_1} \cdot q_2^{n_2} \cdot q_3^{n_3} \cdots$ y luego combinar las soluciones utilizando el Teorema del Resto Chino para obtener la solución del DLP original.
A continuación tienes una implementación del algoritmo PH en Sagemath para un ejemplo muy sencillo. El objetivo es obtener el valor de x tal que $h = g^x \mod p$:

```
g = 7
h=12
p=41
print('El primo p es', p)
F=GF(p)
g1=F(g)
h1=F(h)
N=p-1
qi=[r^N.valuation(r) for r in prime_divisors(N)]
print('Los divisores de p-1: ',qi)
lqi=len(qi)
```

```
Nqi=[N/q for q in qi]
gi=[g1^r for r in Nqi]
hi=[h1^r for r in Nqi]
xi=[discrete_log(hi[i],gi[i]) for i in range(lqi)]
print('Logaritmos discretos x_i=',xi)
x=CRT(xi,qi)
print('El resultado es log_g(h)=',x)
```

Este algoritmo funciona a las mil maravillas siempre y cuando los primos $q_1, q_2, \ldots$ sean pequeños en comparación con p, incluso para primos de cualquier longitud.

4.4.3.3. DH Anónimo: rápido, barato e ¿inseguro?

Por otro lado, DH anónimo puede vulnerarse con un ataque de intermediario: Mallory simplemente necesita interceptar los mensajes y pretender ante Alice ser Bob y ante Bob, ser Alice. Aprenderás cómo protegerse ante estos ataques en la lección 6.

4.4.3.4. Ataque Logjam contra el protocolo TLS

El ataque Logjam permite a un atacante intermediario rebajar las conexiones TLS vulnerables a una criptografía de grado de exportación de 512 bits. Esto permite al atacante leer y modificar cualquier dato que pase por la conexión. El ataque recuerda al ataque FREAK, pero se debe a un fallo en el protocolo TLS en lugar de una vulnerabilidad de implementación, y ataca un intercambio de claves Diffie-Hellman en lugar de un intercambio de claves RSA. El ataque afecta a cualquier servidor que soporte cifrados `DHE_EXPORT`, y afecta a todos los navegadores web modernos. El 8,4 % de los dominios *Top 1 Million* fueron inicialmente vulnerables.

Hasta ahora, todo lo contado se refiere al protocolo clásico de intercambio de secretos Diffie-Hellman en el grupo $\mathbb{Z}_p^*$, donde p es un primo. El protocolo puede generalizarse, en particular a grupos de curvas elípticas (ECC). Esto da lugar a la *criptografía de curva elíptica*, que se ha convertido en un esquema asimétrico muy popular en la práctica y tiene todas las papeletas para sustituir a RSA debido a su potencia y eficiencia. ECC se trata en la Sec. 4.5.

Para profundizar en los ataques a DH, puedes leer la obra ya mencionada RSA, DH and DSA in the Wild.

4.5. El protocolo de acuerdo de secretos de Diffie-Hellman basado en Curvas Elípticas (ECDHKE)

Resulta que el algoritmo DH, recién explicado, puede implementarse en diferentes tipos de grupos, no sólo en los grupos multiplicativos módulo un número primo. También se puede hacer un grupo a partir de curvas elípticas, un tipo de curvas estudiadas en matemáticas. La idea fue propuesta en 1985 por Neal Koblitz y Victor S. Miller, de forma independiente, y mucho más tarde, en el año 2000, fue adoptada cuando empezó la estandarización de los algoritmos criptográficos basados en curvas elípticas (ECC).

La gran ventaja de ECC es que proporciona el mismo nivel de seguridad que RSA o que DH, pero con parámetros considerablemente más cortos: aproximadamente 160-256 bits frente a 1.024-3.072 bits, como se lista en la tabla 4.2. En muchos casos, ECC tiene ventajas de rendimiento (menos cálculos) y de ancho de banda (firmas y claves más cortas) con respecto a los esquemas RSA y de logaritmo discreto (DH), estudiados en las secciones anteriores.

4.5.1. ¿Qué es una curva elíptica

Antes de adentrarte en la criptografía de curva elíptica (ECC), necesitarás equiparte con algunos conocimientos básicos sobre las matemáticas de estas curvas en sus versiones real y discreta.

4.5.1.1. Curvas elípticas sobre los números reales

Una *curva elíptica* es una curva plana definida por una ecuación de dos variables con coeficientes. Las curvas elípticas no son elipses. Se denominan así porque están descritas por ecuaciones cúbicas, similares a las utilizadas para calcular la circunferencia de una elipse. En general, las

ecuaciones cúbicas de las curvas elípticas adoptan la siguiente forma, conocida como *ecuación de Weierstrass*:

$$y^2 + axy + by = x^3 + cx^2 + dx + e$$

donde a, b, c, d, e son números reales y x e y toman valores igualmente reales. En criptografía lo normal es limitarse a las ecuaciones de la forma:

$$y^2 = x^3 + ax + b \tag{4.1}$$

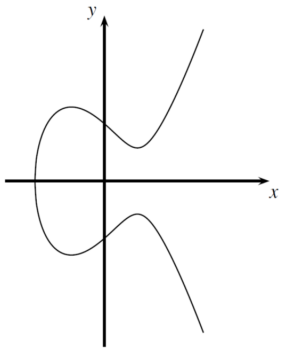


Figura 4.4: Curva elíptica.

Se dice que tales ecuaciones son *cúbicas*, o de grado 3, porque el máximo exponente que contienen es un 3. También se incluye en la definición de una curva elíptica el elemento denotado O y llamado *punto en el infinito* o *punto cero*. Para trazar tal curva, necesitas calcular

$$y = \sqrt{x^3 + ax + b}.$$

Para valores dados de a y b, el gráfico consiste en valores positivos y negativos de y para cada valor de x. Así, cada curva es simétrica alrededor de $y = 0$.

En términos geométricos, las reglas de la suma se pueden establecer de la siguiente manera:

Regla de la suma

Si tres puntos de una curva elíptica se encuentran en una línea recta, su suma es O.

A partir de esta definición, se definen las reglas de suma sobre una curva elíptica:

1. O sirve como identidad adicional. Así $O = -O$; para cualquier punto P de la curva elíptica, $P + O = P$. En lo que sigue, se asume que $P \neq O$ y $Q \neq O$.

2. El negativo de un punto P es el punto con la misma coordenada x, pero el negativo de la coordenada y; es decir, si $P = (x, y)$, entonces $-P = (x, -y)$. Estos dos puntos pueden unirse por una línea vertical. Observa que $P + (-P) = P - P = O$.

3. Para sumar dos puntos P y Q con diferentes coordenadas x, dibuja una línea recta entre ellos y encuentra el tercer punto de intersección R. Verás fácilmente que hay un único punto R, que es el punto de intersección (a menos que la línea sea tangente a la curva en P o Q, en cuyo caso se toma $R = P$ o $R = Q$, respectivamente). Se define que $P + Q$ es la imagen especular (con respecto al eje x) del tercer punto de intersección: $P + Q = -R$. La Fig. 4.5 ilustra esta construcción.

4. La interpretación geométrica del elemento precedente también se aplica a dos puntos, P y $-P$, con la misma coordenada x. Los puntos están unidos por una línea vertical, que puede verse como que también interseca la curva en el punto de infinito. Por lo tanto, $P + (-P) = O$, lo que es consistente con el elemento (2).

5. Para duplicar un punto P, dibuja la línea tangente y encuentra el otro punto de intersección S. Entonces $P + P = 2P = -S$. La Fig. 4.6 ilustra esta construcción.

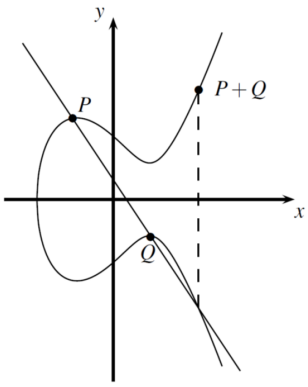


Figura 4.5: Suma de puntos sobre una curva elíptica sobre los números reales.

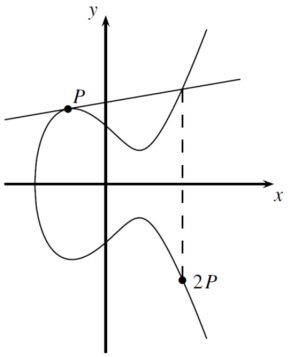
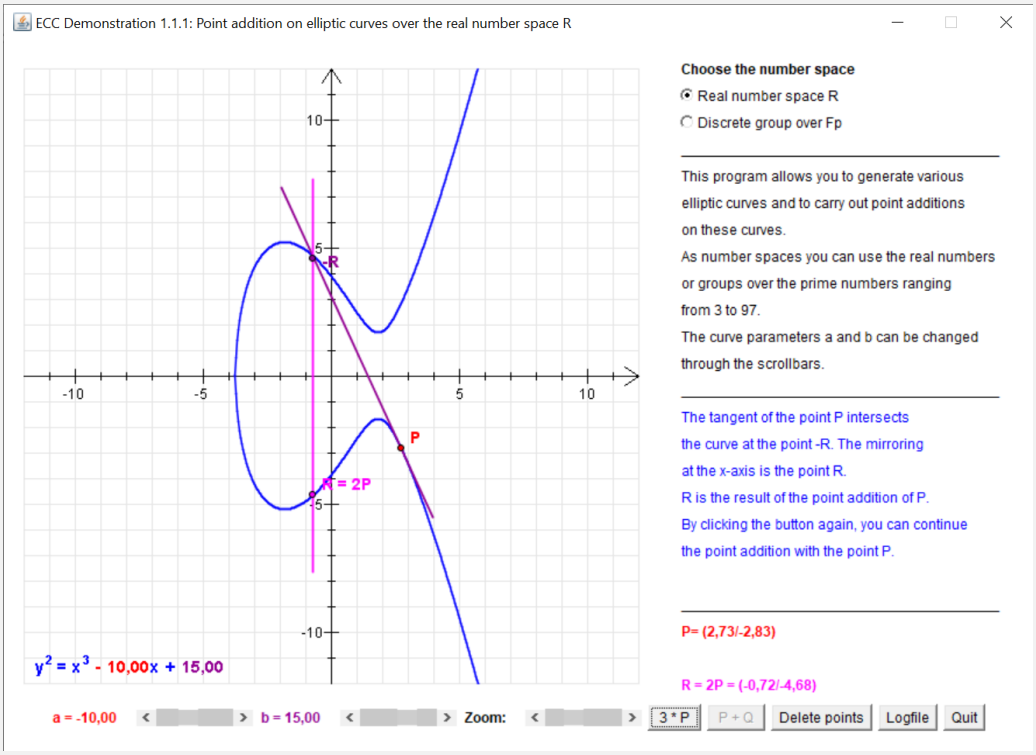


Figura 4.6: Duplicado de puntos sobre una curva elíptica sobre los números reales.

PRÁCTICA 4.17: Experimentando con las curvas elípticas en CrypTool

CrypTool incorpora una herramienta interactiva para visualizar y experimentar con la aritmética sobre curvas elípticas. Selecciona `Procedimientos Indiv. >Teoría de números - Interactiva >Suma de puntos en curvas elípticas`. Seguro que los conceptos anteriores de teoría te van a quedar muuuuuuucho más claros.



También puedes experimentar en la siguiente página web: Elliptic Curve Points.

4.5.1.2. Curvas elípticas sobre cuerpos finitos

La criptografía de curva elíptica hace uso de las curvas elípticas en las que las variables y los coeficientes se restringen a elementos de un cuerpo finito. En las aplicaciones criptográficas se utilizan dos familias de curvas elípticas, siendo las más frecuentes las curvas sobre $\mathbb{F}_p$, con una ecuación cúbica en la que las variables y coeficientes toman todos los valores del conjunto de enteros desde 0 hasta $p-1$ y en la que los cálculos se realizan módulo p, siendo p el tamaño del conjunto, también conocido como *orden*:

$$\mathbb{F}_p = \{0, 1, 2, \ldots, p-1\}$$

Por una serie de razones que se aclararán más adelante, los cuerpos finitos deben tener un orden que sea una potencia de un primo, y que los cuerpos finitos cuyo orden es primo son los que interesan en criptografía.

El problema del logaritmo discreto sobre este conjunto de puntos (ECDLP) se cree que es más difícil de resolver que el correspondiente a los cuerpos finitos (DLP). De esta manera, las longitudes de claves en criptografía de curva elíptica pueden ser más cortas con un nivel de seguridad comparable, como se muestra en la tabla 4.2.

Las curvas elípticas definidas sobre $\mathbb{F}_p$, al igual que con los números reales, se limitan a ecuaciones de la forma de la Ec. 4.1, pero en este caso con coeficientes y variables limitados a $\mathbb{F}_p$:

$$y^2 \mod p = (x^3 + ax + b) \mod p \tag{4.2}$$

Ahora considera el conjunto $E_p(a, b)$ que consiste en todos los pares de números enteros (x, y) que satisfacen la Ec. 4.2, junto con un punto en el infinito O. Los coeficientes a y b y las variables x e y son todos elementos de $\mathbb{F}_p$.

Ejemplo: curva elíptica sobre $\mathbb{F}_p$

Supón que $p = 23$ y considera la curva elíptica $y^2 = x^3 + x + 1$. En este caso, $a = b = 1$. Notarás que esta ecuación es la misma que la de la figura 4.5. La figura muestra una curva continua con todos los puntos reales que satisfacen la ecuación. Para el conjunto $E_{23}(1, 1)$, sólo nos interesan los números enteros no negativos del cuadrante desde $(0, 0)$ hasta $(p-1, p-1)$ que satisfacen la ecuación módulo p. La figura 4.7 traza los puntos

de $E_{23}(1,1)$. Observa que los puntos, con una excepción, son simétricos alrededor de $y = p/2 = 11,5$. La curva no es simétrica con respecto al eje x como en el caso de los números reales porque en un cuerpo finito no hay números negativos.

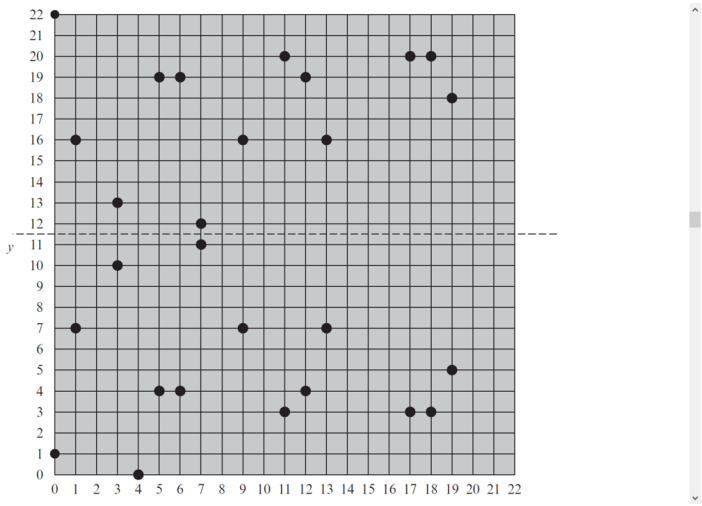


Figura 4.7: La curva elíptica $E_{23}(1,1)$.

Ejemplo: curva elíptica sobre $\mathbb{F}_{103}$

Considera la ecuación $y^2 = x^3 + 7$ sobre $\mathbb{F}_{103}$. Se puede verificar que el punto $(17, 64)$ está sobre la curva calculando ambos lados de la ecuación:

$$y^2 = 64^2 \mod 103 = 79$$

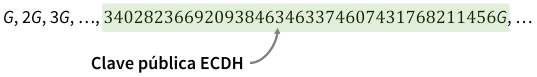
$$x^3 + 7 = 17^3 + 7 \mod 103 = 79$$

Como ejercicio, comprueba si los siguientes puntos se encuentran sobre la curva $y^2 = x^3+7$ sobre $\mathbb{F}_{223}$:
(192,105), (17,56), (200,119), (1,193), (42,99)

4.5.2. Criptografía de Curva Elíptica (ECC)

La operación de suma en ECC es la contrapartida de la multiplicación modular en RSA y la suma múltiple (multiplicación) es la contrapartida de la exponenciación modular. Para formar un sistema criptográfico utilizando curvas elípticas, solo falta encontrar un *problema difícil*, equivalente a la factorización del producto de dos primos o a la extracción del logaritmo discreto.

Considera la ecuación $Q = kP$ donde $Q, P \in E_p(a,b)$ y $k < p$. Es relativamente fácil calcular Q dado k y P, pero es difícil determinar k dado Q y P. Esta dificultad recibe el nombre de *problema del logaritmo discreto para las curvas elípticas*. Para cuerpos finitos y curvas elípticas cuidadosamente escogidos (por criptógrafos) el problema del ECDLP no tiene una solución eficiente.



A estas alturas te habrás convencido de que lo único que importa para la criptografía es contar con un grupo definido con su operación (multiplicación para DH, suma para ECC), y que el logaritmo discreto para este grupo sea difícil. La figura 4.8 muestra la diferencia entre el problema del logaritmo discreto en los dos tipos de grupos estudiados.

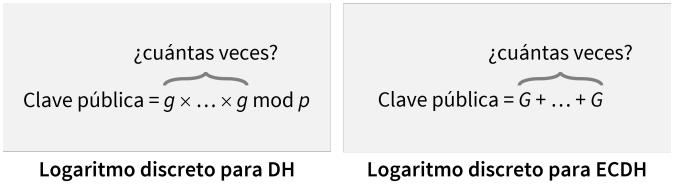


Figura 4.8: Comparación entre el problema del logaritmo discreto módulo grandes primos y el problema del logaritmo discreto en la criptografía de curva elíptica (ECC). Ambos están relacionados con el intercambio de claves DH, ya que el problema consiste en encontrar la clave privada a partir de una clave pública.

Ejemplo: logaritmo discreto para curvas elípticas

Considera el grupo $E_{23}(9, 17)$. Este es el grupo definido por la ecuación

$$y^2 \mod 23 = (x^3 + 9x + 17) \mod 23.$$

¿Cuál es el logaritmo discreto k de $Q = (4, 5)$ a la base $P = (16, 5)$? El método por fuerza bruta consiste en calcular múltiplos de P hasta encontrar Q. Así,
P = (16,5); 2P = (20, 20); 3P = (14, 14); 4P = (19, 20); 5P = (13, 10); 6P = (7, 3); 7P = (8, 7); 8P = (12, 17); 9P = (4, 5)
Dado que $9P = (4, 5) = Q$, el logaritmo discreto $Q = (4, 5)$ en la base $P = (16, 5)$ es $k = 9$. En una aplicación real, k sería tan grande como para volver computacionalmente inviable el enfoque de fuerza bruta.

PRÁCTICA 4.18: Calculando el logaritmo discreto en Sagemath

Vas a reproducir el ejemplo anterior en Sagemath. Simplemente, copia y ejecuta el siguiente código que sigue paso a paso la secuencia del ejemplo.

```
F = FiniteField(23)
C = EllipticCurve(F, [ 9, 17 ])
P=C.point((16,5))
Q=C.point((4,5))
i=1
while Q!=i*P:
    i+=1
i
```

En ECC, cuando multiplicas un punto fijo de la curva G (el punto generador) por cierto número entero k (k puede considerarse como clave privada), obtienes un punto P de la curva (su correspondiente clave pública). Por consiguiente, los elementos requeridos para operar en ECC son:

- Curva elíptica sobre cuerpo finito $\mathbb{F}_p$, definida por los coeficientes a y b.
- G es el punto generador: una constante fija, un punto base en la curva, definido por sus coordenadas G_x y G_y, siendo n el orden del grupo generado, que dicta la longitud de la clave privada. Obviamente, n tiene que ser un valor enorme para disuadir los ataques de fuerza bruta o los ataques de pequeño grupo. **No debe confundirse el *orden* del grupo generado, n, con el *orden* del cuerpo finito, p.**
- k es la clave privada (entero de 256 bits).
- P es la clave pública (punto sobre la curva, definido por sus coordenadas x e y, cada una de 256 bits).

Debido a que el algoritmo más rápido conocido para resolver el ECDLP para claves de tamaño k necesita $\sqrt{k}$ pasos, para lograr una fuerza de seguridad de k bits se necesita al menos una curva de $2k$ bits. Por lo tanto, las curvas elípticas de 256 bits (donde el tamaño de cuerpo p es un número de 256 bits) típicamente proporcionan una fuerza de seguridad de casi 128 bits. Por ejemplo, la

curva secp256k1 ($p = 256$) proporciona una seguridad de $\approx$ 128 bits (127,8 bits para ser exactos) y la curva Curve448-Goldilocks ($p = 448$) proporciona una seguridad de $\approx$ 224 bits (222,8 bits para ser exactos).

ECC en Bitcoin

Bitcoin utiliza una curva elíptica específica y un conjunto de constantes matemáticas, tal y como se define en un estándar llamado secp256k1, establecido por el NIST. La curva secp256k1 está definida por la siguiente función, que produce una curva elíptica:

$$y^2 \mod p = (x^3 + 7) \mod p$$

donde:

- $p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$, un número primo muy grande
- G_x =0x79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798
- G_y =0x483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8
- n =0xfffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141

Aunque 2^{256} es un número enorme, cualquier número inferior a 2^{256} puede almacenarse en 32 bytes, lo que permite almacenar la clave privada con relativa facilidad.

PRÁCTICA 4.19: Generando claves de curva elíptica en OpenSSL

Las dos herramientas para trabajar con ECC en OpenSSL son `ecparam` y `ec`.

1. OpenSSL soporta una gran variedad de curvas elípticas:

   ```
   $ openssl ecparam -list_curves
   ```

2. Genera una pareja de claves pública y privada para la curva secp256k1.

   ```
   $ openssl ecparam -genkey -name secp256k1 -out ECCprivate.key
   $ openssl ec -text -param_enc explicit -noout -in ECCprivate.key
   read EC key
   Private-Key: (256 bit)
   priv:
   02:f2:df:45:d8:ea:dc:b6:3d:e3:86:4f:e3:91:77:
   55:36:ad:5b:0a:ae:4a:ea:74:01:cb:ac:dd:75:59:
   3b:72
   pub:
   04:cb:c7:31:00:0d:52:7e:0f:24:43:ea:64:6b:91:
   2a:40:26:4c:30:de:22:dd:35:7b:ad:d4:80:03:cb:
   e1:3a:f9:62:65:d7:74:58:16:23:01:ba:1e:b4:ec:
   34:1d:85:9b:27:3f:9c:af:1c:21:7a:45:97:b5:5f:
   bd:ff:ce:e5:35
   Field Type: prime-field
   Prime:
   00:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:
   ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:fe:ff:
   ff:fc:2f
   A: 0
   B: 7 (0x7)
   Generator (uncompressed):
   04:79:be:66:7e:f9:dc:bb:ac:55:a0:62:95:ce:87:
   0b:07:02:9b:fc:db:2d:ce:28:d9:59:f2:81:5b:16:
   f8:17:98:48:3a:da:77:26:a3:c4:65:5d:a4:fb:fc:
   0e:11:08:a8:fd:17:b4:48:a6:85:54:19:9c:47:d0:
   8f:fb:10:d4:b8
   Order:
   00:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:
   ff:fe:ba:ae:dc:e6:af:48:a0:3b:bf:d2:5e:8c:d0:
   ```

```
    36:41:41
    Cofactor: 1 (0x1)
```

Observa que OpenSSL muestra el generador G como un solo número. En realidad, lo que hace es representar el punto de la curva como `0x04<G.x><G.y>`, de manera que el punto es realmente:

```
G=(x,y)=
(79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798,
483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8)
```

El mismo razonamiento se aplica al punto de la clave pública, representado por OpenSSL igualmente como `0x04<x><y>`:

```
P=(x,y)
(cbc731000d527e0f2443ea646b912a40264c30de22dd357badd48003cbe13af9,
6265d77458162301ba1eb4ec341d859b273f9caf1c217a4597b55fbdffcee535)
```

Para volcar la clave pública a un archivo:

```
$openssl ec -in ECCPrivate.key -pubout -out ECCPublic.key
read EC key
writing EC key
$ openssl ec -pubin -in ECCPublic.key -text -noout
read EC key
Public-Key: (256 bit)
pub:
04:cb:c7:31:00:0d:52:7e:0f:24:43:ea:64:6b:91:
2a:40:26:4c:30:de:22:dd:35:7b:ad:d4:80:03:cb:
e1:3a:f9:62:65:d7:74:58:16:23:01:ba:1e:b4:ec:
34:1d:85:9b:27:3f:9c:af:1c:21:7a:45:97:b5:5f:
bd:ff:ce:e5:35
ASN1 OID: secp256k1
```

PRÁCTICA 4.20: Creando la clave pública a partir de la clave privada en Sagemath

Copia el siguiente código para reproducir los cálculos realizados por OpenSSL en la práctica anterior. Si prefieres, puedes usar el valor de la clave privada que generaste en tu máquina y verificar que coincide con la clave pública calculada.

```
F = FiniteField(0x00fffffffffffffffffffffffffffffffffffffffffffffffffffffffefffffc2f)
C = EllipticCurve(F, [ 0, 7 ])
G=C.point((0x79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798,
0x483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8))
order = G.order()
d=0x02f2df45d8eadcb63de3864fe391775536ad5b0aae4aea7401cbacdd75593b72
P=d*G
[hex(P[0]),hex(P[1])]
```

4.5.3. Intercambio de claves Diffie-Hellman con curva elíptica (ECDHKE)

El ECDHKE se basa en la siguiente propiedad de los puntos sobre curvas elípticas:

$$(a \cdot G) \cdot b = (b \cdot G) \cdot a$$

Con dos números secretos a y b (dos claves privadas, pertenecientes a Alice y Bob) y una curva elíptica con punto generador G, Alice y Bob pueden intercambiarse sobre un canal inseguro los valores $(a \cdot G)$ y $(b \cdot G)$ (las claves públicas de Alice y Bob) y como resultado podrán derivar un secreto compartido: $(a \cdot G) \cdot b = (b \cdot G) \cdot a$.

El algoritmo ECDHKE es bien sencillo:

1. Alice genera un par de claves ECC aleatorias: $k_{pr_A}, k_{pub_A} = k_{pr_A} \cdot G$.
2. Bob genera un par de claves ECC aleatorias: $k_{pr_B}, k_{pub_B} = k_{pr_B} \cdot G$.

3. Alice y Bob intercambian sus claves públicas a través de un canal inseguro (por ejemplo, a través de Internet).
4. Alice calcula $k_s = k_{pub_B} \cdot k_{pr_A}$
5. Bob calcula $k_s = k_{pub_A} \cdot k_{pr_B}$.
6. Ahora ambos Alice y Bob tienen la misma clave compartida k_s.

PRÁCTICA 4.21: Acordando claves simétricas con OpenSSL

A diferencia de RSA, con EC no se cifra directamente, sino que se usa junto a DH para acordar claves simétricas (ECDHKE).

1. Genera sendas parejas de claves para Alice y Bob con la curva secp256k1, que ambos guardarán a buen recaudo.

```
$ openssl ecparam -name secp256k1 -genkey -noout -out
AlicePrivate.key
$ openssl ecparam -name secp256k1 -genkey -noout -out BobPrivate.key
```

2. Exporta las claves públicas a otro archivo. Como son públicas, Alice y Bob se intercambian tranquilamente estos archivos a través de un canal inseguro.

```
$ openssl ec -in AlicePrivate.key -pubout -out AlicePublic.key
$ openssl ec -in BobPrivate.key -pubout -out BobPublic.key
```

3. Alice deriva el secreto compartido a partir de su clave privada y de la clave pública de Bob.

```
$ openssl pkeyutl -derive -inkey AlicePrivate.key -peerkey
BobPublic.key -out AliceSecret
```

4. Bob deriva el secreto compartido a partir de su clave privada y de la clave pública de Alice.

```
$ openssl pkeyutl -derive -inkey BobPrivate.key -peerkey
AlicePublic.key -out BobSecret
```

5. Por supuesto, ambos secretos deben ser idénticos.

```
$ hexdump AliceSecret
000000 96 b2 b5 dc c8 65 7c a4 16 a2 ec ad e5 18 3a 94
000010 39 dc a2 b2 4d 2d dd c2 fe 76 10 f0 c8 07 9c b5
$ hexdump BobSecret
000000 96 b2 b5 dc c8 65 7c a4 16 a2 ec ad e5 18 3a 94
000010 39 dc a2 b2 4d 2d dd c2 fe 76 10 f0 c8 07 9c b5
```

6. A partir de este momento, pueden usar el secreto compartido para derivar una clave secreta con la cual intercambiarse información cifrada con un algoritmo simétrico como AES.

Encontrarás información adicional sobre la seguridad de ECDH en el trabajo ya mencionado RSA, DH and DSA in the Wild.

4.5.4. Ataques contra ECDH

En el repositorio de Github Known Attacks On Elliptic Curve Cryptography encontrarás una detallada explicación del funcionamiento de las curvas elípticas para intercambio de claves y para firma digital y una sección sobre ataques a ECDH con código en Sagemath.

4.6. Longitudes de clave y niveles de seguridad

Para comparar los diferentes algoritmos, a menudo se considera el nivel de seguridad. Se dice que un algoritmo tiene un nivel de seguridad de n bits si el ataque más conocido requiere 2^n pasos. Esta es una definición bastante natural porque los algoritmos simétricos con un nivel de seguridad de n tienen una clave de longitud n bits.

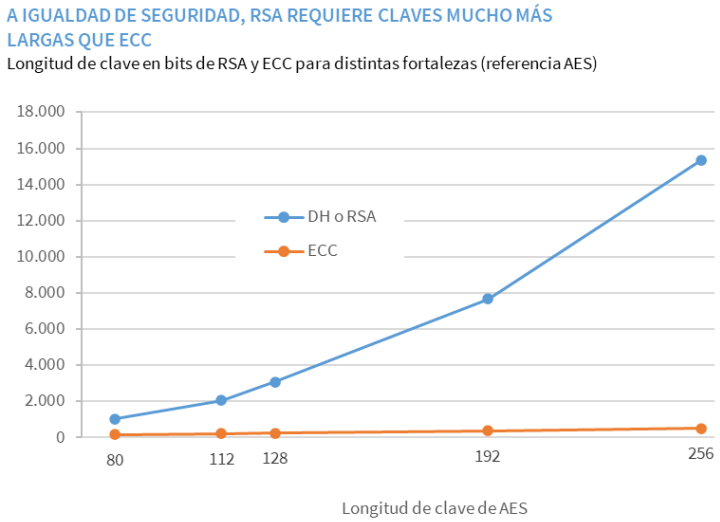


Figura 4.9: A igualdad de seguridad, RSA requiere claves mucho más largas que ECC.

Sin embargo, la relación entre la fuerza criptográfica y la seguridad no es tan directa en el caso asimétrico. La tabla 4.2 muestra las longitudes de bits recomendadas para los algoritmos de clave pública para los cuatro niveles de seguridad 80, 128, 192 y 256 bits. Verás en la tabla que los esquemas tipo RSA y los esquemas de logaritmo discreto requieren operandos y claves muy largas. La longitud de clave de los esquemas de curva elíptica es significativamente menor, pero aun así es el doble de larga que la de los cifradores simétricos con la misma fuerza criptográfica.

Familia de algoritmos	**Criptosistemas**	**Nivel de seguridad (bits)**			
Factorización de enteros	RSA	1.024	3.072	7.680	15.360
Logaritmo discreto	DH, DSA, Elgamal	1.024	3.072	7.680	15.360
Curvas elípticas	ECDH, ECDSA	160	256	384	512
Clave simétrica	AES, 3DES	80	128	192	256

Cuadro 4.2: Longitudes de bits de los algoritmos de clave pública para diferentes niveles de seguridad.

Considerando la potencia de cómputo actual, para que RSA sea seguro en la práctica se recomiendan longitudes para el módulo n de 2.048 bits o más. De hecho, desde 2020 ya se prescribe una longitud de 4.096 bits. Como penalización, las operaciones matemáticas con números tan, tan grandes se ralentizan considerablemente, especialmente en pequeños dispositivos con potencia limitada, como en IoT. RSA es dolorosamente lento.

4.7. Criptografía, aleatoriedad y ataques de distinción

La mayoría de los algoritmos estudiados, tanto de cifrado de clave pública como de clave secreta (de bloque y de flujo), producen como resultado de su ejecución un texto cifrado indistinguible estadísticamente de una secuencia aleatoria. Esta propiedad se conoce como indistinguibilidad.

En la base de todos los cifrados estudiados hay unas funciones deterministas (de lo contrario sería difícil descifrarlas) que toman como parámetro una clave y como entrada un mensaje. Una simple aplicación de las funciones deterministas sobre la clave y el mensaje conduce a un ataque de distinción elemental, ya que todos los mensajes que tienen el mismo valor se cifran en el mismo texto cifrado bajo la misma clave.

Al estudiar el cifrado en bloque se comprobó cómo el cifrado ECB sucumbía fácilmente a este ataque (v. Sec. 3.2.3). Para evitarlo, todos los esquemas de cifrado incluyen un elemento de aleatorización, que puede ser el IV de un modo de funcionamiento de cifrado por bloques o el valor aleatorio en RSA-OAEP, etc.

4.7.1. El cifrado intermitente para evadir la detección estadística del ransomware

La mayoría de las muestras de ransomware operan de forma similar: generan claves para algoritmos de cifrado simétrico y cifran con ellas todos o una selección de los archivos de la víctima, corrompiendo todos los datos. ¿Cómo detectar ransomware y detenerlo antes de que cifre los contenidos de un equipo completo?

Como acabamos de ver, todo archivo cifrado de forma criptográficamente segura aparecerá como aleatorio ante los análisis estadísticos. Por lo tanto, el problema de la detección de ransomware puede reducirse (de forma algo simplista) al problema de detectar la escritura de datos aleatorios en el sistema de archivos. Si el análisis estadístico indica que un archivo ha sido cifrado, el programa antimalware bloqueará el proceso para que no siga modificando otros archivos.

A finales de agosto de 2021 apareció Lockfile, un ransomware que funciona de forma diferente: en lugar de cifrar todo el archivo, cifra intermitentemente 16 bytes cada vez. Como resultado, un archivo de tipo documento de texto sigue siendo parcialmente legible y se parece estadísticamente al original. Para los archivos cuya estructura es importante (como un pdf), corromperá el archivo y lo hará inutilizable, aunque estadísticamente siga pareciéndose al original.

Este truco puede tener éxito contra el software de detección de ransomware basado en la inspección del contenido utilizando el análisis estadístico para detectar el cifrado. La siguiente comparación visual creada por Sophos muestra el mismo documento de texto cifrado por DarkSide, un ransomware convencional, y por el novedoso LockFile:

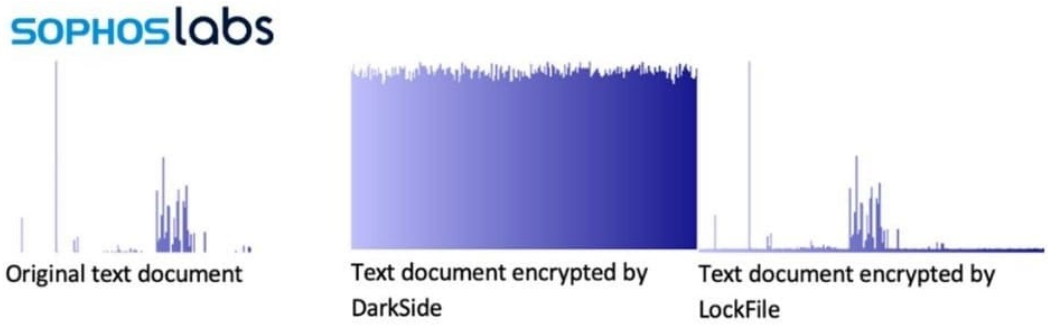


Figura 4.10: Resultado de ejecución del cifrado intermitente.

Como puede verse, el documento de texto cifrado por LockFile se parece mucho estadísticamente al original y seguramente pasaría los tests de detección estadísticos. Esta nueva vuelta de tuerca al ransomware viene a evidenciar cómo el entorno de las ciberamenazas está en perpetuo cambio. No importa cuánto avancen los sistemas de prevención, detección y respuesta, los ciberdelincuentes buscarán nuevas formas de evadir las protecciones. Así que, no olvides mantener copias de seguridad de tus archivos más valiosos para evitar pérdidas de datos ante el ransomware o ante cualquier otra amenaza.

4.8. ¿Qué algoritmos de criptografía asimétrica utiliza el ransomware?

En la tabla 4.3 se observa que todas las muestras de ransomware analizadas utilizan RSA, cuya longitud de claves oscila entre los 1.024 y los 8.192 bits.

4.9. Criptosistemas híbridos: la criptografía simétrica y la asimétrica son como la leche y el cacao en polvo

Entonces, ¿qué es mejor? ¿La criptografía de clave pública o la de clave secreta? Pues ni una ni otra. Las dos juntas, en lo que se conoce como criptosistemas híbridos:

- Dado que **los algoritmos de criptografía de clave secreta son rápidos**, con ellos se cifra el contenido, sin importar lo grande que sea: mensajes, adjuntos, conversaciones de voz y vídeo, etc. Pero presentan un gran inconveniente: ¿cómo compartir claves secretas?

Ransomware	Algoritmo
Ryuk	RSA-2048
Dharma	RSA-1024
LockBit	RSA-2048
SamSam	RSA-2048
GandCrab	RSA-2048
Clop	RSA-1024
Katyusha	RSA-2048
Snatch	RSA-2048
Phobos	RSA-1024
Nemty	RSA-8192/RSA-2048

Cuadro 4.3: Cifradores asimétricos en el ransomware moderno (Fuente: Roderik Ploszek, Peter Švec and Patrik Debnár, Analysis of encryption schemes in modern ransomware, Rad Hazu. Matematicke Znanosti, 25 (546), 1-13, 2021. https://doi. org/10.21857/mn1qgc58gy).

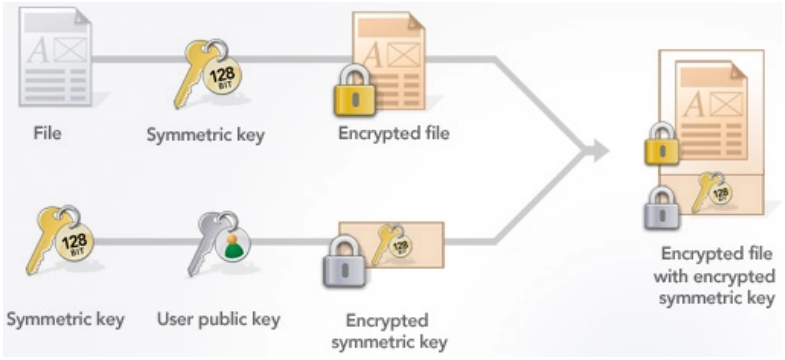


Figura 4.11: Cifrado híbrido usando criptografía asimétrica y simétrica.

- Como **los algoritmos de criptografía de clave pública son lentos**, con ellos se cifra la clave secreta utilizada por los algoritmos de cifrado anteriores, que recordarás que eran cortitas, y así pueden compartirse entre los interlocutores sin peligro de ser interceptadas.

4.9.1. Esquemas de cifrado y métodos de encapsulado de claves

Cuando trabajan mano con mano ambos tipos de criptografía, el mensaje a enviar m se cifra usando una clave secreta de sesión k_s (generada aleatoriamente, ni lo dudes): $c = E_{k_s}(m)$. Esta clave secreta a su vez se cifra con la clave pública del destinatario: $E_{k_{pub}}(k_s)$. Ya que compartir una clave simétrica no es seguro, ésta es diferente para cada sesión, de ahí su nombre. El destinatario descifra la clave secreta de sesión con su propia clave privada y con ella descifra el mensaje original. El proceso se ilustra para el correo electrónico de forma simplificada en las Figs. 4.11 y 4.12. Para una descripción aún más sencilla, también puedes ver el vídeo `VideoCriptografiaHibrida.mp4`.

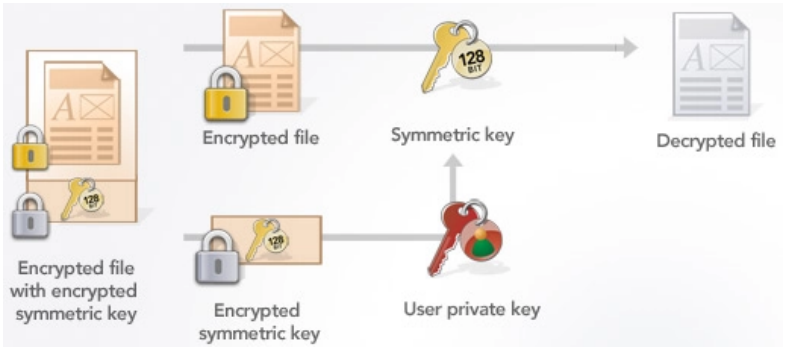


Figura 4.12: Cifrado híbrido usando criptografía asimétrica y simétrica.

4.9.1.1. Esquemas de cifrado integrados

Los esquemas de cifrado integrado (IES) son esquemas modernos de cifrado de clave pública, que combinan cifrados simétricos, asimétricos y algoritmos de derivación de claves para proporcionar un cifrado seguro basado en la clave pública (PKE). En el esquema IES se utilizan algoritmos asimétricos (como RSA o ECC) para cifrar o encapsular una clave simétrica, utilizada posteriormente por los cifradores simétricos (como AES o ChaCha20) para cifrar el mensaje de entrada. Algunos esquemas de IES proporcionan también la autenticación del mensaje. Ejemplos de esquemas IES son el DLIES (Esquema de Cifrado Integrado de Logaritmo Discreto) y el ECIES (Esquema de Cifrado Integrado de Curva Elíptica).

4.9.1.2. Mecanismos de encapsulado de claves (KEM)

Los mecanismos de encapsulado de claves (KEM) son técnicas criptográficas asimétricas que se utilizan para cifrar y encapsular una clave secreta (denominada *clave simétrica efímera*), empleada para cifrar un mensaje de entrada utilizando un cifrado criptográfico simétrico. El KEM encapsula la clave de cifrado simétrica efímera como parte del mensaje cifrado, cifrándola con la clave pública del destinatario. En criptografía, este proceso se conoce como *encapsulado de claves*.

Por supuesto, una clave simétrica nunca debería cifrarse utilizando RSA sin relleno y con un exponente pequeño, porque podría darse el caso de que el resultado del cifrado fuera la propia clave secreta. En lugar de usar relleno, KEM utiliza el siguiente esquema.

Se genera primero un entero m aleatorio, $1 < m < n$, y se deriva una clave simétrica k mediante $k = KDF(m)$, donde KDF es una función de derivación de clave, como por ejemplo una función hash. A continuación, se calcula el texto cifrado c correspondiente a m:

$$c = m^e \mod n$$

La clave simétrica k se recupera a partir de c descifrando en primer lugar el texto anterior:

$$m = c^d \mod n$$

Y luego derivando el mismo valor: $k = KDF(m)$. Este método elimina la complejidad del esquema de relleno y las pruebas necesarias para demostrar que el relleno es seguro.

El resultado de un esquema de cifrado híbrido basado en KEM consiste en un bloque KEM, que contiene la clave simétrica cifrada encapsulada (o ciertos parámetros utilizados para derivarla), y un bloque DEM (mecanismo de encapsulación de datos), que contiene los datos cifrados simétricamente encapsulados (`parámetros de cifrado + texto cifrado + opcionalmente una etiqueta de autenticación`).

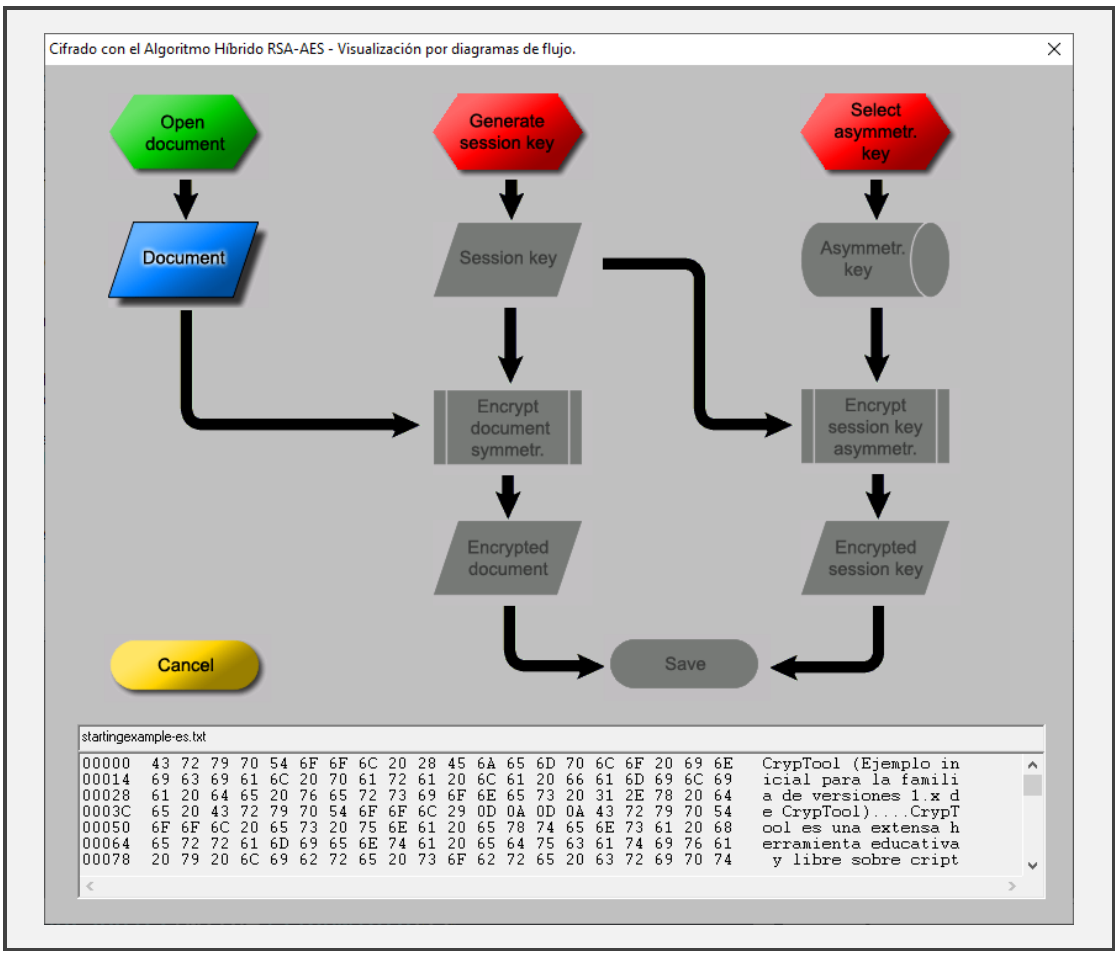
Los mecanismos de encapsulado de claves (KEM) se utilizan en los esquemas de cifrado híbridos y en los esquemas de cifrado integrados, en los que se genera un elemento aleatorio en el criptosistema de clave pública subyacente y se obtiene una clave simétrica a partir de este elemento aleatorio mediante hashing. Este enfoque simplifica el proceso de combinar el cifrado asimétrico y simétrico. Ejemplos de mecanismos modernos de encapsulado de claves son: RSA-KEM, ECIES-KEM y PSEC-KEM.

El encapsulado de claves no debe confundirse con la envoltura de claves.

- El encapsulado de claves (KEM) se refiere al cifrado con clave pública de otra clave (simétrica o asimétrica). Se utiliza para crear esquemas de cifrado híbridos de seguridad probada, por ejemplo, para cifrar una clave secreta AES mediante una clave pública ECC dada.
- La envoltura de clave se refiere al cifrado de clave simétrica de otra clave (que puede ser una clave simétrica o asimétrica). Se utiliza para cifrar, proteger la integridad y transportar claves criptográficas. La envoltura de la clave proporciona protección de la privacidad e integridad de los datos especializados, como las claves criptográficas, sin el uso de *nonces*. Los detalles están estandarizados en el Advanced Encryption Standard (AES) Key Wrap Algorithm (RFC 3394).

PRÁCTICA 4.22: Cifrando mensajes mediante criptografía híbrida con CrypTool

Selecciona `Cifrar / Descifrar >Híbrido >RSA-AES Cifrar` y abre un archivo de texto cualquiera. A partir de ahí, sigue las instrucciones paso a paso.



4.9.2. Secreto perfecto hacia adelante gracias a las claves efímeras

Una clave de sesión es por naturaleza algo efímero: se utiliza para una única sesión de comunicación y luego se descarta permanentemente, para no volver a ser utilizada nunca. Cada vez que se establece una nueva comunicación, se crea un nuevo conjunto de claves. Las claves no se almacenan ni se graban en ningún lugar. Una vez que todos los datos han sido cifrados, se destruyen. Bueno, deberían destruirse, sobrescribiendo la memoria para borrar todo rastro de las mismas. En el caso de WannaCry, por ejemplo, los analistas pudieron recuperar los números primos p y q utilizados para generar las claves públicas. Con Bad Rabbit, las claves quedaban en memoria y se podían leer siempre que no se reiniciara la máquina. CryptoDefense también dejaba rastro de las claves de cifrado usadas.

PRÁCTICA 4.23: Cifrando mensajes mediante criptografía híbrida con OpenSSL

Usando OpenSSL, en esta actividad reproducirás (de manera aproximada) los pasos para enviar un mensaje de correo electrónico a otra persona, llamada Gon.

1. Genera aleatoriamente una clave secreta para AES de 128 bits (16 bytes) y un vector de inicialización IV de 128 bits (16 bytes).

```
$ openssl rand -hex -out iv 16
$ openssl rand -hex -out secret.key 16
$ type secret.key
19941e6371284e15ca0598008fa20498
$ type iv
3e06a051ee17781ff0e84323cfe06bbf
```

2. Cifra el archivo adjunto `GON.txt` mediante el algoritmo AES en modo CBC usando la clave y el IV generados en el paso anterior.

```
$ openssl aes-128-cbc -e -in gon.txt -out gon.enc -iv
3e06a051ee17781ff0e84323cfe06bbf -K 19941e6371284e15ca0598008fa20498
```

```
-nosalt
```

3. Genera una pareja de claves pública y privada para Gon del algoritmo RSA de 1.024 bits de longitud. Este archivo pertenece a Gon y nunca nadie más debería tener acceso al mismo, ya que contiene su clave privada.

   ```
   $ openssl genrsa -out gon.private.key 1024
   ```

4. Exporta la clave pública de Gon a un archivo. Este archivo puede publicarse para que cualquier persona pueda cifrar con su clave pública.

   ```
   $ openssl rsa -in gon.private.key -pubout -out gon.public.key
   ```

5. Cifra la clave secreta de AES mediante la clave pública de Gon recién generada.

   ```
   $ openssl pkeyutl -encrypt -in secret.key -out secret.key.enc -inkey
   public.key -pubin
   ```

6. Imagina que has enviado a Gon el mensaje cifrado mediante AES junto con la clave cifrada mediante RSA. El IV puedes enviarlo en claro. Gon descifra la clave secreta de AES mediante su clave privada de RSA.

   ```
   $ openssl pkeyutl -decrypt -in secret.key.enc -out secret.key.dec
   -inkey private.key
   ```

7. Gon descifra el mensaje original mediante la clave secreta de AES recién descifrada y el IV.

   ```
   $ type secret.key.dec
   19941e6371284e15ca0598008fa20498
   $ openssl aes-128-cbc -d -in gon.enc -out gon.dec.txt -iv
   3e06a051ee17781ff0e84323cfe06bbf -K 19941e6371284e15ca0598008fa20498
   -nosalt
   ```

8. Verifica que el archivo descifrado coincide con el archivo original.

Has experimentado con tus propias manos cómo puedes generar aleatoriamente una clave simétrica, cifrarla con RSA con la clave pública del destinatario y enviársela. Gracias a este protocolo de intercambio de claves, ambas partes compartirán una clave de sesión de forma segura, siempre que el protocolo de intercambio siga ciertas reglas. Incluso hay formas de hacer que ambas partes contribuyan a una clave. Cada parte podría enviar a la otra algunos datos aleatorios y la concatenación de ambos podría ser alimentada a una función de hash para producir la clave de sesión.

Desafortunadamente, el transporte de la clave RSA no proporciona una propiedad realmente fantástica llamada secreto perfecto hacia adelante.

Secreto perfecto hacia adelante

Incluso si una clave resulta comprometida con el tiempo, no revela nada sobre comunicaciones **pasadas**.

¿Echabas en falta a Alice, Bob y Eve? Alice y Bob ya asumen que Eve está interceptando todo lo que se transmite. Precisamente, por este motivo están cifrando la transmisión, ¿no? Así, después de que la transmisión se completa, Eve tiene una copia del texto cifrado que aún no puede descifrar. Pero, en lugar de tirarlo a la basura, Eve lo archiva en un almacén.

Ahora supón que mucho tiempo después, años más tarde, Eve encuentra a Bob. Si Eve logra obtener la clave privada de Bob, ¡puede volver a su copia de la transmisión anterior de Alice y descifrarla! La clave privada de Bob seguirá descifrando las claves de la sesión hasta el final de los tiempos y Eva podrá entonces descifrar toda la transmisión.

En cambio, si un protocolo exhibe secreto hacia adelante, Eve nunca puede recuperar los datos de una sesión terminada sin importar si se hace con las claves privadas en el futuro. Obviamente, el secreto hacia adelante no es posible cuando las claves de sesión se envían directamente cifradas con RSA, porque una vez que la clave privada RSA se ve comprometida, cualquier dato registrado de sesiones anteriores pasa a ser vulnerable.

Con Diffie-Hellman (DH) y Diffie-Hellman de curva elíptica (ECDH), el secreto hacia adelante se logra mediante el uso de claves efímeras. Se genera un nuevo par de claves efímeras para cada

nueva operación de acuerdo de secretos y luego se desechan. El secreto generado también es efímero y se tira después de cada sesión. Debido a que DH y ECDH se utilizan típicamente de esta manera, a menudo se añade una E al final de la sigla (DHE o ECDHE).

Ahora que se utiliza un nuevo par de claves para cada intercambio, comprometer una única clave asimétrica sólo revela una única clave simétrica y, en consecuencia, una única sesión de comunicaciones. Y cuando las claves privadas efímeras de DH y ECDH se eliminan adecuadamente, no queda ninguna clave que Eve pueda comprometer y no hay forma de que pueda descifrar estas sesiones. En cierto modo, es similar al viejo truco del espía de tragarse el papelito con la información secreta.

Queda un escollo por salvar: los métodos de Diffie-Hellman no tienen autenticación. Debido a que las claves son completamente efímeras, no hay forma de vincularlas a una identidad; no se sabe con quién se está hablando. Recuerda que más allá de la comunicación confidencial, es imprescindible saber con quién te estás comunicando confidencialmente. Por sí mismo, el DH y el ECDH no proporcionan ninguna de esas garantías.

Por esta razón, muchos intercambios de claves de DH y ECDH también requieren una clave pública a largo plazo, de RSA o de ECDSA, y esa clave suele estar protegida por un certificado firmado. Estas claves a largo plazo, sin embargo, nunca se utilizan para el cifrado o el transporte de claves y no se utilizan en el intercambio real de datos de claves de ninguna manera. Su único propósito es establecer la identidad de la otra parte, generalmente firmando algunos de los efímeros DH/ECDH intercambiados y asegurar su frescura mediante algún tipo de desafío o *nonce*.

Todo este asunto de las firmas y la autenticación lo verás en profundidad en la última lección.

4.9.3. Estudio de caso #1: cifrado extremo-a-extremo de Whatsapp

Cuando instalas Whatsapp en tu dispositivo, se genera una pareja de claves pública y privada, llamada `Identity Key Pair`, para usar con un algoritmo de criptografía de curva elíptica. A continuación, se genera una segunda pareja de claves, `Signed Pre Key`, firmada con la clave privada de la pareja anterior. Por último, se genera un lote de parejas de claves, `One-Time Pre Keys`, para su uso posterior, y se irán renovando a medida que se agoten.

Cuando te registras con el servidor de Whatsapp, además de tu información de contacto, le envías la parte pública de tu `Identity Key`, la parte pública de tu `Signed Pre Key` y las partes públicas de tu lote inicial de `One-Time Pre Keys`.

Importante: la parte privada de tus claves nunca abandona tu dispositivo y ni Whatsapp ni nadie tiene acceso a ellas (en principio).

Cuando le envías un mensaje a otro usuario de Whatsapp, se produce el siguiente proceso:

1. Pides y el servidor de Whatsapp te envía la parte pública de `Identity Key`, `Signed Pre Key` y de la primera `One-Time Pre Keys` del otro usuario.

2. Tu terminal acuerda con el del otro usuario un secreto maestro derivado a partir de todas estas claves públicas intercambiadas entre ambos. Para ello se utiliza el protocolo de acuerdo de claves ECDH. Esta clave maestra se utiliza para crear mediante un algoritmo de HKDF una `Root Key` y las `Chain Keys` para cifrar posteriormente los mensajes intercambiados con un algoritmo de clave secreta.

3. Cuando a tu amigo le llegue tu primer mensaje, se repetirá el proceso: recibirá la información pública de tus claves; con esa información y la de sus propias claves regenerará el secreto maestro y, a partir del mismo, la `Root Key` y las `Chain Keys`. Finalmente, borra la `One-Time Pre Key` utilizada en el proceso, ya que, como su nombre indica, estas `One-Time Pre Keys` se usan una sola vez. A partir de este momento, ya está todo listo para empezar a cifrar la información intercambiada.

Los mensajes y sus adjuntos se cifran mediante el algoritmo AES en modo CBC, con claves de 256 bits de longitud y HMAC-SHA256 para autenticación. Vamos, criptografía robusta donde la haya.

Estas claves son efímeras: se derivan de las `Chain Keys`, las cuales se actualizan tras cada nuevo mensaje, a partir de su valor previo y de la `Root Key`. Para cifrar cada mensaje intercambiado se crea su correspondiente clave efímera y, una vez usada, ni se reutiliza ni puede reconstruirse. Proporcionan así el ya mencionado secreto hacia adelante: el compromiso de una clave no compromete la seguridad de las claves usadas con anterioridad y no permite por tanto descifrar los mensajes anteriores.

Si te interesa el asunto, te animo a leer el documento técnico completo.

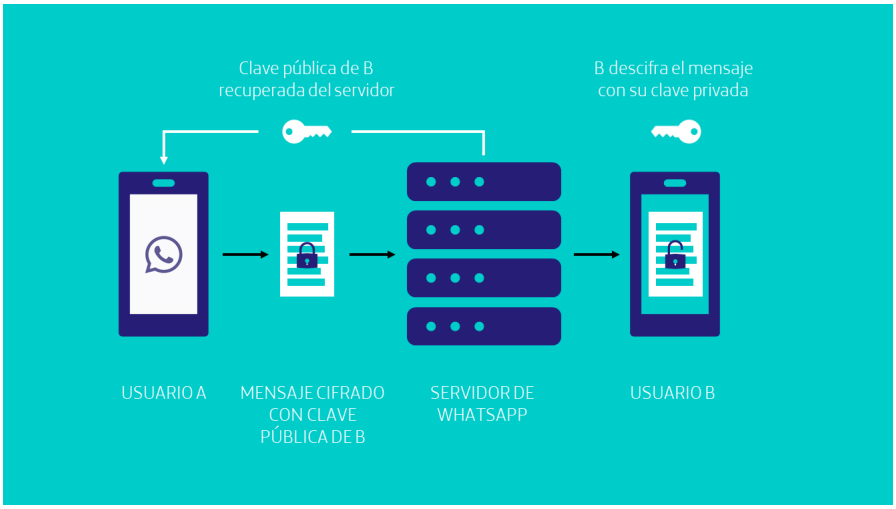


Figura 4.13: Esquema simplificado del cifrado extremo-a-extremo de Whatsapp.

4.9.4. Estudio de caso #2: ransomware

La criptografía detrás de las primeras muestras de ransomware fue simplista e ingenua. Típicamente, el ransomware cifraba cada archivo con una clave diferente de AES (o algoritmos débiles, de todo había), pero todas las claves AES se almacenaban en el sistema en un archivo. El descifrador del ransomware podía encontrar fácilmente las claves y descifrar cada archivo, pero también lo podían hacer los investigadores de seguridad mediante *reversing*. Si no quieres que alguien descifre un archivo, es una mala idea dejar las claves tiradas por ahí (bajo el felpudo, por así decirlo).

Los autores de ransomware se volvieron lógicamente hacia el cifrado asimétrico en busca de solución. La ventaja más evidente de la criptografía asimétrica es que una clave pública puede estar en el sistema de la víctima y su correspondiente clave privada puede estar en otro lugar. Ahora bien, por las razones de eficiencia ya mencionadas en esta lección, los archivos no pueden cifrarse con RSA directamente.

En cambio, el ransomware podría cifrar todas las claves AES individualmente. Después de todo, una clave AES es sólo 16 bytes para el AES-128 y 32 bytes para el AES-256. Cada clave puede cifrarse fácilmente con RSA antes de ser almacenada en el sistema de la víctima. RSA las cifra con la clave pública, así que mientras la clave privada no esté disponible para la víctima, no podrá descifrar las claves AES.

Existen dos variantes ingenuas de este enfoque, ambas problemáticas.

1. El primer enfoque consiste en generar el par de claves con antelación y codificar la clave pública en el propio malware. Una vez que el malware ha cifrado todas las claves AES con la clave pública, la víctima tiene que pagar el rescate para que se le envíe la clave privada para su descifrado. El fallo obvio de este diseño es que la *misma* clave privada desbloquearía todos los sistemas atacados por el ransomware, ya que cada copia del malware incorporaba la misma clave pública.

2. El segundo enfoque consiste en que el ransomware genere el par de claves RSA en el sistema de la víctima y transmita la clave privada al servidor de comando y control (C&C). Ahora una clave pública única por cada muestra de malware cifra las claves AES. Cuando el atacante libera la clave privada para descifrar, sólo desbloquea los archivos de la víctima específica. El problema aquí es que el sistema tiene que estar *en línea* para deshacerse de la clave privada y muchos sistemas de monitorización de red detectarán transmisiones a IPs de riesgo, desde las cuales operan los servidores de C&C. La mera transmisión de la clave privada podría revelar la presencia del ransomware antes incluso de que haya empezado a cifrar los archivos del sistema. Es más sigiloso hacerlo todo en local, hasta que el sistema esté completamente bloqueado.

El ransomware moderno resuelve todos estos problemas con un enfoque bastante inteligente.

1. El atacante genera en su propio sistema una pareja de claves asimétricas para uso a largo plazo, a las que llamaremos claves asimétricas *permanentes*.

2. El atacante crea un malware y codifica la clave pública *permanente* en el malware. Cuando el malware se activa en la máquina de la víctima, lo primero que hace es generar en local un nuevo par de claves asimétricas, a las que llamaremos el par de claves *local*. Cifra inmediatamente la nueva clave privada *local* generada usando la clave pública *permanente* del atacante, incrustada en el malware. Antes de seguir, elimina la clave privada *local* no cifrada.
3. El malware comienza a cifrar los archivos del disco utilizando AES-CTR o AES-CBC. Cada archivo se cifra con una clave diferente, generada aleatoriamente, y luego cada clave se cifra con la clave pública local. La versión sin cifrar de cada clave simétrica se destruye tan pronto como el archivo termina de ser cifrado.
4. Una vez concluido todo el proceso, los archivos de la víctima han acabado cifrados con claves AES que a su vez están cifradas con la clave pública *local*. Estas claves AES se pueden descifrar con la clave privada RSA *local*, pero esa clave se guarda cifrada con la clave pública *permanente* del atacante y la correspondiente clave privada no está en la máquina de la víctima, sino en la del atacante.
5. El atacante contacta con la víctima y exige el rescate. Si la víctima acepta y paga el rescate (normalmente a través de Bitcoin), el atacante proporciona algún tipo de código de autenticación al malware. El malware transmite la clave privada *local* cifrada al atacante. Utilizando su clave privada *permanente*, descifra la clave privada *local* y la envía de vuelta a la víctima. Ahora se pueden descifrar todas las claves AES y, con ellas, todos los archivos cifrados de esa máquina (pero de ninguna otra).

Lo inteligente de este protocolo es que el atacante nunca revela su clave privada *permanente*. Sigue siendo privada. El atacante se limita a descifrar una clave privada secundaria para que la víctima la use para desbloquear el resto del sistema. Como esta clave privada local es distinta para cada sistema comprometido, su revelación solo afecta al sistema en cuestión.

En el vídeo `VideoRansomware.mp4` explico con detalle cómo ha ido evolucionando la criptografía detrás del ransomware.

PRÁCTICA 4.24: Creando un ransomware de andar por casa

En esta práctica reproducirás con OpenSSL los pasos del creador de ransomware, para que te quede claro el proceso. Imagina que eres el cibercriminal. Tienes una pareja de claves pública y privada, tu *clave maestra*, y has creado un malware que cifra con `aes-128-ctr` los archivos del disco de la víctima, cada uno con una clave aleatoria distinta. Estas claves las guarda cifradas con RSA en la cabecera de los archivos. Para cifrarlas, usa una clave pública generada aleatoriamente al infectar el equipo. Esta clave es única para cada equipo: su *clave local*. La correspondiente clave privada la cifra con tu clave pública maestra y la destruye acto seguido. ¡Manos a la obra!

1. Antes de iniciar la campaña, crea tu pareja de claves pública y privada.

```
$ openssl genrsa -out master.key
```

2. Ya has infectado un equipo. Crea su pareja de claves pública y privada y guarda la pública en un archivo separado.

```
$ openssl genrsa -out local.key
$ openssl rsa -in local.key -pubout -out local.public.key
writing RSA key
```

3. Copia y guarda el exponente privado (d) en un archivo separado y asegúrate de que sea un archivo binario y no de texto. Una forma rápida y práctica de hacerlo es mediante la página web Hexadecimal – file (binary).

```
$ openssl rsa -in local.key -text -noout
...
privateExponent:
56:c3:94:7c:5a:a1:77:bf:96:cd:bf:fe:c7:d9:de:
70:24:aa:7b:5a:d2:10:c7:86:8d:db:63:cd:02:c3:
...
$ hexdump local.private.key
000000 56 c3 94 7c 5a a1 77 bf 96 cd bf fe c7 d9 de 70
```

```
000010 24 aa 7b 5a d2 10 c7 86 8d db 63 cd 02 c3 82 da
...
```

4. Cífralo con tu clave pública maestra.

```
$ openssl pkeyutl -encrypt -inkey master.key -in local.private.key
-out local.private.key.enc -pkeyopt rsa_padding_mode:none
$ hexdump local.private.key.enc
000000 02 40 86 20 ce ea 03 c6 54 9b f8 f1 1b 3b f8 84
000010 4a 51 98 f2 e3 fd 40 f7 0c eb 46 bf e3 63 b9 ca
...
```

   Nota: si el valor de la clave privada local es mayor que el valor del módulo de la clave maestra, se producirá un error de cifrado. Si te ocurriera, genera otra pareja de claves locales, hasta que $d_{local} < n_{maestra}$.

5. Destruye el archivo con la clave privada en claro (en realidad, nunca se volcaría en un archivo, pero habría que borrarla de memoria para no dejar ningún rastro).

6. Tienes tu primer archivo que cifrar: `file001.txt`. Genera aleatoriamente su clave de cifrado y su IV.

```
$ openssl rand -hex 16
d15f39ab9222f6925cec70629532f8a9
$ openssl rand -hex 16
4a60c495cce741b2d0bd478073644522
```

7. Cífralo con esos parámetros.

```
$ openssl aes-128-ctr -iv d15f39ab9222f6925cec70629532f8a9
-K 4a60c495cce741b2d0bd478073644522 -in file001.txt -out
file001.txt.enc
```

8. Cifra la clave simétrica con la clave pública local. Aunque en realidad no se escribe la clave simétrica en disco, supón que tú sí lo habías hecho.

```
$ hexdump file001.txt.key
000000 4a 60 c4 95 cc e7 41 b2 d0 bd 47 80 73 64 45 22
$ openssl pkeyutl -encrypt -inkey local.public.key -pubin -in
file001.txt.key -out file001.txt.key.enc
```

9. Destruye todo rastro de la clave simétrica y conserva solo el IV y la clave simétrica cifrada. Y a por el siguiente archivo.

Para rescatar los archivos, se procede en el orden inverso.

1. El malware envía al C&C la clave privada local cifrada con la clave pública maestra.
2. El C&C descifra la clave privada y la devuelve al malware.
3. El malware recorre el disco duro en busca de archivos cifrados. Para cada uno:
   a) Descifra la clave simétrica usando la clave privada local recibida del C&C.
   b) Con la clave simétrica y el IV, descifra el contenido del archivo.

4.10. Estándares de cifrado asimétrico

Al igual que con los algoritmos de cifrado simétrico, los organismos de estandarización también han publicado recomendaciones sobre cómo utilizar los algoritmos de cifrado asimétrico más seguros.

- NIST
  - SP 800-56B Rev. 2: Recommendation for Pair-Wise Key-Establishment Using Integer Factorization Cryptography, Date Published: March 2019.
- IETF

  - PKCS#1: RSA Cryptography Specifications Version 2.2
  - PKCS#3: Diffie-Hellman Key Agreement Method
- ISO
  - ISO/IEC 18033-2:2006: Information technology — Security techniques — Encryption algorithms — Part 2: Asymmetric ciphers
  - ISO/IEC 18033-6:2019: IT Security techniques — Encryption algorithms — Part 6: Homomorphic encryption
- BSI
  - BSI TR-02102-1: "Cryptographic Mechanisms: Recommendations and Key Lengths" Version: 2022-1
- CCN
  - Guía de Mecanismos Criptográficos autorizados por el CCN (CCN-STIC 221)

Lección 5

Funciones hash

Objetivos de aprendizaje

- Comprender las propiedades fundamentales de las funciones hash criptográficas y la relevancia de la resistencia a preimágenes, segunda preimagen y colisiones.
- Distinguir las diferentes construcciones de funciones hash (Merkle-Damgård, Davies-Meyer) y sus implicaciones de seguridad en la práctica.
- Conocer el papel de los algoritmos MAC (HMAC, CBC-MAC) en la autenticación de mensajes y cómo se integran en protocolos criptográficos.
- Examinar el cifrado autenticado (AEAD) como una combinación de cifrado y autenticación, analizando modos como GCM y sus requisitos de uso seguro.

Competencias a desarrollar

- Ser capaz de utilizar herramientas y bibliotecas para generar y verificar hashes (SHA-2, SHA-3, etc.) y códigos de autenticación de mensajes en distintos entornos.
- Saber elegir la función hash adecuada (por ejemplo SHA-256 o SHA3-256) según los requisitos de seguridad de un proyecto, teniendo en cuenta la resistencia a ataques y el rendimiento.
- Implementar y configurar MAC (HMAC, CBC-MAC) y modos AEAD (AES-GCM) en protocolos de comunicación y almacenamiento de datos.
- Identificar y mitigar vulnerabilidades asociadas al uso inadecuado de funciones hash, MAC y modos de cifrado autenticado, evitando prácticas inseguras como la reutilización de nonces.

Las funciones hash son una auténtica navaja suiza criptográfica. Ya sea que estés cifrando un correo electrónico, enviando un mensaje desde tu teléfono móvil, conectándote a un sitio web HTTPS, conectándote a una máquina remota a través de IPSec o SSH o minando bitcoins, hay una función de hash en algún lugar bajo el capó. Las funciones hash son, con mucho, el más versátil y ubicuo de todos los algoritmos de criptografía, como se representa en la Fig. 5.1.

A diferencia de todos los demás algoritmos de criptografía introducidos hasta ahora en este curso, **las funciones hash no tienen una clave**.

5.1. Funciones hash criptográficas

Una función hash es una función matemática H que acepta un bloque de datos de longitud variable m como entrada y produce un valor hash de tamaño fijo $h = H(m)$ a la salida. Una función hash *buena* tiene la propiedad de que los resultados de aplicar la función a un gran conjunto de entradas producirán salidas distribuidas uniformemente y aparentemente aleatorias. En términos generales, el objeto principal de una función hash es la integridad de los datos. Un cambio de cualquier bit o bits en m resulta, con alta probabilidad, en un cambio del 50 % de los bits del hash.



Figura 5.1: Las funciones de hash son una auténtica navaja suiza criptográfica.

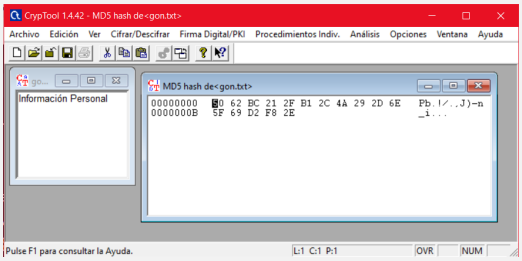
Ejemplo: avalancha en las funciones de hash

Cambiando un solo bit en cada valor pasado al hash, el resultado cambia significativamente:

```
SHA256(‘‘2’’) =
d4 73 5e 3a 26 5e 16 ee e0 3f 59 71 8b 9b 5d 03 01 9c 07 d8 b6 c5 1f 90
da 3a 66 6e ec 13 ab 35
SHA256(‘‘3’’) =
4e 07 40 85 62 be db 8b 60 ce 05 c1 de cf e3 ad 16 b7 22 30 96 7d e0 1f
64 0b 7e 47 29 b4 9f ce
```

PRÁCTICA 5.1: Experimenta con el efecto de avalancha de los hashes en CrypTool

1. Abre un archivo cualquiera con CrypTool o escribe texto en uno nuevo.
2. Selecciona `Procedimientos Indiv. >Hash >Demostración Hash`.
3. Cambia un único carácter en el texto. El hash resultante habrá cambiado alrededor de un 50 % de sus bits.



Una función hash exhibe las siguientes tres propiedades, representadas en la Fig. 5.2:

- Su entrada puede ser cualquier cadena de cualquier tamaño.
- Compresión: produce una salida de tamaño fijo.
- Cálculo sencillo: calcular el hash de una cadena de n bits debería ejecutarse en $O(n)$.

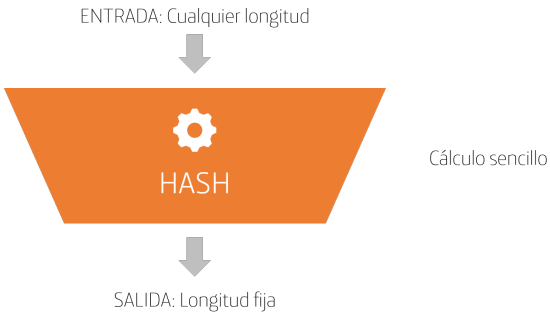


Figura 5.2: Las funciones hash aceptan un valor a la entrada que puede tener cualquier longitud: bits, megabits, terabits, lo que sea. Procesan los datos y devuelven un valor de longitud fija de entre 256 y 512 bits.

Ejemplo: aproximación ingenua al hasing

Se puede utilizar como función hash algo tan simple como sumar el valor de cada letra de un mensaje:

$$h = H(m) = \sum_{i=1}^{n} m_i$$

Usando esta función hash, el hash del mensaje "CRYPTO" se calcula como:

$$C + R + Y + P + T + O = 2 + 17 + 24 + 15 + 19 + 14 = 91$$

Te habrás dado cuenta de que viola la segunda condición, compresión. Para evitar que el valor del hash crezca sin límites con mensajes cada más largos, se puede añadir la operación módulo:

$$h = H(m) = \sum_{i=1}^{n} m_i \mod N,$$

donde N es el número de letras del alfabeto dado. De esta manera, $0 \leq h \leq N$. Puedes comprobar que esta función hash ahora sí satisface las tres propiedades anteriores: su entrada puede ser cualquier cadena de cualquier tamaño, produce una salida de tamaño fijo y es fácil de calcular.

El problema de la función hash del ejemplo es que cualquiera podría encontrar mensajes diferentes con el mismo hash. Por tanto, para que una función hash sea *criptográficamente segura*, debe presentar además las siguientes tres propiedades, representadas en la Fig. 5.3:

1. **Resistencia a la preimagen**: dado un hash, es imposible encontrar un mensaje cualquiera que produzca el mismo hash.
2. **Resistencia a la segunda preimagen**: dados un mensaje y su hash, es imposible encontrar otro mensaje que produzca el mismo hash.
3. **Resistencia a colisiones**: es imposible encontrar dos mensajes cualesquiera que produzcan el mismo hash.

Será mejor estudiarlas con más detalle.

5.1.1. Propiedad 1: resistencia a la preimagen

Para un valor de hash $h = H(x)$, se dice que x es la preimagen de h. Es decir, x es un bloque de datos cuyo valor hash, usando la función H, es h. Como H es una función de compresión, para cualquier valor hash dado h, habrá en general múltiples preimágenes.

La resistencia a la preimagen exige que, para esencialmente todas las salidas preespecificadas, es computacionalmente inviable encontrar cualquier entrada cuyo hash sea esa salida, es decir, resulta inviable encontrar cualquier preimagen x tal que $H(x) = y$, dado cualquier y para el que no se conoce una entrada correspondiente.

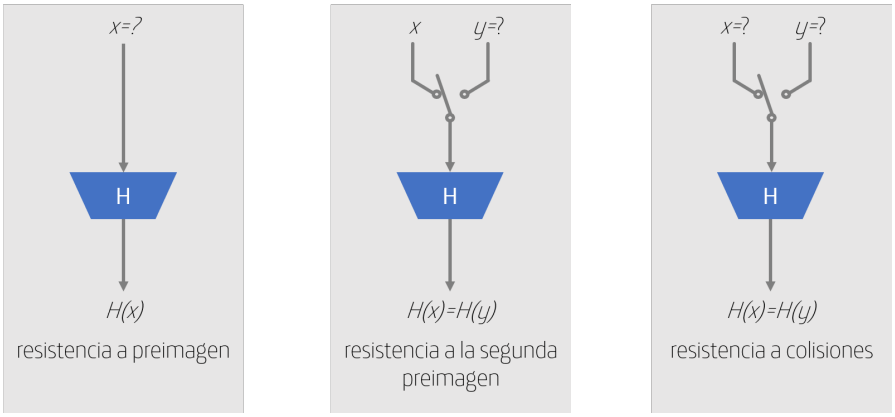


Figura 5.3: Para que una función hash se considere segura para su uso en aplicaciones criptográficas, tiene que cumplir con tres requisitos.

5.1.2. Propiedad 2: resistencia a la 2ª preimagen

Es computacionalmente inviable encontrar cualquier segunda entrada que tenga la misma salida que cualquier entrada especificada, es decir, dado x, resulta inviable encontrar una segunda imagen $x' \neq x$ tal que $H(x) = H(x')$.

5.1.3. Propiedad 3: resistencia a las colisiones

Se produce una colisión cuando dos entradas distintas originan la misma salida. Una función hash H es resistente a las colisiones si nadie puede encontrar una colisión. Formalmente, se dice que una función hash H es resistente a colisiones si no es posible encontrar dos valores, x e y, tales que $H(x) = H(y)$, a pesar de que $x \neq y$. (Nótese que aquí ambas entradas pueden elegirse libremente).

Si lees sobre el tema, en la literatura te encontrarás posiblemente con términos alternativos como:

- Resistencia a la preimagen $\longleftrightarrow$ Unidireccionalidad.
- Resistencia a la 2ª preimagen $\longleftrightarrow$ Resistencia débil a colisiones.
- Resistencia a colisiones $\longleftrightarrow$ Resistencia fuerte a colisiones.

La resistencia fuerte a las colisiones es más difícil de conseguir que la débil, ya que un atacante disfruta de dos grados de libertad: ambos mensajes pueden ser alterados para lograr valores de hash idénticos.

La tabla 5.1 resume los requisitos generalmente aceptados para una función hash criptográfica.

Las tres primeras propiedades son requisitos para la aplicación práctica de una función hash. La cuarta propiedad, resistente a las preimágenes, es la propiedad de unidireccionalidad. La quinta propiedad, resistente a la segunda preimagen, garantiza que no es posible encontrar un mensaje alternativo con el mismo valor de hash que un mensaje dado.

Una función hash que satisface las cinco primeras propiedades de la tabla 5.1 se denomina *función hash débil*. Si la sexta propiedad, resistente a las colisiones, también se satisface, entonces se denomina *función de hash fuerte*. Por ejemplo, una función de hash fuerte protege contra un ataque en el que una parte genera un mensaje para que otra parte lo firme.

Las funciones hash criptográficas se utilizan comúnmente para la derivación de claves y la generación de números pseudoaleatorios. Por lo tanto, tiene sentido verificar que, de hecho, una determinada función hash produce una salida pseudoaleatoria.

En resumen, ¿qué tienes que recordar sobre los hashes?

- El hash tiene una longitud fija.
- Datos idénticos siempre devuelven el mismo hash.
- Datos diferentes siempre devuelven un hash diferente (dentro de las limitaciones prácticas).
- Es fácil de calcular.

Requisito	Descripción
Tamaño de entrada variable	H puede aplicarse a un bloque de datos de cualquier tamaño.
Tamaño de salida fijo	H produce una salida de longitud fija.
Eficiencia	$H(x)$ es relativamente fácil de calcular para cualquier x dada, permitiendo que tanto las implementaciones de hardware como de software sean prácticas.
Resistente a preimágenes (propiedad de unidireccionalidad)	Para cualquier valor hash dado h, es computacionalmente inviable encontrar x tal que $H(x) = h$.
Resistente a la segunda preimagen (resistencia débil a colisiones)	Para cualquier x dado, es computacionalmente inviable encontrar $y \neq x$ with $H(y) = H(x)$.
Resistente a las colisiones (resistencia fuerte a colisiones)	Es computacionalmente inviable encontrar cualquier par (x, y) con $x \neq y$, tal que $H(x) = H(y)$.
Pseudoaleatoriedad	La salida de H supera las pruebas estándar de pseudoaleatoriedad.

Cuadro 5.1: Requisitos de seguridad para una función hash criptográfica H.

- Es inviable convertir el hash en datos.
- Un pequeño cambio en los datos conduce a un gran cambio en el hash.

PRÁCTICA 5.2: Analizando la seguridad de una función hash sencilla

Considera la siguiente función hash, donde los mensajes están formados por secuencias de letras, cada una codificada por números naturales, {A=0, B=1,..., Z=25} y $M = (m_1, m_2, \ldots, m_t)$. El valor de hash h se calcula como

$$h = H(M) = \sum_{i=1}^{t} m_i^2 \mod 26$$

1. ¿Cuántos requisitos de la tabla 5.1 satisface esta función?

   La función hash sólo cumple los siguientes requisitos:

   - Tamaño de entrada variable
   - Tamaño de salida fijo
   - Eficiencia

   Como averiguarás a continuación, no cumple con ningún requisito de seguridad. Por lo tanto, no es criptográficamente segura.

2. Calcula el hash del mensaje `M=UCAM ROCKS` (ignorando los espacios).

   $$H(\texttt{UCAMROCKS}) = 400 + 4 + 0 + 144 + 289 + 196 + 4 + 100 + 324 \mod 26 = 5$$

3. ¿Puedes encontrar una preimagen para el valor $h = 1$?

   $H(\texttt{B}) = 1 \mod 26$, $H(\texttt{AB}) = 0{+}1 \mod 26$ o cualquier mensaje que incluya una letra `B` y tantas letras `A` como se desee. Por supuesto, muchas otras combinaciones más complejas servirían, como `DADADA`, `FABABA`, `FEDABABAB`, etc. ¿Se te ocurren otras?

4. ¿Puedes encontrar una segunda preimagen para la palabra `GON`?

   $$H(\texttt{GON}) = 36 + 196 + 169 \mod 26 = 11$$

   Hay infinitas posibilidades. La más fácil es `DABAB`. ¿Cuáles más se te ocurren?

5. ¿Puedes encontrar un algoritmo general fácil para encontrar colisiones?

   Hay muchas maneras de encontrar colisiones en esta función. Por ejemplo, dado que `A = 0`, puedes añadir tantos caracteres `A` como quieras; o puedes sustituir un carácter dado por 4 veces su raíz cuadrada (cuando es un número entero), como `E = CCCC`, porque $16 = 4+4+4+4$, o `C = BBBB`, ya que $4 = 1+1+1+1$; `F = DE`, ya que $25 = 9+16$; etc. ¿Has pensado en alguna otra combinación?

Por último, considera el siguiente método simple para encontrar una colisión para una función hash con un tamaño de salida de n bits: escoge $2^n + 1$ valores distintos, calcula los hashes de cada uno de ellos y comprueba si dos salidas cualesquiera son iguales. Dado que hay más entradas que posibles salidas, necesariomente un par de valores debe colisionar. Por supuesto, este procedimiento será extremadamente lento, ya que el tamaño de la salida crece con n. Pero, ¿realmente se necesita comprobar cada valor de salida posible antes de encontrar una colisión con una probabilidad del 50 % o más?

5.2. La paradoja del cumpleaños

¿Cuántas personas crees que hace falta meter en una sala para que la probabilidad de que dos de ellas cumplan años el mismo día sea superior al 50 %? Intuitivamente, pensamos que hará falta un número muy grande: cincuenta, cien personas, mil. ¡Pues no! Basta con 23 personas. Es lo que se conoce como la paradoja del cumpleaños: es una paradoja no porque contradiga la lógica, sino nuestra intuición.

La clave es que el problema pide que *dos personas cumplan años el mismo día*, no que haya alguien que cumpla el mismo día que otra persona en concreto. En primer caso, 23 personas pueden formar 253 parejas distintas, y todas son candidatas. Si nos refiriéramos sólo a una de las personas en concreto sería de 1/365 para cada una de las 22 personas, $(22/365) \approx 6\,\%$ en total, que es mucho menos.

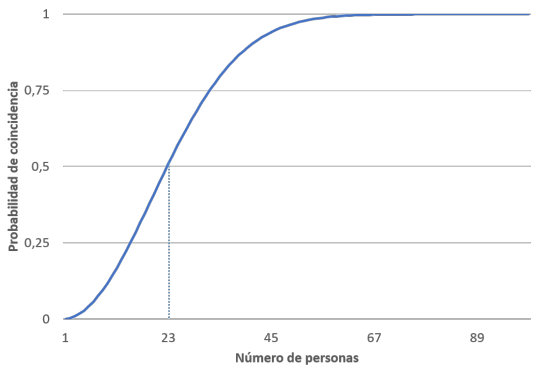


Figura 5.4: La paradoja del cumpleaños: bastan 23 personas para que la probabilidad de que dos cumplan años el mismo día será superior al 50 %.

Para obtener la respuesta exacta usando cáclculo de probabilidades, es más fácil calcular primero la probabilidad de que dos personas no cumplan años el mismo día, es decir, que no se produzca una colisión de sus cumpleaños. Para una sola persona, la probabilidad de que no haya colisión es de $\frac{365}{365} = 1$, lo cual resulta trivial, ya que un solo cumpleaños no puede colisionar con el de nadie más. Para la segunda persona, la probabilidad de no colisionar con la primera es de $\frac{364}{365}$, ya que sólo hay un día, el cumpleaños de la primera persona, para colisionar:

$$Pr(\neg\text{colisión entre dos personas}) = \frac{365}{365} \cdot \frac{364}{365}$$

Si una tercera persona se une a la fiesta, puede colisionar con uno de los dos que ya están allí, por lo tanto:

$$Pr(\neg\text{colisión entre tres personas}) = \frac{365}{365} \cdot \frac{364}{365} \cdot \frac{363}{365}$$

Por consiguiente, la probabilidad de que t personas no cumplan años el mismo día viene dada por:

$$Pr(\neg\text{colisión entre } t \text{ personas}) = \frac{365}{365} \cdot \frac{364}{365} \cdot \frac{363}{365} \cdots \frac{365-t+1}{365} = \frac{365!}{365^t(365-t)!}$$

Si generalizamos para cualquier número de elementos, la probabilidad de que no haya colisión entre t elementos en una muestra de N elementos es:

$$Pr(\neg\text{colisión entre } t \text{ personas}) = \frac{N!}{N^t(N-t)!}$$

Y en correspondencia, la probabilidad de que haya una colisión (dos cumplan años el mismo día) es:

$$Pr(\text{colisión}) = 1 - Pr(\neg\text{colisión})$$

Puedes comprobar fácilmente que para $N = 365$ y $t = 23$, dicha probabilidad es 50,7 %.

Ahora bien, si N es muy grande, entonces la probabilidad de que haya una repetición en el conjunto puede aproximarse como la raíz cuadrada de N:

$$Pr(\text{colisión}) = \sqrt{N}$$

Por lo tanto, para un hash de n bits, si seleccionas bloques de datos al azar, podrás esperar encontrar dos bloques de datos con el mismo valor hash con probabilidad superior al 50 % en un número de intentos igual a $\sqrt{2^n} = 2^{n/2}$. Por consiguiente, si el hash tiene una longitud de 160 bits, un ataque de fuerza bruta o de búsqueda exhaustiva exige probar en promedio 2^{80} posibles valores de hash para encontrar una colisión. Es como reducir la longitud efectiva del hash a la mitad.

Por supuesto, si la función de hash es débil, hay otros métodos para encontrar colisiones, como has comprobado en la práctica anterior.

Si te ha picado la curiosidad por este tema de la paradoja del cumpleaños, te recomiendo que te pases por este Simulador de la Paradoja del Cumpleaños, creado por Brain Feast.

5.3. Construcción de funciones hash

Hasta aquí, los requisitos para las funciones hash. Ahora, aprenderás cómo se construyen realmente. Para empezar, hay dos tipos generales de funciones hash:

1. **Funciones hash dedicadas**: son algoritmos diseñados específicamente para servir como funciones hash.
2. **Funciones hash basadas en el cifrado en bloque**: también es posible utilizar cifradores en bloque como AES para construir funciones hash.

5.3.1. Funciones hash dedicadas: la transformación de Merkle-Damgård (M-D)

Las funciones hash dedicadas son algoritmos diseñados a medida. Las funciones hash se construyen a menudo diseñando primero una función de compresión resistente a las colisiones que maneje entradas de longitud fija y luego usando *extensión de dominio* para manejar entradas de longitud arbitraria. La transformación Merkle-Damgård (M-D) es un enfoque común para extender una función de compresión a una función de hash completa, manteniendo la propiedad de resistencia a las colisiones de la primera.

Para calcular el hash de un mensaje, la construcción M-D divide el mensaje en bloques de idéntico tamaño y mezcla estos bloques con un estado interno usando una función de compresión F, como se muestra en la Fig. 5.5. Aquí, H_0 es el valor inicial (denotado por IV) del estado interno, los valores $H_1, H_2, \ldots$ se llaman los valores de encadenamiento y el valor final del estado interno es el valor del hash del mensaje.

Prácticamente todas las funciones de hash desarrolladas desde los años 80 hasta los 2010 se basan en la construcción (M-D): MD4, MD5, SHA-1 y la familia SHA-2, así como las menos conocidas funciones de hash RIPEMD y Whirlpool. La construcción M-D no es perfecta, pero es simple y ha demostrado ser lo suficientemente segura para muchas aplicaciones.

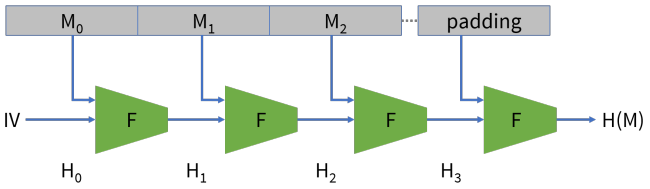


Figura 5.5: Construcción de Merkle–Damgård: hash iterativo usando una función de compresión que transforma una entrada en una salida más pequeña.

Algoritmo	Tamaño de la salida	Uso legado	Uso futuro
SHA-2	256, 384, 512, 512/256	✓	✓
SHA-3	256, 384, 512	✓	✓
SHA-3	SHAKE128, SHAKE256	✓	✓
Whirlpool	512	✓	✓
BLAKE	256, 384, 512	✓	✓
RIPEMD-160	160	✓	✗
SHA-2	224, 512/224	✓	✗
SHA-3	224	✓	✗
MD5	128	✗	✗
RIPEMD-128	128	✗	✗
SHA-1	160	✗	✗

Cuadro 5.2: Seguridad de las funciones hash más populares (Fuente: Algorithms, Key Size and Protocols Report (2018), ECRYPT-CSA).

PRÁCTICA 5.3: Calcula hashes con OpenSSL

Utiliza OpenSSL para calcular hashes.

```
$ openssl dgst -md5 file.txt
```

5.3.2. Funciones hash a partir de cifradores en bloque

Es posible construir una función de compresión resistente a las colisiones a partir de un cifrado en bloque que satisfaga ciertas propiedades adicionales. Hay varias maneras de hacerlo, siendo una de las más comunes a través de la construcción de *Davies-Meyer*. Sea E un cifrador en bloque con longitud de clave de n bits y longitud de bloque de l bits. Se define la función de compresión como se ve en la Fig. 5.6:

$$H_i = E_{M_i}(H_{i-1}) \oplus H_{i-1}.$$

El bloque del mensaje M_i actúa como la clave de cifrado del bloque y el valor de encadenamiento H_{i-1} actúa como su bloque de texto claro. Mientras el cifrado en bloque sea seguro, la función de compresión resultante es segura y resistente a las colisiones y a las preimágenes. Sin el XOR del valor de encadenamiento precedente ($\oplus H_{i-1}$), Davies-Meyer sería inseguro porque podría invertirse, pasando del nuevo valor de encadenamiento al anterior utilizando la función de descifrado del bloque de cifrado.

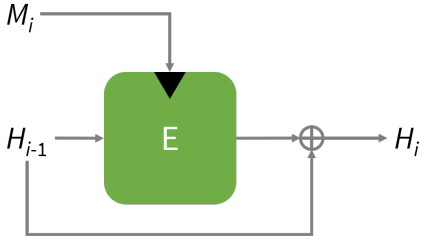


Figura 5.6: La construcción de Davies–Meyer.

5.3.3. Crónica de la muerte anunciada de SHA-1

Las siglas SHA provienen de *Secure Hash Algorithm*. SHA constituye toda una familia de hashes. En 1993, el NIST americano creó SHA, el primer miembro de la familia, hoy llamado **SHA-0** para distinguirlo de los siguientes: **SHA-1**, del que luego hablaremos; **SHA-2**,basada en la construcción de Merkle-Damgård, que a su vez comprende SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224 y SHA-512/256; y **SHA-3**, en sus versiones de 224, 256, 384 y 512 bits, así como las versiones de tamaño arbitrario de salida SHAKE128 y SHAKE256.

En concreto, SHA-1, de la familia MD4, fue diseñado por la NSA en 1996 como parte de su Algoritmo de Firma Digital (DSA). Durante prácticamente dos décadas SHA-1 ha sido el algoritmo de hash más popular, presente de forma predeterminada o como opción en todas las aplicaciones criptográficas habidas y por haber, incluyendo TLS y SSL, OpenSSL, PGP, SSH, S/MIME, Ipsec, sistemas de control de revisiones como Git, todas las bibliotecas criptográficas y, en general, todas aquellas aplicaciones en las que se usan hashes.

Por desgracia, se le fueron descubriendo diferentes vulnerabilidades con los años. De hecho, ya desde 2010 se desaconsejó su uso y se fue eliminando paulatinamente de protocolos y estándares. El golpe más duro lo recibió en 2017 cuando Google encontró una colisión para SHA-1 completamente documentada en SHAttered, implementando por primera vez, gracias a la evolución de la capacidad de cómputo, un ataque puramente teórico publicado en 2005, con complejidad de 2^69 en lugar de 2^80.

El último clavo en el ataúd se lo pusieron dos investigadores en 2020 implementando otro ataque teórico propuesto en 2019 por ellos mismos. Su ataque permite encontrar colisiones con una complejidad de $2^{61,2}$, en dos meses de cálculos, con un coste de computación de 11.000 dólares: tan asequible que SHA-1 se considera muerto y enterrado.

Para demostrar la viabilidad del ataque, crearon dos peticiones de firma de certificados PGP/GnuPG: uno con el nombre de la víctima y otro con el nombre y la foto del atacante, de forma que ambos originaban el mismo hash SHA-1. El atacante podía solicitar una firma de su clave y su imagen a un tercero (de la Web of Trust o de una CA) y transferir la firma a su certificado. A partir de ahí, el atacante podía hacerse pasar por la víctima y firmar cualquier documento en su nombre. Tras este ataque devastador, SHA-1 se considera completamente inseguro, al mismo nivel que MD5, por lo que se deberían reemplazar inmediatamente por alternativas más seguras como SHA-256 o SHA-3.

5.4. Aplicaciones de las funciones hash criptográficas

Las funciones hash se utilizan en una amplia variedad de aplicaciones de seguridad y protocolos de Internet como, por ejemplo:

- Autenticación de mensajes
- Firmas digitales
- Otras aplicaciones

5.4.1. Autenticación de mensajes

La autenticación de mensajes es un mecanismo o servicio utilizado para verificar la integridad de un mensaje. La autenticación de mensajes garantiza que los datos recibidos son exactamente como se han enviado, es decir, que no se modifican, insertan, eliminan o repiten. En muchos casos, se exige que el mecanismo de autenticación asegure que la supuesta identidad del remitente sea válida. Cuando se utiliza una función hash para proporcionar la autenticación de un mensaje, el valor de la función hash suele denominarse *resumen* del mensaje.

La esencia de la utilización de una función hash para la integridad del mensaje es la siguiente. El emisor calcula el hash de los bits del mensaje y transmite tanto el hash como el mensaje. El receptor realiza el mismo cálculo de hash sobre los bits del mensaje y compara este valor con el hash recibido. Si no coinciden, el receptor sabe que el mensaje (o posiblemente el hash) ha sido alterado.

El hash debe transmitirse de forma segura. Es decir, el hash debe estar protegido de manera que, si un adversario altera o reemplaza el mensaje, no sea factible que el adversario también altere el hash para engañar al receptor. Esta seguridad se proporciona gracias a las firmas digitales.

5.4.2. Firmas digitales

En el caso de la firma digital, el hash de un mensaje se cifra con la clave privada del usuario. Cualquiera que conozca la clave pública del usuario puede verificar la integridad del mensaje asociado a la firma digital. En este caso, un atacante que desee alterar el mensaje necesitaría conocer la clave privada del usuario. Como verás en la siguiente lección, las implicaciones de las firmas digitales van más allá de la mera autenticación del mensaje.

5.4.3. Otras aplicaciones

Y hay mucho más:

- Las funciones de hash se usan comúnmente para crear un **archivo de contraseñas unidireccional**: se almacena en el sistema el hash de la contraseña en lugar de la propia contraseña. Por lo tanto, la contraseña real no es recuperable por un atacante que obtenga acceso al archivo de contraseñas. En términos sencillos, cuando un usuario introduce una contraseña, el hash de esa contraseña se compara con el valor de hash almacenado para su verificación. Este enfoque de la protección de contraseñas es utilizado por la mayoría de los sistemas operativos.
- Las funciones hash pueden usarse para la **detección de intrusiones y de virus**. Se almacena el hash $H(F)$ por cada archivo F en el sistema y se aseguran los valores de hash, por ejemplo, escribiéndolos en un CD-R que se mantiene a buen recaudo. Posteriormente, se puede determinar si un archivo ha sido modificado volviendo a calcular $H(F)$ para cada archivo. Un intruso necesitaría cambiar F sin que cambie $H(F)$.
- Pueden usarse para construir una función pseudoaleatoria (PRF). Una aplicación común para una PRF basada en hash es para la generación de claves simétricas. La función de derivación de clave (KDF) deriva una o más claves secretas a partir de un valor secreto como una clave maestra, una contraseña o una frase de contraseña utilizando una función pseudoaleatoria. Las KDF pueden utilizarse para extender claves en otras más largas o para obtener claves en un cierto formato. PKCS#5 es un estándar propuesto por la antigua RSA Data Security como uno de una serie de estándares de criptografía de clave pública. Define dos algoritmos KDF llamados PBKDF1 y PBKDF2. El primero es un estándar antiguo que originalmente estaba destinado a ser usado con DES y como tal tiene entradas de tamaño fijo. PBKDF2 es más flexible y es el KDF recomendado del estándar PKCS#5.
- También se utilizan en criptografía basada en hashes, el término genérico para construcciones de primitivas criptográficas basadas en la seguridad de las funciones hash. Resultan de interés como un tipo de criptografía postcuántica.
- Para crear **puzles en Blockchain**. Los puzles basados en hashes constituyen la piedra angular de las pruebas de trabajo, ya que sólo pueden resolverse por fuerza bruta, a base de prueba y error. Para una explicación muy clara y con código sobre cómo funciona Blockchain, te recomiendo que te pases por blockchaindemo.io.
- A medida que se crean, almacenan y manipulan cada vez más datos en línea, a veces es crucial verificar cuándo se creó o modificó por última vez un documento o registro digital. Los procedimientos de **estampillado o sellado de tiempo**, estudiados en la Sec. 6.1.7, confían en las funciones hash para conseguir que el sello dependa únicamente de los datos del documento en cuestión y que no sea susceptible de manipulación, ni de los datos, ni de la fecha y hora.

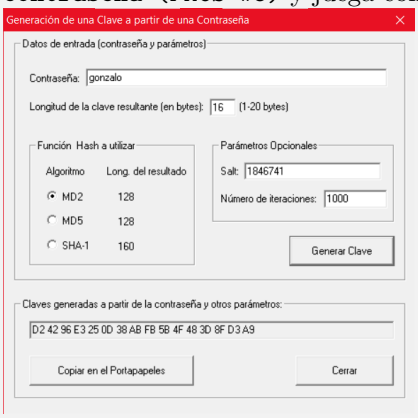
PRÁCTICA 5.4: Derivando claves simétricas a partir de contraseñas mediante hashes en CrypTool

La función de derivación de claves PBKDF2 admite cinco parámetros de entrada:
`DK = PBKDF2(PRF, Contraseña, Sal, c, dkLen)`
donde:

- `PRF` es una función pseudoaleatoria de dos parámetros, por ejemplo, un HMAC con clave.
- `Contraseña` es la contraseña maestra a partir de la cual se genera una clave derivada.

- `Sal` es una secuencia de bits, conocida como sal criptográfica.
- `c` es el número de iteraciones deseadas.
- `dkLen` es la longitud de bits deseada de la clave derivada.
- `DK` es la clave derivada generada.

Para ponerlo todo en práctica, selecciona `Procedimientos Indiv. >Hash >Generación de una clave desde una contraseña (PKCS #5)` y juega con los parámetros a tu gusto.



PRÁCTICA 5.5: Derivando claves simétricas a partir de secretos en OpenSSL

A partir de la versión 3.0, OpenSSL incluye la posibilidad de operar con funciones de derivación de claves o KDF. Por ejemplo, para crear una clave de 32 bytes a partir de un secreto (expresado en binario) usando una función hash de 256 bits con un salt aleatorio y un campo de info (expresado como cadena de texto), puedes usar el siguiente comando:

```
$ openssl kdf -keylen 32 -kdfopt digest:SHA2-256
-kdfopt hexkey:0ca8cea5573c8371dc5ca36b518e5f8ec92a45e1df0e25efcf8c469b4ecfb564
-kdfopt hexsalt:359fcec597dfdd12c87702759cfdd5a6
-kdfopt info:‘‘etiqueta adicional’’ hkdf
FA:3C:71:0E:11:24:92:13:96:B0:49:65:83:AD:AC:1A:E3:B3:EA:7C:5D:F6:8B:59:
8A:B8:08:96:00:F6:AF:CF
```

5.5. Códigos de autenticación de mensajes

Un código de autenticación de mensajes (*message authentication code*, MAC) es un pequeño fragmento de información que puede utilizarse para comprobar la autenticidad y la integridad de un mensaje. Estos códigos se denominan a menudo etiquetas (*tags*). Un algoritmo de MAC toma un mensaje de longitud arbitraria m y una clave secreta de longitud fija k y produce la etiqueta t.

$$t = MAC(k, m)$$

El algoritmo MAC también viene con un algoritmo de verificación que toma un mensaje, la clave y una etiqueta y decide si la etiqueta era válida o no. No siempre basta con volver a calcular una etiqueta y comprobar si son iguales; muchos algoritmos de MAC seguros son aleatorios y producirán etiquetas diferentes cada vez que los apliques.

Los objetivos de seguridad de un algoritmo MAC son diferentes a los de una función hash. Aquí, en lugar de tratar de asegurar la integridad de un mensaje, se trata de establecer su autenticidad. Estos son objetivos distintos, pero comparten muchos puntos en común. En ambos casos, se trata de determinar la corrección o, más específicamente, la pureza de un mensaje. Donde los conceptos difieren es en que el objetivo de la autenticidad trata también de establecer un origen para el mensaje.

Por ejemplo, si te digo que el hash SHA-1 de un archivo es la cadena h de 160 bits y luego te doy el archivo, o mejor aún, tú mismo lo recuperas, entonces puedes determinar si el archivo es original (sin modificar) si el hash del mensaje calculado por ti coincide con el que te di. Pero no sabrás quién creó el archivo, el hash del mensaje no te dirá eso. Ahora supongamos que estamos

en medio de una comunicación y que ambos tenemos una clave secreta compartida K. Si te envío un archivo y la etiqueta MAC producida con la clave K, puedes verificar si el mensaje fue originado por mí verificando la etiqueta MAC.

Se pueden distinguir tres mecanismos de autenticación de la información: algoritmos MAC, códigos de autenticación y firmas digitales. Los dos primeros mecanismos se basan en una clave secreta compartida entre el remitente y el destinatario, lo que significa que si el remitente autentica un mensaje con un algoritmo MAC o un código de autenticación y más tarde lo niega, no hay forma de demostrar que ha autenticado el mensaje, ya que el destinatario también podría haberlo hecho. En términos técnicos, los algoritmos MAC y los códigos de autenticación no pueden proporcionar un *no repudio del origen*. Las firmas digitales, en cambio, permiten establecer de manera irrefutable el origen y el contenido de la información digital. Las estudiarás en la siguiente lección.

El algoritmo MAC consta de tres componentes:

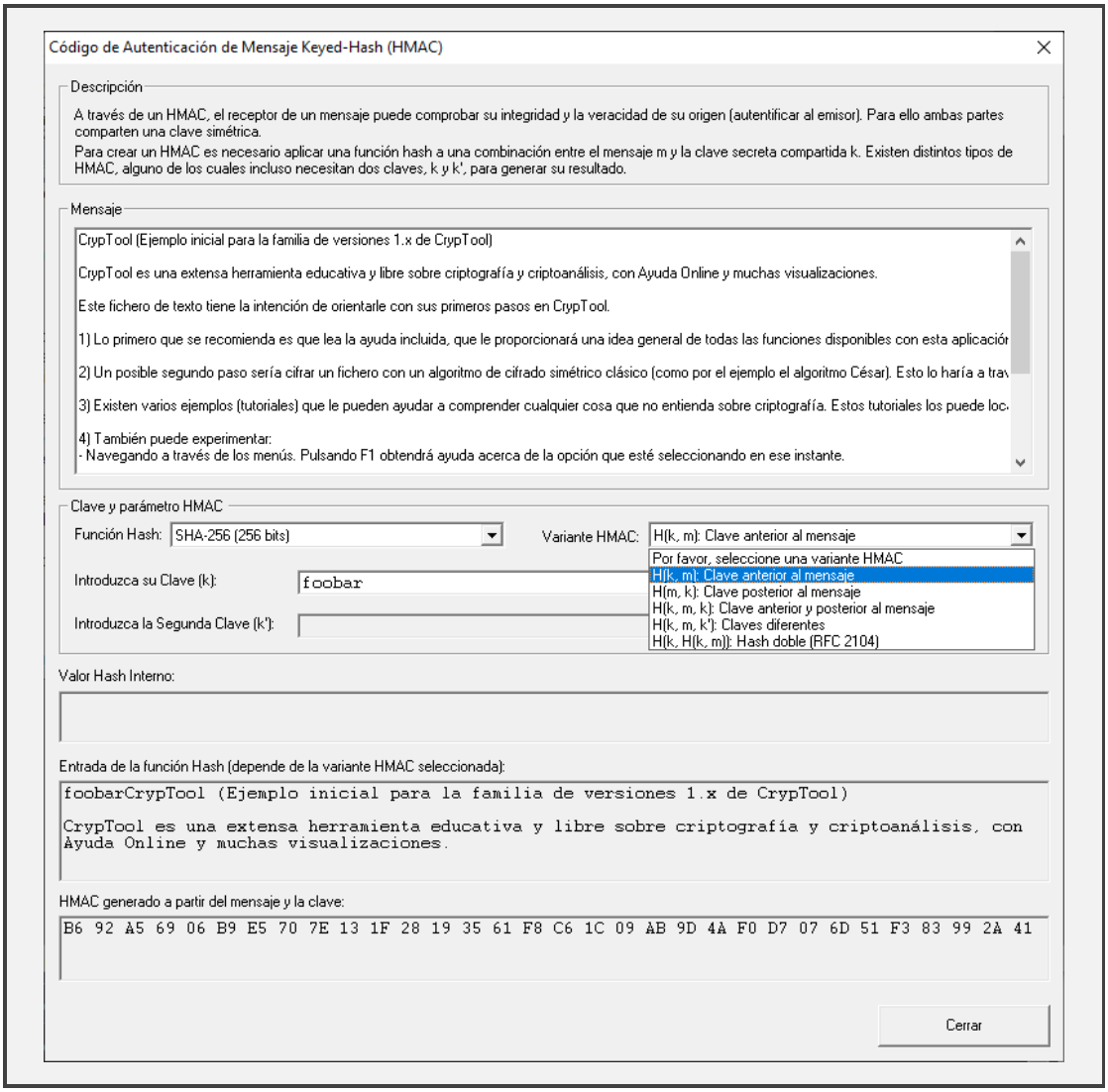
- **Un algoritmo de generación de claves**: para la mayoría de los algoritmos de MAC la clave secreta k es una cadena de bits aleatorios de longitud entre 56 y 256 bits.
- **Un algoritmo de generación de MAC**: este algoritmo calcula a partir de la entrada de texto m y de la clave secreta k un valor $t = MAC(k, m)$, que es una cadena de bits de longitud fija, típicamemte entre 24 y 96 bits.
- **Un algoritmo de verificación MAC**: al introducir el texto m, el valor t y la clave k (y posiblemente una cadena aleatoria), el algoritmo verifica si el valor MAC es correcto o no.

En un contexto de comunicación, el emisor y el receptor se pondrán de acuerdo sobre una clave secreta utilizando un protocolo de acuerdo de claves, como por ejemplo Diffie-Hellman, estudiado en la lección anterior.

Las funciones de MAC suelen construirse a partir de otras primitivas criptográficas, como los cifradores en bloque y los hashes, que son los que estudiarás a continuación en esta lección.

PRÁCTICA 5.6: Explorando los detalles de los HMACs con CrypTool

1. Selecciona `Procedimientos Indiv. >Hash >Generación de MACs`.
2. En `Variante HMAC` tienes distintas opciones para elegir.



5.5.1. MAC basado en hash criptográfico o HMAC

5.5.1.1. Un intento ingenuo con funciones hash

Muchas formas de construir MAC implican funciones hash. Una de las formas más simples que pueden imaginarse consiste simplemente en prefijar el mensaje con la clave secreta y hacer el hash completo:

$$t = H(k||m)$$

Este esquema se llama más comúnmente *Prefijo-MAC*, porque es un algoritmo MAC que funciona usando la clave secreta como prefijo. La función de hash criptográficamente segura H garantiza algunas propiedades importantes:

- La etiqueta t será fácil de calcular; la función de hash H en sí misma es típicamente muy rápida. En muchos casos podrás calcular la parte de la clave común con antelación, así que sólo tendrás que hacer un hash del mensaje en sí.
- Dada cualquier cantidad de etiquetas, no hay forma de que un atacante *invierta* la función de hash para recuperar k, lo que le permitiría falsificar mensajes arbitrarios.

Una pequeña advertencia: estamos asumiendo que la clave secreta k tiene suficiente entropía. De lo contrario, surge el mismo problema que para el almacenamiento de contraseñas usando las funciones de hash: un atacante podría intentar calcular el hash de cada contraseña posible hasta que una de ellas coincida. Una vez que lo ha hecho, es casi seguro que ha encontrado la k correcta. Sin embargo, eso no es realmente un fallo de la MAC: si tu clave secreta contiene tan poca entropía que es factible que un atacante las pruebe todas, ya has perdido, sin importar el algoritmo de MAC que elijas.

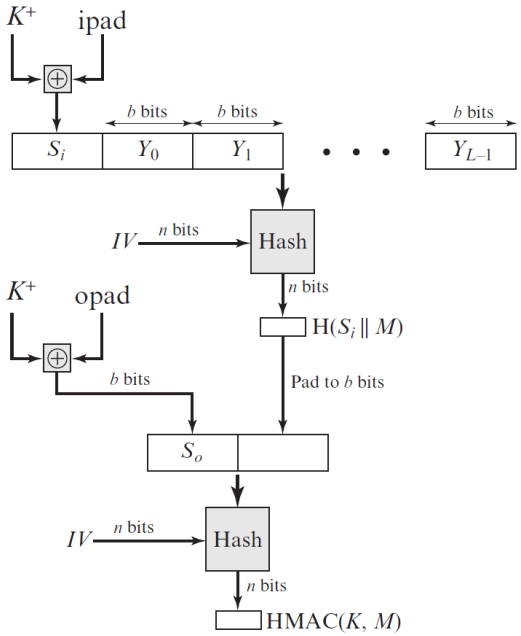


Figura 5.7: Estructura de la función HMAC.

5.5.1.2. HMAC

HMAC es la variante más popular de MACs basados en funciones hash. Utiliza una construcción anidada con claves con relleno:

$$MAC_K(m) = H((K^+ \oplus opad)||H((K^+ \oplus ipad)||m))$$

donde K es la clave del algoritmo MAC (rellenada con ceros a un bloque de b bits) y *opad* e *ipad* son cadenas constantes de b bits (obtenidas repitiendo los valores hexadecimales `0x36` y `0x5c`, respectivamente). El valor MAC resultante puede truncarse opcionalmente. El HMAC ofrece la ventaja de que puede implementarse sin hacer ninguna modificación en el código de la propia función hash. La estructura completa se representa en la Fig. 5.7, donde n representa la longitud del hash interno y K, la clave secreta. K^+ es la clave rellena con ceros hasta llegar a los b bits.

La seguridad de HMAC está garantizada si la función hash es resistente a las colisiones para un valor secreto H_0 y si la propia función de compresión es un MAC seguro para un bloque (con la clave secreta en la entrada H_i y el texto en la entrada x_i).

PRÁCTICA 5.7: Calculando HMACs con OpenSSL

OpenSSL utiliza nuevamente el comando `dgst`. Hay que ser cuidadoso a la hora de introducir la clave secreta para el HMAC, ya que OpenSSL la interpreta como una cadena de texto. En este ejemplo he usado como clave "key". Si quieres introducirla en hexadecimal, necesitas el parámetro `-macopt hexkey`. Te muestro un ejemplo para que veas cómo interpreta la clave `key` como una cadena de texto. El valor `0x6b6579` es el valor binario de esa misma cadena. Las tres formas de introducir la clave conducen al mismo resultado, obviamente.

```
$ openssl dgst -md5 -hmac key file.txt
HMAC-MD5(file.txt)= 54b8790e4d85bd130d6cbb63b466ed82
$ openssl dgst -md5 -mac hmac -macopt key:key file.txt
HMAC-MD5(file.txt)= 54b8790e4d85bd130d6cbb63b466ed82
$ openssl dgst -md5 -mac hmac -macopt hexkey:6b6579 file.txt
HMAC-MD5(file.txt)= 54b8790e4d85bd130d6cbb63b466ed82
```

PRÁCTICA 5.8: Fisgando dentro de un monedero Ethereum

En las criptomonedas como Bitcoin y Ethereum, las claves privadas de los titulares se almacenan en monederos criptográficos (*cryptowallets*). Técnicamente, estos *criptomonederos* no son más que archivos en el disco duro local cifrados por una contraseña.
En el caso concreto de Ethereum, los criptomonederos se almacenan internamente en un formato especial de cifrado conocido como *UTC/JSON Wallet (Keystore File)*. ¿Quieres crear uno y fisgar en su interior?

1. Conéctate a la herramienta online MyEtherWallet y pulsa el botón enorme `Create A New Wallet`.
2. Selecciona el método de creación `By Keystore File`. Sí, sí, no lo recomiendan, pero paciencia, te permitirá ver lo que pasa por dentro. Introduce una contraseña y pulsa `Next`.
3. Generarán un monedero para ti. Descárgatelo y guárdalo a buen recaudo.
4. Ahora ya puedes fisgar en su interior. Ábrelo con un editor de texto, preferiblemente que resalte con colores las palabras clave de JSON. Yo uso Notepad++. La parte emocionante de los contenidos del almacén de claves está bajo `crypto`:
   a) `ciphertext`: la clave privada ECDSA cifrada con la clave simétrica. En la próxima lección aprenderás cómo funciona ECDSA.
   b) `cipher`: el algoritmo utilizado para cifrar, en este caso AES de 128 bits de clave en modo CTR, como recordarás de la lección 3.
   c) `cipherparams`: los parámetros de cifrado, en este caso, el IV.
   d) `kdf`: el algoritmo para derivar la clave simétrica a partir de la contraseña que has utilizado para proteger tu clave privada, en este caso, scrypt.
   e) `kdfparams`: los parámetros para inicializar scrypt. Si te pica la curiosidad, en la Wikipedia tienes una bonita descripción.
   f) `mac`: un código usado para verificar que tu contraseña es correcta. A fin de cuentas, la clave privada es un número aleatorio y podría haberse descifrado mal.


```
{
"version":3,
"id":"fd9c176a-739b-4339-a932-285f8d9d911e",
"address":"30c6cf54a961ca048c68a63ededea13686796239",
"crypto":{
    "ciphertext":"bd11d79ee34c75698b4b5f6926a3c5dcd67fd6f7c256fbdb02f74cc69b477974",
    "cipherparams":{"iv":"633ece06ca4ed4134594f1587617812b"},
    "cipher":"aes-128-ctr",
    "kdf":"scrypt",
    "kdfparams":{
        "dklen":32,
        "salt":"3073b68301518295b3302c3991648dc3d2cf430a52b130898d403abc2220bbb4",
        "n":131072,
        "r":8,
        "p":1
        },
    "mac":"09f2ff256c49f268770c0b473e024bf8d919726ae50ca1f32b404f19cbe01b86"
    }
}
```


5.5.2. MAC basado en cifradores en bloque

El algoritmo MAC más antiguo y popular es ciertamente CBC-MAC, que se basa en un cifrado en bloque y que ha sido ampliamente estandarizado. CBC-MAC es un algoritmo MAC iterado. El método de relleno más común consiste en añadir un bit, seguido de entre 0 y $n-1$ bits a cero, de tal manera que la cadena resultante tiene una longitud de entrada que es un múltiplo de n. Denotando la cadena resultante como $x_1, x_2, \ldots, x_t$, la función de compresión para CBC-MAC tiene la siguiente forma, representada esquemáticamente en la Fig. 5.8:

$$H_i = E_K(H_{i-1} \oplus x_i), 1 \leq i \leq t.$$

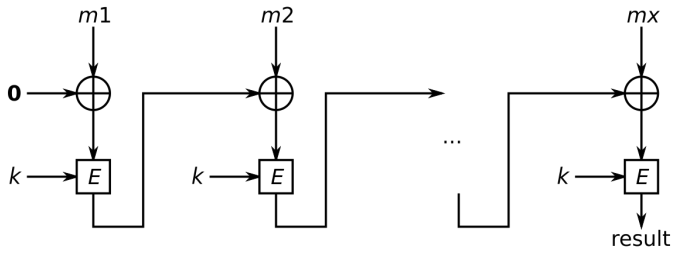


Figura 5.8: Estructura de la función CBC-MAC (Fuente: Wikipedia).

$H_0 = IV$ es un valor inicial fijo, que se establece a cero. El MAC se calcula entonces como $MAC_K(x) = g(H_t)$, donde g es la transformación de salida.

5.6. Cifrado autenticado

En la lección 3 te hiciste con varios modos de operación del cifrado en bloque. Un modo de funcionamiento que proporciona un cifrado autenticado con datos asociados (AEAD) cifrará y autenticará un mensaje y, al mismo tiempo, autenticará (pero no cifrará) una cadena de datos asociados. En la práctica, un uso típico de un algoritmo AEAD es proteger un paquete cifrando y autenticando el cuerpo del paquete de datos y autenticando (pero no cifrando) la cabecera del paquete.

Hay tres formas comunes de enfocar el cifrado autenticado.

1. **Autenticar y cifrar**: autenticas y cifras el texto claro por separado. Así es como lo hace SSH o PuTTY. $c = E_{K_c}(m), t = MAC(K_m, P)$, y envías tanto el texto cifrado c como la etiqueta t.

2. **Autenticar y luego cifrar**: autenticas el texto claro y luego cifras la combinación del texto claro y la etiqueta de autenticación. Así es como lo hace normalmente TLS. $t = MAC(K_m, m), c = E_{K_c}(m||t)$, y sólo envías c. No necesitas enviar t porque ya es una parte cifrada de c.

3. **Cifrar y luego autenticar**: cifras el texto claro y luego calculas el MAC de ese texto cifrado. Así es como lo hace IPSec. $c = E_{K_c}(m), t = MAC(K_m, c)$, y envías tanto c como t.

Se recomienda como forma más segura cifrar y luego autenticar (la prueba queda fuera del alcance de este curso).

5.6.1. GCM

El modo Galois/Counter (GCM) protege la confidencialidad del texto claro m mediante el uso de cifrado en bloque en modo CTR. Además, GCM protege no sólo la autenticidad del texto claro m, sino también la autenticidad de una cadena AAD llamada *datos autenticados adicionales*. Estos datos autenticados, a diferencia del texto claro, se dejan en claro en este modo de operación. En la práctica, la cadena AAD podría incluir direcciones y parámetros en un protocolo de red.

GCM se basa en el uso de un algoritmo de cifrado en bloque y un multiplicador de campo de Galois con el que se realizan las dos funciones del GCM: *cifrado autenticado* y *descifrado autenticado*. El cifrado debe tener un tamaño de bloque de 128 bits como el AES. En el lado del remitente, GCM cifra los datos utilizando el modo de contador (CTR) seguido del cálculo de un valor MAC.

Para el cifrado, mostrado en la parte superior de la Fig. 5.9, primero se obtiene un contador inicial a partir de un IV y un número de serie. Luego se incrementa el valor del contador inicial y este valor se cifra y se hace XOR con el primer bloque de texto claro. Para los siguientes textos claros, el contador se incrementa y luego se cifra. Observa que el cifrado del bloque subyacente sólo se utiliza en modo de cifrado. GCM permite el precálculo de la función de cifrado en bloque si se conoce de antemano el vector de inicialización. Hasta aquí, todo funciona como en el modo CTR.

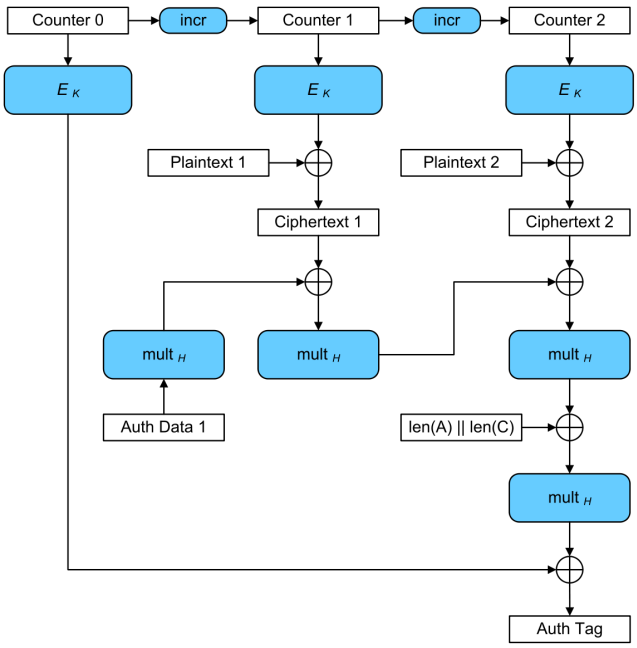


Figura 5.9: Estructura del modo de operación Galois/Counter Mode (GCM) (Fuente: Wikipedia).

Para la autenticación, mostrada en la parte inferior de la Fig. 5.9, el GCM realiza una multiplicación encadenada de campo de Galois. Para cada texto claro m_i se deriva un parámetro de autenticación intermedio g_i. g_i se calcula como la suma XOR del texto cifrado actual c_i y g_i y se multiplica por la constante H. El valor H es una subclave hash que se genera mediante el cifrado de la entrada todo a ceros al cifrador en bloque. Todas las multiplicaciones se realizan en el campo de Galois de 128 bits $GF(2^{128})$ con el polinomio irreducible $p(x) = x^{128} + x^7 + x^2 + x + 1$. Como sólo se requiere una multiplicación por cada bloque siendo cifrado, el modo GCM añade muy poca sobrecarga computacional al cifrado.

1. Cifrado:
   a) Deriva un valor de contador CTR_0 a partir del IV y calcula $CTR_1 = CTR_0 + 1$.
   b) Calcula el bloque cifrado: $c_i = E_K(CTR_i) \oplus m_i$.
2. Autenticado:
   a) Genera una subclave de autenticación $H = E_K(0)$.
   b) Calcula $g_0 = AAD \times H$ (multiplicación de campo de Galois).
   c) Calcula $g_i = (g_{i-1} \oplus c_i) \times H, 1 \leq i \leq n$ (multiplicación de campo de Galois).
   d) Etiqueta final de autenticación: $T = (g_n \times H) \oplus E_K(CTR_0)$.

El receptor del paquete $[(c_1, \ldots, c_n), T, ADD]$ descifra el texto cifrado aplicando también el modo CTR. Para comprobar la autenticidad de los datos, el receptor también calcula una etiqueta de autenticación T' usando como entrada el texto cifrado recibido y ADD. Emplea exactamente los mismos pasos que el emisor. Si T y T' coinciden, el receptor se asegura de que el texto cifrado (y ADD) no fueron manipulados en tránsito y que sólo el emisor pudo haber generado el mensaje.

El artículo AES-GCM and breaking it on nonce reuse demuestra que **la reutilización del nonce en AES-GCM lleva a fallos de seguridad catastróficos**, permitiendo a un atacante recuperar el keystream y falsificar mensajes. Al reutilizar el nonce, un atacante puede deducir el keystream a partir de un pareja de texto claro y cifrado conocidos, lo que le permite descifrar cualquier otro mensaje cifrado con el mismo nonce. Además, al usar la factorización de polinomios, el atacante puede recuperar la clave H utilizada en la autenticación de AES-GCM, permitiéndole crear tags de autenticación válidos para cualquier mensaje, incluso sin conocer la clave secreta. En resumen, **la reutilización del nonce en AES-GCM compromete completamente la seguridad del sistema, permitiendo el descifrado y la falsificación de mensajes**.

5.7. Estándares de funciones Hash

El uso de las funciones de hash también ha sido estandarizado por varios organismos para garantizar su uso seguro en todo tipo de aplicaciones.

- NIST
  - FIPS 180-4, Secure Hash Standard
    - SHA-1 (Secure Hash Algorithm-1)
    - SHA-2 family of hash algorithms: SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224, and SHA-512/256
  - FIPS 202, SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions.
    - Four fixed-length hash algorithms: SHA3-224, SHA3-256, SHA3-384, and SHA3-512; and
    - Two closely related, "extendable-output" functions (XOFs): SHAKE128 and SHAKE256.
- ISO
  - ISO/IEC 10118-1:2016: Information technology — Security techniques — Hash-functions — Part 1: General
- BSI
  - BSI TR-02102-1: "Cryptographic Mechanisms: Recommendations and Key Lengths" Version: 2022-1
- CCN
  - Guía de Mecanismos Criptográficos autorizados por el CCN (CCN-STIC 221)

Termino con este artículo Choosing a hash function for 2030 and beyond: SHA-2 vs SHA-3 vs BLAKE3 que compara la velocidad, seguridad y la facilidad de implementación de distintos algoritmos de hash.

Lección 6

Firma digital

Objetivos de aprendizaje

1. Comprender los fundamentos de las firmas digitales y sus propiedades de verificación del autor, autenticación del contenido y validez ante terceros.
2. Conocer los pasos esenciales de un esquema de firma (generación de claves, firma y verificación) y valorar el uso de certificados digitales.
3. Analizar diferentes algoritmos de firma (RSA, DSA, ECDSA) y sus ventajas, limitaciones y requisitos de seguridad.
4. Identificar cómo se integran las firmas digitales con los certificados X.509 y la infraestructura de clave pública para evitar ataques de intermediario.

Competencias a desarrollar

1. Ser capaz de generar y gestionar pares de claves y certificados para firmar y validar documentos en distintos estándares (OpenSSL, CrypTool).
2. Aplicar correctamente algoritmos como RSA-PSS, DSA o ECDSA para firmar mensajes y verificar firmas de forma segura y eficiente.
3. Integrar sistemas de firma digital en aplicaciones y protocolos (p. ej. S/MIME) entendiendo la importancia de la correcta gestión de claves y certificación.
4. Evaluar y mitigar posibles vulnerabilidades de implementación y uso (reutilización de k, ataques de intermediario, etc.) en esquemas de firma digital.

Las firmas digitales comparten alguna funcionalidad con las firmas manuscritas. Deseamos tres propiedades de las firmas digitales que se corresponden bien con la analogía de la firma manuscrita.

- Debe verificar el autor (y la fecha y hora de la firma).
- Debe autenticar el contenido en el momento de la firma.
- Debe ser verificable por terceros, para resolver disputas.

Sobre la base de estas propiedades, podemos formular los siguientes requisitos para una firma digital:

- La firma debe ser un patrón de bits que depende del mensaje que se firme.
- La firma debe usar alguna información que sólo conozca el remitente para prevenir tanto la falsificación como el repudio.
- Debe ser relativamente fácil producir la firma digital.
- Debe ser relativamente fácil reconocer y verificar la firma digital.

- Debe ser computacionalmente inviable falsificar una firma digital, ya sea mediante la elaboración de un nuevo mensaje para una firma digital existente o mediante la elaboración de una firma digital fraudulenta para un mensaje determinado.
- Debe ser práctico conservar una copia de la firma digital.

6.1. Esquema general de firma digital

Un esquema general de firma digital consiste en los siguientes tres algoritmos:

- $(k_{pr}, k_{pub}) \leftarrow generateKeys(keysize)$ El método $generateKeys$ toma un tamaño de clave $keysize$ y genera un par de claves. La clave privada k_{pr} se mantiene en privado y se usa para firmar mensajes. k_{pub} es la clave de verificación pública que se reparte a todo el mundo. Cualquiera con esta clave puede verificar su firma.
- $s \leftarrow sign(k_{pr}, m)$ El método de firma toma un mensaje m y una clave privada, k_{pr}, como entradas y produce como salida una firma s para el mensaje mediante k_{pr}.
- $isValid \leftarrow verify(k_{pub}, m, s)$ El método de verificación toma un mensaje, una firma y una clave pública como entrada. Devuelve un valor booleano, $isValid$, que será verdadero si s es una firma válida para el mensaje bajo la clave pública k_{pub}, y falso en caso contrario.

Se requiere que se mantengan las siguientes dos propiedades:

1. Las firmas válidas deben verificar: $verify(k_{pub}, m, sign(k_{pr}, m)) == true$.
2. Las firmas son existencialmente infalsificables: un adversario que conoce la clave pública de una víctima y ve sus firmas en otros mensajes no puede falsificar su firma en un mensaje para el cual no ha visto su firma. En otras palabras, el atacante no podrá crear una pareja (m, s) válida bajo la clave pública de la víctima k_{pub}.

Es importante señalar que una firma digital por sí misma no sirve de nada si no va acompañada del mensaje. Una firma digital sin el mensaje es el equivalente a una firma manuscrita en una tira de papel sin el contrato o un cheque que se supone que está firmado.

El algoritmo $generateKeys$ debe ser aleatorio, porque debe generar claves diferentes para personas diferentes. Los algoritmos de firma pueden o no estar aleatorizados (en particular, el que se usa en Bitcoin lo está) y, por lo tanto, se requiere una buena fuente de aleatoriedad. Por el contrario, $verify$ siempre será determinista.

6.1.1. Primera aproximación a la firma digital

La Fig. 6.1 representa esquemáticamente una primera aproximación a la firma digital usando criptografía de clave pública.

1. Para firmar un mensaje m, éste podría cifrarse con la clave privada de su propietario k_{pr}, dando como resultado $s = sign(k_{pr}, m) = E_{k_{pr}}(m)$. Como nadie más tiene acceso a dicha clave privada, solo él, y nadie más que él, puede realizar esa operación.
2. Para verificar su firma, el mensaje original y el mensaje firmado (cifrado con la clave privada) (m, s), se comparten públicamente para que cualquiera pueda realizar la operación de verificación. Como su clave pública k_{pub} es pública, cualquier persona puede conocerla, lo que significa que cualquier persona podrá descifrar la firma s con la clave pública: $m' = D_{k_{pub}}(s)$.
3. Para validarla, basta con comparar el resultado m' con el mensaje original m. Si ambos coinciden, $m = m'$, la firma se ha validado correctamente.

¿Qué implica esta validación positiva? Significa que aporta tres servicios de seguridad:

1. INTEGRIDAD: sabes que el documento original en claro no ha sido manipulado, porque no habría coincidido con el documento recuperado al descifrar la copia cifrada.
2. AUTENTICACIÓN: verificas la identidad del firmante, ya que sabes que el documento fue firmado por el legítimo propietario de la clave privada, operación que solo pudo realizar él y nadie más.

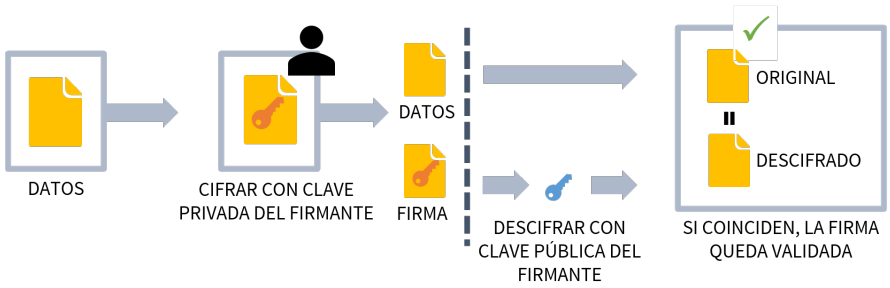


Figura 6.1: Primera aproximación a la firma digital.

3. NO REPUDIO: en el futuro, el firmante no podrá echarse atrás y decir que él no lo firmó. Después de todo, ¿quién sino él podría haberlo cifrado con su clave privada, solo conocida por él?

En el esquema de firma recién descrito, la firma es al menos tan larga como el mensaje, lo que supone una desventaja cuando el mensaje es largo. Recordarás que los algoritmos de clave pública son muy lentos y, por consiguiente, adecuados sólo para cifrar pequeñas cantidades de texto; mientras que los algoritmos de clave secreta son muy rápidos y, por tanto, ideales para cifrar grandes volúmenes de datos.

Pero en este caso, para firmar un documento, lo estás cifrando mediante criptografía de clave pública. Si el documento a firmar es muy grande, de gigas, ¡tardarás una eternidad! ¿Cómo resolverlo? La respuesta está en los hashes.

Para remediar la situación, se utiliza una función hash y el esquema de firma se aplica entonces al hash del mensaje, en lugar de al mensaje mismo.

6.1.2. Segunda aproximación a la firma digital

La Fig. 6.2 representa esta segunda aproximación más eficiente.

1. Comenzando con un mensaje m, Alice calcula el hash $h = H(m)$. Esta salida h es significativamente más pequeña que el mensaje m y, por lo tanto, firmar el hash puede hacerse más rápidamente que firmar el mensaje completo.

2. Alice cifra el hash del mensaje con su clave privada $s = sign(k_{pr}, h) = E_{k_{pr}}(h)$ y lo utiliza como firma del mensaje s. El par (m, s) ahora transmite básicamente el mismo conocimiento que el esquema de firma del primer enfoque, pero con la ventaja de que es más rápido de crear (bajo el supuesto razonable de que la operación de hash es rápida) y requiere menos recursos para la transmisión o el almacenamiento. Si la función de hash es unidireccional, a un atacante le resultará difícil (¿imposible?) encontrar un mensaje diferente m' tal que $H(m') = H(m)$. Alice ya tiene el documento firmado digitalmente.

3. Si Bob quiere verificar la firma, por un lado, calcula el hash de los datos recibidos $h_1 = H(m)$.

4. Por otro lado, Bob descifra con la clave pública de Alice el hash que había sido cifrado por Alice con su clave privada $h_2 = D_{k_{pub}}(s)$.

5. Por último, Bob compara ambos hashes: ¿$h_1 = h_2$? Si coinciden, entonces nadie ha manipulado los datos ni la firma (INTEGRIDAD), sabe quién los ha firmado (AUTENTICACIÓN) y Alice no podrá repudiar dicha firma en el futuro (NO REPUDIO).

Ahora todo funciona de maravilla. ¿O quizás no?

6.1.3. Ataque de intermediario (o de la persona en el medio)

Hasta ahora hemos asumido que cuando dos usuarios se intercambian sus claves públicas, éstas son auténticas y no han sido manipuladas. Pero ¿cómo estar seguro de que una clave pública pertenece a una persona? Sin una garantía segura de la identidad del propietario, se podría realizar el ataque de intermediario (o de la persona en el medio), explicado en el vídeo `VideoAtaqueIntermediario.mp4` y esquematizado en la Fig. 6.3.

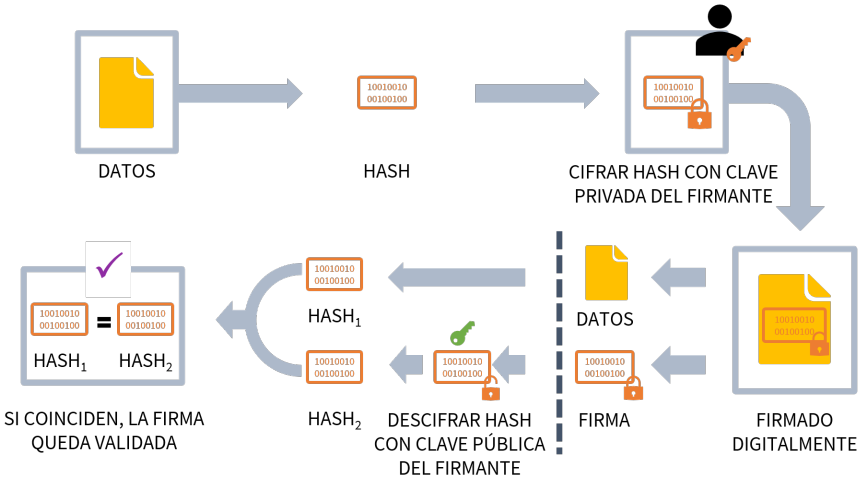


Figura 6.2: Segunda aproximación a la firma digital.

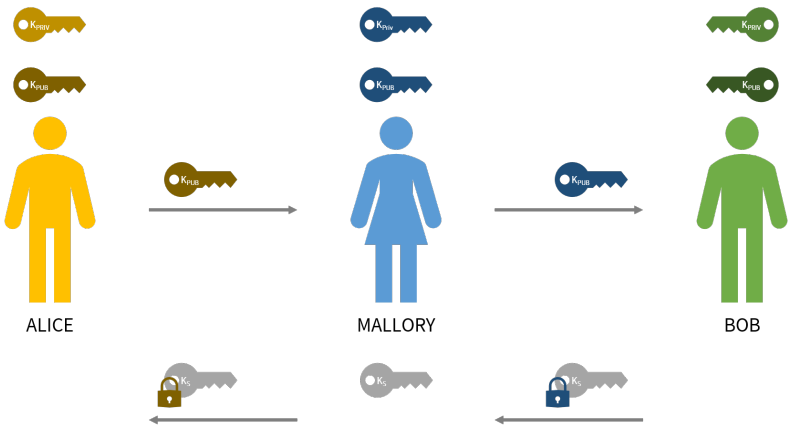


Figura 6.3: Ataque de intermediario (o de la persona en el medio).

Imagina que un atacante, Mallory, puede interceptar y alterar las comunicaciones entre Alice y Bob. Por supuesto, tendrá su propia pareja de claves pública y privada, (k_{pub_M}, k_{pr_M}). Cuando Alice le envía a Bob su clave pública k_{pub_A}, Mallory la intercepta y le envía a Bob su propia clave pública k_{pub_M}, haciéndole creer a Bob que se trata de la clave de Alice.

Ahora Bob genera por ejemplo una clave secreta k_s de un algoritmo de criptografía simétrica para cifrar grandes volúmenes de información. Para transmitir la clave k_s de manera segura, pretende cifrarla con la clave pública de Alice k_{pub_A}. Por desgracia, como no es consciente del cambio, en realidad la está cifrando con la clave pública del atacante k_{pub_M}.

Una vez cifrada, Bob se la envía a Alice y es interceptada por Mallory. Como Mallory posee obviamente la correspondiente clave privada k_{pr_M}, la descifra sin dificultad. Ya conoce la clave secreta k_s.

A continuación Mallory la cifra de nuevo, pero ahora usando la clave pública original que había interceptado a Alice, k_{pub_A} y se la envía.

Alice la descifra con su correspondiente clave privada k_{pr_A}, y obtiene la clave secreta k_s con la que cifrará los mensajes intercambiados con Bob. Ninguno de los dos sospecha que el atacante ha obtenido también esa misma clave secreta y podrá espiar todas sus comunicaciones.

¿Cómo evitar este ataque? Mediante los certificados digitales. Constituyen la solución al gravísimo problema de confianza que sufre la criptografía de clave pública.

6.1.4. Certificados digitales

La criptografía de clave pública plantea varios interrogantes, que generan un serio problema de confianza:

- ¿Cómo tener certeza de que la clave pública de un usuario corresponde realmente a ese

individuo y no ha sido falsificada por otro?

- ¿Por qué fiarse de esa clave pública antes de confiarle algún secreto?
- ¿Quién verifica la identidad del poseedor de la clave pública?

Aunque los esquemas de clave pública no requieren un canal seguro, sí requieren canales autenticados para la distribución de las claves públicas.

La respuesta a todos estos interrogantes reside en los certificados digitales. Son documentos electrónicos que garantizan la identidad de una persona y que ésta posee una determinada clave pública.

Una tercera parte de confianza, llamada la Autoridad de Certificación (AC), se ocupa de validar esta identidad y de emitir los certificados.

Un certificado contiene la siguiente información:

- **Versión del estándar X509**, actualmente es 3.
- **Número de serie usado por la autoridad de certificación**. Hoy se exige que sean aleatorios, no secuenciales, con una entropía mínima de 20 bits.
- **Identificador del algoritmo**, que debe ser el mismo que el *Algoritmo de firma del certificado*, descrito más abajo. Se ubica en esta posición para ser protegido por la firma.
- **Identificación de la autoridad de certificación**, mediante un nombre distinguido (*Distinguished Name*, DN) único:
  - CN - CommonName
  - L - LocalityName
  - ST - StateOrProvinceName
  - O - OrganizationName
  - OU - OrganizationalUnitName
  - C - CountryName
- **El período de validez del certificado**: desde cuándo hasta cuándo.
- **El nombre distinguido único del titular del certificado**. Puede tratarse de una persona física o jurídica. La autoridad verifica esta identidad de forma más o menos fehaciente, como se verá.
- **Información de la clave pública del titular**, especificada por:
  - Algoritmo
  - Tamaño de la clave
  - Exponente
  - Módulo
- **Extensiones (opcional)**: se introdujeron en la versión 3. Una AC puede utilizar extensiones para emitir un certificado sólo para un propósito específico, por ejemplo, sólo para servidores HTTP. Las extensiones pueden ser *críticas* o *no críticas*. Las no críticas pueden ser ignoradas, mientras que las críticas deben ser aplicadas y todo el certificado debe ser rechazado si el sistema no reconoce una extensión crítica.
- **Algoritmo de firma del certificado**. Existen multitud de algoritmos seguros en uso: RSA, DSA, ECC, etc.
- **Firma del certificado**: el valor de la firma. Este valor corresponde al resultado de cifrar con la clave privada de la AC el hash de la información del certificado. **Esta firma permite verificar que la clave pública que aparece en el certificado efectivamente pertenece al titular del certificado**, porque así lo certifica la autoridad de certificación.

En resumen, un certificado digital recoge la garantía de una autoridad que da fe de que una clave pública pertenece a una persona (física o jurídica) dada. ¿Cuánto puedes fiarte de su palabra? Veámoslo.

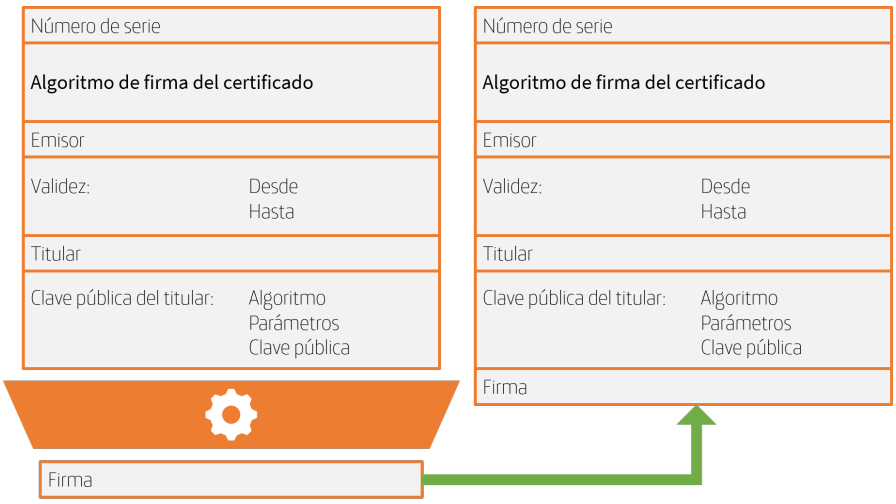


Figura 6.4: Proceso de generación del certificado digital de un usuario por la Autoridad de Certificación.

6.1.4.1. No todos los certificados fueron creados con el mismo nivel de confianza

La misión principal de una autoridad de certificación es validar la identidad de la entidad que solicita un certificado. Y no todos los certificados ofrecen el mismo grado de confianza en la identidad del titular, porque el proceso de validación puede variar enormemente de unos tipos de certificado a otros.

Existen tres niveles de validación para sitios web, cuyas características se resumen en la Fig. 6.5:

- **Validación de dominio**: los certificados de validación de dominio (DV) representan el nivel más bajo de validación, ya que la AC solo verifica que quien solicita el certificado posee el dominio para el que lo pide y que dicho dominio está registrado en Whois. La propiedad del dominio se comprueba típicamente a través de un correo electrónico de confirmación enviado a una dirección en dicho dominio. Si el destinatario sigue el enlace del correo electrónico, se emite el certificado. **La emisión de certificados DV está totalmente automatizada y puede ser muy rápida**. La duración depende en gran medida de la rapidez con la que se responda al correo electrónico de confirmación. **No incluyen el nombre de la entidad poseedora del dominio**, solo el nombre de dominio. Por ejemplo, los certificados de Let's Encrypt pertenecen a esta categoría. ¿Qué garantías aportan sobre la identidad del titular del dominio? ¡Ninguna! Un sitio web con un certificado DV es seguro en el sentido de proteger la confidencialidad, pero no garantiza en absoluto la identidad del sitio. Sabes que tus datos viajan cifrados, pero no tienes ni idea de a quién se los envías. **Son ideales para sitios web de particulares, pymes, blogs, foros**, sin pretensión de ofrecer servicios de pago y sin tratar datos personales.

- **Validación de organización**: los certificados de organización validada (OV) requieren verificar la identidad del titular del dominio. Las prácticas de validación de la identidad del solicitante varían drásticamente de unas autoridades de certificación a otras, existiendo una gran inconsistencia y diversidad de garantías, aunque todas **exigen el envío de algún documento oficial** para verificar el dominio, el CIF, el teléfono, la dirección física y algún otro requisito. **El proceso puede tardar de uno a tres días laborables**. **Incluyen el nombre de la organización en el contenido del certificado**. Estos certificados resultan **indicados para instituciones y empresas que desean una presencia sólida en Internet**, pero sin ofrecer servicios transaccionales a través de la web.

- **Validación extendida**: los certificados de validación extendida (EV) ofrecen **el mayor grado de confianza**. También requieren la verificación de la identidad y la autenticidad, pero con requisitos muy estrictos. Se introdujeron para solucionar la falta de coherencia de los certificados OV. Exigen incluso más documentación que los certificados OV para verificar fehacientemente la identidad del solicitante y sus operaciones de negocio. También **incluyen en el certificado el nombre de la organización** a la que se le ha emitido e incluso **se ve la organización al hacer clic sobre el candadito**, sin necesidad de

hurgar en los detalles del certificado, facilitando así la verificación de su identidad por parte de los visitantes a su sitio web. Es el más **indicado para aquellos sitios web que ofrecen servicios que involucren transacciones financieras**. Normalmente lo utilizan los bancos, grandes organizaciones y tiendas online de renombre. Debido a los controles implantados, la obtención de un certificado EV suele llevar **de 5 a 15 días**.



Figura 6.5: Diferencias entre los distintos niveles de validación de la información de los certificados.

Es importante resaltar que todos ellos ofrecen la misma fortaleza criptográfica: 256 bits de cifrado simétrico y 2.048 bits de cifrado de clave pública. La diferencia entre unos y otros no reside en la criptografía, idéntica para todos ellos, sino en los procesos de validación, más o menos estrictos, lo que ofrece más o menos garantías sobre la identidad de la persona (física o jurídica) detrás de un sitio web.

6.1.4.2. Cómo funcionan los certificados digitales

Para la generación del certificado, el usuario calcula su propio par de claves asimétricas y se limita a solicitar a la AC que firme la clave pública, como se muestra en el siguiente protocolo sencillo para un usuario llamado Alice:

1. Para generar una clave pública certificada, Alice primero genera un par de claves públicas/privadas (k_{pub_A}, k_{pr_A}) para algún criptosistema de clave pública, como un esquema de cifrado de clave pública (p.e. RSA) o un esquema de firma (p.e. DSA).

2. A continuación, Alice presenta su clave pública k_{pub_A} a la AC. La AC verifica que Alice es quien dice ser, y una vez que la AC está convencida de que está hablando con Alice, la AC construye una declaración m que dice "la clave pública $k_{pub,A}$ pertenece a Alice": $m = (k_{pub_A}, ID_A)$.

3. Finalmente, la AC firma el mensaje m usando su propia clave privada $k_{pr_{CA}}$ y devuelve a Alice tanto el mensaje m como la firma $s = E_{k_{privCA}}(m)$.

$$\text{Cert}_A = [(k_{pub_A}, ID_A), E_{k_{pr,CA}}(k_{pub_A}, ID_A)]$$

Este par Cert_A es el certificado para k_{pub_A}. Cuando Bob necesita la clave pública de Alice, primero obtiene el certificado de Alice y verifica la firma de la AC en el certificado. Si la firma es válida, Bob confía en que k_{pub_A} es la clave pública de Alice. El propósito principal de la firma digital de la AC es probar a Bob que la declaración m fue emitida por la AC. Por supuesto, para verificar la firma de la AC, Bob necesita la clave pública de la AC $k_{pub_{CA}}$. Normalmente, las claves públicas de las ACs vienen preinstaladas en el sistema operativo o en el navegador web. En otras palabras, simplemente asumimos que la clave pública de la AC ya está disponible en la máquina de Bob.

Ahora, si Mallory intenta reemplazar k_{pub_A} por k_{pub_M}, será detectado. Así, se dice que los certificados vinculan la identidad de un usuario a su clave pública.

Por supuesto, lo anterior puede generalizarse de manera que el certificado de la AC para Alice asocie varias claves públicas con su identidad, por ejemplo, claves públicas tanto para cifrar como para firmar.

PRÁCTICA 6.1: Creando una Autoridad de Certificación (AC) en OpenSSL

1. Genera una pareja de claves para la AC, a la que llamarás miAC.

   ```
   $ openssl genrsa -out miAC.key 2048
   ```

2. Inspecciona las claves recién creadas.

   ```
   $ openssl rsa -in miAC.key -noout -text
   ```

3. Genera un certificado autofirmado para la AC. Como las ACs están en la raíz de la jerarquía de certificación, lógicamente se firman el certificado a sí mismas con su propia clave privada.

   ```
   $ openssl req -new -x509 -key miAC.key -out miAC.crt
   -----
   Country Name (2 letter code) [AU]:ES
   State or Province Name (full name) [Some-State]:MADRID
   Locality Name (eg, city) []:MADRID
   Organization Name (eg, company) [Internet Widgits Pty Ltd]:11P
   Organizational Unit Name (eg, section) []:Cert
   Common Name (e.g. server FQDN or YOUR name) []:miAC
   Email Address []:miAC@11paths.com
   ```

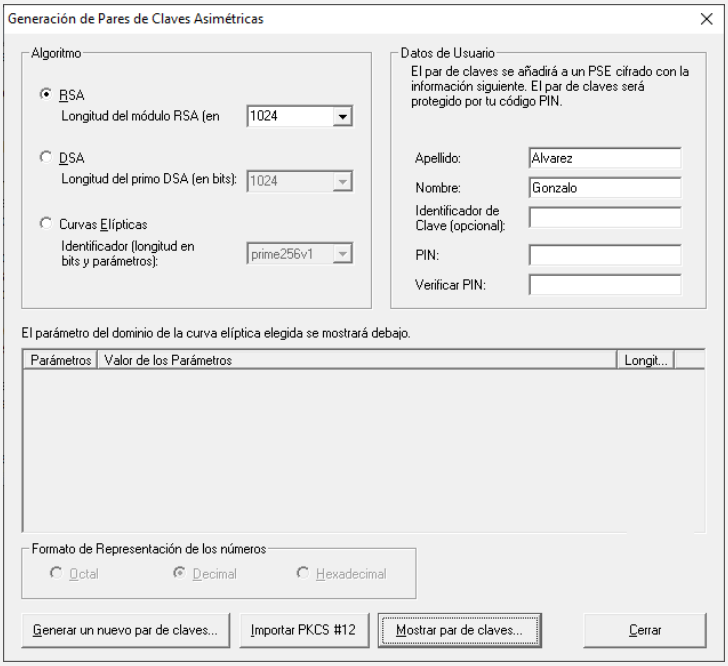
4. Examina los contenidos del certificado autofirmado de tu AC.

   ```
   $ openssl x509 -in miAC.crt -noout -text
   ```

Ya estás listo para firmar peticiones de certificado de usuarios y sitios web.

PRÁCTICA 6.2: Generando certificados digitales en CrypTool

1. Selecciona `Firma digital / PKI >PKI >Generar / Importar claves`.



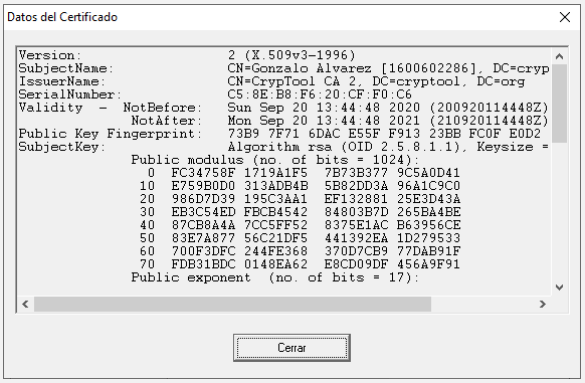
2. Selecciona el tipo de clave deseado: RSA (v. 6.2), DSA (v. 6.3), ECC (v. 6.4) y sus parámetros correspondientes.

3. Rellena los datos de usuario.

4. Pulsa `Generar un nuevo par de claves`.

5. Pulsa `Mostrar par de claves`. Aparece un listado con todos los pares de claves disponibles, entre ellos, el que acabas de crear.

6. Pulsa `Mostrar certificado`. Aparecen los datos del certificado.



7. Pulsa `Exportar PSE (PKCS#12)`. Te pedirá el PIN que introdujiste anteriormente y exportará los datos del certificado a un archivo de PKCS#12 y extensión `.p12`.

8. Abre el archivo con OpenSSL.

```
$ openssl pkcs12 -in [Alvarez][Gonzalo][RSA-1024][1600602286].p12
-nodes -passin pass:1234
```

PRÁCTICA 6.3: Generando un certificado para un servidor web con tu propia AC en OpenSSL

1. Crea un archivo de petición de firma de certificado para tu servidor web, alojado en example.com.

```
$ openssl req -newkey rsa:2048 -keyout example.com.key -out
example.com.csr -nodes
Generating a RSA private key
...........+++++
writing new private key to 'example.com.key'
-----
Country Name (2 letter code) [AU]:ES
State or Province Name (full name) [Some-State]:MADRID
Locality Name (eg, city) []:MADRID
Organization Name (eg, company) [Internet Widgits Pty
Ltd]:example.com
Organizational Unit Name (eg, section) []:web
Common Name (e.g. server FQDN or YOUR name) []:www.example.com
Email Address []:info@example.com
```

2. Inspecciona el contenido de la petición.

```
$ openssl req -in example.com.csr -noout -text
```

3. Crea y firma el nuevo certificado para el sitio web a partir de la petición de firma contenida en el archivo CSR.

```
$ openssl x509 -req -in example.com.csr -CA miAC.crt -CAkey miAC.key
-CAcreateserial -out example.com.crt
Signature ok
subject=C = ES, ST = MADRID, L = MADRID, O = example.com, OU = web,
CN = www.example.com, emailAddress = info@example.com
Getting CA Private Key
```

El parámetro `-CAcreateserial` le indica a OpenSSL que crea un archivo para llevar

un recuento de los certificados emitidos. Si no se especifica, el nombre predeterminado es el de la AC, con la extensión .srl.

```
$ type miAC.srl
758D0C41D664CDB9C7FE5EE1031D6E1B0E497FB2
```

4. Examina el contenido del certificado para el sitio web y observa quién lo ha emitido y a quién.

```
$ openssl x509 -in example.com.crt -text -noout
...
Issuer: C = ES, ST = MADRID, L = MADRID, O = 11P, OU = Cert, CN =
miAC, emailAddress = miAC@11paths.com
...
Subject: C = ES, ST = MADRID, L = MADRID, O = example.com, OU = web,
CN = www.example.com, emailAddress = info@example.com
```

5. Crea un servidor web rudimentario a la escucha en el puerto 4430, usando el certificado recién creado y firmado.

```
$ openssl s_server -cert example.com.crt -key example.com.key -www
-accept 4430
```

6. Conéctate con tu navegador a https://localhost:4430/.

   Te saltará un aviso, advirtiéndote de que el certificado del sitio web no es de confianza, lógico porque ha sido firmado por una AC que no está en su lista de autoridades raíz.

6.1.5. Tercera aproximación a la firma digital

Ahora que cuentas en tu arsenal criptográfico con certificados digitales, puedes repetir el proceso de firma de la segunda aproximación, pero con una ventaja: estarás seguro de que la clave pública pertenece a quien dice ser su propietario y no podrá ocurrir el ataque de intermediario sin que sea detectado. Este nuevo esquema se representa en la Fig. 6.6.

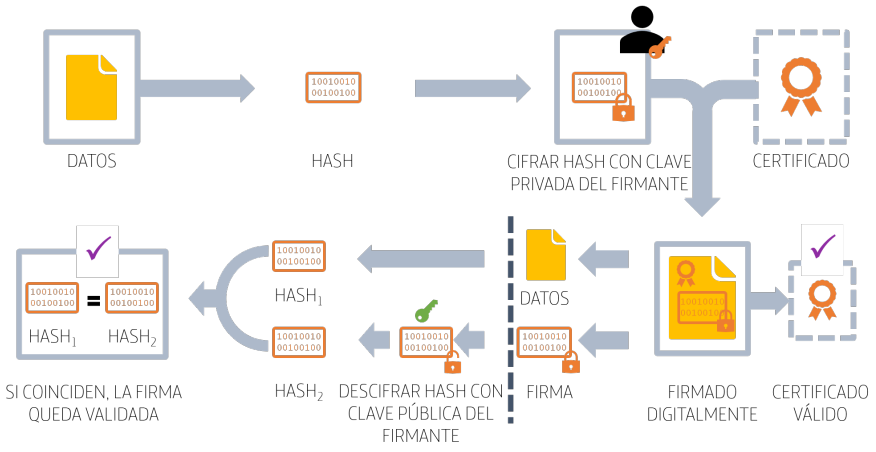


Figura 6.6: Tercera aproximación a la firma digital.

1. Alice calcula el hash de los datos $h = H(m)$ y lo cifra con su clave privada $s = E_{k_{pr}}(h)$. La correspondiente clave pública está dentro de un certificado emitido por una autoridad de certificación de confianza.

2. Si Bob quiere verificar la firma, empezará por validar el certificado. Ahora que ya tiene certeza de que la identidad del firmante es realmente Alice, está preparado para validar que los datos han sido firmados correctamente.

3. Por un lado, se calcula el hash de los datos recibidos $h_1 = H(m)$.

4. Por otro, con la clave pública de Alice extraída del certificado descifra el hash que había sido cifrado por Alice con su clave privada $h_2 = D_{k_{pub}}(s)$.

5. Finalmente, compara ambos hashes. Si coinciden, entonces Bob ha obtenido los servicios de INTEGRIDAD, AUTENTICACIÓN y NO REPUDIO.

Si se desea añadir CONFIDENCIALIDAD, entonces basta con cifrar los datos usando un algoritmo de cifrado simétrico. Obviamente, habrá que cifrar a su vez la clave secreta con la clave pública del destinatario, la cual se extrae de su certificado digital, tal y como se vio al explicar la criptografía híbrida.

PRÁCTICA 6.4: Tercera aproximación a la firma digital con OpenSSL

1. Crea un archivo de petición de firma de certificado (CSR) creando a la vez una nueva pareja de claves RSA.

```
$ openssl req -new -newkey rsa:1024 -keyout gonzalo.key -nodes -out
gonzalo.csr
You are about to be asked to enter information that will be
incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name
or a DN.
There are quite a few fields but you can leave some blank For some
fields there will be a default value, If you enter '.', the field
will be left blank.
-----
Country Name (2 letter code) [AU]:ES
State or Province Name (full name) [Some-State]:MADRID
Locality Name (eg, city) []:MADRID
Organization Name (eg, company) [Internet Widgits Pty Ltd]:11P
Organizational Unit Name (eg, section) []:CRYPTO
Common Name (e.g. server FQDN or YOUR name) []:Gonzalo Alvarez
Email Address []:gonzalo.alvarezmaranon@telefonica.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
```

2. Verifica la información dentro del archivo de petición de firma (CSR).

```
$ openssl req -verify -in gonzalo.csr -noout -text
verify OK
Certificate Request:
Data:
Version: 1 (0x0)
Subject: C = ES, ST = MADRID, L = MADRID, O = 11P,
OU = CRYPTO, CN = Gonzalo Alvarez, emailAddress =
gonzalo.alvarezmaranon@telefonica.com
Subject Public Key Info:
Public Key Algorithm: rsaEncryption
RSA Public-Key: (1024 bit)
Modulus:
00:ce:de:19:cd:ae:ba:e7:93:e9:96:77:e2:f5:6b:
...
Exponent: 65537 (0x10001)
Attributes:
challengePassword :1234
Signature Algorithm: sha256WithRSAEncryption
52:c3:5a:8e:17:ad:bb:5f:96:47:2b:54:c6:34:d7:7c:ab:23:
...
```

3. Imagina que le envías este archivo CSR a una AC, por ejemplo, a la que has creado antes, miAC, para que ella lo firme.

```
$ openssl x509 -req -in gonzalo.csr -CA miAC.crt -CAkey miAC.key
-out gonzalo.crt
Signature ok subject=C = ES, ST = MADRID, L = MADRID, O
= 11P, OU = CRYPTO, CN = Gonzalo Alvarez, emailAddress =
gonzalo.alvarezmaranon@telefonica.com Getting CA Private Key
```

4. Verifica el contenido del certificado.

```
$ openssl x509 -in gonzalo.crt -text -noout
```

5. Cualquier persona que tenga tu certificado podrá verificar su información y sabrá (asumiendo que miAC fuera una autoridad reconocida) que tú eres el legítimo propietario de la clave pública listada dentro del certificado y, por consiguiente, de su correspondiente clave privada. Por supuesto, tu clave privada solo debería abandonar el dispositivo en el que ha sido creada viajando bien protegida. Puedes exportarla a un archivo de intercambio de claves en formato PKCS#12 (PFX), protegiéndolo con una contraseña de calidad, que te pedirán en el momento.

```
$ openssl pkcs12 -export -inkey gonzalo.key -in gonzalo.crt -out
gonzalo.pfx
Enter Export Password:
Verifying - Enter Export Password:
```

6. Puedes importar el certificado dentro de Windows sin más que hacer doble clic sobre el archivo `gonzalo.pfx`.

7. Finalmente, imagina que alguien quiere enviarte una clave simétrica cifrada AES de 128 bits con tu clave pública. Seguirá los siguientes pasos.

   a) Genera la clave secreta.

   ```
   $ openssl rand -out secret.key 16
   ```

   b) Extrae la clave pública del certificado.

   ```
   $ openssl x509 -pubkey -in gonzalo.crt -noout -nocert -out
   gonzalo.public.key
   ```

   c) Cifra la clave secreta con la clave pública extraída del certificado.

   ```
   $ openssl pkeyutl -encrypt -in secret.key -inkey gonzalo.pub
   -pubin -out secret.enc
   ```

6.1.6. Estudio de caso #1: cifrado/firmado de correos electrónicos con SMIME

S/MIME (Secure Multipurpose Internet Mail Extensions) proporciona una forma coherente de enviar y recibir datos MIME seguros, basado en el estándar MIME de Internet. S/MIME ofrece los siguientes servicios de seguridad criptográfica para aplicaciones de mensajería electrónica:

- Autenticación, integridad de los mensajes y no repudio del origen (mediante *firmas digitales*).
- Privacidad y seguridad de los datos (mediante *cifrado*).

Si examinas el código fuente de un correo electrónico firmado, encontrarás las siguientes cabeceras de S/MIME:

```
Content-Type: multipart/signed; protocol="application/pkcs7-signature";
micalg=sha1; boundary=""
This is a cryptographically signed message in MIME format.
[Texto del correo]
Content-Type: application/pkcs7-signature; name="smime.p7s"
Content-Transfer-Encoding: base64
Content-Disposition: archivo adjunto; filename="smime.p7s"
Content-Description: Firma criptográfica S/MIME
```

Mientras que en el caso de un correo electrónico cifrado, las cabeceras son:

```
Content-Type: application/pkcs7-mime; name="smime.p7m"; smime-type=enveloped-data
Content-Transfer-Encoding: base64
Content-Disposition: archivo adjunto; filename="smime.p7m"
Content-Description: Mensaje cifrado S/MIME
[Número y letras aleatorios]
```

Se ofrecen los siguientes casos de uso:

- Para crear un mensaje firmado por S/MIME, necesitas una pareja de claves RSA y un certificado X.509 con la clave pública.
- Para crear un mensaje cifrado por S/MIME, necesitas un certificado X.509 para cada destinatario. De esta manera, podrás cifrar con la clave pública de cada destinatario la clave de sesión con la que se cifra el contenido del mensaje.
- Para crear un mensaje firmado y cifrado por S/MIME, primero tienes que crear un mensaje firmado y, a continuación, cifrar el mensaje firmado con los certificados de los destinatarios.

PRÁCTICA 6.5: Cifrando y firmando con S/MIME usando OpenSSL.

Cifrar:

```
$ openssl smime -encrypt -in message.txt -aes256 -out mail.msg
gonzalo.pem
```

Descifrar:

```
$ openssl smime -decrypt -in mail.msg -recip gonzalo.pem -inkey
gonzalo.key
```

Firmar:

```
$ openssl smime -sign -in message.txt -text -out mail.msg -inkey priv.key
-signer gonzalo.pem
```

Verificar firma:

```
$ openssl smime -verify -in mail.msg -CAfile gonzalo-chain.pem -signer
gonzalo.pem -out signedtext.txt
```

Puedes listar los certificados contenidos en el archivo PKCS7:

```
$ openssl smime -pk7out -in mail.msg | openssl pkcs7 -print -text -noout
```

El resultado de la ejecución del comando muestra el contenido del mensaje cifrado en SMIME con cada una de sus partes:

```
PKCS7:
  type: pkcs7-envelopedData (1.2.840.113549.1.7.3)
  d.enveloped:
    version: 0
    recipientinfo:
        version: 0
        issuer_and_serial:
          issuer: C=RU, ST=Moscow, L=Kremlin, O=ChernayaZvezda,
          CN=Yeguery Burkov
          serial: 0x273F35DA474CC0B5B5269A48903FC81A4863D4C4
        key_enc_algor:
          algorithm: rsaEncryption (1.2.840.113549.1.1.1)
          parameter: NULL
        enc_key:
          0000 - 2a a8 55 67 11 73 a3 55-65 fa 65 04 d6 74 5a
          000f - 91 f5 48 19 46 25 25 72-32 8b c4 98 b4 ed 7b
          001e - e2 5a 51 a8 f1 35 6f c8-88 de f8 e3 83 8a 09
          002d - 05 ab 00 0b 02 2e 96 76-c1 c6 4f 18 9f bc 6c
          ...
    enc_data:
      content_type: pkcs7-data (1.2.840.113549.1.7.1)
      algorithm:
        algorithm: aes-256-cbc (2.16.840.1.101.3.4.1.42)
```

```
        parameter: OCTET STRING:
          0000 - cf 9e d8 66 06 6c 3d 30-8b 07 cc 7e 94 3b 7e
          000f - a0
      enc_data:
        0000 - 09 03 70 f5 da 7d 3f a3-3d 66 d8 49 b3 97 e5
        000f - 4b 3e 5a 99 5c 08 31 34-19 66 f8 30 9b b4 41
        001e - 18 19 7c 58 f4 a3 c5 24-66 c1 aa d1 7d 1d 50
        002d - 90 33 45 60 1c b7 c8 5b-f2 2f ee 5c c6 0c 6d
        ...
```

Fíjate bien en lo que significa cada campo:

- `enc_key`: contiene la clave simétrica AES de 256 bits cifrada con la clave pública del destinatario.
- `parameter: OCTET STRING`: contiene el IV.
- `enc_data`: contiene el mensaje de correo cifrado con la clave simétrica AES en modo CBC.

Puedes visualizar los contenidos del archivo cifrado/firmado en SMIME usando otro comando de OpenSSL:

```
$ openssl cms -in mail.msg -inform SMIME -cmsout -print
```

O puedes verificar una firma sin comprobar la cadena de certificados usando:

```
$ openssl cms -verify -in mail.msg -inform SMIME -noverify -out contenido.txt
```

Puedes usar el comando `cms` para prácticamente lo mismo que `smime`.

6.1.7. Estudio de caso #2: el estampillado de tiempo

En muchas situaciones es necesario certificar la fecha de creación o de última modificación de un documento. Por ejemplo, en la gestión de la propiedad intelectual, en la firma de acuerdos de confidencialidad o en la contabilización de transacciones financieras. Las marcas de tiempo que se basan en los relojes del sistema no son suficientes, ya que no es difícil alterar la fecha y la hora localmente en una máquina.

El RFC 3161 describe los requisitos que debe cumplir una entidad para poder operar como Autoridad de Estampillado de Tiempo (TSA, *Time Stamping Authority*). Las TSA utilizan tecnología de infraestructura de clave pública (PKI) para aplicar las marcas de tiempo.

La idea básica es la siguiente. Un tercero de confianza T (el agente de la marca de tiempo) añade una marca de tiempo t_1 a un documento digital o archivo de datos presentado D, firma el documento compuesto (garantizando así el momento de su existencia), y devuelve el documento firmado incluyendo t_1 al remitente. La verificación posterior de la firma de T establece entonces, basándose en la confianza en T, la existencia del documento en el momento t_1. Si los datos presentados para el sellado de tiempo son el hash de un documento, no es necesario revelar el contenido del documento en el momento del sellado de tiempo. Esto también proporciona una protección de la privacidad frente a los fisgones en el caso de envíos a través de un canal no seguro, y reduce el ancho de banda y los costes de almacenamiento para los documentos de gran tamaño.

A continuación se presenta un resumen de los pasos descritos, ilustrados en la Fig. 6.7.

- La aplicación cliente A crea un hash como identificador único de los datos o el archivo D que necesita un sello de tiempo y lo envía a T: $h(D)$.
- T anota la fecha y la hora t_1 de recepción. A partir de este momento, cualquier cambio, incluso por un solo bit de información, en el archivo original requerirá la comunicación de los cambios al servidor de la TSA.
- T combina el hash $h(D)$ y t_1. El resultado se firma digitalmente con la clave privada de T, creando una estampilla o sello de tiempo que se envía al cliente: $s = sign_{kpriv_T}(h(D), t_1)$. Esta estampilla s contiene la información que la aplicación cliente necesitará para verificar la hora más adelante.
- La aplicación cliente recibe la estampilla y la registra en el documento.

Cuando en el futuro se abran los datos o el archivo con la estampilla de tiempo resultante, la aplicación cliente utilizará la clave pública de la TSA para autenticar la TSA (es decir, para validar que la marca de tiempo procede de una TSA de confianza) y recalcular el hash de los datos originales. Este nuevo hash se compara con el creado originalmente (paso 1). Si se ha realizado algún cambio en los datos desde que se aplicó la marca de tiempo, esta comprobación del hash fallará y se mostrarán mensajes de advertencia indicando que los datos han sido alterados y que no se debe confiar en ellos.

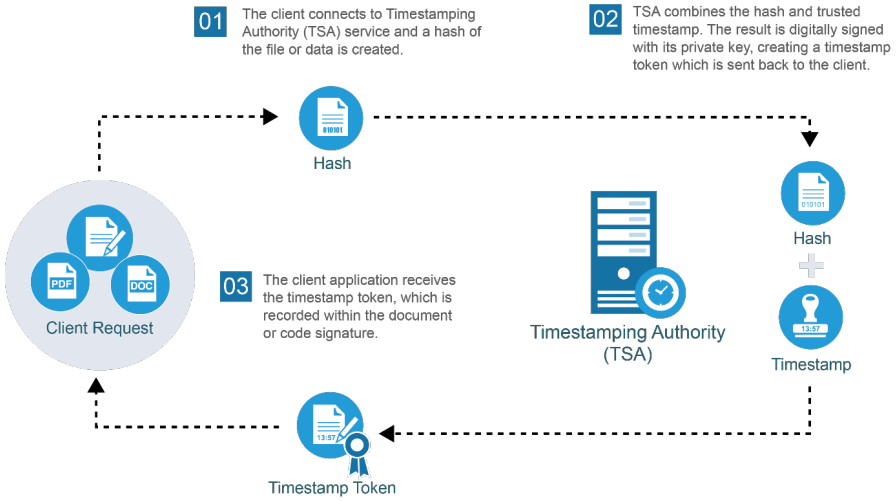


Figura 6.7: Esquema de funcionamiento del estampillado de tiempo digital (fuente: Real World Application of Cryptography).

6.2. Esquema de firma digital con RSA

El esquema de firma RSA se basa en el algoritmo RSA introducido en la lección 4. Su seguridad se basa en la dificultad de factorizar un producto de dos grandes primos (el problema de la factorización de enteros). El protocolo de firma real se muestra a continuación. El mensaje m siendo firmado está en el rango $(1, 2, \ldots, n-1)$.

1. Bob envía su clave pública a Alice $k_{pub} = (e, n)$.
2. Bob calcula la firma del mensaje como $s = m^d \mod n$.
3. Bob envía el mensaje m y la firma s a Alice.
4. Alice verifica la firma de Bob:

$$m' \equiv s^e \mod n$$

$$m' = \begin{cases} \equiv m & \mod n \Rightarrow \text{firma válida} \\ \not\equiv m & \mod n \Rightarrow \text{firma inválida} \end{cases}$$

Como puedes apreciar en el protocolo, Bob calcula la firma s para un mensaje m cifrando m con su clave privada k_{pr}. Bob es el único que puede aplicar k_{pr} y, por lo tanto, la propiedad de k_{pr} lo autentica como el autor del mensaje firmado. Bob añade la firma s al mensaje m y envía ambos a Alice. Alice recibe el mensaje firmado y RSA descifra s usando la clave pública de Bob k_{pub}, obteniendo m. Si m y m' coinciden, Alice sabe dos cosas importantes:

1. El autor del mensaje estaba en posesión de la clave secreta de Bob y, si sólo Bob ha tenido acceso a la clave, debe ser Bob quien firmó el mensaje (*autenticación* de mensajes y *no repudio*).
2. El mensaje no ha sido cambiado en tránsito, por lo que se garantiza la *integridad* del mensaje.

El papel de las claves públicas y privadas se intercambia en comparación con el esquema de cifrado RSA. Mientras que el cifrado RSA aplica la clave pública al mensaje m, el esquema de firma aplica la clave privada. Al otro lado del canal de comunicación, el cifrado RSA requiere el uso de la clave privada por el receptor, mientras que en el esquema de firma digital éste aplica la clave pública para la verificación.

Ejemplo: firma de mensajes con RSA

Bob quiere firmar el mensaje $m = 4$ y enviárselo a Alice. Seguirá los siguientes pasos:

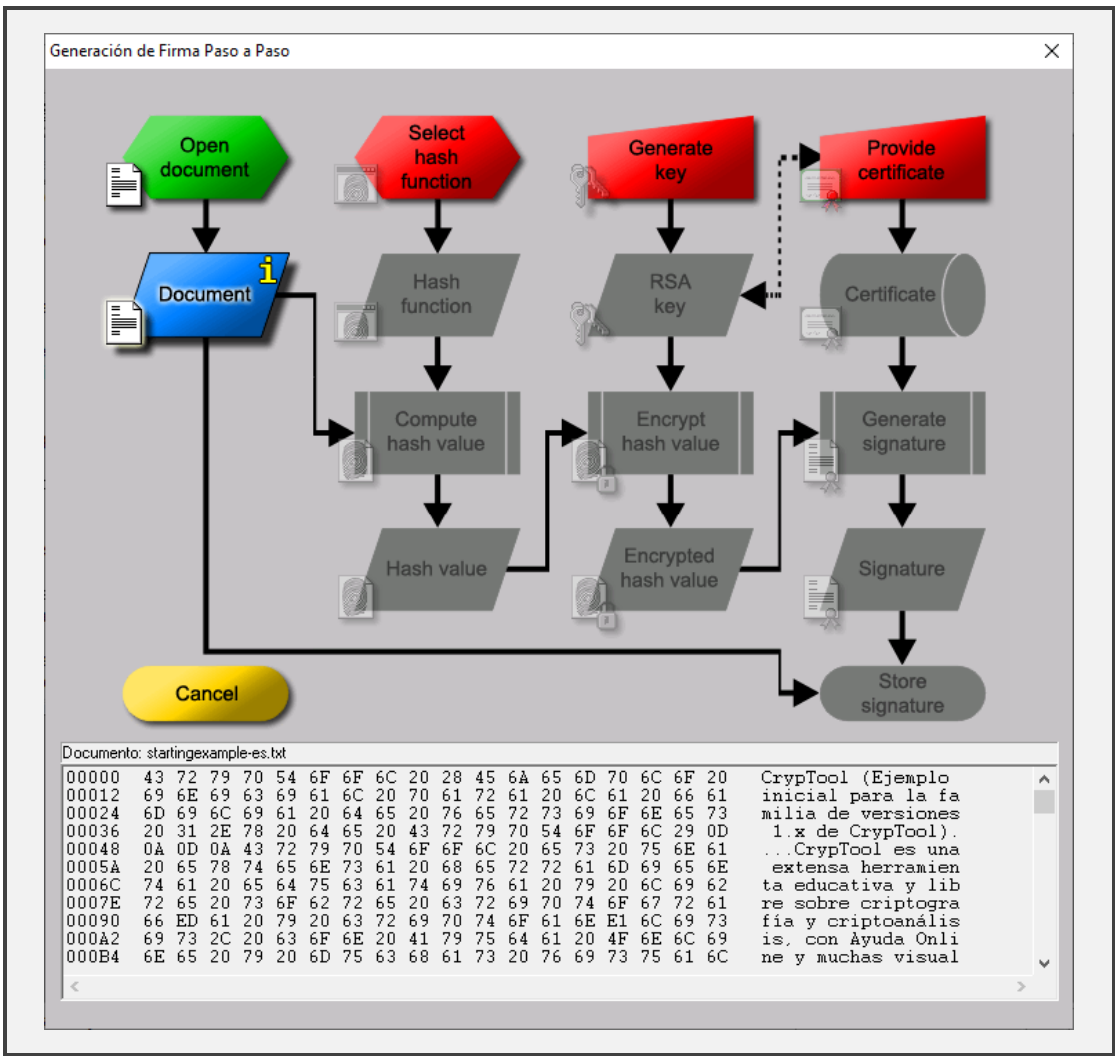
1. Bob genera una pareja de claves RSA:
   a) Elige aleatoriamente dos números primos $p = 3$ y $q = 11$.
   b) Calcula el módulo público: $n = p \cdot q = 33$.
   c) Calcula $\phi(n) = (3-1)(11-1) = 20$.
   d) Elige un exponente público: $e = 3$.
   e) Calcula el exponente privado: $d \equiv e^{-1} \equiv 7 \mod 20$.
2. Bob envía $(n, e) = (33, 3)$ a Alice.
3. Bob calcula la firma del mensaje $m = 4$ como: $s = m^d \equiv 4^7 \equiv 16 \mod 33$.
4. Bob envía el mensaje m y la firma s a Alice: $(m, s) = (4, 16)$.
5. Alice verifica la firma:

$$m' = s^e \equiv 16^3 \equiv 4 \mod 33$$

$$m' \equiv m \mod 33 \Rightarrow \text{firma válida}$$

PRÁCTICA 6.6: Demostración paso a paso de firma digital de RSA con CrypTool

Una vez abierto un archivo cualquiera a firmar, selecciona `Firma digital / PKI >Demostración de firma (generación de una firma)` y sigue las instrucciones que te guiarán paso a paso, desde el hash del archivo, a la generación de los parámetros RSA, el cifrado del hash y su verificación. ¡Muy ilustrativo!



6.2.1. Relleno de RSA

Algunos ataques a las firmas RSA de *libro de texto* pueden prevenirse permitiendo sólo ciertos formatos de mensajes. En términos generales, el formato impone una regla que permite al verificador distinguir entre los mensajes válidos e inválidos; esto se llama *relleno* o *padding*.

6.2.1.1. Relleno PKCS#1 v1.5

Recordarás de la Sec. 4.3.2.5.1, que en PKCS#1 v1.5, el relleno se compone de las siguientes partes:

00	BT	RELLENO ALEATORIO	00	MENSAJE

Las Reglas Básicas de Codificación (BER) especifican cómo se deben codificar los datos para su transmisión, independientemente del tipo de máquina, el lenguaje de programación o la representación dentro de un programa de aplicación. Opcionalmente (no es necesario), se puede crear la codificación ASN.1 BER de un valor MD5, SHA-1, SHA-256 o SHA-512, entre otros, adjuntando estas cadenas al principio de los valores hash de 16 o 20 bytes, respectivamente:

- **MD5** - `3020300C 06082A86 4886F70D 02050500 0410`
- **SHA-1** - `30213009 06052B0E 03021A05 000414`
- **SHA-256** - `3031300D 06096086 48016503 04020105 000420`
- **SHA-384** - `3041300D 06096086 48016503 04020105 000430`
- **SHA-512** - `3051300D 06096086 48016503 04020105 000440`

PRÁCTICA 6.7: Rellenando con PKCS#1 v1.5 para firma digital RSA con OpenSSL

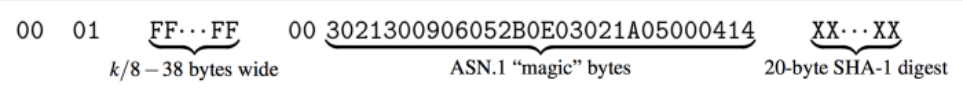
1. Genera una pareja de claves de 512 bits, por simplicidad, y exporta la clave pública a otro archivo.

```
$ openssl genrsa -out private.key 512
$ openssl rsa -in private.key -out public.key -pubout
```

2. Calcula el hash SHA1 de un archivo cualquiera.

```
$ openssl dgst -sha1 -out gon.sha1 -binary gon.txt
$ hexdump gon.sha1
000000 5d b6 42 59 94 3d 1e 0c 97 b1 31 0a af e8 cc 2e
000010 69 7c 32 b4
```

3. Crea el relleno para SHA-1 (puedes ayudarte de Cryptool, como en lecciones pasadas), que tiene la siguiente pinta:




```
$ hexdump gon.sha1.padded
000000 00 01 ff ff ff ff ff ff ff ff ff ff ff ff ff ff
000010 ff ff ff ff ff ff ff ff ff ff ff ff 00 30 21 30
000020 09 06 05 2b 0e 03 02 1a 05 00 04 14 5d b6 42 59
000030 94 3d 1e 0c 97 b1 31 0a af e8 cc 2e 69 7c 32 b4
```

4. Realiza la operación de firmar: $m^d \mod n$.

```
$ openssl pkeyutl -decrypt -inkey private.key -in gon.sha1.padded
-out signed -pkeyopt rsa_padding_mode:none
$ hexdump signed
000000 a4 73 ee 3f 57 43 cc 6f 2a 32 1a 1a bb 2d b6 68
000010 c8 d6 b4 03 15 8f 65 f3 6a 03 cf 66 a8 4f dd 4c
000020 a1 01 be 35 ba f6 c0 f0 c0 e7 7c 7d be f0 9d a9
000030 98 58 1a 7d 07 e8 7f c8 a1 24 d2 be 76 35 8a 72
```

La siguiente concatenación de comandos usando las opciones `-sign` y `-verifyrecover` del comando `pkeyutl` muestra perfectamente el relleno PKCS#1 v1.5 usado internamente:

```
$ type gon.txt | openssl dgst -sha1 -binary | openssl pkeyutl -sign
-inkey private.key | openssl pkeyutl -verifyrecover -inkey private.key
-hexdump -pkeyopt rsa_padding_mode:none
0000 - 00 01 ff ff ff ff ff ff-ff ff ff ff ff ff ff ff
0010 - ff ff ff ff ff ff ff ff-ff ff ff ff ff ff ff ff
0020 - ff ff ff ff ff ff ff ff-ff ff ff ff ff ff ff ff
0030 - ff ff ff ff ff ff ff ff-ff ff ff ff ff ff ff ff
0040 - ff ff ff ff ff ff ff ff-ff ff ff ff ff ff ff ff
0050 - ff ff ff ff ff ff ff ff-ff ff ff ff ff ff ff ff
0060 - ff ff ff ff ff ff ff ff-ff ff ff 00 5d b6 42 59
0070 - 94 3d 1e 0c 97 b1 31 0a-af e8 cc 2e 69 7c 32 b4
```

La limitación del uso del firmado con el comando `pkeyutl` es que solo firma datos del tamaño de la clave. Por tanto, podría usarse para firmar un hash, pero no un texto largo.

PRÁCTICA 6.8: Firmando con RSA PKCS#1 v1.5 un archivo en OpenSSL

1. Firma el mismo archivo de la práctica anterior, usando las mismas claves RSA y el mismo algoritmo de hash.

```
$ openssl dgst -sha1 -sign private.key -out signature gon.txt
$ hexdump signature
000000 a4 73 e3f 57 43 cc 6f 2a 32 1a 1a bb 2d b6 68
```

```
000010 c8 d6 b4 03 15 8f 65 f3 6a 03 cf 66 a8 4f dd 4c
000020 a1 01 be 35 ba f6 c0 f0 c0 e7 7c 7d be f0 9d a9
000030 98 58 1a 7d 07 e8 7f c8 a1 24 d2 be 76 35 8a 72
```

Por supuesto, este valor tiene que coincidir con el que has calculado en la práctica anterior.

2. Verifica que el valor de la firma es correcto.

```
$ openssl dgst -sha1 -verify public.key -signature signature gon.txt
Verified OK
```

6.2.1.2. The Probabilistic Signature Standard (PSS)

En la actualidad se utiliza el esquema de firma probabilística (RSA-PSS), estandarizado como parte de PKCS#1 v2.1. Combina la firma y la verificación con una codificación del mensaje. Este enfoque, a diferencia de otros esquemas basados en RSA como PKCS#1 v1.5, introduce un proceso de aleatorización que permite demostrar que la seguridad del método está estrechamente relacionada con la seguridad del propio algoritmo RSA. Esto hace que RSA-PSS sea más deseable como opción para la firma digital basada en RSA y haya sido adoptado por TLS 1.3.

PRÁCTICA 6.9: Firmando con RSA-PSS en OpenSSL

1. Genera una pareja RSA de claves pública y privada de 1.024 bits y exporta la clave pública a otro archivo.

```
$ openssl genrsa -out private.key 1024
$ openssl rsa -in private.key -out public.key -pubout
```

2. Firma un archivo cualquiera, prestando atención a los parámetros opcionales siguientes:

   - `rsa_padding_mode:mode`: los valores aceptables para `mode` son: `pkcs1` para el relleno PKCS#1, `sslv23` para el relleno SSLv23, `none` para ningún relleno, `oaep` para el modo OAEP, `x931` para el modo X9.31 y `pss` para PSS. Para el modo `oaep` sólo se admite el cifrado y el descifrado. Para el modo `pss` sólo se admiten la firma y la verificación.
   - `rsa_pss_saltlen:len`: solo para el modo `pss`, esta opción especifica la longitud del *salt*. Se admiten tres valores especiales: `digest` establece la longitud del *salt* a la longitud del digest, `max` establece la longitud del *salt* al valor máximo permitido. Con `-1` no usa *salt*. Al verificar `auto` hace que la longitud del *salt* se determine automáticamente en base a la estructura de bloques del PSS.
   - `rsa_mgf1_md:digest`: para el relleno de PSS y OAEP establece el digest MGF1. Si el digest MGF1 no está configurado explícitamente en modo PSS, entonces se utiliza el digest de firma.

3. Firma el mensaje usando RSA-PSS y SHA-256 como hash.

```
$ openssl dgst -sha256 -sigopt rsa_padding_mode:pss -sigopt
rsa_pss_saltlen:-1 -sign private.key -out PSSsignature gon.txt
```

4. Verifica la firma anterior.

```
$ openssl dgst -sha256 -sigopt rsa_padding_mode:pss -sigopt
rsa_pss_saltlen:-1 -signature PSSsignature -verify public.key gon.txt
Verified OK
```

6.2.2. Firmas ciegas con RSA

Para conseguir que una persona firme digitalmente un documento, pero sin que pueda ver el contenido del documento firmado, se utilizan igualmente valores aleatorios que ciegan al firmante, ocultando el contenido del mensaje a firmar. Posteriormente, conocido el valor aleatorio, puede verificarse el valor de la firma. Es el equivalente digital a firmar un documento poniendo encima un papel de calco: impide ver el documento a firmar, pero transfiere perfectamente la firma

al documento. ¿Y quién querría firma algo sin verlo antes? Estos protocolos de firma ciega su utilizan por ejemplo en aplicaciones de voto electrónico y de dinero digital.

El proceso de una firma ciega con RSA es el siguiente:

1. Calcula el hash del mensaje que quieres que te firmen ciegamente, $h(m)$, y un nonce, r.
2. Conocida la clave pública del firmante calcula el valor:

$$b = h(m)r^e \mod n$$

3. El firmante calcula el valor de la firma:

$$sig(b) = b^d \mod n$$

4. Una vez devuelto este valor por el firmante, extrae la firma ciega de tu mensaje:

$$sig(m) = sig(b)/r = h(m)^d r^{e^d}/r = h(m)^d \mod n$$

5. Si proporcionas $sig(m)$ y m a cualquier persona, podrá verificar que:

$$sig(m)^e \mod n = h(m)$$

Has conseguido que el firmante firme tu mensaje sin llegar a verlo, conservando así su confidencialidad, de manera que cualquiera pueda verificar su firma. Puedes aprender más sobre las firmas ciegas en esta entrada del blog Cryptographic Engineering.

6.2.3. Ataques a las firmas RSA

No solo el cifrado con RSA en modo libro de texto es vulnerable a un sinfín de ataques, sino también las firmas.

6.2.3.1. Obtención del módulo a partir de varias firmas

Dados un conjunto de parejas de textos y sus firmas, $S = \{(m_1, s_1), (m_2, s_2), \ldots, (m_i, s_i)\}$, con $|S| = k$, creadas con la misma clave, el objetivo es encontrar el valor del módulo, n, sin ninguna información adicional.

Obviamente, $s_i^e \mod n = m_i$ o, lo que es lo mismo, $s_i^e = m_i + r_i n$, de donde se deduce que $t_i = s_i^e - m_i = r_i n$. Observa que todos los t_i comparten el mismo factor n, que podrás recuperar con una cierta probabilidad sin más que calcular el máximo común divisor de todos los t_i:

$$n' = gcd(t_1, t_2, \ldots, t_k).$$

Concretamente, esta probabilidad puede calcularse como:

$$Pr[n' = n] = 1/\zeta(k)$$

donde ζ representa la función zeta de Riemann. Por ejemplo, para $k = 2$ la probabilidad de dar con el valor de n correcto es 0,61 y para $k \geq 14$ es 0,9999.

Puedes profundizar en estos ataques en el trabajo RSA, DH and DSA in the Wild.

6.3. Esquema de firma digital con DSA

En 1991, el NIST propuso un algoritmo de firma digital llamado tal cual: Digital Signature Algorithm (DSA). Posteriormente fue adoptado como la norma FIPS 186. El algoritmo es una variante del esquema de ElGamal. Al igual que Diffie-Hellman, su seguridad se basa en el Problema del Logaritmo Discreto (DLP). DSA utiliza un algoritmo que está diseñado para proporcionar sólo la función de firma digital. A diferencia de la RSA, no puede utilizarse para el cifrado o el intercambio de claves.

Acabas de ver cómo en la segunda aproximación a la firma digital primero se calcula el hash del mensaje a firmar, el cual se cifra utilizando la clave privada de Alice para crear la firma. A continuación, Alice transmite tanto el mensaje como la firma. Bob toma el mensaje y calcula su hash. Bob también descifra la firma usando la clave pública de Alice. Si el hash calculado coincide

con la firma descifrada, Bob acepta la firma como válida. Debido a que sólo Alice conoce la clave privada, sólo Alice podría haber producido una firma válida.

El enfoque DSA también hace uso de una función de hash. El código hash se proporciona como entrada a una función de firma junto con un número aleatorio k generado para esta firma en particular. La función de firma también depende de la clave privada del remitente (k_{prA}) y de un conjunto de parámetros conocidos por los participantes. Este conjunto constituye una clave pública global (k_{pubG}). El resultado es una firma que consta de dos componentes, etiquetados s y r.

En el extremo receptor, Bob genera el hash del mensaje entrante. El hash y la firma constituyen las entradas para una función de verificación. La función de verificación también depende de la clave pública global, así como de la clave pública del remitente (k_{pubA}), que se empareja con su clave privada. La salida de la función de verificación es un valor que es igual al componente r de la firma si la firma es válida. La función de firma es tal que sólo Alice, con conocimiento de la clave privada, podría haber producido la firma válida.

Y ahora, a por los detalles de los tres algoritmos: generación de claves, firma y verificación.

6.3.0.1. Generación de claves

La generación de claves tiene dos fases. La primera fase es una elección de parámetros de algoritmos que pueden ser compartidos por diferentes usuarios del sistema, mientras que la segunda fase calcula un único par de claves para un usuario.

Generación de parámetros

1. Elige una función hash segura, H. En el estándar original DSS se usaba SHA-1, pero, como recordarás de la Sec. 5.3.3, ya no se considera seguro. La versión actual del estándar utiliza SHA-2.
2. Elige un número primo p de L bits, tal que $512 \leq L \leq 1024$ y L es divisible por 64.
3. Elige un número primo q de 160 bits, tal que $p-1 = qz$, es decir, debe ser un factor primo de $p-1$.
4. Elige aleatoriamente h, donde $1 < h < p-1$, tal que $h^z \mod p > 1$.
5. Calcula $g = h^z \mod p$.

Los datos públicos son (p, q, g).

Generación de claves de usuario

1. Elige x de forma aleatoria, donde $1 < x < q-1$.
2. Calcula $y = g^x \mod p$.

x es la clave privada y y es la clave pública.

6.3.0.2. Firma

1. Elige un número aleatorio k, donde $1 < k < q$.
2. Calcula $r = (g^k \mod p) \mod q$.
3. Calcula $s = k^{-1}(H(m) + rx) \mod q$, donde $H(m)$ es la función hash aplicada al mensaje m.

La firma es el par (r, s). Si r o s es cero, se vuelve a repetir el procedimiento.

6.3.0.3. Verificación de firma

1. Calcula $w = s^{-1} \mod q$.
2. Calcula $u_1 = H(m)w \mod q$.
3. Calcula $u_2 = rw \mod q$.
4. Calcula $v = (g^{u_1} y^{u_2} \mod p) \mod q$.

La firma es válida si $v = r$.

Dada la dificultad de tomar logaritmos discretos, es inviable que un oponente recupere k a partir de r o que recupere x a partir de s.

Otro punto destacable es que la única tarea exigente desde el punto de vista computacional en la generación de firmas es el cálculo exponencial $g^k \mod p$. Como este valor no depende del mensaje que se va a firmar, se puede calcular con antelación. De hecho, un usuario podría precalcular una serie de valores de r que se utilizarían para firmar documentos según fuera necesario. La única otra tarea algo exigente es el cálculo de inversos multiplicativos modulares, k^{-1}. Una vez más, se pueden precalcular varios de estos valores.

PRÁCTICA 6.10: Firmando y verificando firmas con DSA en OpenSSL

1. Genera los parámetros públicos (p, q, g) para DSA de 2.048 bits.

```
$ openssl dsaparam -out dsaparam.pem 2048
$ openssl dsaparam -in dsaparam.pem -text -noout
P:
00:b1:d3:f3:23:99:56:46:b4:d7:6a:15:3d:c2:fe:
2a:d7:d7:f8:fa:05:e2:5f:82:34:56:87:2e:d1:d4:
...
Q:
00:9e:4b:c6:b7:e0:ec:af:21:00:e5:d8:98:d3:a2:
a4:47:56:26:78:e2:f9:bb:ec:57:25:d4:06:1c:64:
ac:42:87
G:
01:6d:bc:b7:0c:8a:5b:2c:ba:df:ac:9d:59:2b:6c:
b4:58:40:78:8c:5f:5f:7a:4e:2d:3c:60:ea:f3:4f:
...
```

2. Genera la clave privada DSA a partir de los parámetros anteriores.

```
$ openssl gendsa -out dsaprivate.key dsaparam.pem
Generating DSA key, 2048 bits
$ openssl dsa -in dsaprivate.key -text -noout
read DSA key
Private-Key: (2048 bit)
priv:
69:55:8d:14:68:39:ad:9a:20:72:e6:4c:b7:7d:2a:
36:20:48:ac:fb:54:78:5f:99:f1:aa:d5:55:8d:7f:
f2:c6
pub:
34:c8:9d:58:0d:85:7d:01:13:ea:6e:f2:dc:78:00:
b0:80:dd:ba:e9:a3:1f:64:fb:41:e9:57:b7:88:9a:
...
P:
00:b1:d3:f3:23:99:56:46:b4:d7:6a:15:3d:c2:fe:
...
```

3. Exporta la clave publica a otro archivo.

```
$ openssl dsa -in dsaprivate.key -out dsapublic.key -pubout
```

4. Firma un archivo cualquier de tu disco con la clave privada usando como función hash SHA-3/512.

```
$ openssl dgst -sha3-512 -sign dsaprivate.key -out signature gon.txt
```

5. Verifica la firma.

```
$ openssl dgst -sha3-512 -verify dsapublic.key -signature signature
gon.txt
Verified OK
```

6.4. Esquema de firma digital con ECDSA

El algoritmo estándar utilizado para la firma con ECC es ECDSA. Este algoritmo está sustituyendo a las firmas RSA y a las firmas DSA en muchas aplicaciones. Es, por ejemplo, el único algoritmo de firma usado en Bitcoin y es soportado por muchas implementaciones TLS y SSH.

Como con todos los esquemas de firma, ECDSA consiste en un algoritmo de generación de firmas que el firmante utiliza para crear una firma usando su clave privada y un algoritmo de verificación que un verificador utiliza para comprobar la corrección de una firma dada la clave pública del firmante. El firmante posee un número, d, como clave privada, y los verificadores poseen la clave pública, $P = dG$. Ambos saben de antemano qué curva elíptica utilizar, así como las coordenadas de un punto base, G, y su orden (n, el número de puntos de la curva).

6.4.1. Generación de firmas con ECDSA

Para firmar un mensaje, sigue los siguientes pasos:

1. Calcula el hash h del mensaje usando una función hash H previamente acordada, típicamente de 256 bits, que se interpreta como un número entre 0 y $n - 1$ (se trunca si fuera necesario). En el caso de Bitcoin, el hash se efectúa dos veces seguidas, para dificultar los ataques de cumpleaños (v. 5.2).

2. A continuación, elige un número aleatorio, k, entre 1 y $n - 1$. Si utilizas ECDSA determinista, entonces k se deriva con un HMAC a partir de $h + d$.

3. Calcula $R = kG$, un punto con las coordenadas (R_x, R_y), y establece $r = R_x$.

4. Calcula la prueba de firma como: $s = k^{-1}(h + rd) \mod n$ (recuerda que d es tu clave privada) y usa estos dos valores como la firma (r, s).

Esta firma codifica el punto aleatorio $R = kG$, junto con una prueba s, que confirma que el firmante conoce el mensaje h y la clave privada d. La prueba s es verificable mediante la correspondiente clave pública.

La longitud de la firma dependerá de las longitudes de las coordenadas que se utilicen. Por ejemplo, cuando se trabaja con una curva en la que las coordenadas son números de 256 bits, r y s tendrán ambos 256 bits de longitud, lo que arrojará una firma de 512 bits de longitud.

6.4.2. Verificación de la firma con ECDSA

El algoritmo de verificación de ECDSA utiliza la clave pública de un firmante para verificar la validez de una firma. Para verificar una firma ECDSA (r, s) y el hash de un mensaje, h:

1. El verificador comprueba primero que r y s son enteros en el rango $[1, n - 1]$, ya que en caso contrario, la firma se descarta inmediatamente como inválida.

2. A continuación calcula $w = 1/s$, el inverso de s en la firma, que es igual a $k/(h + rd)$ mód n, ya que s se define como $s = (h + rd)/k$.

3. Después, el verificador multiplica w por h para encontrar u según la siguiente fórmula:

$$wh = hk/(h + rd) = u$$

4. A continuación, el verificador multiplica w por r para encontrar v:

$$wr = rk/(h + rd) = v$$

5. Dados u y v, el verificador calcula el punto R según la siguiente fórmula:

$$R = uG + vP$$

6. Aquí, P es la clave pública del firmante, que es igual a dG y el verificador sólo acepta la firma si la coordenada x de R es igual al valor r de la firma.

Este proceso funciona porque, como último paso, se calcula el punto R sustituyendo la clave pública P por su valor real dG:

$$uG + vdG = (u + vd)G$$

Al sustituir u y v por sus valores reales, se obtiene:

$$u + vd = hk/(h + rd) + drk/(h + rd) = (hk + drk)/(h + rd) = k(h + dr)/(h + rd) = k$$

Esto revela que $(u + vd)$ es igual al valor k, elegido durante la generación de la firma y que $uG + vdG$ es igual al punto kG. En otras palabras, el algoritmo de verificación logra calcular el punto kG, el mismo punto calculado durante la generación de la firma. La validación se completa una vez que un verificador confirma que la coordenada x de kG es igual a la r recibida; de lo contrario, la firma es rechazada como inválida.

PRÁCTICA 6.11: Firmando y verificando firmas con ECDSA en Sagemath

Vamos a firmar un mensaje usando valores muy pequeños, para facilitar el seguimiento del ejercicio. Copia el siguiente código en Sagemath:

```
p = 17
F = FiniteField(p)
C = EllipticCurve(F, [ 2, 2 ])
G = C.point((5,1))
n = G.order()
d = 7
P = d*G
h = 26
k = 10
R = k*G
I = Integers(n)
r = I(R[0])
s = (h+r*d)/k
w = 1/s
u = w*h
v = w*r
Q = int(u)*G+int(v)*P
```

Puedes pintar la curva mediante la siguiente instrucción en Sagemath:

```
plot(C, aspect_ratio = 1) + plot(p/2,(x,0,p), color='red',linestyle='--')
```

¡A que mola!

¿Por qué funciona la verificación de firma ECDSA?

La firma ECDSA (r, s) funciona por la siguiente sencilla razón:

1. La firma codifica un punto aleatorio R (representado sólo por su coordenada x) mediante transformaciones de curva elíptica utilizando la clave privada d y el hash del mensaje h en un número s, que es la prueba de que el firmante del mensaje conoce la clave privada d. La firma (r, s) no puede revelar la clave privada debido a la dificultad del problema ECDLP.

2. La verificación de la firma decodifica el número de prueba s de la firma de vuelta a su punto original R, utilizando la clave pública P y el hash del mensaje h y compara la coordenada x de la R recuperada con el valor r de la firma.

PRÁCTICA 6.12: Firmando y verificando firmas con ECDSA en OpenSSL

1. Crea una pareja de claves ECC y exporta la clave pública a un archivo separado.

```
$ openssl ecparam -name secp256k1 -out ECC.private.key -genkey
$ openssl ec -in ECC.private.key -out ECC.public.key -pubout
```

2. Firma el archivo con la clave privada y SHA-256.

```
$ openssl dgst -sha256 -sign ECC.private.key -out ecdsa gon.txt
```

3. Observa el contenido de la firma:

```
$ hexdump ecdsa
000000 30 45 02 20 7c df 70 74 1b 02 cb 36 67 e4 19 d1
000010 fa 48 dc 45 19 e8 14 23 99 23 66 a6 d9 e6 8c 00
000020 3d 70 91 e0 02 21 00 90 62 a6 4e 3d 59 bf 50 2a
000030 f6 6b 46 30 76 dd 4d 38 2a 64 e4 70 8c ac 40 3d
000040 96 fb 91 7e 32 42 0f
```

4. Verifica la firma con la clave pública.

```
$ openssl dgst -sha256 -verify ECC.public.key -signature ecdsa
gon.txt
Verified OK
```

6.4.3. Codificación de firmas ECDSA

El estándar para serializar firmas se llama Distinguished Encoding Rules (DER). El formato de firma se define de la siguiente manera en DER:

1. Comienza con el byte 0x30.
2. Codifica la longitud del resto de la firma (normalmente 0x44 o 0x45).
3. Añade el byte marcador, 0x02.
4. Codifica r como un entero big-endian, pero prefíjalo con el byte 0x00 si el primer byte de $r \geq 0x80$. Antepón la longitud resultante a r.
5. Añade el byte marcador, 0x02.
6. Codifica s como un entero big-endian, pero prefíjalo con el byte 0x00 si el primer byte de $s \geq 0x80$. Antepón la longitud resultante a s.

A modo de ejercicio, ¿cómo codificarías en DER la siguiente firma?
r = 0x37206a0610995c58074999cb9767b87af4c4978db68c06e8e6e81d282047a7c6
s = 0x8ca63759c1157ebeaec0d03cecca119fc9a75bf8e6d0fa65c841c8e2738cdaec

6.4.4. La importancia de k

Una vez más, los siguientes ataques ponen de manifiesto la importancia de la aleatoriedad en criptografía.

6.4.4.1. Nunca reutilices el número aleatorio

Al generar las firmas ECDSA, es importante **mantener k realmente en secreto**. Si usas el mismo k para todas las firmas o si tu generador de números aleatorios resulta predecible, ¡un atacante sería capaz de averiguar tu clave privada!

Este es el tipo de error que cometió Sony hace unos años. Básicamente, la consola PlayStation 3 sólo puede ejecutar juegos firmados por Sony con ECDSA. De esta manera, si quisieras crear un nuevo juego para PlayStation 3, no podrías distribuirlo al público sin la firma de Sony. El problema es: todas las firmas hechas por Sony fueron generadas... redoble de tambores... ¡usando un valor para k estático!

En esta situación, podrías recuperar fácilmente la clave privada de Sony comprando sólo dos juegos firmados, extrayendo sus hashes h_1 y h_2 y sus firmas (r_1, s_1) y (r_2, s_2), junto con los parámetros del dominio:

1. Observa que $r_1 = r_2$ porque $r = Q_x \mod n$ y $P = kG$ son iguales para ambas firmas.
2. Considera que de la ecuación de s se deduce que $(s_1 - s_2) \mod n = k^{-1}(h_1 - h_2) \mod n$.
3. Ahora multiplica cada término de la ecuación por k: $k(s_1 - s_2) \mod n = (h_1 - h_2) \mod n$.
4. Divide por $(s_1 - s_2)$ para obtener $k = (h_1 - h_2)(s_1 - s_2)^{-1} \mod n$.

5. Finalmente, puedes extraer la clave privada usando la ecuación para s:

$$s = k^{-1}(h + rd) \mod n$$
$$d = r^{-1}(sk - h) \mod n$$

6.4.4.2. Usa números aleatorios, sin partes conocidas

Basta conocer unos pocos bits de k para recuperar su valor con un número pequeño de firmas. De hecho, si se conoce el tamaño de k incluso el conocimiento de uno solo de sus bits puede conducir a romper el esquema de firma ECDSA con el número suficiente de firmas. Los ataques más conocidos se basan en análisis de Fourier, el Problema del Número Oculto o en el algoritmo de simplificación de bases de retículos de Lenstra–Lenstra–Lovász (LLL).

El ataque más sencillo de entender es el basado en el algoritmo LLL. Imagina que has creado dos firmas, (r_1, s_1) y (r_2, s_2), usando dos nonces k_1 y k_2, respectivamente, con la misma clave privada, d, sobre dos mensajes distintos, m_1 y m_2. En concreto, estos nonces comparten el mismo prefijo fijo, lo que significa que sus bits más significativos son siempre los mismos. Si utilizas una serie de firmas ECDSA para construir una matriz de una manera determinada, el algoritmo LLL dará como resultado una matriz que permite recuperar la clave privada ECDSA: una de las filas de la salida de LLL contendrá todos los nonces de las firmas. Una vez recuperados los nonces, puedes utilizar el ataque básico descrito anteriormente para recuperar la clave secreta.

Para este ejemplo, piensa en una curva común, como la secp256k1. Por simplificar, limita los nonces a $b = 128$ bits, es decir, sus bits más significativos son cero. Con toda la información proporcionada puedes crear la siguiente matriz:

$$\begin{vmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ r_1 s_1^{-1} & r_2 s_2^{-1} & b/n & 0 \\ m_1 s_1^{-1} & m_2 s_2^{-1} & 0 & b \end{vmatrix}$$

Si introduces esta matriz en el algoritmo LLL, obtendrás una matriz reducida, una de cuyas filas contiene los nonces originales. Con ellos, podrás recuperar la clave secreta. Puedes encontrar más información y código en Python en ECDSA: Handle with Care.

PRÁCTICA 6.13: Aplicando el algoritmo LLL con Sagemath

Curva secp256r1:

```
p = 0xFFFFFFFF00000001000000000000000000000000FFFFFFFFFFFFFFFFFFFFFFFF
F = FiniteField(p)
a = 0xFFFFFFFF00000001000000000000000000000000FFFFFFFFFFFFFFFFFFFFFFFC
b = 0x5AC635D8AA3A93E7B3EBBD55769886BC651D06B0CC53B0F63BCE3C3E27D2604B
C = EllipticCurve(F, [ a, b ])
Gx = 0x6B17D1F2E12C4247F8BCE6E563A440F277037D812DEB33A0F4A13945D898C296
Gy = 0x4FE342E2FE1A7F9B8EE7EB4A7C0F9E162BCE33576B315ECECBB6406837BF51F5
G = C.point((Gx,Gy))
n = G.order()
I = Integers(n)
d = randint(1, n)
P = d*G
bits=127
h1 = randint(1, n)
k1 = randint(1, 2**bits)
h2 = randint(1, n)
k2 = randint(1, 2**bits)
R1 = k1*G
r1 = I(R1[0])
s1 = (h1+r1*d)/k1
R2 = k2*G
r2 = I(R2[0])
s2 = (h2+r2*d)/k2
M=matrix(QQ,[ [n,0,0,0], [0,n,0,0], [r1/s1,r2/s2,2**(bits+1)/n,0],
[h1/s1,h2/s2,0,2**(bits+1)] ])
```

```
print("k1 = ", k1, "k2 = ", k2)
L=M.LLL()
for row in L.column(0):
      print(row)
```

El resultado de una ejecución con valores aleatorios es:

```
k1 = 504307698777534046728140620768284109 0
k2 = 142017304139478330389273328685452790824
-468557429561481841588482287522395325 93
0
142017304139478330389273328685452790824
469540357363997595816247403921037980211
```

¡Puede observarse una coincidencia en la tercera fila con el segundo nonce!

Si quieres saber más sobre estos ataques, te animo a leer la referencia ya mencionada RSA, DH and DSA in the Wild.

6.4.4.3. Asegúrate de comprobar los valores de r y s

El algoritmo de verificación de firma explícitamente establece que es necesario validar que los dos valores r y s son enteros mayores o igual a uno antes de la verificación de firma. Java 15, de 2020, omitió esa comprobación en el momento en el que se reescribió de C++ a Java el código relacionado con las curvas elípticas. Seguro que se evitaron fallos relacionados con el manejo de memoria... pero se les olvidó un paso y ningún criptógrafo lo comprobó.

El fallo se notificó en noviembre de 2021. Oracle publicó la corrección en abril de 2022.

6.4.4.4. Más ataques contra ECDSA

En el repositorio de Github Known Attacks On Elliptic Curve Cryptography encontrarás una detallada explicación del funcionamiento de las curvas elípticas para intercambio de claves y para firma digital y una sección sobre ataques contra ECDSA con muchos ejemplos en Sagemath.

6.5. Comparación de la eficiencia de firma entre RSA y ECDSA

La criptografía de curva elíptica se considera a menudo como alternativa a RSA para criptografía de clave pública, pero ECC y RSA tienen poco en común. RSA se utiliza únicamente para cifrar y firmar, mientras que ECC constituye toda una familia de algoritmos que puede utilizarse para cifrar, generar firmas, acordar secretos y ofrecer funciones criptográficas avanzadas como el cifrado basado en la identidad (un tipo de cifrado que utiliza claves de cifrado derivadas de un identificador personal, como una dirección de correo electrónico).

Al comparar los algoritmos de firma de RSA y ECC, recordarás que en las firmas RSA, el firmante utiliza su clave privada d para calcular una firma como $s = m^d \mod n$, donde m es el mensaje a firmar y s es la firma. La verificación utiliza la clave pública e para confirmar que $s^e \mod n$ es igual a m, un proceso claramente más simple que el de ECDSA.

El proceso de verificación de RSA suele ser más rápido que la generación de firmas de ECC porque utiliza una clave pública pequeña e. Pero ECC presenta dos grandes ventajas sobre RSA:

- **Firmas más cortas**: dado que ECC trabaja con números más cortos, produce firmas más cortas que RSA (cientos de bits de largo, no miles de bits), lo que es una ventaja evidente si hay que almacenar o transmitir numerosas firmas, en especial en dispositivos constreñidos.
- **Velocidad de firma**: firmar con ECDSA también es mucho más rápido que firmar con RSA (aunque la verificación de firmas es casi igual de rápida) porque ECDSA trabaja con números mucho más pequeños que RSA para un nivel de seguridad similar.

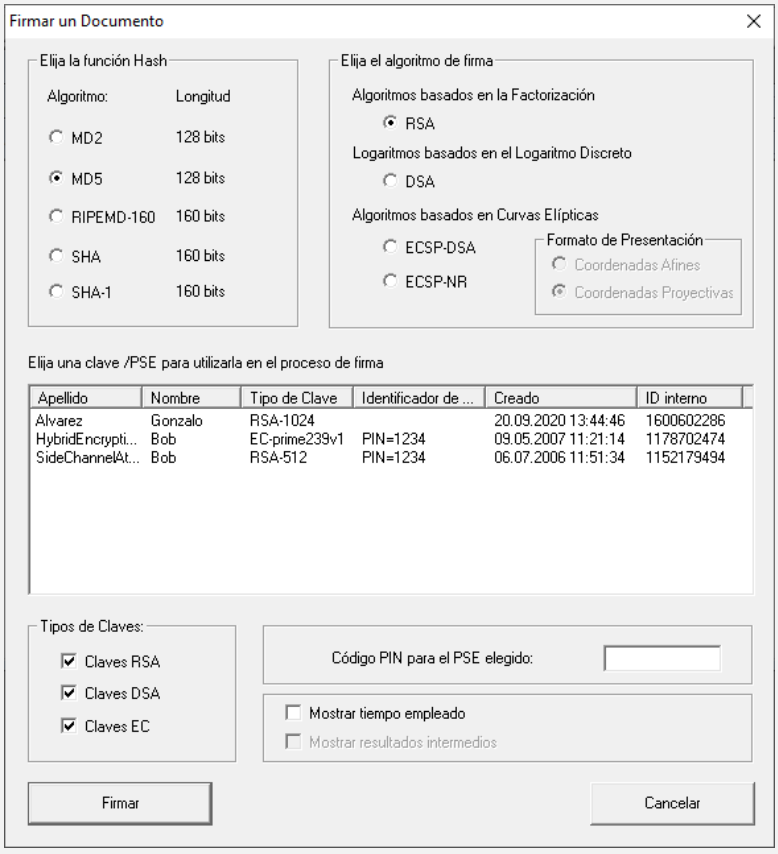
PRÁCTICA 6.14: Verificando la velocidad de las firmas RSA y ECDSA con OpenSSL

```
$ openssl speed ecdsap256 rsa4096
Doing 4096 bits private rsa's for 10s: 1612 4096 bits private RSA's in
9.03s
```

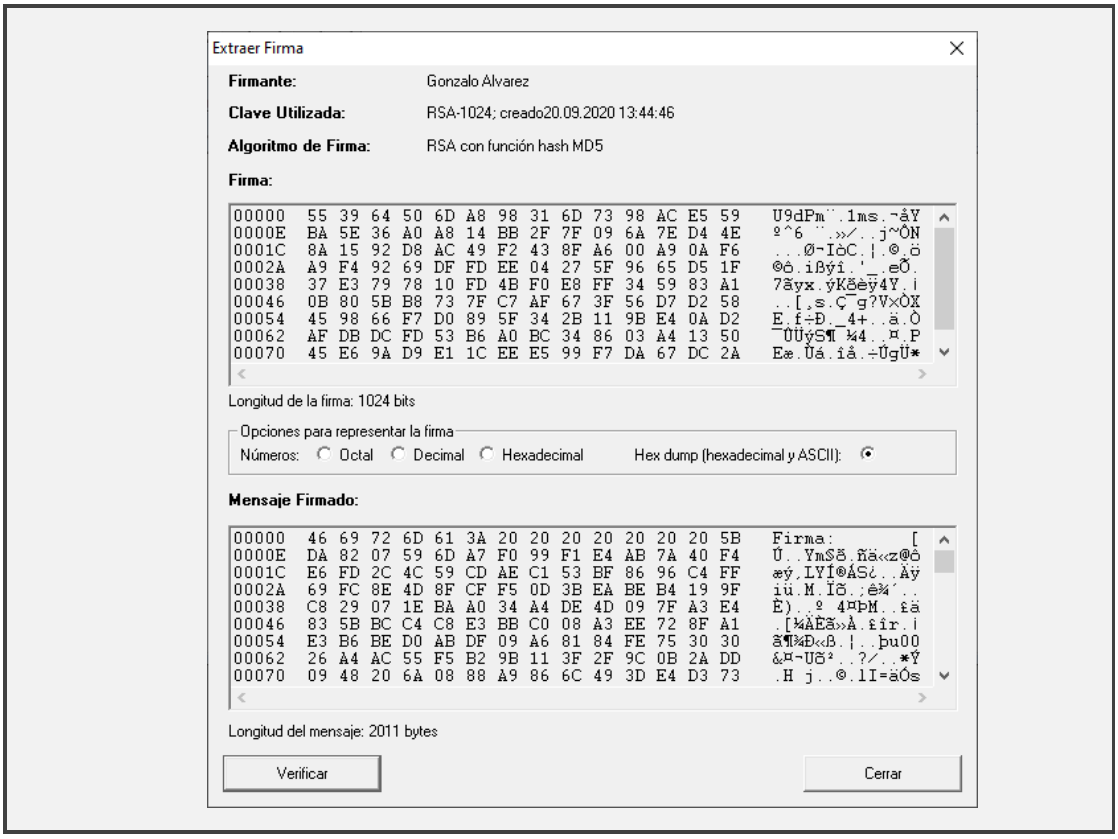
```
Doing 4096 bits public rsa's for 10s: 63969 4096 bits public RSA's in
9.98s
Doing 256 bits sign ecdsa's for 10s: 161519 256 bits ECDSA signs in 9.95s
Doing 256 bits verify ecdsa's for 10s: 59707 256 bits ECDSA verify in
9.23s
OpenSSL 1.1.1g 21 Apr 2020
sign verify sign/s verify/s
rsa 4096 bits 0.005603s 0.000156s 178.5 6406.9
sign verify sign/s verify/s
256 bits ecdsa (nistp256) 0.0001s 0.0002s 16228.0 6465.7
```

PRÁCTICA 6.15: Firmando documentos con RSA, DSA y ECDSA en CrypTool

1. Selecciona `Firma digital / PKI >Firmar documento`.



2. Elige la función hash.
3. Selecciona el tipo de firma: RSA, DSA o ECDSA. En función de si en prácticas anteriores has creado certificados para estos esquemas de firma, te aparecerán o no en listado de certificados. Selecciona el certificado con la identidad deseada para firmar el documento.
4. Introduce el código PIN y pulsa `Firmar`.
5. En una nueva ventana aparecerá toda la información de la firma, junto con el mensaje firmado.
6. Para facilitar su examen, selecciona `Firma digital / PKI >Extraer firma`.



6.6. Criptografía con umbral y el futuro de las firmas digitales

Como bien sabes, para firmar un mensaje m con RSA, realizas el cálculo $s = m^d \mod n$. Verificar la firma es muy sencillo por cualquier persona que conozca la clave pública, realizando la operación $s^e \mod n$.

¿Cómo conseguir que un grupo de personas coopere para firmar un mensaje? En lugar de firmar el mensaje una sola persona con la clave privada d, se puede separar esta clave en varias, por ejemplo, en tres: d_1, d_2, d_3, tales que:

$$d_1 + d_2 + d_3 = d \mod (p-1)(q-1)$$

Ahora, cada una de las partes puede firmar por su cuenta el mismo mensaje m:

$$s_1 = m^{d_1}, s_2 = m^{d_2}, s_3 = m^{d_3}$$

De manera que la firma total será el producto de las tres firmas:

$$s = s_1 \cdot s_2 \cdot s_3$$

Es fácil verificar que

$$s_1 \cdot s_2 \cdot s_3 = m^{d_1} \cdot m^{d_2} \cdot m^{d_3} = m^{d_1+d_2+d_3} = m^d \mod n$$

En otras palabras, sólo puede crearse una firma completa si cada una de las partes firma el mensaje con su parte de la clave privada. Así se protege la clave privada, d, ya que no se almacena completa en ningún servidor ni en ninguna memoria. Ni siquiera es necesario reunir las tres partes de la clave para firmar síncronamente todos los mensajes, ya que cada operación de cifrado de cada parte es independiente del resto. Podría comprometerse una parte de la clave o incluso dos y, aun así, la clave completa se mantendría segura.

Otros esquemas de criptografía con umbral más sofisticados poseen la propiedad k de n, ya mencionada en la Sec.3.6. Esta propiedad aporta:

- **Tolerancia a fallos**: una parte de la clave podría perderse o verse comprometida y, aun así, se podría realizar la operación criptográfica con la parte restante.

- **Necesidad de cooperación**: ninguna parte podrá realizar la operación criptográfica completa; al menos k partes han de ponerse de acuerdo.

Desde la perspectiva de un atacante, comprometer una parte de la clave no le servirá de nada: necesitará comprometer al menos k partes.

Como ves, la criptografía con umbral elimina los puntos únicos de fallo en criptografía, permitiendo redistribuir la responsabilidad de la custodia de las claves. Y no vayas a creer que todo queda en ejercicios matemáticos para cursos de postgrado: los productos de gestión de claves de Sepior y de Unbound constituyen los ejemplos más avanzados de soluciones basadas en criptografía con umbral de la actualidad. Como los otros campos de estudio, está en constante expansión y aparecerán nuevos resultados próximamente.

Puedes profundizar sobre las firmas con umbral en este artículo sobre el futuro de las firmas digitales para proteger criptodivisas.

6.7. Estándares de firma digital

El uso de certificados y firmas digitales también está debidamente estandarizado para garantizar la interoperabilidad.

- NIST
  - FIPS 186-4, Digital Signature Standard (DSS), july 2013
    - DSA, RSA, and ECDSA
- IETF
  - RFC7292: PKCS#12: Personal Information Exchange Syntax v1.1
  - RFC 3161: Internet X.509 Public Key Infrastructure Time-Stamp Protocol
- ISO
  - 20009-2:2013: Information technology — Security techniques — Anonymous entity authentication — Part 2: Mechanisms based on signatures using a group public key
- BSI
  - BSI TR-02102-1: Cryptographic Mechanisms: Recommendations and Key Lengths - Version: 2022-1
- CCN
  - Guía de Mecanismos Criptográficos autorizados por el CCN (CCN-STIC 221)
- ETSI
  - ETSI TS 119 312 V1.5.1 (2024-12): Electronic Signatures and Trust Infrastructures (ESI); Cryptographic Suites

Apéndice A

Una cálida bienvenida a ASN.1

Resulta esencial para un criptógrafo o practicante de ciberseguridad conocer el rol y la importancia de ASN.1 en muchos sistemas y protocolos criptográficos.

Para empezar, ASN.1 (Abstract Syntax Notation One) es un estándar internacional que define una notación para describir de forma abstracta tipos de datos, estructuras de datos y valores, es decir, totalmente independiente del hardware, del sistema operativo y del lenguaje de programación. Su propósito principal es asegurar la interoperabilidad entre sistemas y lenguajes de programación diferentes.

ASN.1 es fundamental en la definición de certificados X.509, vitales en las infraestructuras de clave pública (PKI). También se utiliza en listas de revocación de certificados (CRLs).

Si bien ASN.1 define estructuras de forma abstracta, estas estructuras necesitan codificarse para su transmisión o almacenamiento. Las codificaciones como BER, DER y PER son derivadas de ASN.1. En particular, DER es crucial en contextos criptográficos debido a su naturaleza canónica.

Dado que muchas bibliotecas y herramientas de ciberseguridad utilizan o procesan estructuras definidas en ASN.1, te será útil conocer al menos algunas herramientas que permiten trabajar con estos formatos, como decodificadores o verificadores. En concreto, OpenSSL permite trabajar con ASN.1 mediante el comando `asn1parse`.

PRÁCTICA A.1: Decodificando archivos en formato DER con OpenSSL

Si tienes datos codificados en DER o PEM que están definidos usando ASN.1 y quieres ver su contenido en un formato más legible, puedes utilizar la herramienta asn1parse de OpenSSL. Por ejemplo, para decodificar un archivo en formato DER:

```
$ openssl asn1parse -in private_key.pem
$ openssl ecparam -name secp256k1 -text -noout -param_enc explicit
openssl ecparam -name secp256k1 -out ECC.private.key -genkey
$ openssl ec -in ECC.private.key -out ECC.public.key -pubout
$ openssl dgst -sha256 -sign ECC.private.key -out firma.pem gon.txt
$ openssl dgst -sha256 -verify ECC.public.key -signature firma.pem
gon.txt
$ openssl asn1parse -in ECC.private.key
```

Esto te dará una representación desglosada de la estructura ASN.1 del archivo de clave privada.

Se puede usar para ver los valores de una firma:

```
$ openssl asn1parse -inform DER -in firma.pem
```

PRÁCTICA A.2: Decodificando claves RSA

```
$ openssl genrsa -out rsa_pri.key 512 openssl asn1parse -in rsa_pri.key -i
$ openssl asn1parse -in rsa_pri.key -strparse 22
$ openssl pkey -in rsa_pri.key -text -noout
$ openssl pkey -in rsa_pri.key -pubout -out rsa_pub.key
$ openssl pkey -pubin -in rsa_pub.key -text -noout
$ openssl asn1parse -in rsa_pub.key -i
$ openssl asn1parse -in rsa_pub.key -strparse 17
```

PRÁCTICA A.3: Construyendo claves RSA a partir de archivo ASN.1

Puedes construir una clave pública RSA a partir de un archivo de configuración de OpenSSL en formato ASN.1, si conoces todos los componentes de la clave (p, q, e, d y sus derivados):

```
$ type asn1_rsa_pub_key.cnf
asn1 = SEQUENCE:rsa_pub_key
[rsa_pub_key]
algorithm =SEQUENCE:rsa_alg
pubkey =BITWRAP,SEQUENCE:rsa_pub_components

[rsa_alg]
algorithm =OID:rsaEncryption
parameter =NULL

[rsa_pub_components]
n =INTEGER:0xBB6FE79432CC6EA2D8F970675A5A87BFBE1AFF0BE63E879F2AF
FB93644D4D2C6D000430DEC66ABF47829E74B8C5108623A1C0EE8BE217B3AD8D36D5EB4FCA1D9
e =INTEGER:0x010001

$ openssl asn1parse -genconf asn1_rsa_pub_key.cnf -noout -out
rsa_pub_key.der
$ openssl pkey -pubin -in rsa_pub_key.der -inform DER -text -noout
Public-Key: (512 bit)
Modulus:

00:bb:6f:e7:94:32:cc:6e:a2:d8:f9:70:67:5a:5a:
87:bf:be:1a:ff:0b:e6:3e:87:9f:2a:ff:b9:36:44:
d4:d2:c6:d0:00:43:0d:ec:66:ab:f4:78:29:e7:4b:
8c:51:08:62:3a:1c:0e:e8:be:21:7b:3a:d8:d3:6d:5e:b4:fc:a1:d9
Exponent: 65537 (0x10001)
```

Para una clave privada:

```
$ type asn1_rsa_pri_key.cnf
asn1 = SEQUENCE:rsa_pri_key

[rsa_pri_key]
version=INTEGER:0
n =INTEGER:0xBB6FE79432CC6EA2D8F970675A5A87BFBE1AFF0BE63E879F2AFFB9
3644D4D2C6D000430DEC66ABF47829E74B8C5108623A1C0EE8BE217B3AD8D36D5EB4FCA1D9
e =INTEGER:0x010001
d =INTEGER:0x6F05EAD2F27FFAEC84BEC360C4B928FD5F3A9865D0FCAAD291E2A5
2F4AF810DC6373278C006A0ABBA27DC8C63BF97F7E666E27C5284D7D3B1FFFE16B7A87B51D
p =INTEGER:0xF3929B9435608F8A22C208D86795271D54EBDFB09DDEF539AB083D
A912D4BD57
q =INTEGER:0xC50016F89DFF2561347ED1186A46E150E28BF2D0F539A1594BBD7F
E46746EC4F
exp1 =INTEGER:0x9E7D4326C924AFC1DEA40B45650134966D6F9DFA3A7F9D698CD4AB
EA9C0A39B9
exp2 =INTEGER:0xBA84003BB95355AFB7C50DF140C60513D0BA51D637272E355E3977
79E7B2458F
coeff =INTEGER:0x30B9E4F2AFA5AC679F920FC83F1F2DF1BAF1779CF989447FABC2F5
628657053A

$ openssl asn1parse -genconf asn1_rsa_pri_key.cnf -noout -out
rsa_pri_key.der
$ openssl pkey -in rsa_pri_key.der -inform DER -text -noout
Private-Key: (512 bit)
modulus:
00:bb:6f:e7:94:32:cc:6e:a2:d8:f9:70:67:5a:5a:
```

```
87:bf:be:1a:ff:0b:e6:3e:87:9f:2a:ff:b9:36:44:
d4:d2:c6:d0:00:43:0d:ec:66:ab:f4:78:29:e7:4b:
8c:51:08:62:3a:1c:0e:e8:be:21:7b:3a:d8:d3:6d:
5e:b4:fc:a1:d9
publicExponent: 65537 (0x10001)
privateExponent:
6f:05:ea:d2:f2:7f:fa:ec:84:be:c3:60:c4:b9:28:
fd:5f:3a:98:65:d0:fc:aa:d2:91:e2:a5:2f:4a:f8:
10:dc:63:73:27:8c:00:6a:0a:bb:a2:7d:c8:c6:3b:
f9:7f:7e:66:6e:27:c5:28:4d:7d:3b:1f:ff:e1:6b:
7a:87:b5:1d
prime1:
00:f3:92:9b:94:35:60:8f:8a:22:c2:08:d8:67:95:
27:1d:54:eb:df:b0:9d:de:f5:39:ab:08:3d:a9:12:
d4:bd:57
prime2:
00:c5:00:16:f8:9d:ff:25:61:34:7e:d1:18:6a:46:
e1:50:e2:8b:f2:d0:f5:39:a1:59:4b:bd:7f:e4:67:
46:ec:4f
exponent1:
00:9e:7d:43:26:c9:24:af:c1:de:a4:0b:45:65:01:
34:96:6d:6f:9d:fa:3a:7f:9d:69:8c:d4:ab:ea:9c:
0a:39:b9
exponent2:
00:ba:84:00:3b:b9:53:55:af:b7:c5:0d:f1:40:c6:
05:13:d0:ba:51:d6:37:27:2e:35:5e:39:77:79:e7:
b2:45:8f
coefficient:
30:b9:e4:f2:af:a5:ac:67:9f:92:0f:c8:3f:1f:2d:
f1:ba:f1:77:9c:f9:89:44:7f:ab:c2:f5:62:86:57:
05:3a
```

PRÁCTICA A.4: Decodificando claves públicas en DSA

```
$ openssl genpkey -genparam -algorithm DSA -out dsa_par.pem -pkeyopt
dsa_paramgen_bits:1024
$ openssl genpkey -paramfile dsa_par.pem -out dsa_pri.pem
$ openssl pkey -in dsa_pri.key -text -noout
$ openssl asn1parse -in dsa_pri.key
$ openssl asn1parse -in dsa_pri.key -strparse 318
$ openssl pkey -in dsa_pri.key -pubout -out dsa_pub.key
$ openssl pkey -pubin -in dsa_pub.key -text -noout
$ openssl asn1parse -in dsa_pub.key -i
$ openssl asn1parse -in dsa_pub.key -strparse 315
$ type asn1_dsa_pub_key.cnf
asn1 = SEQUENCE:dsa_pub_key

[dsa_pub_key]
algorithm =SEQUENCE:dsa_alg
public =BITWRAP,INTEGER:0x424A04378703B51FAD65CEC2C59BF6FA2CD40AA64C
43975B564004519798E5B2FCC74D4DE407B73812341ABBCE0878CD2CA23A4501992E6384
2531BCAA7A9702

[dsa_alg]
algorithm =OID:dsaEncryption
parameter =SEQUENCE:dsa_param

[dsa_param]
p =INTEGER:0xB2A22FE456DD10930CEED27DE2E58E69E27CB52E14BE655C5D0F4E
```

```
DC4A627A800A211DABD9B07D974EF3C997DD198E31C2B238696CBF5E9984B1EE3D11B2D9E3
q =INTEGER:0xA17145E13E0034692FC1BD85BDF88E1AA578818D
g =INTEGER:0x79F4E4FB123458BFDD388E0CC90D776C3CBB9C82E10668C71318D3
22AF5968EE8FB9F64F9AC09A9AAA24AD8DED795B7CE4BB81E02047AAC4D3E95513AA8FD5DD

$ openssl asn1parse -genconf asn1_dsa_pub_key.cnf -noout -out
dsa_pub_key.der

$ openssl pkey -pubin -in dsa_pub_key.der -inform DER -text -noout
pub:
42:4a:04:37:87:03:b5:1f:ad:65:ce:c2:c5:9b:f6:
fa:2c:d4:0a:a6:4c:43:97:5b:56:40:04:51:97:98:
e5:b2:fc:c7:4d:4d:e4:07:b7:38:12:34:1a:bb:ce:
08:78:cd:2c:a2:3a:45:01:99:2e:63:84:25:31:bc:
aa:7a:97:02
P:
00:b2:a2:2f:e4:56:dd:10:93:0c:ee:d2:7d:e2:e5:
8e:69:e2:7c:b5:2e:14:be:65:5c:5d:0f:4e:dc:4a:
62:7a:80:0a:21:1d:ab:d9:b0:7d:97:4e:f3:c9:97:
dd:19:8e:31:c2:b2:38:69:6c:bf:5e:99:84:b1:ee:
3d:11:b2:d9:e3
Q:
00:a1:71:45:e1:3e:00:34:69:2f:c1:bd:85:bd:f8:
8e:1a:a5:78:81:8d
G:
79:f4:e4:fb:12:34:58:bf:dd:38:8e:0c:c9:0d:77:
6c:3c:bb:9c:82:e1:06:68:c7:13:18:d3:22:af:59:
68:ee:8f:b9:f6:4f:9a:c0:9a:9a:aa:24:ad:8d:ed:
79:5b:7c:e4:bb:81:e0:20:47:aa:c4:d3:e9:55:13:
aa:8f:d5:dd
```

PRÁCTICA A.5: Decodificando claves privadas en DSA

```
type asn1_dsa_pri_key.cnf
asn1 = SEQUENCE:dsa_pri_key

[dsa_pri_key]
version =INTEGER:0
algorithm =SEQUENCE:dsa_alg
private =OCTWRAP,INTEGER:0x6A9C8BEFEB21DDD3F16FB3A0A202E718A70D19F8

[dsa_alg]
algorithm =OID:dsaEncryption
parameter =SEQUENCE:dsa_param

[dsa_param]
p =INTEGER:0xB2A22FE456DD10930CEED27DE2E58E69E27CB52E14BE655C5D0F4E
DC4A627A800A211DABD9B07D974EF3C997DD198E31C2B238696CBF5E9984B1EE3D11B2D9E3
q =INTEGER:0xA17145E13E0034692FC1BD85BDF88E1AA578818D
g =INTEGER:0x79F4E4FB123458BFDD388E0CC90D776C3CBB9C82E10668C71318D3
22AF5968EE8FB9F64F9AC09A9AAA24AD8DED795B7CE4BB81E02047AAC4D3E95513AA8FD5DD

$ openssl asn1parse -genconf asn1_dsa_pri_key.cnf -noout -out
dsa_pri_key.der

$ openssl pkey -in dsa_pri_key.der -inform DER -text -noout
Private-Key: (512 bit)
priv:
6a:9c:8b:ef:eb:21:dd:d3:f1:6f:b3:a0:a2:02:e7:
```

```
18:a7:0d:19:f8
pub:
42:4a:04:37:87:03:b5:1f:ad:65:ce:c2:c5:9b:f6:
fa:2c:d4:0a:a6:4c:43:97:5b:56:40:04:51:97:98:
e5:b2:fc:c7:4d:4d:e4:07:b7:38:12:34:1a:bb:ce:
08:78:cd:2c:a2:3a:45:01:99:2e:63:84:25:31:bc:
aa:7a:97:02
P:
00:b2:a2:2f:e4:56:dd:10:93:0c:ee:d2:7d:e2:e5:
8e:69:e2:7c:b5:2e:14:be:65:5c:5d:0f:4e:dc:4a:
62:7a:80:0a:21:1d:ab:d9:b0:7d:97:4e:f3:c9:97:
dd:19:8e:31:c2:b2:38:69:6c:bf:5e:99:84:b1:ee:
3d:11:b2:d9:e3
Q:
00:a1:71:45:e1:3e:00:34:69:2f:c1:bd:85:bd:f8:
8e:1a:a5:78:81:8d
G:
79:f4:e4:fb:12:34:58:bf:dd:38:8e:0c:c9:0d:77:
6c:3c:bb:9c:82:e1:06:68:c7:13:18:d3:22:af:59:
68:ee:8f:b9:f6:4f:9a:c0:9a:9a:aa:24:ad:8d:ed:
79:5b:7c:e4:bb:81:e0:20:47:aa:c4:d3:e9:55:13:
aa:8f:d5:dd
```

PRÁCTICA A.6: Para claves públicas en DH

```
$ openssl genpkey -genparam -algorithm DH -out dhp.pem -pkeyopt
dh_paramgen_prime_len:1024
$ openssl genpkey -paramfile dhp.pem -out dh_pri.key
$ openssl pkey -in dh_pri.key -text {noout
$ openssl asn1parse -in dh_pri.key -i
$ openssl asn1parse -in dh_pri.key -strparse 159
$ openssl pkey -in dh_pri.key -pubout -out dh_pub.key
$ openssl pkey -pubin -in dh_pub.key -text {noout
$ openssl asn1parse -in dh_pub.key -i
$ openssl asn1parse -in dh_pub.key -strparse 156
$ type asn1_dh_pub_key.cnf
asn1 = SEQUENCE:dh_pri_key

[dh_pri_key]
algorithm =SEQUENCE:dh_alg
private =BITWRAP,INTEGER:0x4CC9C94CEB5B13F2B4B3ABB9F20CB830FFC3BB5DC6
428C08B18CF3E5301908AD

[dh_alg]
algorithm =OID:dhKeyAgreement
parameter =SEQUENCE:dh_param

[dh_param]
prime =INTEGER:0x8B58E1395DF5CF48DA87B088C6525EFF4AECBDA5C269EF2378
A31A4A62B1647B
generator =INTEGER:0x02

$ openssl asn1parse -genconf asn1_dh_pub_key.cnf -noout -out dh_pub_key.der
$ openssl pkey -pubin -in dh_pub_key.der -inform DER -text -noout
DH Public-Key: (256 bit)
public-key:
4c:c9:c9:4c:eb:5b:13:f2:b4:b3:ab:b9:f2:0c:b8:
30:ff:c3:bb:5d:c6:42:8c:08:b1:8c:f3:e5:30:19:
08:ad
```

```
prime:
00:8b:58:e1:39:5d:f5:cf:48:da:87:b0:88:c6:52:
5e:ff:4a:ec:bd:a5:c2:69:ef:23:78:a3:1a:4a:62:
b1:64:7b
generator: 2 (0x2)
```

PRÁCTICA A.7: Para claves privadas en DH

```
$ type asn1_dh_pri_key.cnf
asn1 = SEQUENCE:dh_pri_key

[dh_pri_key]
version =INTEGER:0
algorithm =SEQUENCE:dh_alg
private =OCTWRAP,INTEGER:0x4233D0A221A766BA2CE0492F2B8F6A822BE38CF33E
3A6850C806C6031D78457E

[dh_alg]
algorithm =OID:dhKeyAgreement
parameter =SEQUENCE:dh_param

[dh_param]
prime =INTEGER:0x8B58E1395DF5CF48DA87B088C6525EFF4AECBDA5C269EF2378
A31A4A62B1647B
generator =INTEGER:0x02

$ openssl asn1parse -genconf asn1_dh_pri_key.cnf -noout -out dh_pri_key.der
$ openssl pkey -in dh_pri_key.der -inform DER -text -noout
DH Private-Key: (256 bit)
private-key:
42:33:d0:a2:21:a7:66:ba:2c:e0:49:2f:2b:8f:6a:
82:2b:e3:8c:f3:3e:3a:68:50:c8:06:c6:03:1d:78:
45:7e
public-key:
4c:c9:c9:4c:eb:5b:13:f2:b4:b3:ab:b9:f2:0c:b8:
30:ff:c3:bb:5d:c6:42:8c:08:b1:8c:f3:e5:30:19:
08:ad
prime:
00:8b:58:e1:39:5d:f5:cf:48:da:87:b0:88:c6:52:
5e:ff:4a:ec:bd:a5:c2:69:ef:23:78:a3:1a:4a:62:
b1:64:7b
generator: 2 (0x2)
```

Si tienes interés en profundizar en este estándar, la guía A Warm Welcome to ASN.1 and DER te sumergirá de lleno en su funcionamiento.

Apéndice B

Biblioteca Jedi sobre criptografía para todos los niveles de inquietud y de conocimiento

Nivel Joven Padawan: lecturas ligeras para introducirse en la criptografía con rigor y diversión

1. Luis Hernández Encinas, La criptografía, CSIC, 2016.
2. Simon Singh, The Code Book: Science of Secrecy from Ancient Egypt to Quantum Cryptography, Random House, 2000.
3. Joshua Holden, The Mathematics of Secrets: Cryptography from Caesar Ciphers to Digital Encryption, Princeton University Press, 2017.

Nivel Jedi: lecturas técnicas que no requieren conocimientos de matemáticas

1. Christof Paar & Jan Pelzl, Understanding Cryptography: A Textbook for Students and Practitioners, Springer, 2010.
2. William Stallings, Cryptography and Network Security: Principles and Practice, Pearson Education, 2017.
3. Seth James Nielson, Christopher K. Monson, Practical Cryptography in Python: Learning Correct Cryptography by Example, Apress, 2019.

Nivel Emperador de la Galaxia: lecturas hardcore que requieren conocimientos avanzados de matemáticas

1. Serge Vaudenay, A Classical Introduction to Cryptography: Applications for Communications Security, Springer, 2005.
2. Alfred J. Menezes, Paul C. van Oorschot, Scott A. Vanstone, Handbook of Applied Cryptography (Discrete Mathematics and Its Applications), CRC Press, 1996.
3. Jonathan Katz, Yehuda Lindell, Introduction to Modern Cryptography, Chapman and Hall/CRC, 2014.